

RK-XC7Z020 FPGA 开发教程

文档变更记录		
版本	时间	描述
V0.1	2024.10.22	初版

目录

一. 初识 ZYNQ	7
1. ZYNQ 介绍	7
2. ZYNQ 芯片命名	9
3. ZYNQ 简介	10
3.1 PL 简介	10
3.2 PS 简介	12
二. ZYNQ 开发板注意事项	16
1. ZYNQ IO 分配	16
1.1 PL 端 IO 分配	16
1.2 PS 端 IO 分配	18
2. 开发板使用注意事项	19
三. PL 的“Hello World”——点亮 LED 灯	20
1. 点亮 LED 灯	20
1.1 实验目标	20
1.2 LED 硬件介绍	20
1.3 新建工程	21
1.4 LED 驱动设计	27
1.5 LED 驱动仿真	28
1.6 综合与管脚约束	30
1.7 在线逻辑分析仪使用	31
1.8 时序约束	38
1.9 上板验证	39
2. LED 实验总结	42
四. 按键控制 LED 灯	43
1. 实验目标	43
2. 按键硬件电路	43
3. 按键驱动程序设计	43
4. 按键驱动仿真	46
5. 添加内部逻辑分析仪	47
6. 添加时序约束	49
7. 添加管脚约束	50
8. 上板验证	51
9. 实验总结	53
五. 呼吸灯实验	54
1. 呼吸灯实现原理	54
2. 呼吸灯硬件设计	54
3. 实验任务	54
4. PWM 驱动程序设计	54
5. PWM 驱动仿真	57
6. 添加内部逻辑分析仪	58
7. 添加时序约束	60
8. 添加管脚约束	61

9. 上板验证	61
10. 实验总结	63
六. IP 核之 PLL 实验	64
1. PLL IP 核简介	64
2. 实验任务	65
3. 程序设计	65
4. PLL 仿真设计及结果	70
5. 添加内部逻辑分析仪	71
6. 添加时序约束	72
7. 添加管脚约束	72
8. 上板验证	73
9. 实验总结	74
七. IP 核之单端口 RAM 实验	75
1. RAM 简介	75
2. 硬件设计	76
3. 实验任务	76
4. 程序设计	76
4.1 新建单端口 RAM IP	76
4.2 RAM 控制程序设计	79
5. RAM 仿真设计及结果	80
6. 添加内部逻辑分析仪	82
7. 添加时序约束	83
8. 添加管脚约束	84
9. 上板验证	84
10. 实验总结	85
八. IP 核之双端口 RAM 实验	86
1. 实验任务	86
2. 状态机简介	86
3. 程序设计	87
3.1 新建双端口 RAM IP	87
3.2 双端口 RAM 读写设计时序	89
3.3 双端口 RAM 读写模块设计	90
4. RAM 仿真设计及结果	95
5. 添加内部逻辑分析仪	97
6. 添加时序约束	98
7. 添加管脚约束	99
8. 上板验证	100
九. IP 核之 FIFO 实验	102
1. FIFO IP 简介	102
2. 实验任务	104
3. 程序设计	105
3.1 新建异步 FIFO IP 核	105
3.2 异步 FIFO 时序	108
3.3 FIFO 读写模块设计	110

4. FIFO 仿真设计及结果	113
5. 添加内部逻辑分析仪	115
6. 添加时序约束	116
7. 添加管脚约束	117
8. 上板验证	118
9. 实验总结	119
十. UART 串口通信实验	120
1. UART 串口简介	120
2. 实验任务	122
3. 程序设计	122
3.1 UART_RX 模块	123
3.2 UART_TX 模块	127
4. UART 驱动仿真及仿真结果	129
5. 添加内部逻辑分析仪	131
6. 添加时序约束	133
7. 添加管脚约束	134
8. 上板验证	134
9. 实验总结	136
十一. RS-485 串口通信实验	137
1. RS485 简介	137
2. RS485 硬件设计	138
3. 实验任务	139
4. 程序设计	139
5. RS485 驱动仿真及仿真结果	143
6. 添加时序约束	145
7. 添加管脚约束	146
8. 上板验证	147
9. 实验总结	148
十二. HDMI 彩条显示实验	149
1. HDMI 简介	149
2. 硬件设计	152
3. 实验任务	153
4. 程序设计	153
4.1 video_driver 驱动模块	154
4.2 video_display 模块	156
4.3 RGB2DVI 模块	156
4.4 编码模块	158
4.5 Serializer 模块设计	160
4.6 顶层模块设计	163
5. HDMI 彩条实验仿真及仿真结果	165
6. 添加内部逻辑分析仪	166
7. 添加时序约束	167
8. 添加管脚约束	168
9. 上板验证	169

10. 实验总结	170
十三. HDMI 方块移动实验	171
1. 程序设计	171
1.1 HDMI 显示模块设计	171
1.2 HDMI 方块移动顶层模模块	174
2. HDMI 方块移动仿真及仿真结果	175
3. 上板验证	177
4. 实验总结	178
十四. IIC 协议的 EEPROM 读写实验	179
1. EEPROM 及 IIC 简介	179
2. 硬件设计	187
3. 实验任务	189
4. 程序设计	189
4.1 IIC 驱动模块设计	189
4.2 E2PROM 读写模块	196
4.3 E2PROM 读写测试结果显示模块	198
5. 基于 I2C 协议的 EEPROM 仿真	199
5.1 添加 EEPROM 仿真文件	199
5.2 EEPROM 仿真结果	201
6. 添加内部逻辑分析仪	202
7. 添加时序约束	203
8. 添加管脚约束	204
9. 上板验证	205
10. 实验总结	208
十五. 以太网 UDP 协议栈实验	209
1. 网络传输格式简介	209
2. 硬件设计	214
3. 实验任务	215
4. 程序设计	215
4.1 MAC 层模块	215
4.2 ARP 模块	221
4.3 UDP 模块	226
4.4 TOP 层模块	229
5. 协议栈模块仿真	230
5.1 ARP 仿真验证	230
5.2 UDP 仿真验证	234
6. 添加内部逻辑分析仪	237
7. 添加时序约束	239
8. 添加管脚约束	240
9. 上板验证	241
9.1 硬件连接及修改 IP 地址	242
9.2 ICMP 验证	245
9.3 UDP 环回验证	248
10. 本章总结	250

一. 初识 ZYNQ

本篇将详细介绍开发板的硬件，通过该篇的介绍，将了解到开发板的功能与特点。为了让读者更好的使用此块开发板，本篇还将介绍一些使用开发板时的注意事项，请读者耐心阅读，避免因操作不当而造成不必要的损失。

1.ZYNQ 介绍

进入 21 世纪，FPGA 的发展遭遇了瓶颈，单纯的提升 FPGA 的容量已经不能满足各类应用的需求。由于更多客户开始追求更高的性价比，FPGA 不得不从单纯的可编程门阵列，逐步转变为拥有复杂功能的可编程片上系统。

Zynq-7000 系列就是一片带有可编程片上系统的芯片，它是由 Xilinx 公司于 2010 年 4 月推出的行业第一个可扩展处理平台，旨在为视频监控、汽车驾驶员辅助以及工厂自动化等高端嵌入式应用提供所需的处理能力与计算性能。这款基于 ARM 处理器的 SoC 可满足复杂嵌入式系统的高性能、低功耗和多核处理能力等要求。

Zynq 的全称是 Zynq-7000 All Programmable SoC，是由一个双核 ARM Cortex-A9 处理器和一个传统的现场可编程门阵列（FPGA）逻辑部件组成的。由于该新型器件的可编程逻辑部分基于 Xilinx 28nm 工艺的 7 系列 FPGA，因此该系列产品的名称中添加了“7000”，以保持与 7 系列 FPGA 的一致性，同时也方便日后本系列新产品的命名。

Zynq 实际上是一个片上系统（System on Chip, SoC）。那么，什么是“SoC”？一个能够实现一定功能的电路系统由多个模块构成，如处理器、接口、存储器、模数转换器等。这些功能模块可以由分立的器件来实现，然后在印刷电路板（PCB）上组合起来，最终形成板上系统（System-on-a-Board）。如下图所示：

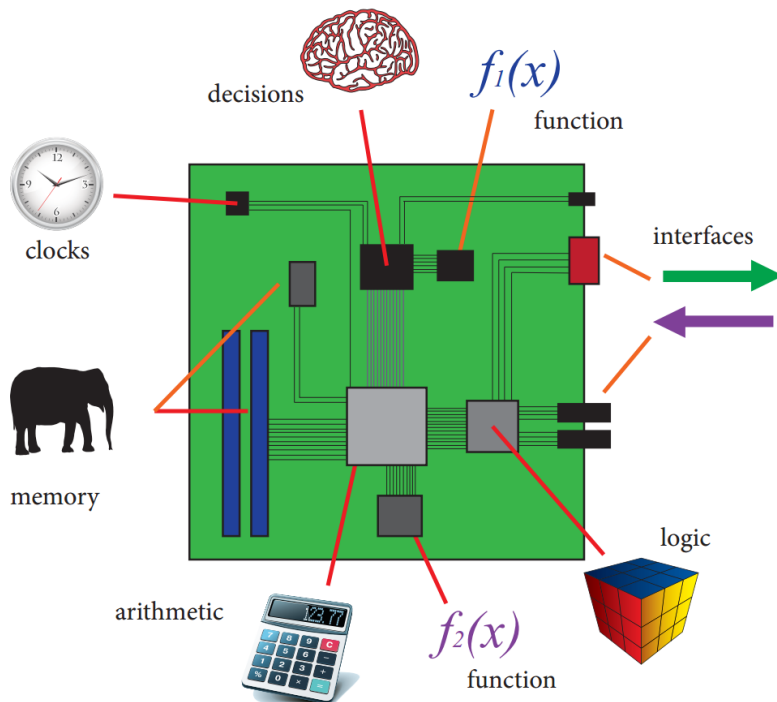


图 1-1 板上系统

在上图所示的板上系统中，绿色的矩形代表印刷电路板（PCB），上面各种颜色的小矩

形代表了系统中各个功能模块，如存储器等。这些模块的功能都由一个个独立的硅芯片分别实现的，它们之间通过 PCB 上的金属走线连接，最终构成一个完整的系统。而片上系统（System-on-Chip）指的是在单个硅芯片就可以实现整个系统的功能，其示意图如下所示：

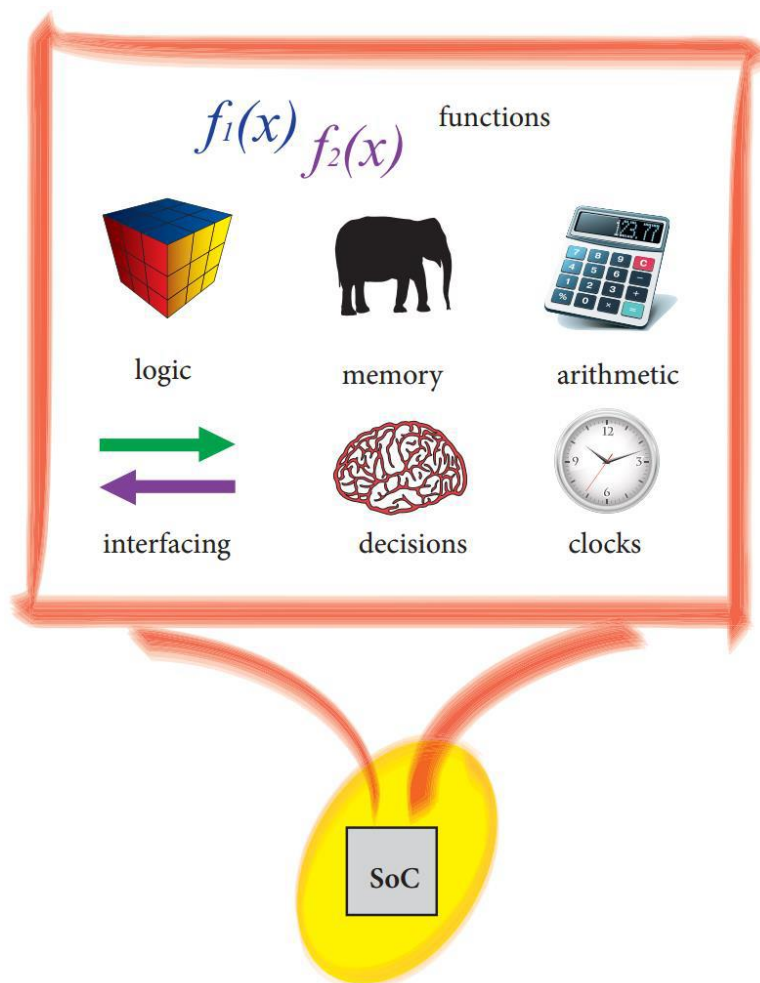


图 1-2 片上系统

如上图所示，片上系统 SoC 在一个芯片里就实现了存储、处理、逻辑和接口等各个功能模块，而不是像板上系统那样，需要用几个不同的物理芯片来实现。与板上系统相比，SoC 的解决方案成本更低，能在不同的系统单元之间实现更快更安全的数据传输，具有更高的整体系统速度、更低的功耗、更小的物理尺寸和更好的可靠性。

在过去，SoC 这个术语专指专用集成电路 (Application Specific Integrated Circuit, ASIC)。基于 ASIC 的 SoC 的代表性例子包括在 PC、平板和智能手机上使用的处理器，基于 ASIC 的 SoC 的主要缺点有两个：1、开发周期长且成本巨大；2、缺乏灵活性。ASIC 的开发工程不具有重复性且工程投入成本也是巨大的，使得这种 SoC 类型只适合于大批量而且寿命有限的产品中。因此，ASIC SoC 的局限性导致它们不适用于很多应用，特别是产品的灵活性和升级能力已经成为快速投入市场关键因素时。对于小批量或中批量的产品，ASIC SoC 也不是好的解决方案。

可编程片上系统 (SOPC, System-on-Programmable-Chip) 为上述应用提供了一个更灵活的解决方案：一种在可编程、可重新配置的芯片上实现的 SoC，其中可编程的芯片指的就是 FPGA。FPGA 天生的灵活性使其可以被随心所欲地重新配置，以实现不同系统的功能，包括嵌入式处理器。和使用 ASIC 来实现 SoC 相比，FPGA 能构成更为基础灵活的平台，方便系统的升级。

相比于 SOPC, Zynq 为实现灵活的 SoC 提供了一个更加理想的平台, Xilinx 将其打造成“全可编程片上系统 (APSoC, All-Programmable SoC)”。它将处理器的软件可编程性与 FPGA 的硬件可编程性进行完美整合, 以提供无与伦比的系统性能、灵活性与可扩展性。

Zynq 是由两个主要部分组成的: 一个是由双核 ARM Cortex-A9 为核心构成的处理系统 (PS, Processing System), 和一个等价于一片 FPGA 的可编程逻辑 (PL, Programmable Logic) 部分组成。Zynq 架构的简化模型如下图所示:

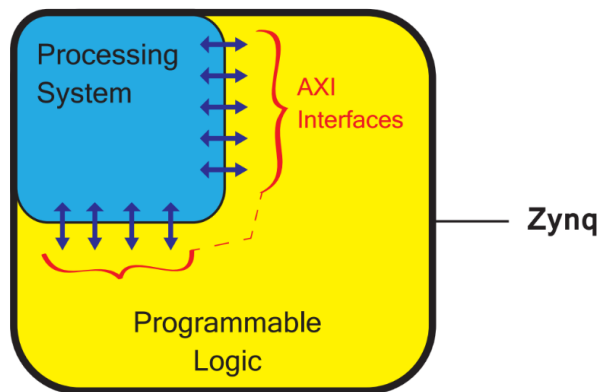


图 1-3 ZYNQ 架构简化模型

从上图中可以看出, 对于 Zynq 来说其实分为两个部分, PS 部分和 PL 部分。PL: 可编程逻辑 (Programmable Logic), 也就是 FPGA 部分。PS: 处理系统 (Processing System), 就是 ARM 的 SOC 的部分。

在 Zynq 上, ARM Cortex-A9 是一个应用级的处理器, 能运行像 Linux 这样的操作系统, 而可编程逻辑是基于 Xilinx 7 系列的 FPGA 架构。Zynq 架构实现了工业标准的 AXI 接口, 在芯片的两个部分之间实现了高带宽、低延迟的连接。这意味着处理器和逻辑部分各自都可以发挥最佳的用途, 而不会产生在两个分立器件之间的接口开销。与此同时, 又能获得系统被简化为单一芯片所带来的好处, 包括物理尺寸和整体成本的降低。

2.ZYNQ 芯片命名

通过第一小节大家已经了解了什么是 ZYNQ 并且也知道了开发板上的搭载的芯片型号分别是 XC7Z020-CLG484-2, 以 Xilinx 的官方文档来给做个简单的介绍, 方便大家以后自己去选型的时候可能会用得着这个小知识。

先来看下我从官方文档中截的图, 从这张图中我们可以看到它是以 ZYNQ-7000 系列的芯片为例的, 其实 Artix7、Kintex7 和 Virtex7 这三个系列的命名规则是通用的:

Zynq®-7000 Family Device Ordering Information

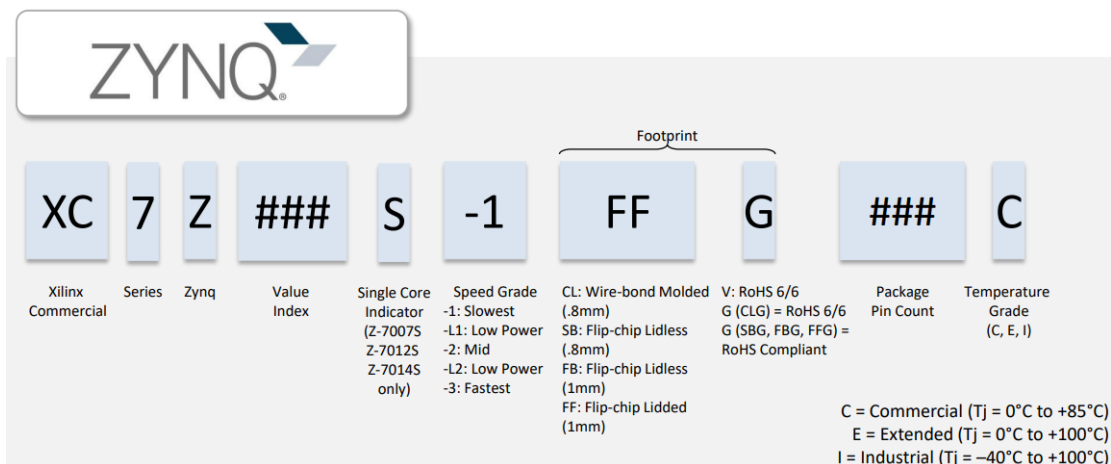


图 1-4 Zynq-7000 系列设备命名方式

以板载的 ZYNQ7020 芯片——“XC7Z020-CLG484-2”为例，“XC”是“Xilinx 公司”；“7”是 7 系列 (Series)；“Z”是型号是 zynq；“020”是价值索引 (ValueIndex)，该数值越大对应芯片内部资源越丰富，价值越高；“clg484”芯片封装信息，“CLG”是芯片制造工艺信息，“484”是芯片封装引脚计数 (Package Pin Count)；“-2”是速度等级 (Speed Grade)；“-I”是温度等级 (Temperature Grade)，温度等级有三种标准，分别是“C”、“E”与“I”。

C：商业级，表示芯片可以工作在 0°C 到+85°C；

E：扩展级，表示芯片可以工作在 0°C 到+100°C；

I：工业级，表示芯片可以工作在-40°C 到+125°C。

所以对照这个规则我们 ZYNQ-7020 芯片为例，ZYNQ-7020 是一块 Xilinx 公司 ZYNQ-7000 家族第七代芯片，该芯片为 7020 芯片有 484 个 IO 引脚，它的速度等级为 2，并且采用的是 CLG 封装方式。

3.ZYNQ 简介

3.1 PL 简介

ZYNQ PL 部分等价于 Xilinx 7 系列 FPGA，以开发板选用的 XC7Z020 芯片为例，其 PL 部分等价于 Xilinx Artix7 系列 FPGA。

FPGA 基本结构一般由六部分组成，分别为可编程输入/输出单元、基本可编程逻辑单元、底层嵌入功能单元、布线资源、嵌入式块 RAM 和内嵌专用硬核。FPGA 基本结构如下图所示：

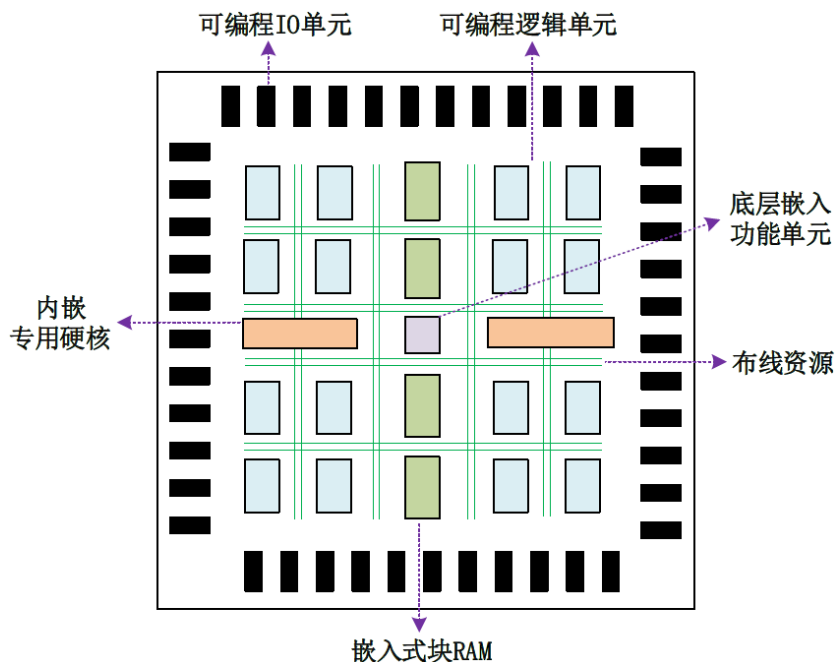


图 1-5 FPGA 内部基本结构

不同器件商或不同器件族的内嵌块 RAM 的结构不同, Zynq-7000 里的块 RAM 和 Xilinx 7 系列 FPGA 里的 BRAM 是等同的, 它们可以实现 RAM、ROM 和先入先出 (First In First Out, FIFO) 缓冲器。每个块 RAM 可以存储最多 36KB 的信息, 并且可以被配置为一个 36KB 的 RAM 或两个独立的 18KB RAM。默认的字宽是 18 位, 这样的配置下每个 RAM 含有 2048 个存储单元。RAM 还可以被“重塑”来包含更多更小的单元 (比如 4096x9 位或 8192x4 位), 或是另外做成更少更长的单元 (如 1024x36 位或 512x72 位)。把两个或多个块 RAM 组合起来可以形成更大的存储容量。PL 中的块 RAM 示意图如下所示:

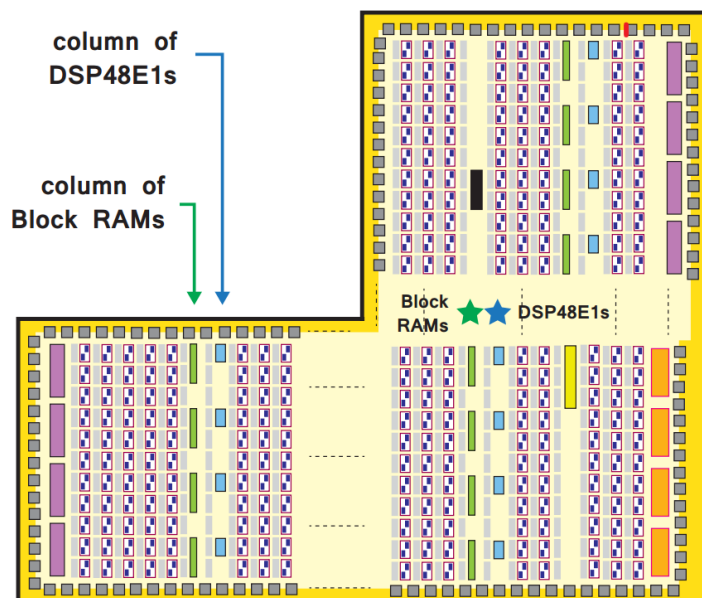


图 1-5 FPGA 中的 Block RAM 结构

除了块 RAM, 还可以将 LUT 配置成 RAM、ROM、FIFO 等存储结构, 这种技术被称为分布式 RAM。根据设计需求, 块 RAM 的数量和配置方式也是器件选型的一个重要标准。而内嵌专用硬核与前面的底层嵌入单元是有区分的, 这里讲的内嵌专用硬核主要指那些通用性相对较弱, 不是所有 FPGA 器件都包含硬核。如在 ZYNQ 的 PL 端有一个数模混合模块—

—XADC，它就是一个硬核。XADC 包含两个模数转换器（ADC），一个模拟多路复用器，片上温度和片上电压传感器等。我们可以利用这个模块监测芯片温度和供电电压，也可以用来测量外部的模拟电压信号。

最后需要注意的是：所有的 ZYNQ 器件的 PL 部分都是可以单独当作 FPGA 去使用，其开发流程与纯 FPGA 器件的开发流程没有任何区别。

3.2 PS 简介

Zynq 实际上是一个以处理器为核心的系统，PL 部分可以看作是它的一个外设。我们可以通过使用 **AXI (Advanced eXtensible Interface)** 接口的方式调用 IP 核，系统通过 **AXI** 接口将 IP 与处理器连接，也就是实现 **PL** 与 **PS** 互联。Zynq-7000 系列的亮点在于它包含了完整的 ARM 处理器系统，且处理器系统中集成了内存控制器和大量的外设，使 Cortex-A9 处理器可以完全独立于可编程逻辑单元。实际上在 Zynq 中，PL 和 PS 两部分的供电电路是独立的，这样 PS 或 PL 部分不被使用的话就可以被断电。

Zynq 中集成的是一颗“硬核”处理器，它是硅芯片上专用且经过优化的硬件电路，硬核处理器的优势是它可以获得相对较高的性能。另外，Zynq 中的硬件处理器和软核处理器并不冲突，我们完全可以使用 PL 的逻辑资源搭建一个软核处理器，来和 ARM 硬核处理器协同工作。

需要注意的是，Zynq 处理器系统里并非只有 ARM 处理器，还有一组相关的处理资源，共同形成了一个应用处理器单元（Application Processing Unit, APU），另外还有扩展外设接口、cache 存储器、存储器接口、互联接口和时钟发生电路等。

Zynq 处理器系统（PS）示意图如下所示，其中红色高亮区域为 APU。

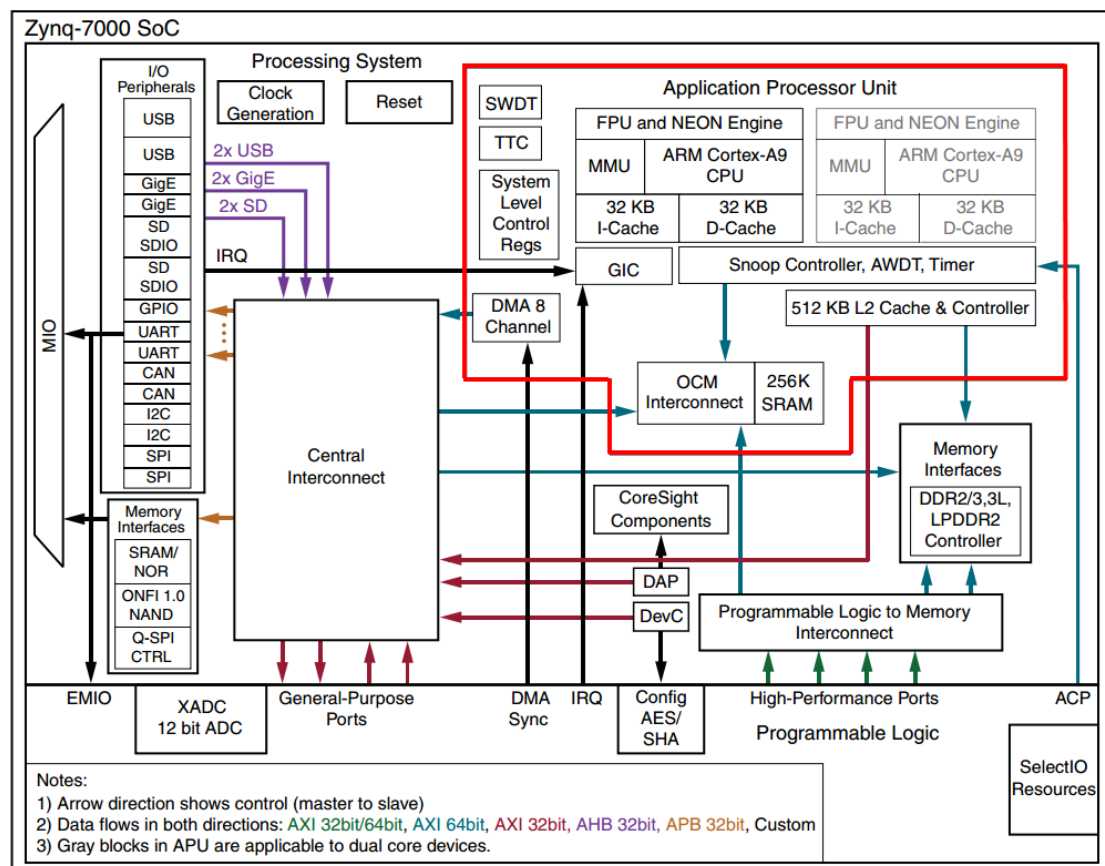


图 1-6 PS 系统示意图

(1) APU

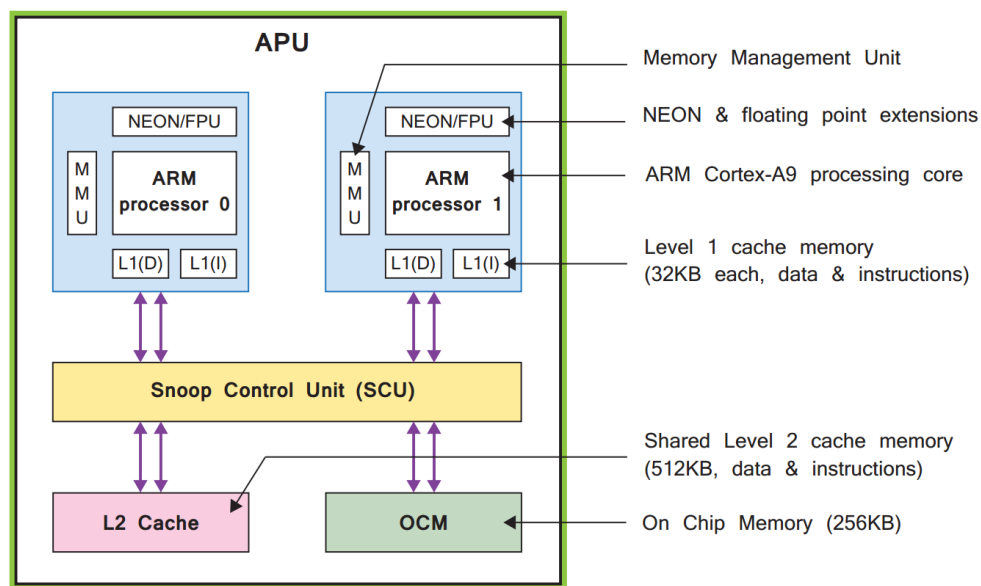


图 1-7 APU 简化示意图

如上图所示是 APU 的简化框图。APU 主要是由两个 ARM 处理器核组成的，每个都关联了一些可计算的单元：一个 NEONTM 媒体处理引擎 (Media Processing Engine, MPE) 和浮点单元 (Floating PointUnit, FPU)；一个内存管理单元 (Memory Management Unit, MMU)；和一个一级 cache 存储器 (分为指令和数据两个部分)。APU 里还有一个二级 cache 存储器，再往下还有片上存储器 (On Chip Memory, OCM)，这些是两个 ARM 处理器共用的。最后，由一个一致性控制单元 (Snoop Control Unit, SCU) 在 ARM 核和二级 cache 及 OCM 存储器之间形成了桥连接。SCU 还部分负责与 PL 对接，图中没有标出这个接口。

(2) 外部接口

如图 1-6 中所示，ZYNQ PS 端实现了众多接口，既有 PS 和 PL 之间的，也有 PS 和外部部件之间的。PS 和外部接口之间的通信主要是通过复用的输入/输出 (Multiplexed Input/Output, MIO) 实现的，它提供了可以灵活配置的 54 个引脚，这表明外部设备和引脚之间的映射是可以按需定义的。当需要扩展超过 54 个引脚的时候可以通过扩展 MIO (Extended MIO, EMIO) 来实现，EMIO 并不是 PS 和外部连接之间的直接通路，而是通过共用了 PL 的 I/O 资源来实现的。

I/O 接口	说明
SPI (x2)	串行外设接口(Serial Peripheral Interface) 基于 4 引脚接口的串行通信的事实标准, 可以用于主机或从机模式。
I ² C (x2)	I ² C 总线与 I ² C 总线标准第二版兼容。支持主机和从机模式。
CAN (x2)	控制器区域网络(Controller Area Network)兼容 IS0118980-1,CAW2.0A 和 CAN2.0B 标准的接口控制器。
UART (x2)	通用异步收发器(Universal Asynchronous Receiver Transmitter)用于串行通信的低速数据调制解调器接口。常用于和主机 PC 的终端连接。
GPIO	通用输入/输出(General Purpose Input./Output)有 4 组 GPIO,每组 32 位。
SD (x2)	用于和 SD 卡存储器对接。
USB (x2)	通用串行总线(Universal Serial Bus)兼容 USB2.0,可以做主机、设备或灵活配置(“on-the-go”也就是 OTG 模式, 意思是它可以在主机和设备模式之间做切换)。
Ethernet (x2)	以太网 MAC 外设, 支持 10Mbps、100Mbps 和 1Gbps 模式。

(3) 存储器接口

ZYNQ-7000 SoC 上的存储器接口单元包括一个动态存储器控制器和几个静态存储器接口模块。动态存储器控制器可以用于 DDR3、DDR3L、DDR2 或 LPDDR2。静态存储器控制器支持一个 NAND 闪存接口、一个 Quad-SPI 闪存接口、一个并行数据总线和并行 NOR 闪存接口

(4) 片上存储器

片上存储器包括 256kB 的 RAM (OCM) 和 128kB 的 ROM (Boot ROM)，OCM 支持两个 64 位 AXI 从机接口端口，一个端口专用于通过 APU SCU 的 CPU/ACP 访问，而另一个是由 PS 和 PL 内其他所有的总线主机所共享的。Boot ROM 是 ZYNQ 芯片上的一块非易失性存储器，它包含了 ZYNQ 所支持的配置器件的驱动。Boot ROM 对于用户是不可见的，专门保留且只用于引导的过程。

(5) AXI 接口

ZYNQ 将高性能 ARM Cortex-A 系列处理器与高性能 FPGA 在单芯片内紧密结合，为设计带来了如减小体积和功耗、降低设计风险、增加设计灵活性等诸多优点。在将不同工艺特征的处理器与 FPGA 融合在一个芯片上之后，片内处理器与 FPGA 之间的互联通路就成了 ZYNQ 芯片设计的重中之重。如果 Cortex-A9 与 FPGA 之间的数据交互成为瓶颈，那么处理器与 FPGA 结合的性能优势就不能发挥出来，所以熟练掌握 AXI 接口将是使用 ZYNQ 的重中之重。

AXI 协议是一种高性能、高带宽和低延迟的片内总线，该总线是 ARM 公司所提出的 AMBA (Advanced Microcontroller Bus Architecture) 协议的一部分。AXI4 协议支持以下三种类型的接口：

- a. AXI4：高性能存储映射接口。
- b. AXI4-Lite：简化版的 AXI4 接口，用于较少数据量的存储映射通信。
- c. AXI4-Stream：用于高速数据流传输，非存储映射接口。

在这里首先我们首先解释一下存储映射 (Memory Map) 这一概念。如果一个协议是存储映射的，那么主机所发出的会话（无论读或写）就会标明一个地址。这个地址对应于系统存储空间中的一个地址，表明是针对该存储空间的读写操作。

AXI4 协议支持突发传输，主要用于处理器访问存储器等需要指定地址的高速数据传输场景。在 PS 和 PL 之间的主要连接是通过一组 9 个 AXI 接口，每个接口有多个通道组成。这些形成了 PS 内部的互联以及与 PL 的连接，其结构如下图所示：

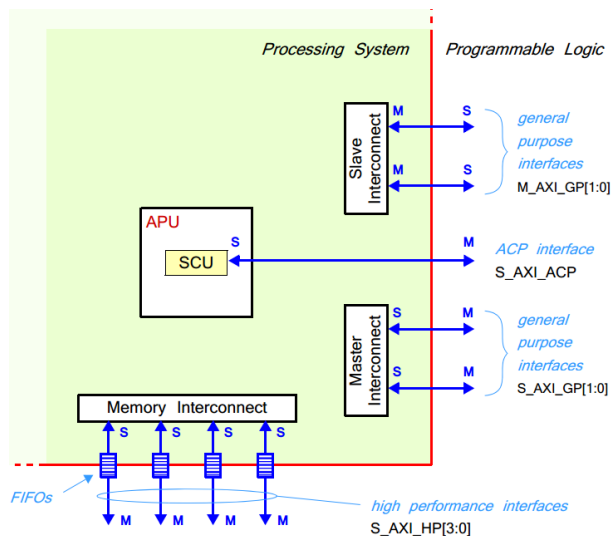


图 1-8 PS 与 PL 的 AXI 互联

将图 1-8 中的互联接口总结后如下所示：

Interface Name	Interface Description	Master	Slave
M_AXI_GP0	General Purpose (AXI_GP)	PS	PL
M_AXI_GP1		PS	PL
S_AXI_GP0	General Purpose (AXI_GP)	PL	PS
S_AXI_GP1		PL	PS
S_AXI_ACP	Accelerator Coherency Port (ACP), cache coherent transaction	PL	PS
S_AXI_HP0	High Performance Ports (AXI_HP) with read/write FIFOs. (Note that AXI_HP interfaces are sometimes referred to as AXI Fifo Interfaces, or AFIs).	PL	PS
S_AXI_HP1		PL	PS
S_AXI_HP2		PL	PS
S_AXI_HP3		PL	PS

图 1-9 PS 与 PL 的 AXI 接口

上图给出了每个接口的简述，标出了主机和从机（按照惯例，主机是控制总线并发起会话的，而从机是做响应的）。需要注意的是，接口命名的第一个字母表示的是 PS 的角色，也就是说，第一个字母“M”表示 PS 是主机，而第一个字母“S”表示 PS 是从机。

图 1-9 中 PS 和 PL 之间的 9 个 AXI 接口可以分成三种类型：

a. **通用 AXI** (General Purpose AXI)：一条 32 位数据总线，适合 PL 和 PS 之间的中低速通信。接口是不带缓冲的透传。总共有四个通用接口：两个 PS 做主机，另两个 PL 做主机。

b. **加速器一致性端口** (Accelerator Coherency Port)：在 PL 和 APU 内的 SCU 之间的单个异步连接，总线宽度为 64 位。这个端口用来实现 APU cache 和 PL 的单元之间的一致性，PL 是做主机的。

c. **高性能端口** (High Performance Ports)：四个高性能 AXI 接口，带有 FIFO 缓冲来提供“批量”读写操作，并支持 PL 和 PS 中的存储器单元的高速率通信。数据宽度是 32 或 64 位，在四个接口中 PL 都是做主机的。

上面的每条总线都是由一组信号组成的，这些总线上的会话是根据 AXI4 总线协议进行通信的。有关 AXI4 协议更详细的内容，将在后续的章节进行介绍。

二. ZYNQ 开发板注意事项

本章, 将向大家介绍此块开发板的板载资源, 让大家对开发板板载的硬件有个初步理解, 并向大家介绍开发板的使用注意事项, 为后面的学习做好准备。

1.ZYNQ IO 分配

开发板的主控芯片为 XC7Z020CLG484-2, XC7Z020 芯片有 6 个 BANK (比 XC7Z010 多一个 BANK) 分配给用户使用和最大 253 个用户 I/O, 了解器件的 IO 分配方式, 有助于我们对设计进行合理的 IO 分配, 减少硬件出错的可能性。ZYNQ 的 IO 口分成了 PL 和 PS 两部分, 我们将分别介绍 PL 和 PS 两部分的 IO 分配。

1.1 PL 端 IO 分配

ZYNQ 的 PL 侧和传统 FPGA 一样, 可以灵活地分配到不同的 IO 口上。在 XC7Z020 中, PL 端的 IO 被分成了 4 组, 分别是 BANK 13、BANK33、BANK34、BANK35。同一个 Bank 中的所有 IO 供电相同, 而各个 Bank 的 IO 供电可以不同, 如开发板上是将 BANK13、BANK33 连接到 3.3V 的电源上, BANK34 与 BANK35 连接的电源为: 2.5V。电源电路如下图所示, 而其中 ADJ1 电压为 2.5V, ADJ2 电压为 3.3V。

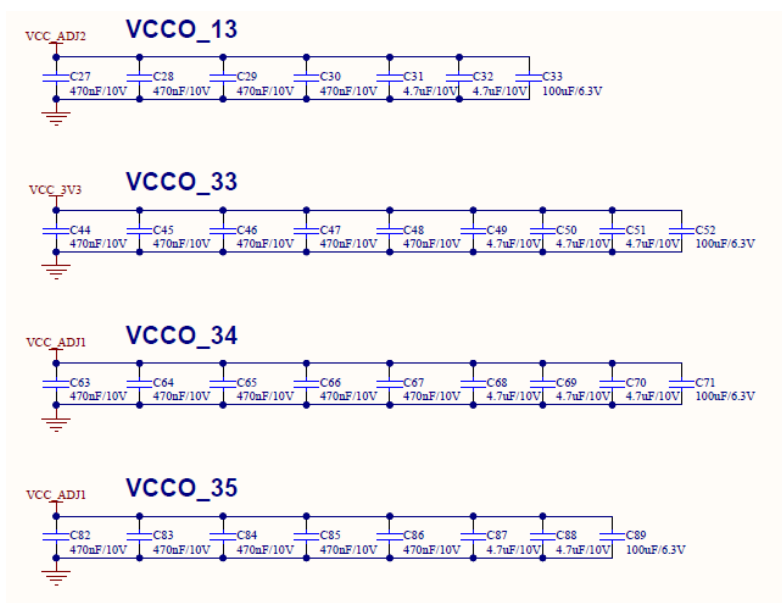


图 2-1 用户 BANK 电源电压

以 BANK 33 为例, BANK13 连接资源如下图所示, 从图中可以看出 BANK 33 连接了各种各样的外设, 主要有以太网、HDMI 接口、按键等。

U2B

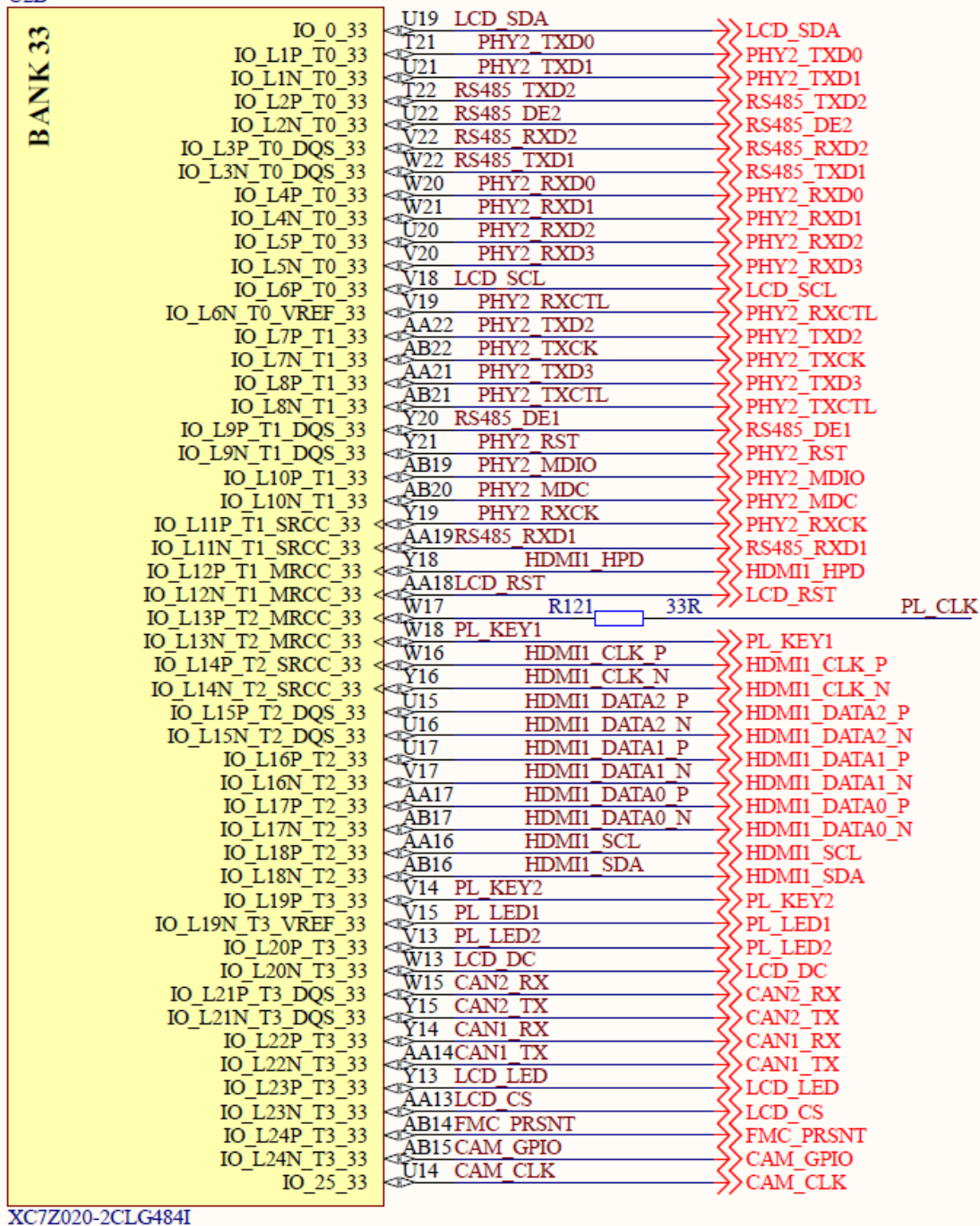


图 2-2 BANK 33 外设

但从图 2-2 中可以看出，若对着硬件原理图一个一个找外设管脚十分麻烦，也很容易出错。为快速约束 PL 端的管脚，可参考开发板资料中→硬件资料文件夹→管脚定义。该文件管脚是与硬件板卡相一致的，方便后续开发使用。该文件路径如下图所示：



图 2-3 硬件管脚定义

1.2 PS 端 IO 分配

在 ZYNQ PS 端同样也包含了 3 个 IO BANK, 主要为 BANK500、BANK501 及 BANK502, 以 BANK500 和 BANK501 为例, 其外设如下图所示:

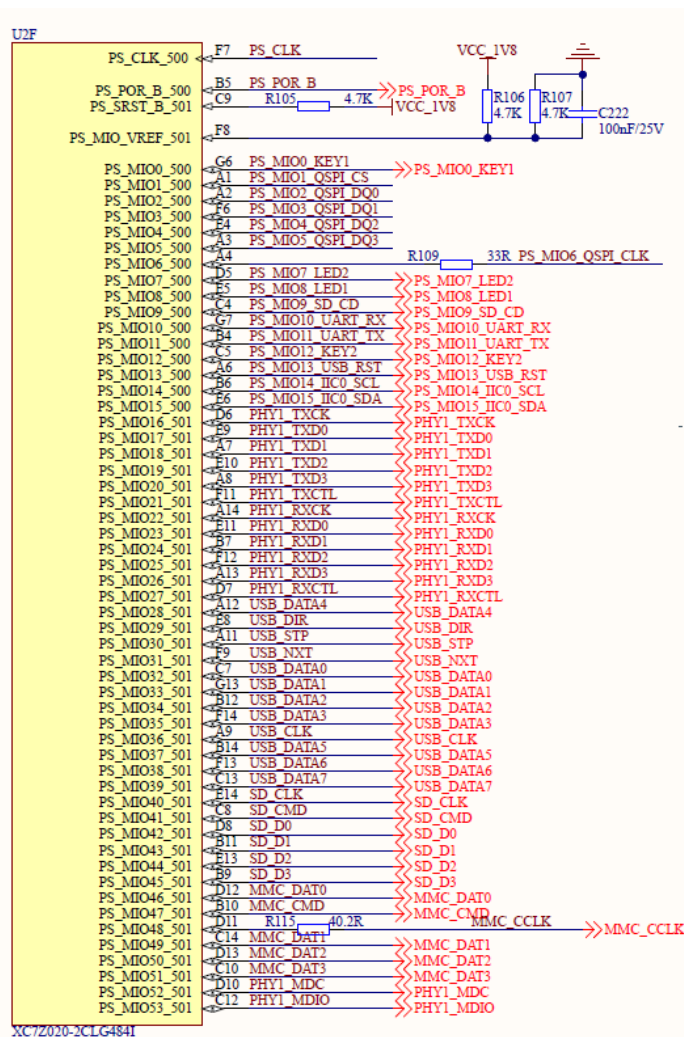


图 2-4 BANK 500 及 501 外设

根据 PS 端连接不同的外设来选择不同的供电电压。BANK500、BANK501 和 BANK502 分别采用 3.3V、1.8V 和 1.5V 供电。

与 PL 端 IO BANK 不同的是，PS 端的 IO 端口是相对固定的，用户不能够像 PL 引脚那样，将 PS 端的 IO 随意分配到某个外设。PS 端外设的 IO 口与 MIO（多路复用 IO）之间，具有固定关系的映射，某个外设的 IO 口可能会映射到不同的 MIO 上，某个 MIO 也有可能具有多个外设的 IO 口映射到其上面。PS 具体 IO 端口连接哪个 MIO 管脚可以参考原理图，同样可以参考图 2-3 中的管脚定义。

2.开发板使用注意事项

为了让大家更好的使用体验，在这里总结该开发板使用的时候尤其要注意的一些问题，希望大家在使用的时候多多注意，以减少不必要的问题。

- （1）由于 ZYNQ 芯片工作电流较大，建议大家使用配套的电源适配器供电。
- （2）当使用某个 IO 口用作其他用处的时候，请先看看开发板的原理图，该 IO 口是否有连接在开发板的某个外设上，如果有，需要确认该外设的这个信号是否会对你的使用造成干扰，先确定无干扰，再使用这个 IO。
- （3）在使用 40 PIN 拓展口的时，尤其需要注意电源以及地的 IO 管脚，请在使用子卡或其他小模块时需与原理图对应，防止由于接错导致的电路损坏。

三. PL 的“Hello World”——点亮 LED 灯

前面章节对 ZYNQ 以及 Verilog 进行了一个基本的介绍，虽然整体上对 ZYNQ 有了基础的了解，但对如何使用该芯片仍一头雾水，本章则以点亮开发板上 PL 端板载的 LED 灯为例，带领使用者熟悉整个 FPGA 的开发流程。

在 FPGA 的开发过程中会产生很多不同功能的文件，为了方便使用者能随时找到对应的文件，首先对工程内部的各种文件进行分类管理，后续其他实验的目录与下图目录一致，分类目录如下：

名称	修改日期	类型	大小
Bit	2024/10/26 9:33	文件夹	
Constraint	2024/10/26 9:31	文件夹	
Project	2024/10/26 9:31	文件夹	
Simulation	2024/10/26 9:31	文件夹	
Source	2024/10/26 9:10	文件夹	

图 3-1 文件夹分类

其中，Bit 文件夹存放历程生成的文件，Constraint 文件夹存放历程中产生的管脚约束，时序约束以及 Debug 约束，Project 文件夹存放此次历程的工程文件，Source 文件夹存放例程的源文件，Simulation 文件夹存放例程的仿真文件，。

1.点亮 LED 灯

1.1 实验目标

ZYNQ 开发板上焊接有两颗 LED，此次实验的目标是控制 PL_LED1 和 PL_LED2 灯亮灭，要求 PL_LED1 亮，PL_LED2 灭，PL_LED2 亮，PL_LED2 灭，时间间隔为 0.5S。

1.2 LED 硬件介绍

底板上的 PL_LED1、PL_LED2 电路原理图如图 3-2 所示：

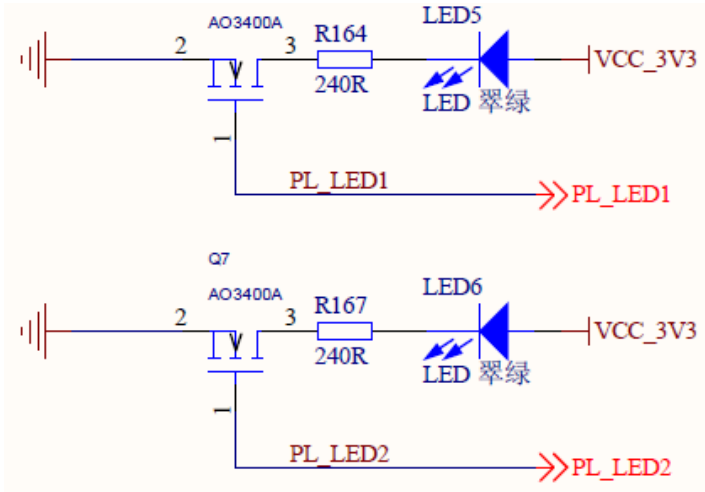


图 3-2 PL LED 电路图

由图 3-2 可知, 两组 LED 灯的阴极通过 240 欧姆的电阻连到 N 沟道 MOS 管的漏极上, 起到限流的作用, 以防止在 MOS 管导通时电流过大烧坏 LED 灯; 两组 LED 灯的阳极连接在 VCC 3.3V, 而 ZYNQ PL 的 IO 管脚连接至 MOS 管栅极, N 沟道 MOS 管得导通条件为栅极电压高于源极电压则 MOS 管导通, 那么当 ZYNQ PL 端的 IO 输出高电平 1 时, 点亮 LED 灯; 当 ZYNQ PL 端的 IO 输出低电平 0 时, LED 灯熄灭。

1.3 新建工程

开发板使用的开发工具为 Vivado, 版本为 2023.1。可以通过 Vivado 开发工具创建一个全新的工程。新建工程流程如下:

(1) 双击桌面上的 Vivado 软件图标, 打开 Vivado 软件, Vivado 软件启动界面如下图所示, 点击“Create Project 来创建一个新的工程”。

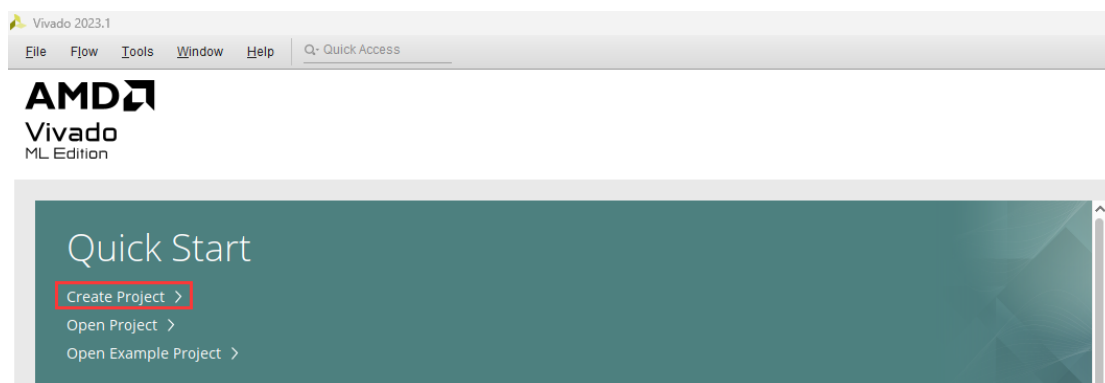


图 3-3 Vivado 软件启动界面

(2) 出现下图所示窗口, 直接点击“Next”, 如下图所示:

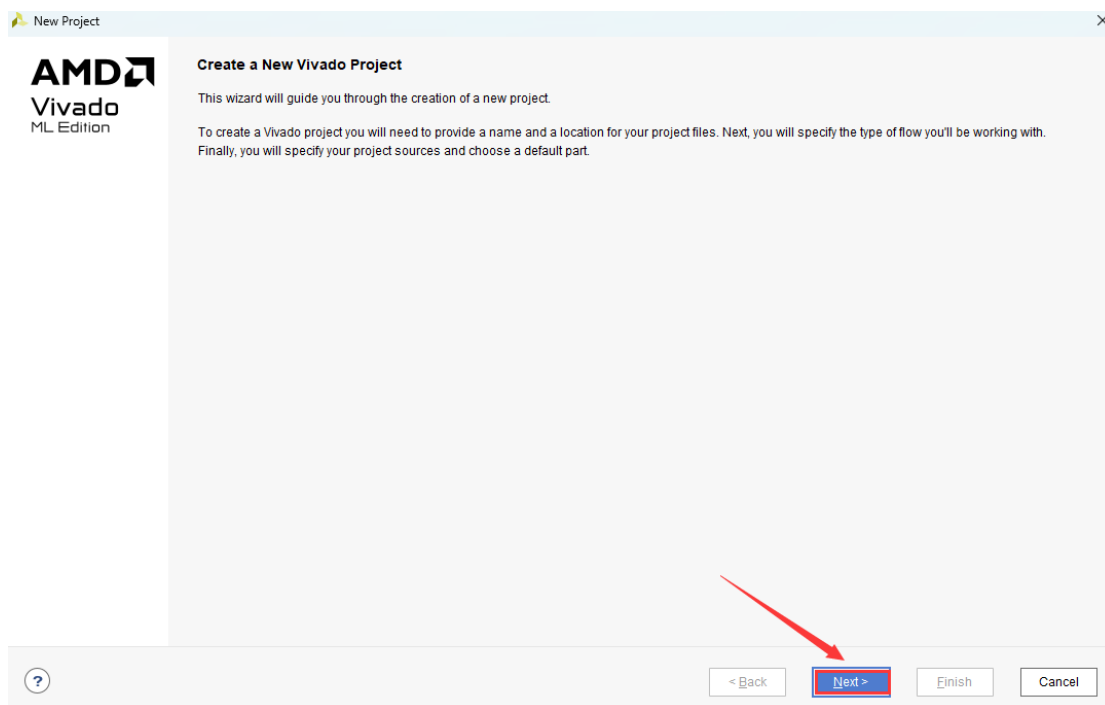


图 3-4 新建工程向导

(3) 接下来输入工程的名称和路径。名称要能反应出工程所实现的功能, 本次工程实现了点亮 LED 的功能, 因此项目名称命名为“led”。工程路径是上文中提到的工程管理的

01_LED/Project 文件夹，取消勾选 “Create project subdirectory”选项。需要注意的是，工程路径不能包含中文、空格或者其它一些特殊的符号，尽量使用英文、数字和下划线，否则工程会创建失败。

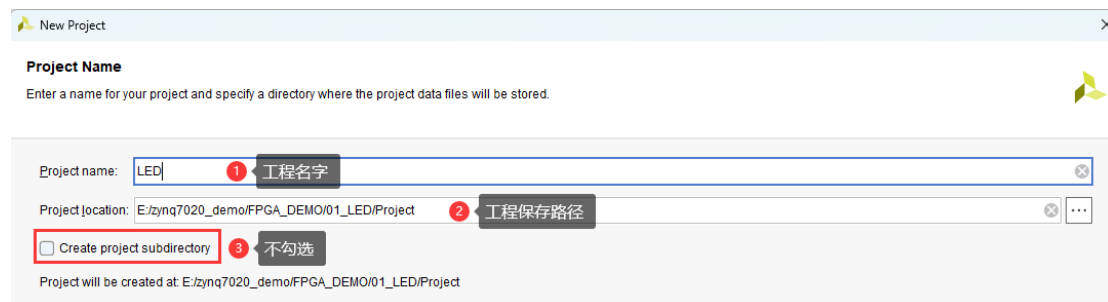


图 3-5 输入工程名称以及路径

(4) 继续点击“Next”按钮，接下来是工程类型的选择，选择“RTL Project”，如下图所示：

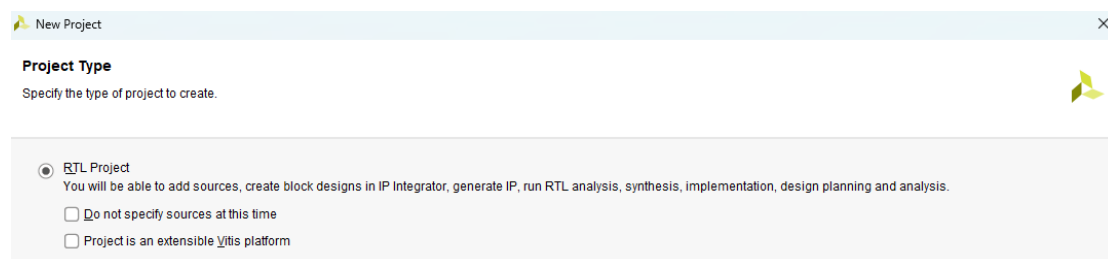


图 3-6 工程类型及选择

RTL Project 是按正常设计流程所选择的类型，这也是常用的一种类型。其他的设计类型一般不会使用，在此教程中不对其进行介绍。RTL Project 下方的选项含义为：

a. “Do not specify sources at this time”：用于设置是否在创建工程向导的过程中添加设计文件，如果勾选后，则不创建或者添加设计文件，后续是需要添加设计文件，所以不勾选该选项。

b. “Project is an extensible Vitis platform”：创建的工程是否需要扩展 Vitis 开发平台，点亮 LED 等实验不需要 Vitis 开发平台。所以这个选项也不需要勾选。

(5) 选择了“RTL Project”后，点击“Next”，进入添加源文件页面。需要注意的是若勾选图中图 3-6 中“RTL Project”下的“Do not specify sources at this time”，则不会出现添加源文件的界面。该界面不需要创建文件，直接点击“next”

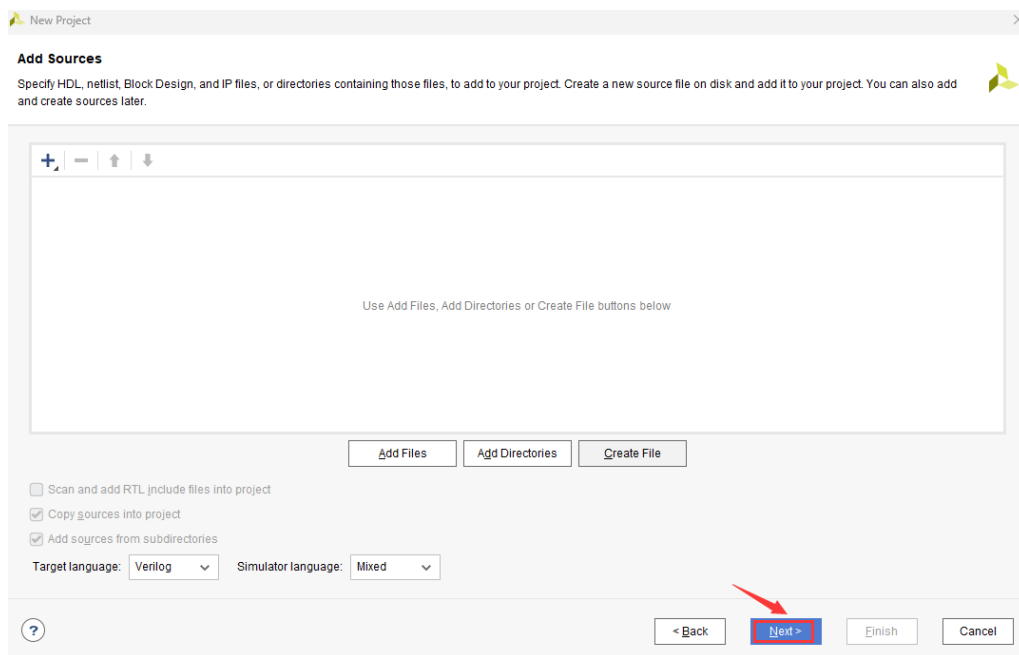


图 3-7 添加源文件

(6) 图 3-7 点击 Next 后会弹出添加约束的窗口，由于并未进行代码设计，因此也直接点击“Next”。

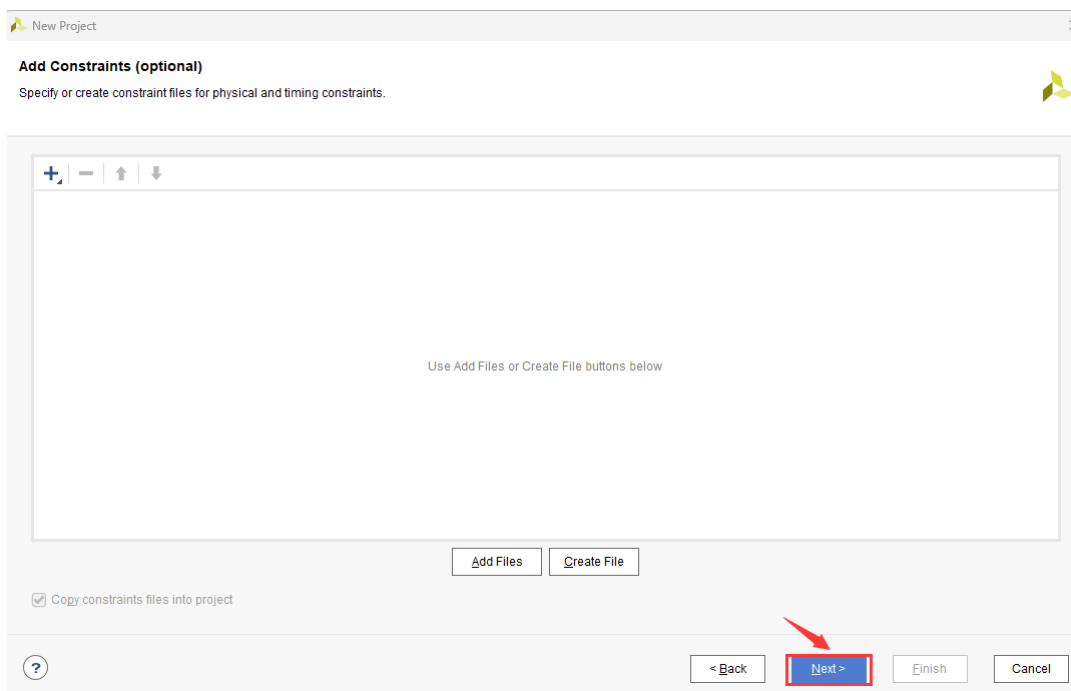


图 3-8 添加约束文件

(7) 接下来选择开发板的芯片型号，可以直接在搜索框中输入完整的芯片型号，根据自己所使用的 ZYNQ 核心板型号进行选择。可直接输入“xc7z020clg484-2”，如下图所示：

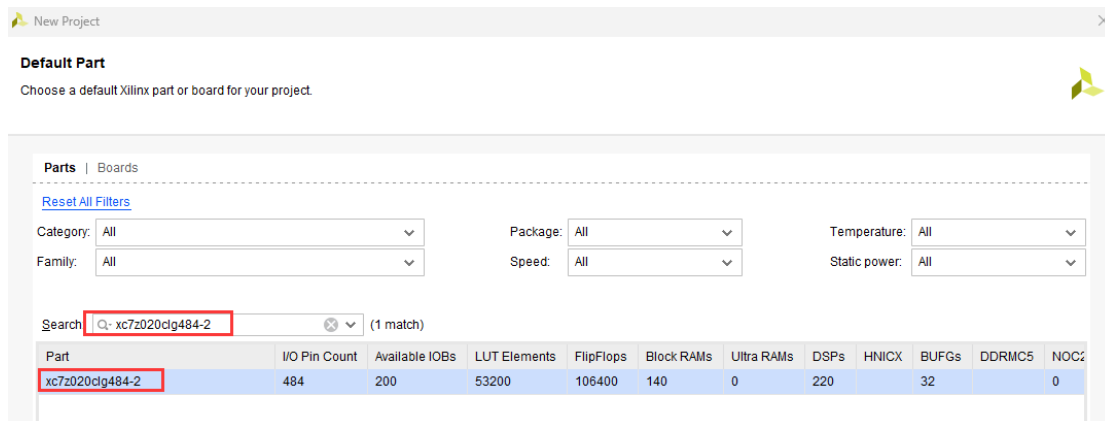


图 3-9 选择 ZYNQ 开发板的芯片型号

在搜索框中输入完整的芯片型号后，在“Part”一栏会出现唯一匹配的型号，单击选中“Part”一栏的芯片型号就会选中该型号芯片。当然，用户也可以根据芯片本身的所在系列、封装、速度等级以及工/商业级等信息进行选择。

(8) 芯片型号选择完成，点击“Next”进入下一个界面的设置，如下图所示：

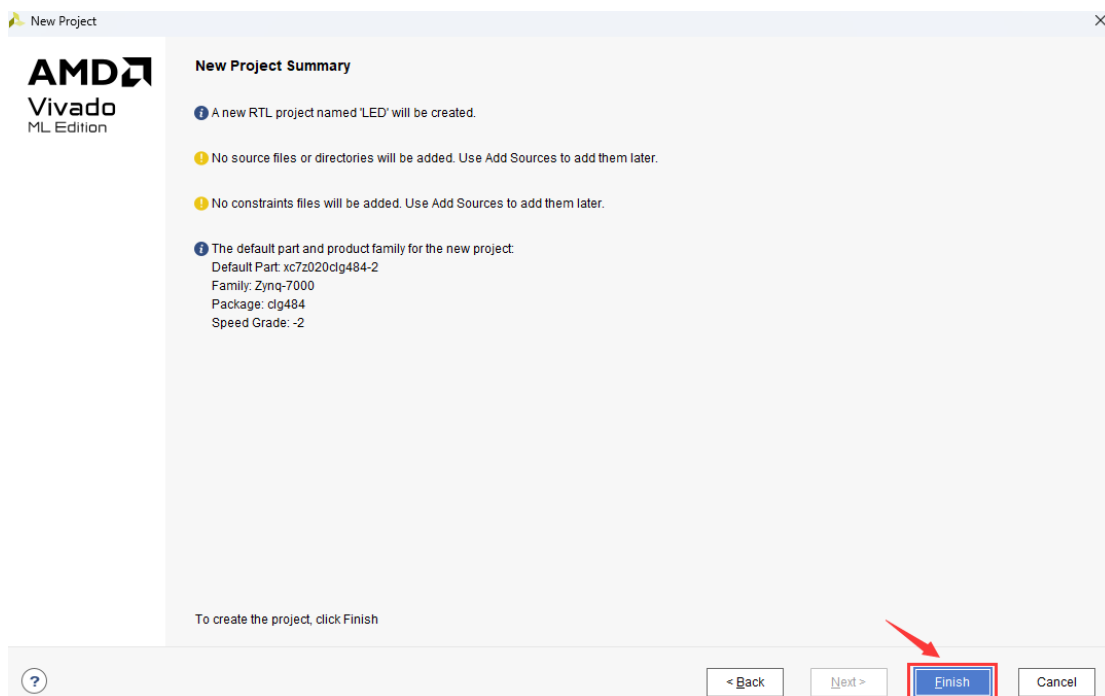


图 3-10 工程概览（Summary）页面

工程概览页面是将之前几个步骤中的设置全部列了出来，供用户检查，选择不同的芯片型号，概览页面列举的芯片型号也不一样，直接点击“Finish”按钮完成工程的创建。工程创建完成后，就进入了 Vivado 的工程主界面，如下图所示：

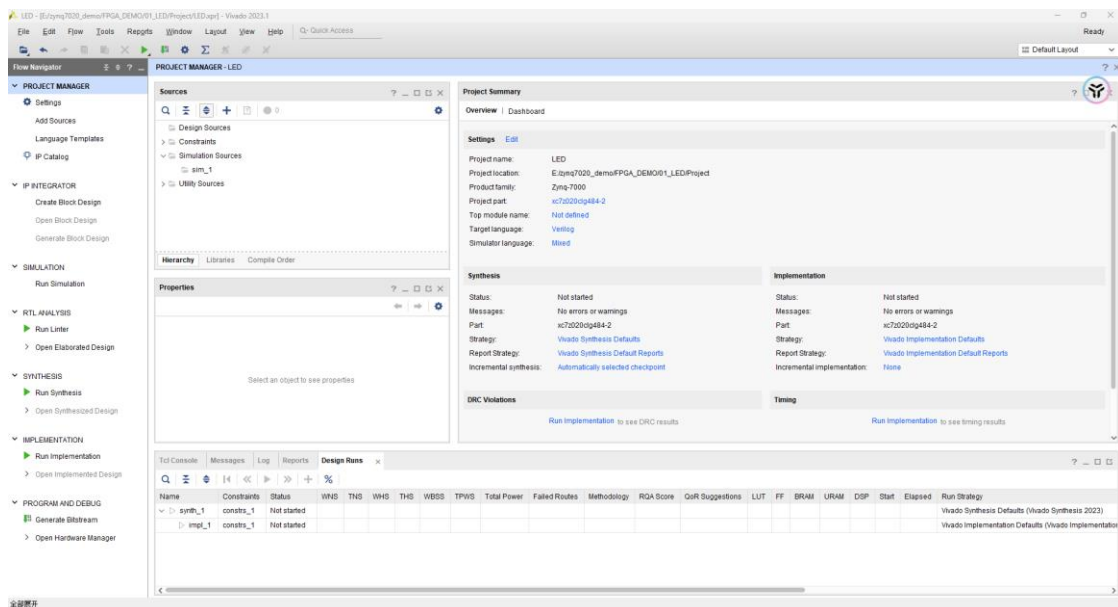


图 3-11 Vivado 工程主界面

(9) 新建 Source 文件，创建方式点开下图红框处可任意一处可创建 source 文件

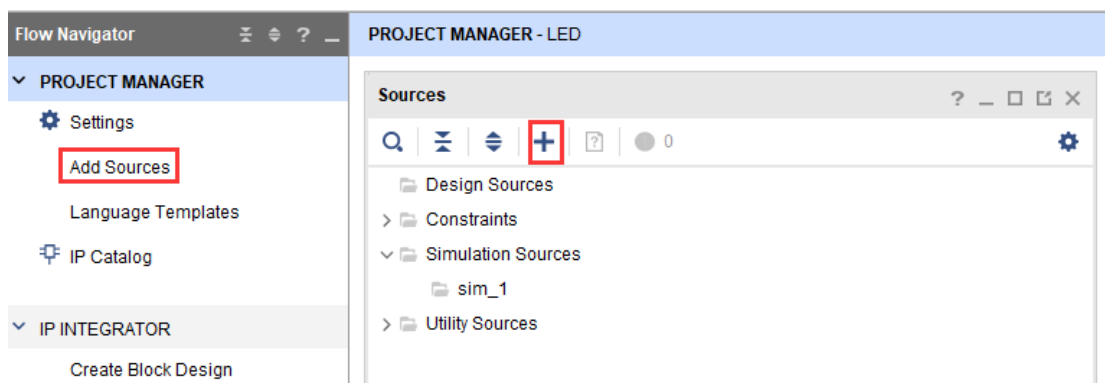


图 3-12 新建 Source

(10) 点击图 3-12 中的红框后，会弹出以下界面，若创建 Verilog 文件则点击下图的 2 处。
(注意：若需要仿真，则需要点击下图 3 处。) 随后点击 Next。



图 3-13 选择新建文件类型

(11) 点击 Next 会出现以下图片，创建 Verilog 文件，直接点击下图 1 处。

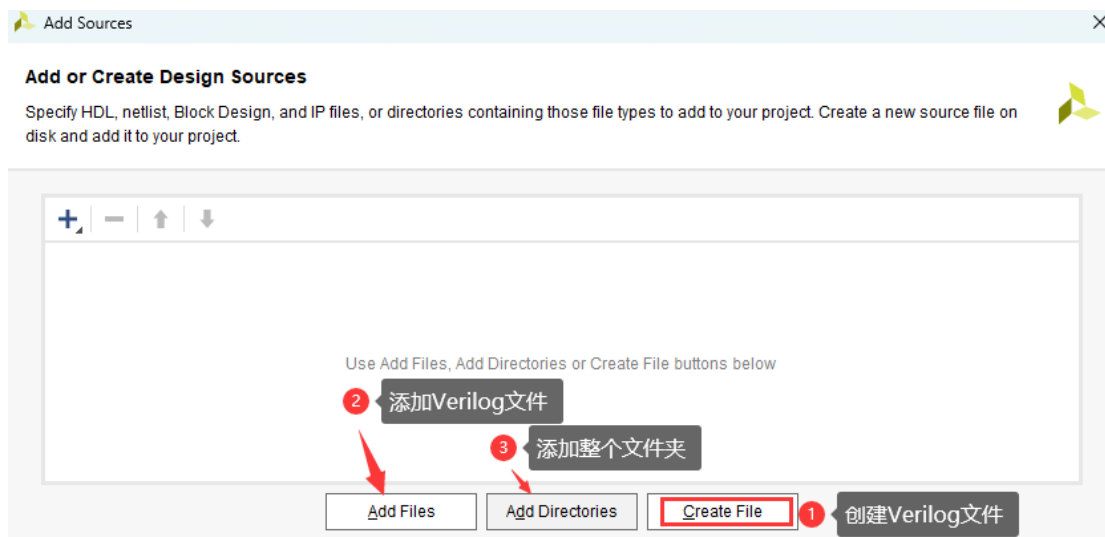


图 3-14 新建 Verilog 文件

(12) 在弹出的界面中，输入 Verilog 文件的名称以及选择保存位置后，点击 ok

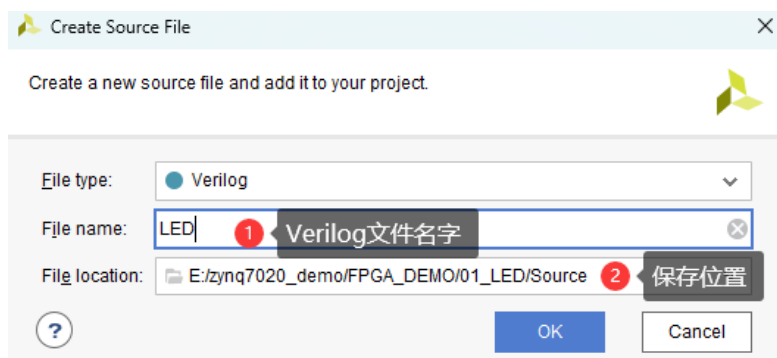


图 3-15 Verilog 文件的名称及保存位置

(13) 在下图界面中，直接点击 OK，完成 Verilog 文件的创建。

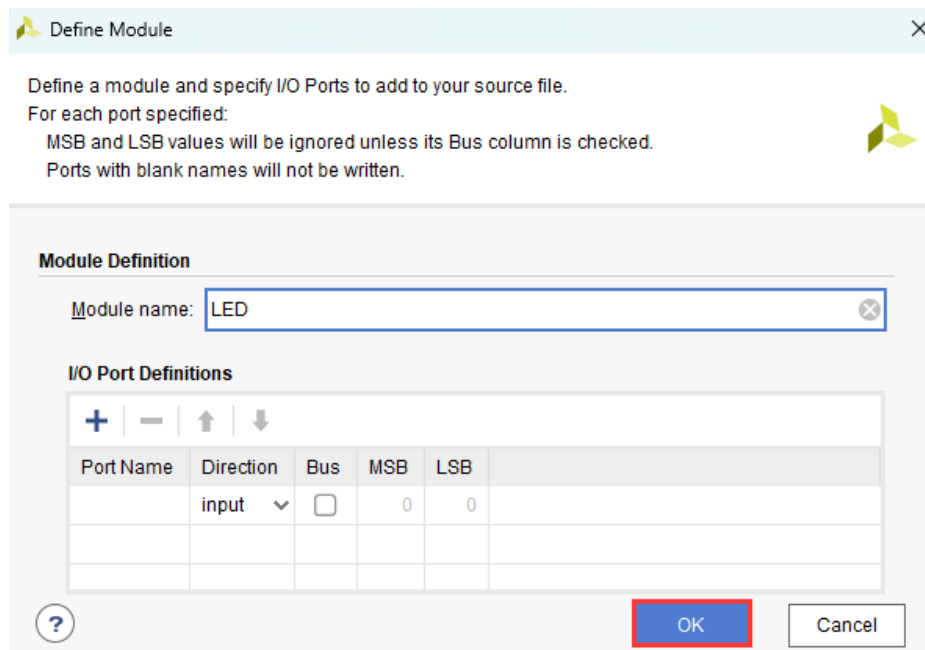


图 3-16 结束工程创建

单击 OK 后，Source 界面如下所示，表示当前 Verilog 文件创建完成。



图 3-17 Source 文件创建结束界面

至此，一个包含一个名为 LED 的 Verilog 的 Vivado 工程已经新建完成了。

1.4 LED 驱动设计

由 1.1 章以及 1.2 章的实验目标及驱动方式可以知道本章的目标以及 LED 的驱动方式，那么开始编写 Verilog 代码

(1) 在实验目标中提出，每隔 0.5S 就翻转 LED 的状态，那么首先需要产生一个 0.5S 间隔的计数器。而 ZYNQ PL 端的时钟为 50MHz，需要产生一个 0.5S 的计数器则需要计数到 24999999，Verilog 实现如下：

```
/* 生成一个周期为500ms 的时钟 */
always @(posedge i_clk or negedge i_rst) begin
    if (~i_rst)begin
        r_cnt <= 'd0;
    end else if (r_cnt == P_CNT) begin
        r_cnt <= 'd0;
    end else begin
        r_cnt <= r_cnt + 'd1;
    end
end
end
```

上述代码片段中 P_CNT 是参数化的设计，P_LEN 的参数为 24999999。主要目的是为了在仿真时能加快仿真时间。

(2) 由第一步可以明白已经产生了一个 0.5S 的计数器，那么则需要在每次到达 0.5S 时对 LED 的状态进行翻转，而在 1.2 章节中分析可以知道，LED 的亮是需要 PL 的 IO 输出“1”的情况下才会亮，在复位时，将两颗 LED 的状态赋为“10”，随着计数器到达 0.5S，对 LED 状态进行按位取反，Verilog 实现如下：

```
/* 当计数器到达500ms，Led 状态取反 */
always @(posedge i_clk or negedge i_rst) begin
    if (~i_rst)begin
        ro_led <= 2'b10;
    end else if (r_cnt == P_CNT) begin
        ro_led <= ~ro_led; //LED 按位取反
    end
end
```

```

end else begin
    ro_led <= ro_led;
end
end

```

上述代码实现了 LED 亮灭功能的关键步骤。

1.5 LED 驱动仿真

一般的 FPGA 设计在设计完 Verilog 驱动后，也需要进行仿真，确认当前代码是否符合设计要求，接下来以 LED 驱动为例，实现 LED 闪烁功能的仿真。

(1) 仿真文件的创建与 Verilog 文件创建是一致的，可以参考 1.3 章中 Verilog 的创建过程，创建文件如下图所示：

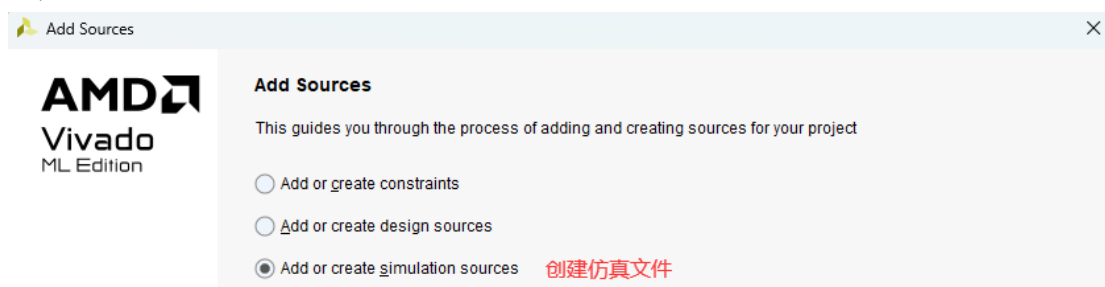


图 3-18 simulation 文件创建

添加仿真文件名称以及保存路径

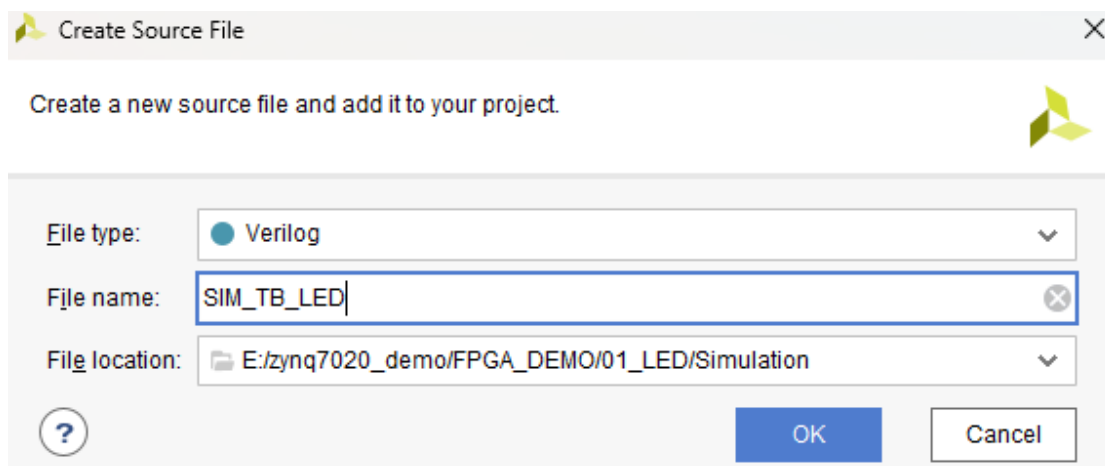


图 3-19 添加仿真文件名称及保存路径

(2) 编写仿真测试激励

其中 P_LEN 的时间可以缩小至 9，这样可以快速实现对 LED 的观测。

```

module SIM_TB_LED();

    reg    clk,rst;    //定义时钟以及复位

    //初始化时钟及复位状态
    initial begin
        clk <= 'd0;
        rst <= 'd0;
    end

```

```

        #100
        rst <= 'd1;
end

//产生测试时钟
always begin
    clk <= 'd0;
    #10;
    clk <= 'd1;
    #10;
end

//例化 LED 驱动代码
LED
#(
    .P_CNT      (9) //参数设置为9，减少仿真时间，观测实验现象
)LED_U0
(
    .i_clk      (clk),
    .i_rst      (rst),

    .o_led      ()
);
endmodule

```

(3) 启动 Vivado 自带的仿真软件，如下图所示：

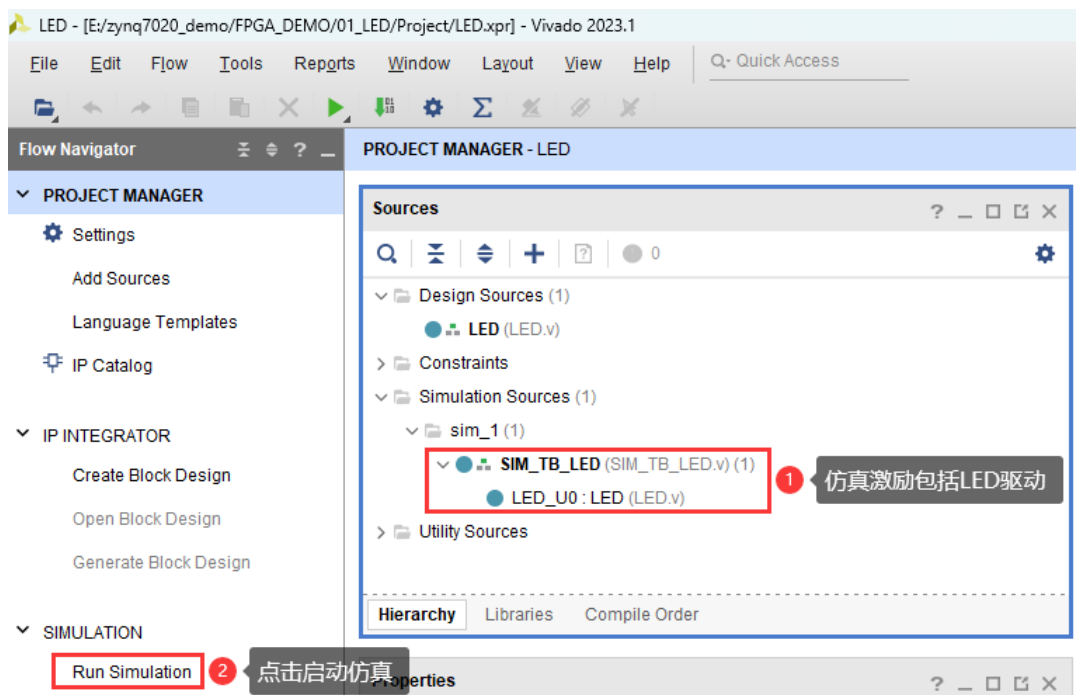


图 3-20 启动仿真

(4) 仿真启动后，仿真界面如下：

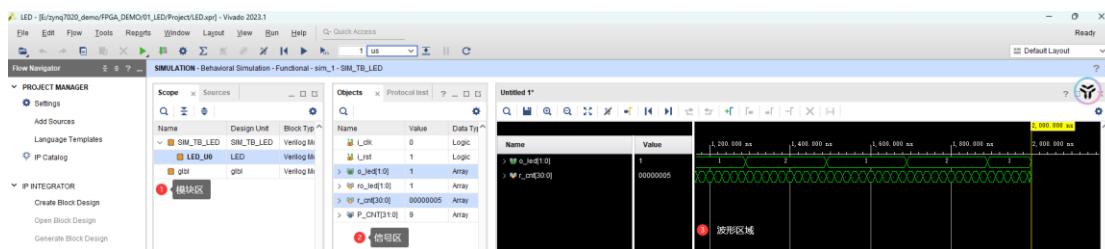


图 3-21 仿真界面

(5) 将仿真时间设置为 1us，运行仿真。在波形区域可以观测需要的信号，仿真结果如下

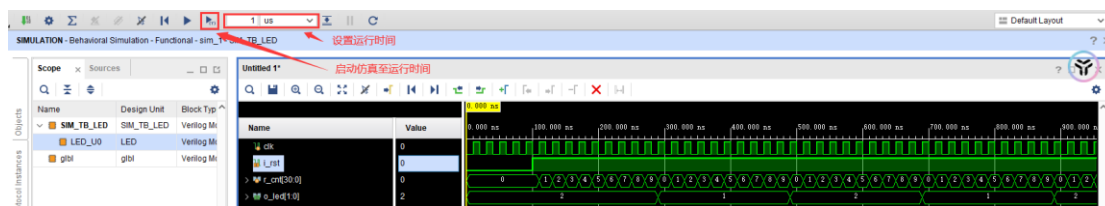


图 3-22 仿真结果

由图 3-22 可以看出，复位信号为低时，计数器一直为 0，同时 LED 输出为“2”，这个 2 是十进制的，转换为 2 进制为“10”，也就是说 LED 是高位亮低位灭。随着计数器增加至 9，LED 状态转换为“1”，转换为 2 进制为“01”，也就是说 LED 是高位灭低位亮，以此循环。基本符合设计要求。

上述六个步骤从 FPGA 代码设计至仿真的一个简单流程，代码基本符合设计要求，接下来则是对整个工程进行综合与约束。

1.6 综合与管脚约束

(1) 综合是对当前的设计进行一个检查，检查当前的设计是否有语法错误等。综合时间与当前代码数量有关，代码数量越大，则运行过程越长。仿真点击下图所示的按钮。

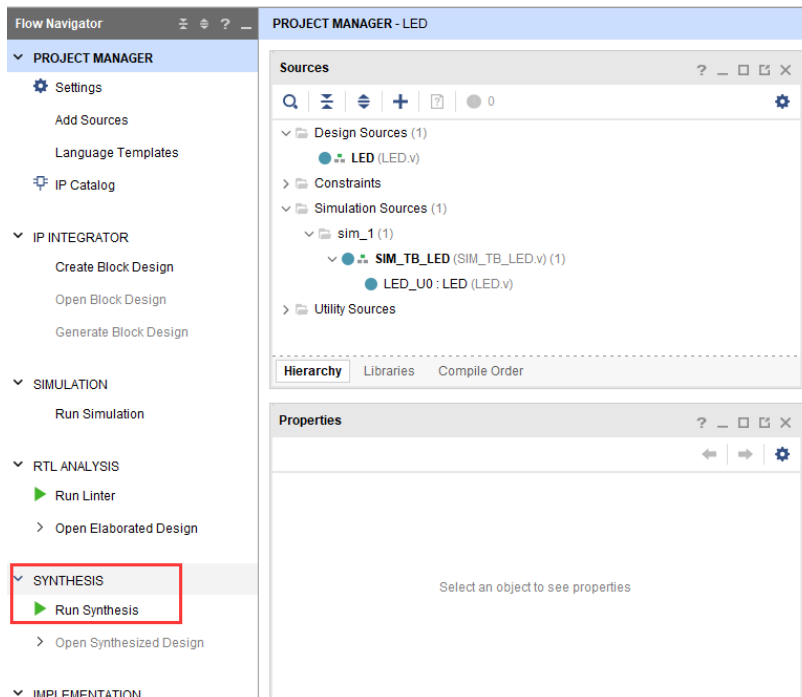


图 3-23 综合过程

(2) 综合通过后，在弹出的界面中点击“Cancel”，如下图所示：

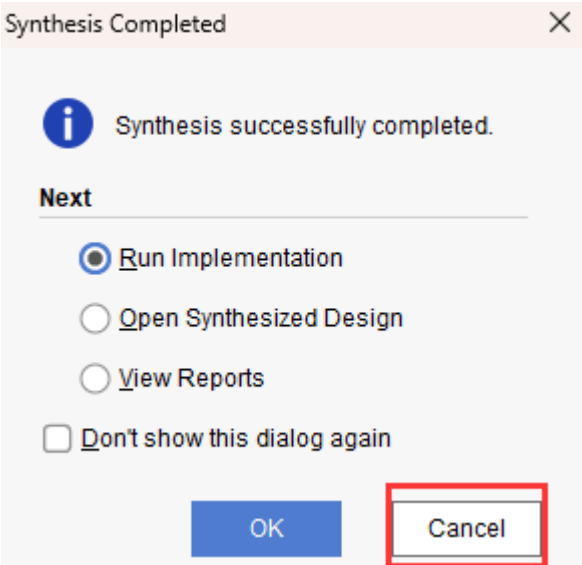


图 3-24 综合通过

(3) 在综合通过后一般会对管脚进行物理约束。点击下图按钮。

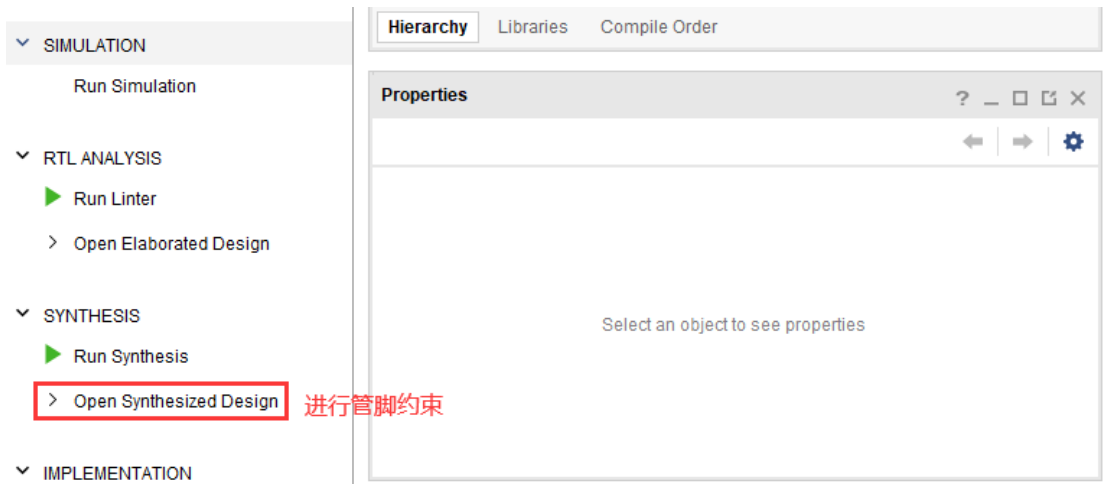


图 3-24 管脚约束按钮

(4) 在弹出界面中约束 CLK、RST 以及 LED，在下图 1 处约束 FPGA 管脚，2 处约束电平，CLK 管脚为 W17，电平标准选择 LVCOMS 33，LED 灯的管脚分别为：V15，V13。电平标准选择 LVCOMS 33；复位按键管脚为：W18，电平标准选择 LVCOMS 33。管脚约束如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
1. FPGA管脚													
o_led[2]	OUT		V13	☒	33	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
o_led[1]	OUT		V15	☒	33	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
o_led[0]	OUT		V15	☒	33	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
2. FPGA管脚电平													
i_clk	IN		W17	☒	33	LVCMOS33*	3.300				NONE	NONE	
i_rst	IN		W18	☒	33	LVCMOS33*	3.300				NONE	NONE	

图 3-25 FPGA 管脚约束

管脚约束完成后，则可以对约束文件进行保存。

1.7 在线逻辑分析仪使用

虽然上述代码已经进行仿真，符合设计要求，但实际上板时会出现各种各样的不确定性，

因此需要借用逻辑分析仪对 FPGA 内部信号进行抓取分析，而传统的 FPGA 板级调试是由外接的逻辑分析仪连接到 FPGA 的控制管脚，然后将内部信号引出至引脚 IO 上，进行板级调试。这种方法的缺点是需要一个逻辑分析仪，而逻辑分析仪一般价格都比较昂贵，且对于需要测试几十个信号的时候，选择使用外接的逻辑分析仪就比较繁琐了。在线逻辑分析仪克服了以上所有的缺点，其借用了传统逻辑分析仪的理念以及大部分的功能，并利用 FPGA 中的逻辑资源，将这些功能植入到 FPGA 的设计当中。在线逻辑分析仪的应用原理框图如下图所示：

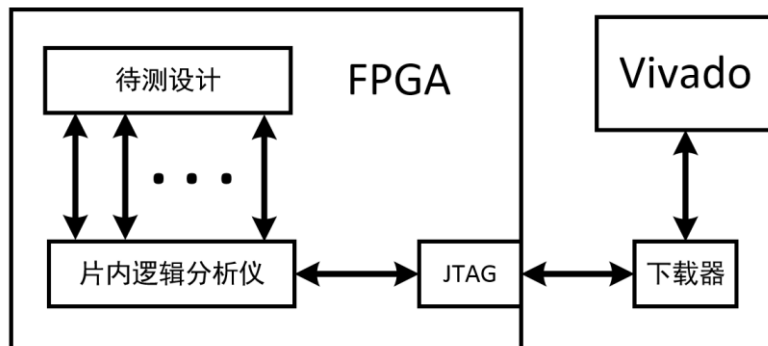


图 3-26 FPGA 内部逻辑分析仪工作原理

其中，待测设计（Design Under Test, DUT）就是用户逻辑，它和片内的在线逻辑分析仪都位于 FPGA 中。在线逻辑分析仪通过一个或多个探针（Probe）来采集希望观察的信号，然后通过片内的 JTAG 硬核组件，将捕获到的数据传送给下载器，进而上传到 Vivado 供用户查看。Vivado 也能够按照上述数据路径，反向地向 FPGA 中的在线逻辑分析仪传送一些控制信息。由此可见，在线逻辑分析仪不需要将待测信号引出至 I/O 上，也不需要电路板走线或者探测点，当然更不需要外部的逻辑分析仪，在 Vivado 中就可以将在线逻辑分析仪添加到设计中。但是在线逻辑分析仪会占用一定数量的内部逻辑资源，如块 RAM、查找表、触发器等等。

上述介绍了在线逻辑分析仪的基本原理，接下来则在 Xilinx 平台上实现两种在线逻辑分析仪的使用。

1.7.1 HDL 实例化调试探针

(1) “HDL 实例化调试探针流程”需要在 HDL 源代码中实例化 ILA IP 核。我们点击“Flow Navigator”窗口中的“IP Catalog”按钮，如下图所示：

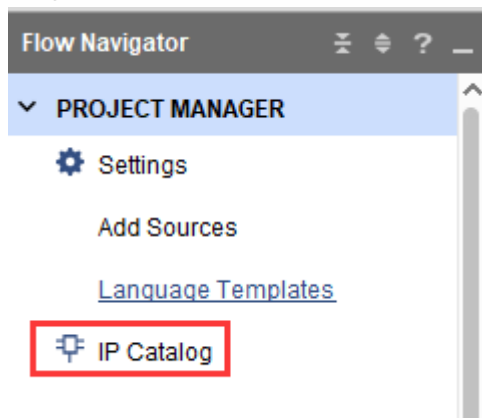


图 3-27 “IP Catalog”按钮

(2) 这时“IP Catalog”窗口就被打开了，并在搜索栏“ILA”。如下图所示：

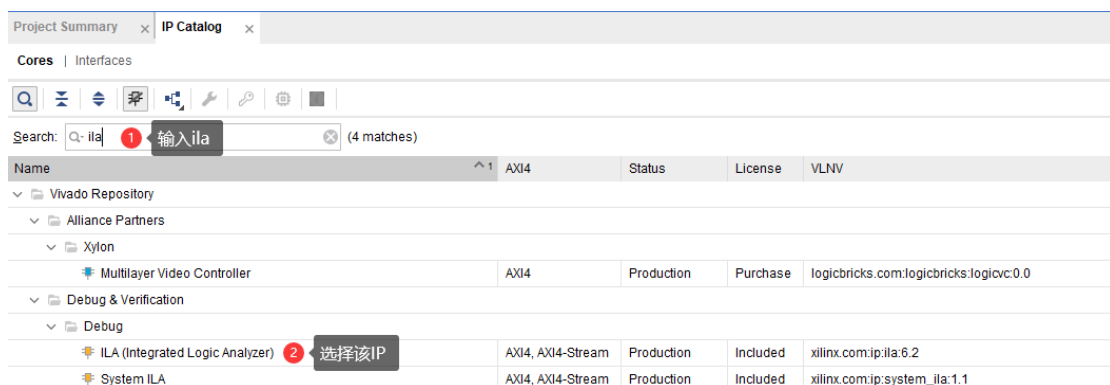


图 3-28 搜索 ila IP

(3) 双击“ILA (Integrated Logic Analyzer)”会弹出“ILA IP”核的配置页面，ILA IP 核的默认配置页面如下图所示：

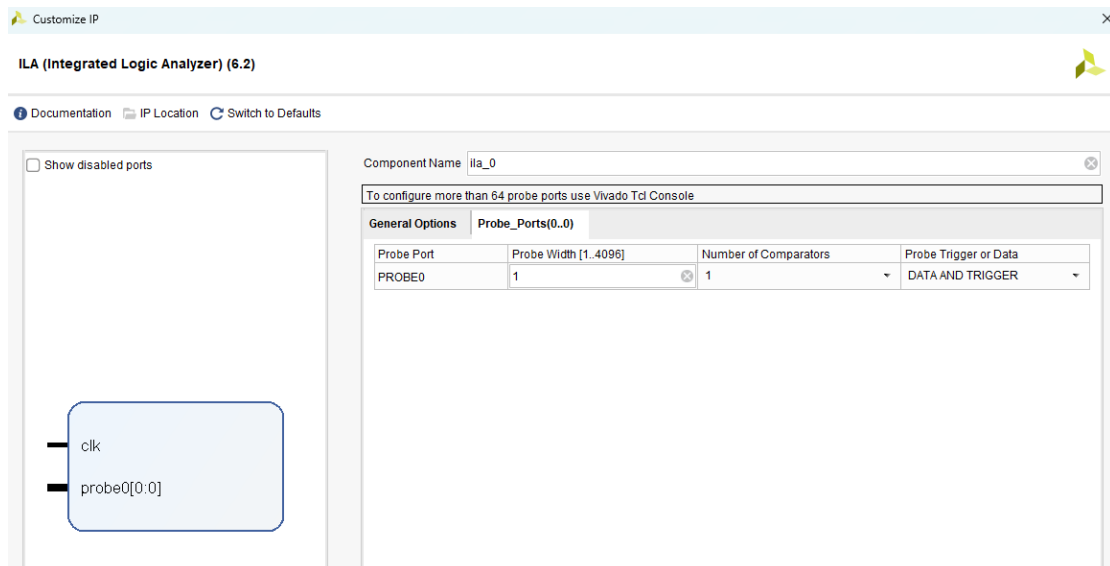


图 3-29 ila IP 默认界面

(4) 本次实验主要抓取计数器信号以及 LED 管脚信号，那么可以设计 2 个探针，位宽分别设置为 31 bit 以及 2 bit，设置如下：



图 3-30 探针及采样深度设置

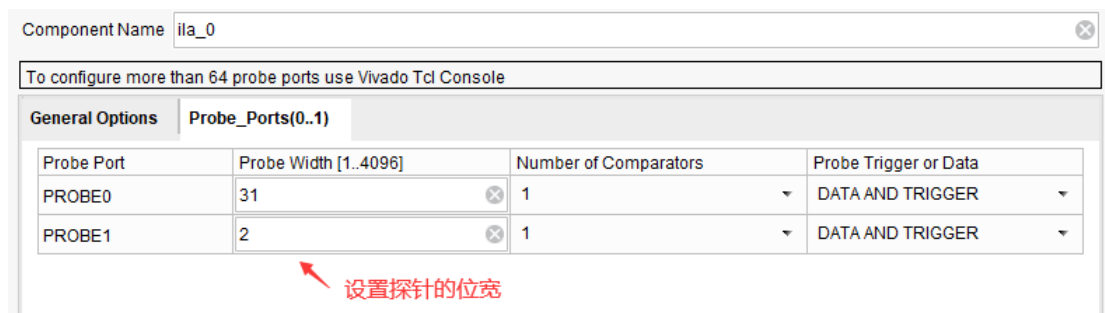


图 3-31 探针的位宽

其他保持默认，直接生成 IP。



图 3-32 IP 生成成功

(5) 将生成的 IP 例化至工程文件中，复制 Xilinx IP 提供的例化模板。

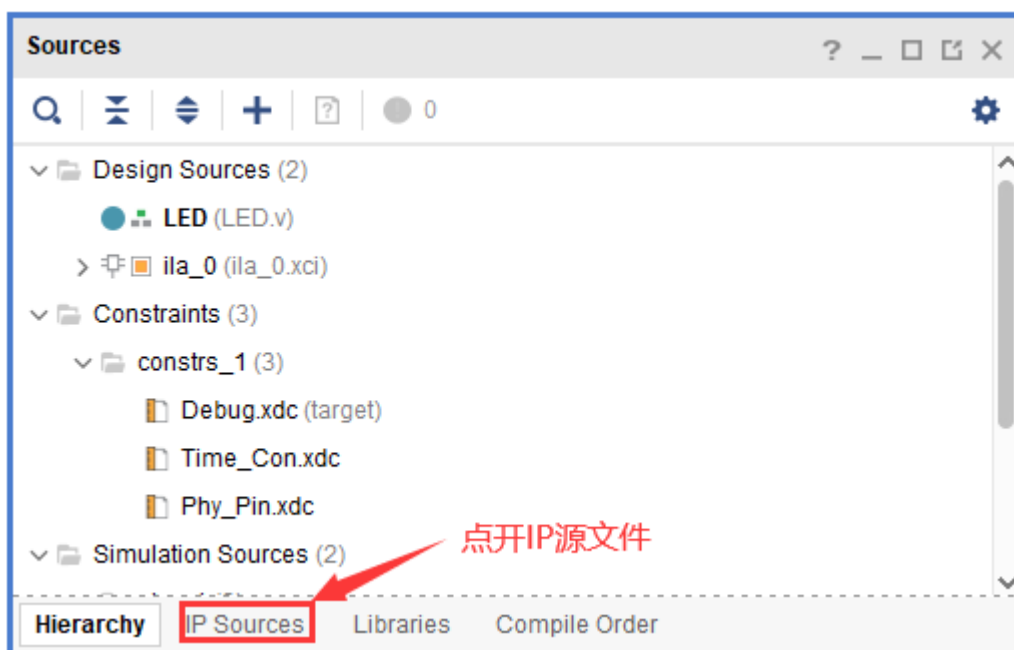


图 3-33 找到 IP 核

随后打开 ILA IP 的例化模板，如下图所示：



图 3-34 打开例化模板

双击打开图 3-34 中的例化模板，可以看出 Xilinx 提供的例化模板如下：

```
ila_0 your_instance_name (
    .clk(clk), // input wire clk

    .probe0(probe0), // input wire [30:0] probe0
    .probe1(probe1) // input wire [1:0] probe1
);
```

图 3-35 例化模板

(6) 将该例化模板复制到工程文件中，并将需要观测的信号连接至 ILA 的信号中，ILA 时钟是系统的 50M 系统时钟，ILA 例化如下图所示：

```
ila_0 ila_0_u0
(
    .clk    (i_clk    ), // input wire clk
    .probe0 (r_cnt    ), // input wire [30:0] probe0
    .probe1 (o_led    )  // input wire [1:0] probe1
);
```

图 3-36 例化 ILA

由此完成了 HDL 实例化调试探针的设计流程。随后可进行时钟约束。

1.7.2 使用 Debug 标记创建 ILA 调试环境

使用实例化 IP 可以直观的了解到需要连接的信号，但该 IP 的使用相对来说不灵活，因

此在大规模的 FPGA 的设计中很少采用 ILA IP 核的设计，更多的是使用 Debug 标记的方式创建 ILA 调试环境。

(1) 使用 Debug 标记创建 ILA 就是在综合之前的 HDL 代码中为想要观察的 reg 或 wire 信号添加“Mark Debug”综合属性，使用方式如下图所示：

```
(* mark_debug = "true" *)
output      wire      [1:0]      o_led
);

reg      [1:0]      ro_led  ;
(* mark_debug = "true" *)
reg      [30:0]     r_cnt   ;
```

图 3-37 Debug 标记创建 ILA 调试环境

(2) 添加 “Make Debug” 属性之后对代码重新保存，然后点击 “Flow Navigator”→“SYNTHESIS”窗口中的“Run Synthesis”进行综合，随后添加 Debug 环境，如下图所示：

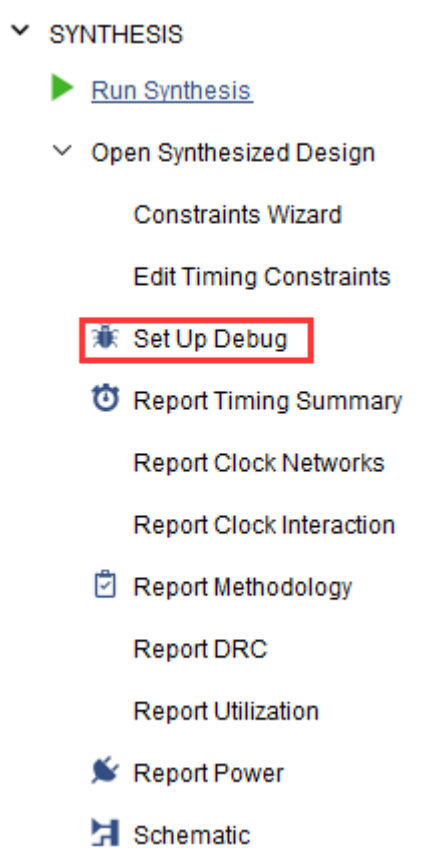


图 3-38 点击“Set Up Debug”按钮

(3) 点击“Set Up Debug”按钮后进入 Debug 信号与探针关联引导页面，如下图所示，直接点击“Next”。

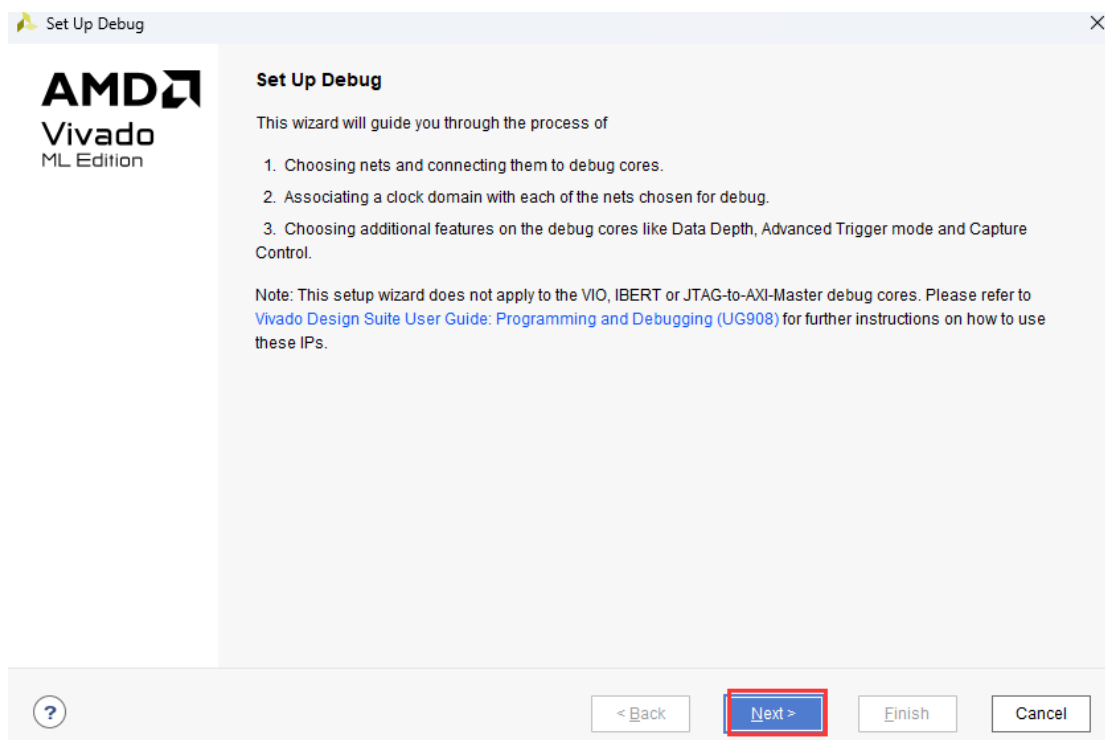


图 3-39 探针关联引导页面

(4) 引导页面点击“Next”后进入引脚配置页面，一般情况下 Vivado 会自动连接 Debug 信号，若有些信号不需要连接，可以自行查找 ILA Debug 详细教程应用。连接情况如下图所示：

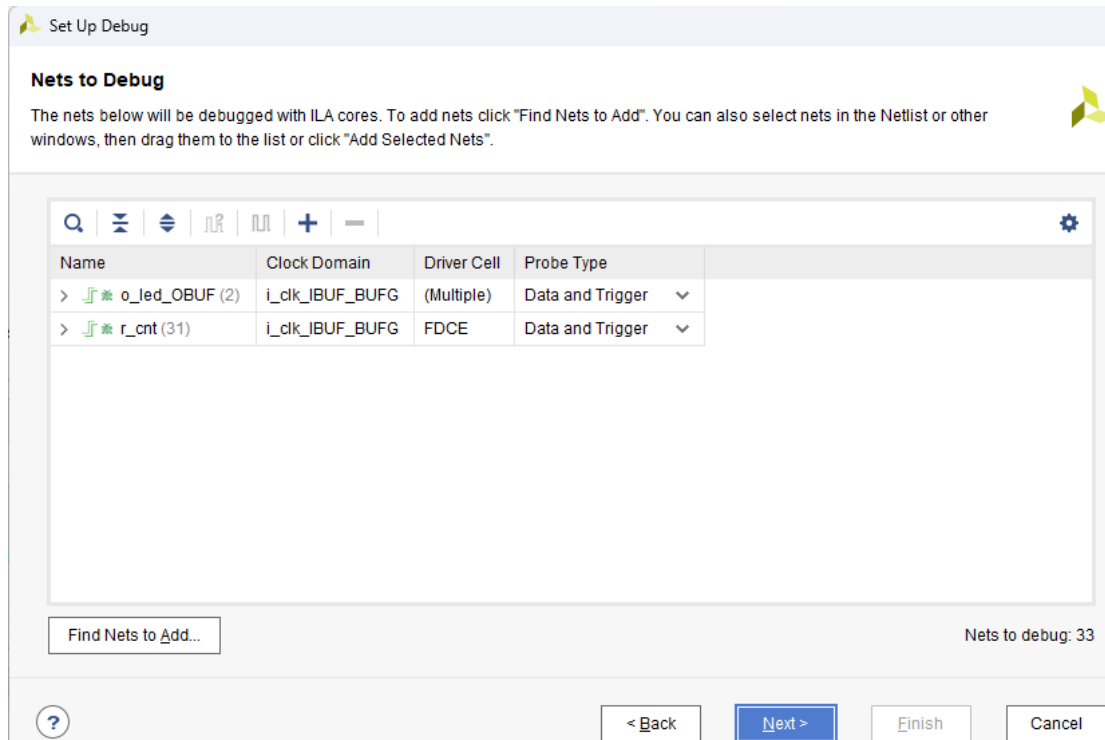


图 3-40 添加 Debug 信号

(5) 随后点击 Next，接下来的页面用于设置 ILA 采集深度，如下图所示：

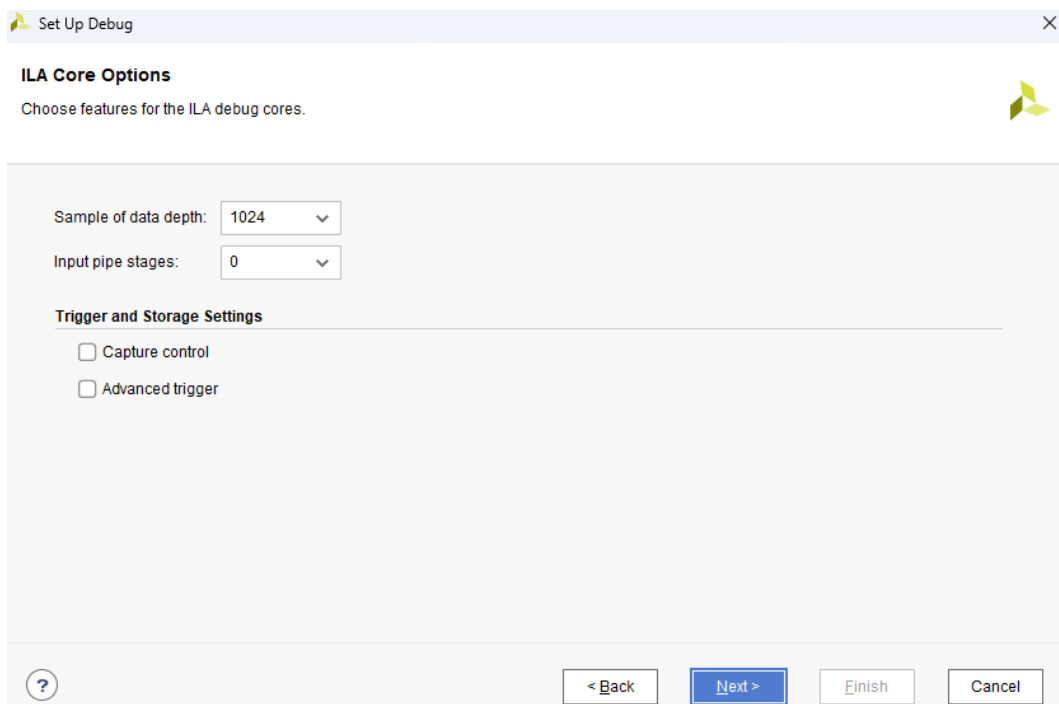


图 3-41 ILA 采样深度

(6) 随后点击 Finish，并保存文件，等待 Vivado 综合完成。随后进行时序约束

1.8 时序约束

时序约束 (Timing Constraints) 用来描述设计人员对时序的要求，比如时钟频率，输入输出的延时等，以满足设计的时序要求。

那么为什么工程中需要添加时序约束呢？对于这个原因，首先给大家说明下，如果有些设计工程不加时序约束，可能会出现功能错误，这个地方加了“可能”两个字，也就是说不一定会出错，有可能某一次编译功能没有问题，下一次编译就出现功能问题。

接下来则以简单 LED 带领大家为主时钟 50M 进行约束，主要目的是“告诉”FPGA 当前时钟的频率、相位等信息。

(1) 在时序约束前首先要对整个工程进行综合，综合完成后，点击下图红框：

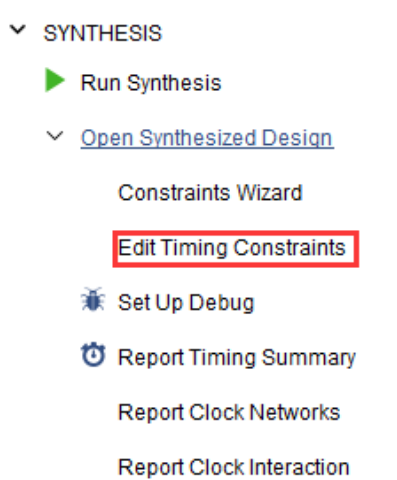


图 3-41 点击开始时序约束

(2) 点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点

击该界面下的“+”号按钮就可以添加时钟约束了，如下图所示：

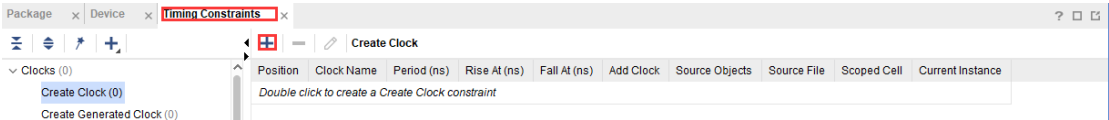


图 3-42 点击“+”号

(3) 点击“+”号后，弹出的“CreateClock”界面，并填入当前约束时钟的信息，如下图所示：

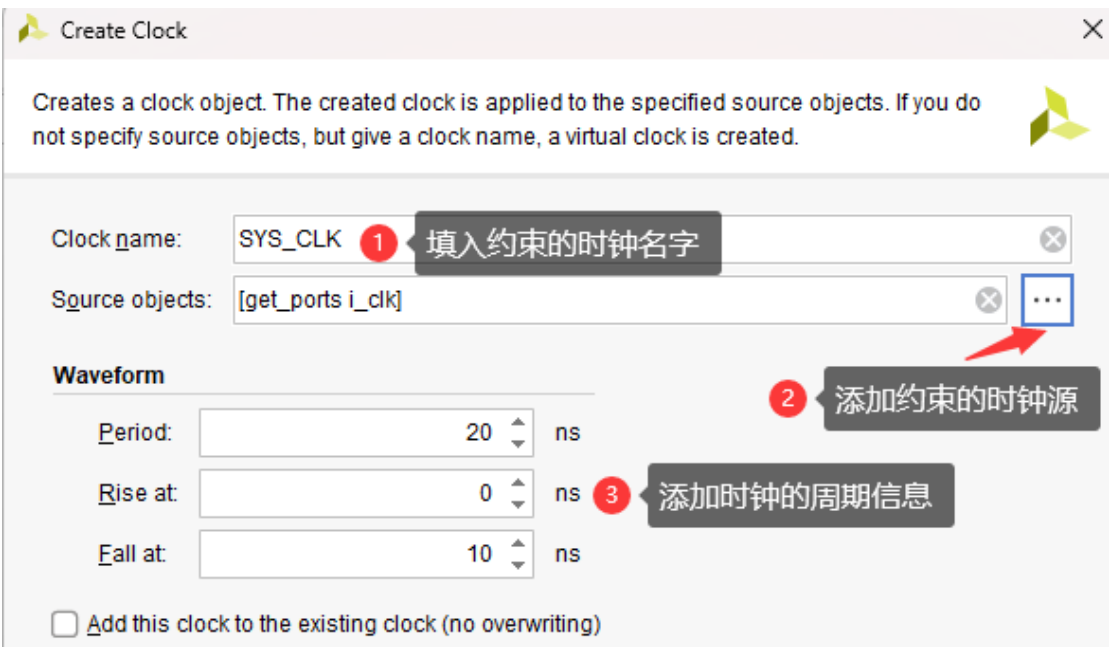


图 3-43 约束主时钟

(4) 点击 OK，随后保存，可以看以下约束文件中是否写入了对主时钟的约束，如下图所示：

```
create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} [get_ports i_clk]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clock [get_nets i_clk_IBUF_BUFG]
```

图 3-44 约束文件中主时钟约束语句

1.9 上板验证

1.9.1 HDL 实例化调试及上板验证

(1) 生成 BIT，点击生成 BIT，如下图所示，一般生成 bit 过程较慢，稍作等待

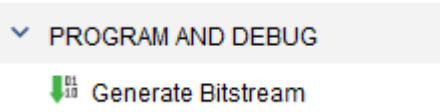


图 3-45 点击生成 BIT 按钮

(2) bit 生成完成后，观察当前的设计，发现并没有时序错误，如下图所示：

Timing	Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	14.036 ns		
Total Negative Slack (TNS):	0 ns		
Number of Failing Endpoints:	0		
Total Number of Endpoints:	3828		
Implemented Timing Report			

图 3-46 时序报告

(3) 将生成的 bit 文件下载至 FPGA 内，打开 HardWare Manager，选择自动连接 JTAG。

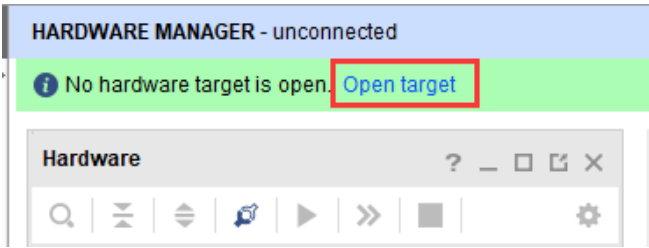


图 3-47 打开 HardWare Manager

连接成功界面如下图所示：



图 3-48 成功连接 JTAG

(4) 选择 FPGA，右击下载，Vivado 会自动检测 BIT 文件所在位置。如下图所示：

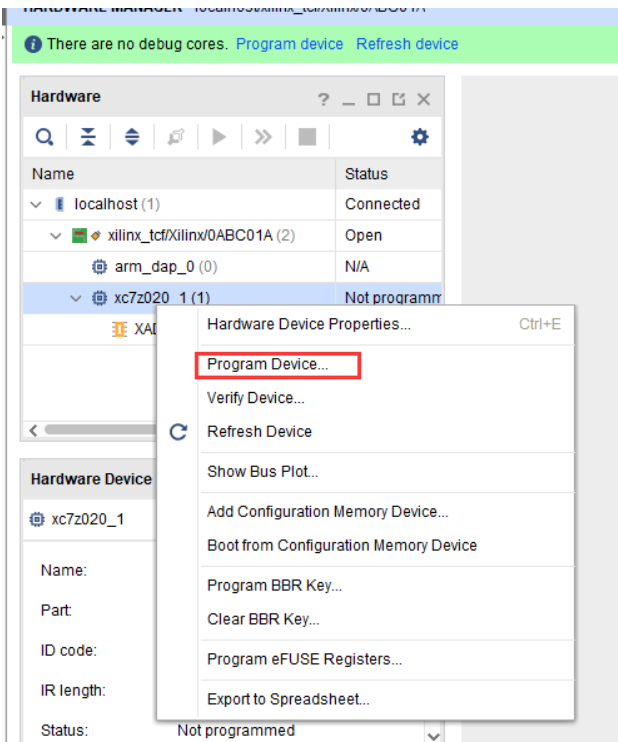


图 3-49 下载 BIT 文件

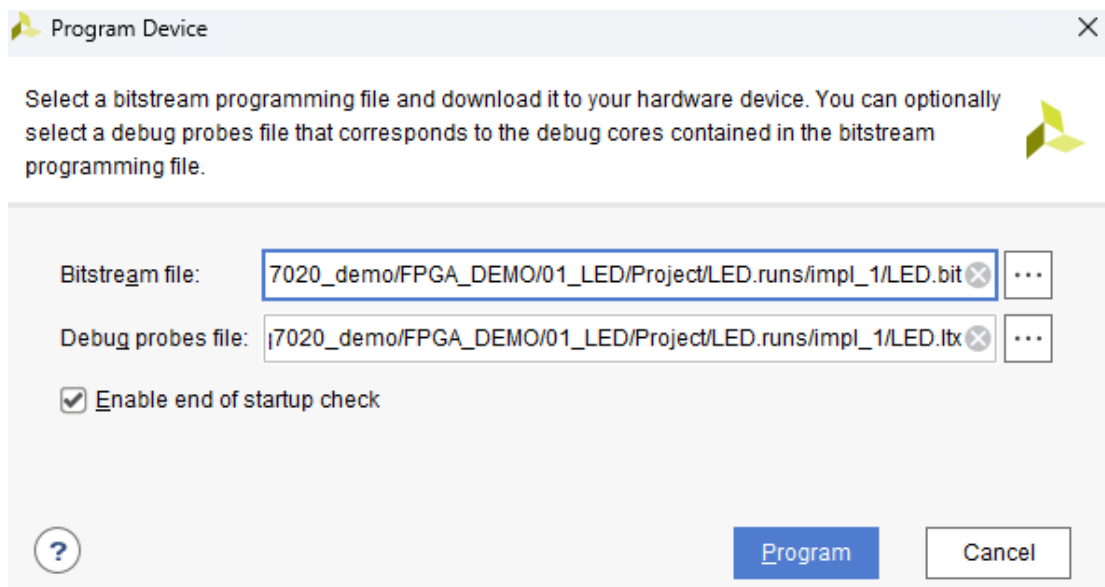


图 3-50 自动检测 BIT 文件以及波形文件位置

(5) 随后点击下载，界面如下图所示：

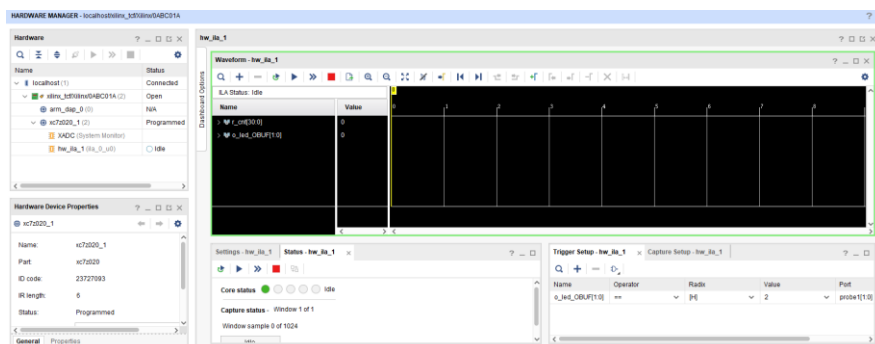


图 3-51 下载成功界面

当下载成功会发现 LED 灯开始闪烁。

(6) LED 灯开始闪烁，是肉眼能观测到的现象，但我们 1.7 章中进行了 ILA 的设置，我们可以观测抓取到的信号是否符合仿真。将 ILA 设置为触发条件为 LED 输出为 2，随后点击触发，如下图所示：

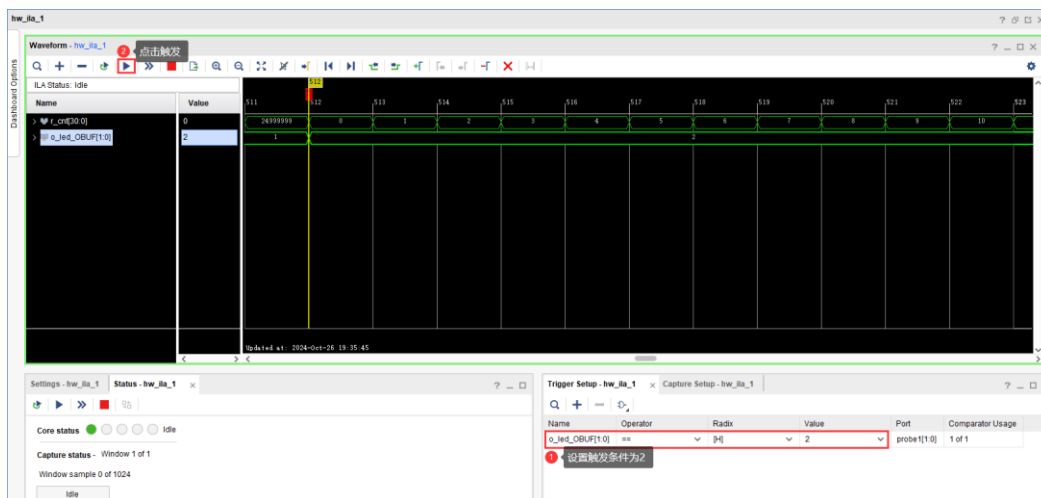


图 3-52 触发成功

由图 3-52 可以发现，LED 在计数器计数到 24999999 时进行了切换，也就是对应了我们的设计要求，LED 开始闪烁，辐射设计要求。

1.9.2 Debug 调试及上板验证

生成 bit 与下载过程与 1.9.1 中是一致的，在这重复写了，直接下载至 FPGA 内，触发信号也如 1.9.1 教程一致，触发成功下图所示。

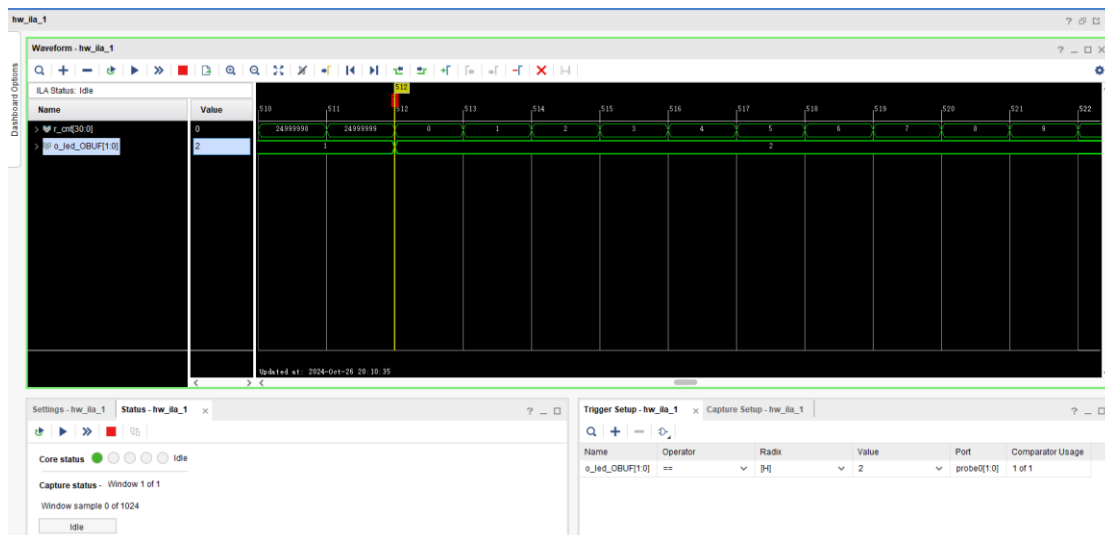


图 3-53 触发成功

整个测试方式与 1.9.1 章节是一致的，整个测试过程是一致的，结果也是一致的。

2.LED 实验总结

本章的内容主要是通过一个简单的“点亮 LED 灯”的例程，将 FPGA 开发的完整流程走一遍，整个 FPGA 开发流程中编写代码、仿真验证、时序约束以及 ILA 使用的步骤是比较重要的，需要熟练掌握操作流程。因为“点亮 LED 灯”的例程比较简单，我们上述步骤的使用都是基础功能，在后续开发的历程中还有更加详细的描述介绍。

四. 按键控制 LED 灯

按键是常用的一种控制器件。生活中我们可以见到各种形式的按键，由于其结构简单，成本低廉等特点，在家电、数码产品、玩具等方面有广泛的应用。本章介绍如何使用按键来控制 LED 灯闪烁。

1. 实验目标

本节实验任务是使用板子上的 PL_KEY1 和 PL_KEY2 按键来控制底板上的 PL_LED1 和 PL_LED2 两个 LED 闪烁。没有按键被按下时，两个 LED 保持常灭；如果按键 PL_KEY2 被按下，则两个 LED 交替闪烁；如果按键 PL_KEY1 被按下，则两个 LED 同时闪烁。

2. 按键硬件电路

开发板上的 PL_KEY1、PL_KEY2 电路原理图如图 4-1 所示：

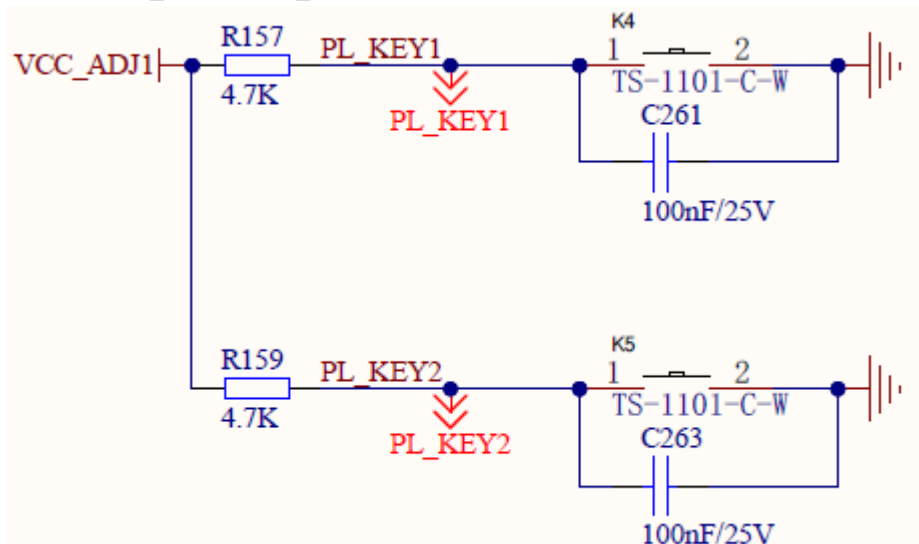


图 4-1 PL 端 KEY 按键电路

由图 4-1 可以看出，当按键未按下时，FPGA 端口检测的电压为高电平，而当按键按下时电路导通，此时 FPGA 端口检测的电压为低电平。串联在电路中的 4.7K 电阻主要做分流作用防止电流过大烧坏电路，并联的 100nF 电容主要做滤波作用，将按键按键及松开过程中产生的抖动信号进行滤除。

LED 电路分析在第三章的实验中有详细介绍，再次不做赘述。

3. 按键驱动程序设计

由实验任务可知，本次实验需要实现的功能是使用两个按键控制两个 LED 灯以不同的方式闪烁。两个 LED 灯闪烁的间隔可以使用计数器实现，因此本次实验需要使用系统时钟，系统复位在 FPGA 系统中也是必不可少的，当程序出现跑飞等异常情况时，可以使程序恢复

至默认状态；由以上分析可知，本次实验需要 3 个输入的端口，分别为系统时钟、系统复位和两位的按键输入，输出为两位的 LED 端口，模块框图如下图所示：

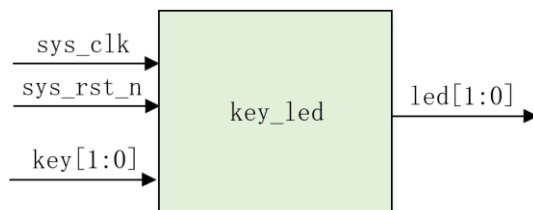


图 4-2 按键控制 LED 系统框图

最终闪烁方式效果图如下所示：

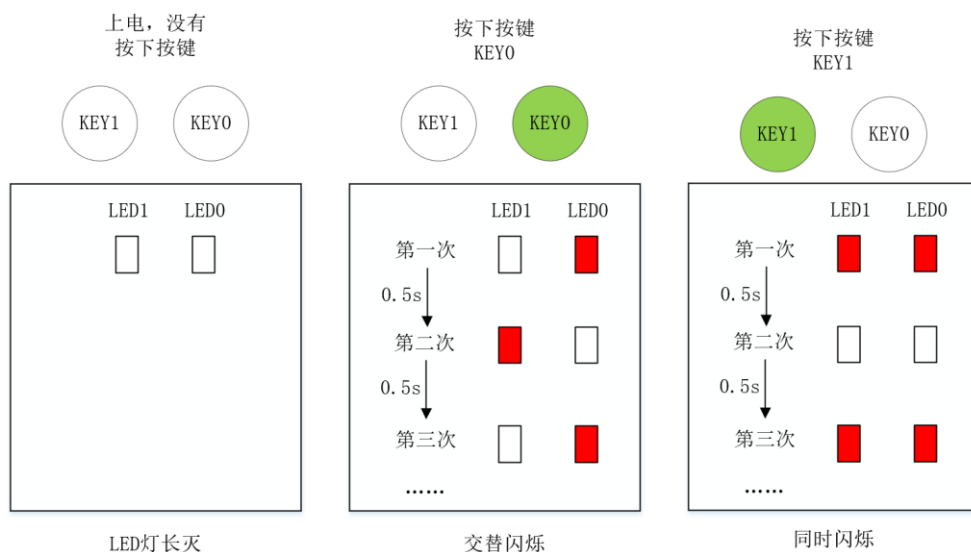


图 4-3 实验效果

在点亮 LED 的实验中介绍过 LED 亮灭的原理，在按键控制 LED 实验中是一致的，实验是通过计数器实现一个 0.5S 翻转的状态位，通过状态位与按键的状态控制驱动 LED。利用 Vivado 创建工程在第三章中有详细教程，因此这部分不进行详细介绍，直接开始编写代码。

(1) 在实验目标中提出，每隔 0.5S 就翻转一个标志，那么首先需要产生一个 0.5S 间隔的计数器。而 ZYNQ PL 端的时钟为 50MHz，需要产生一个 0.5S 的计数器则需要计数到 24999999，P_LEN 是一个参数，主要是加快仿真时间。Verilog 实现如下：

```
//产生0.5s 计数器
always @(posedge i_clk or negedge i_rst_n) begin
    if(~i_rst_n)
        r_cnt <= 'd0;
    else if (r_cnt == P_LEN)
        r_cnt <= 'd0;
    else
        r_cnt <= r_cnt + 'd1;
end
```

(2) 0.5S 的信号产生后，则需要产生一个翻转的标志，该信号主要是在计数器计数到 0.5S 时则进行翻转，用于与按键信号进行同时判断，Verilog 代码的实现如下：

```
// 每0.5S 产生翻转信号
always @(posedge i_clk or negedge i_rst_n) begin
    if(~i_rst_n)
        r_flag <= 'd0;
    else if (r_cnt == P_LEN)
        r_flag <= ~r_flag;
    else
        r_flag <= r_flag;
end
```

(3) 最后则是通过 flag 与输入的按键信号进行 case 的判断，依据不同的输入按键信号输出不同的 LED 效果，Verilog 实现效果如下：

```
// 依据输入 key 信号驱动 LED
always @(posedge i_clk or negedge i_rst_n) begin
    if(~i_rst_n)
        ro_led <= 'd0;
    else
        case (i_key)
            2'b01: // 按键 Key 1 按下, Key 2 未按下, LED 交替闪烁
                begin
                    if(r_flag == 0)
                        ro_led <= 2'b10;
                    else
                        ro_led <= 2'b01;
                end
            2'b10: // 按键 Key 1 未按下, Key 2 按下, LED 同时闪烁
                begin
                    if(r_flag == 0)
                        ro_led <= 2'b00;
                    else
                        ro_led <= 2'b11;
                end
            2'b11 // Key 1、Key 2 都未按下, LED 灯灭
                begin
                    ro_led <= 2'b00;
                end
            default: ro_led <= 'd0;
        endcase
end
```

4.按键驱动仿真

关于如何创建仿真文件的步骤在第三章中有详细教程，过程不再赘述，直接进行仿真代码的编写，并对上述驱动进行仿真，观察仿真是否符合设计要求。

(1) 编写仿真代码，仿真代码中产生了 FPGA 的测试时钟、复位、及每隔一段时间对按键 KEY 的值进行修改，修改后输入按键控制 LED 中，将 **P_LEN** 的值修改为 9，主要的目的是加快仿真时间，能快速观测到仿真效果。

```
module SIM_TB_KEY_LED();

reg clk,rst_n; //定义时钟及复位

reg [1:0] key; //定义按键

//初始化时钟及复位
initial begin
    clk    <= 'd0;
    rst_n <= 'd0;
    #100
    rst_n <= 'd1;
end

//产生测试时钟
always begin
    clk <= 'd0;
    #10;
    clk <= 'd1;
    #10;
end

//产生按键值
initial begin
    key    <= 2'b00;
    #200
    key    <= 2'b10;
    #200
    key    <= 2'b01;
    #200
    key    <= 2'b11;
end
```

```

KEY_LED
#(
    .P_LEN ('d9)
)KEY_LED_U0
(
    .i_clk ( clk      ),
    .i_rst_n ( rst_n   ),

    .i_key ( key      ),
    .o_led  ()
);
endmodule

```

(2) 仿真效果如下图所示：

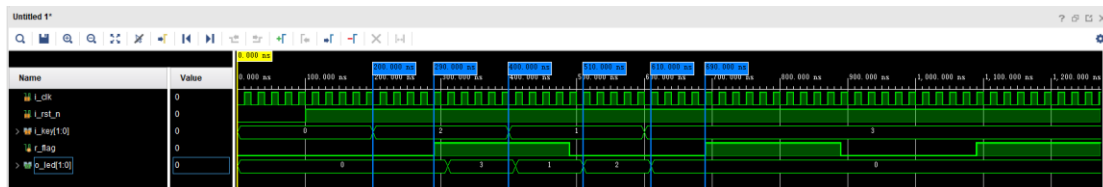


图 4-4 仿真效果

从图 4-4 可以看出，当 200ns 时，按键值改变为 2，Flag 为 0，此时 LED 输出为 0；在 290ns 时，按键值为 2，Flag 从 0 改变成 2，下一个周期 LED 输出 13，表明两个灯都亮了；在 400ns 时，按键值修改为 1，Flag 为 1，LED 输出 1，表明低位 LED 亮了；在 510ns 时，Flag 为 0，LED 输出 2，表明高位 LED 亮了；在 610ns 时，按键值修改为 3，Flag 为 0，LED 输出 0，表明两个 LED 熄灭；在 690ns 时，按键值为 3，Flag 为 1，LED 输出 0，表明两个 LED 熄灭。

从图 4-4 中的仿真结果可以看出，符合第一节中提出的设计要求。

5. 添加内部逻辑分析仪

(1) 由于采用 IP 的方式对信号抓取是不灵活的，后续逻辑分析的添加都是采用 Debug 的方式进行添加。在按键控制 LED 的实验，可以抓取 Flag 信号，输入 Key 信号以及输出 LED 信号，添加信号如下图所示：

```

module KEY_LED
#(
    parameter P_LEN = 'd24_999_999
)
(
    input        wire        i_clk  ,
    input        wire        i_rst_n ,
    (* mark_debug = "true" *)
    input        wire [1:0]    i_key  ,
    (* mark_debug = "true" *)
    output        wire [1:0]    o_led
);

reg [1:0]    ro_led ;
reg [30:0]   r_cnt  ;
(* mark_debug = "true" *)
reg          r_flag ;

```

图 4-5 添加 Debug 信号

(2) 添加完 Debug 信号后，对当前工程进行综合并设置需要抓取的信号

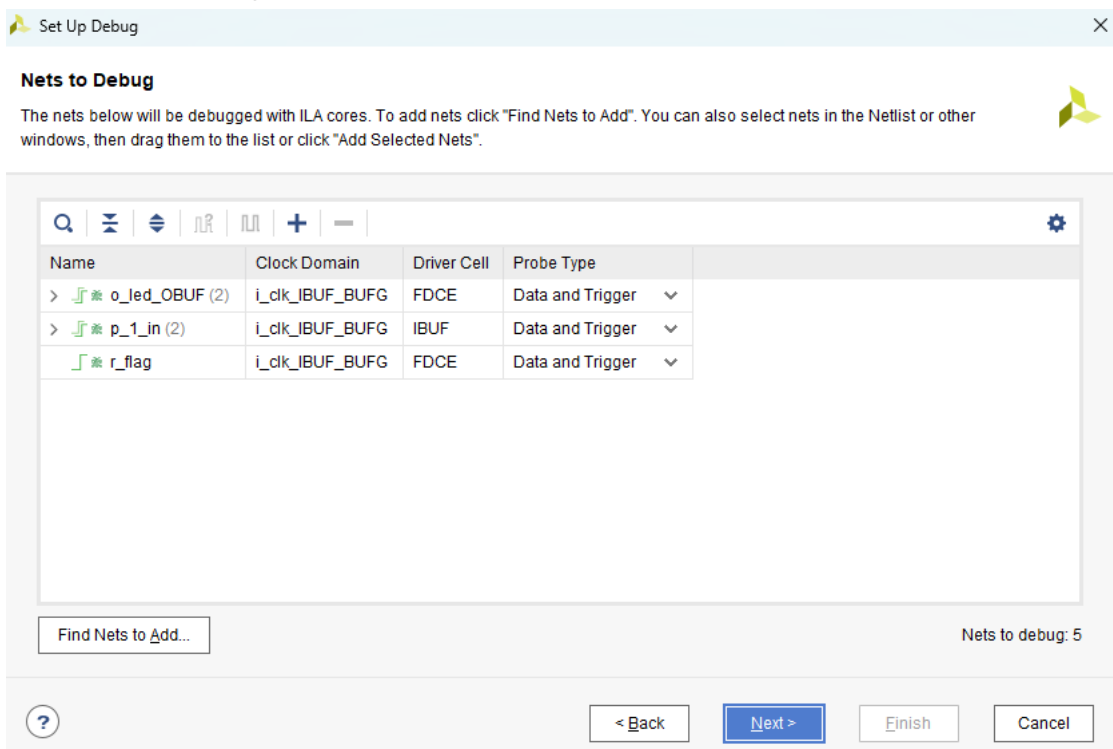


图 4-6 设置 Debug 的信号

设置 ILA 深度为 1024

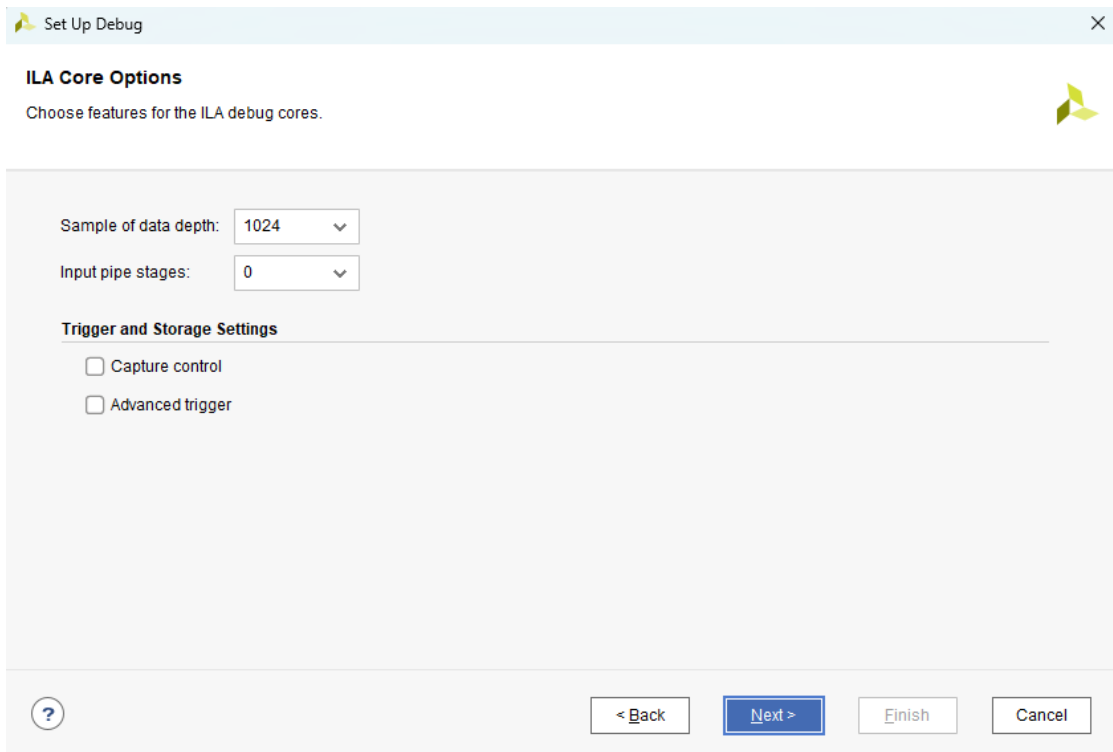


图 4-7 设置 Debug 深度

(3) 查看约束文件是否写入 Debug

```

create_debug_core u_ila_0 ila
set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]
set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
set_property port_width 1 [get_debug_ports u_ila_0/clk]
connect_debug_port u_ila_0/clk [get_nets [list i_clk_IBUF_BUFG]]
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
set_property port_width 2 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets [list {o_led_OBUF[0]} {o_led_OBUF[1]}]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
set_property port_width 2 [get_debug_ports u_ila_0/probe1]
connect_debug_port u_ila_0/probe1 [get_nets [list {p_1_in[0]} {p_1_in[1]}]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
set_property port_width 1 [get_debug_ports u_ila_0/probe2]
connect_debug_port u_ila_0/probe2 [get_nets [list r_flag]]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets i_clk_IBUF_BUFG]

```

图 4-8 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

6. 添加时序约束

由于按键控制 LED 的工程比较简单，因此只需要约束主时钟，虽然 Vivado 在生成 Bit 的过程中能自动约束时钟，进行时序约束的过程是为了一个设计的完整性。

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按钮就可以添加时钟约束了，如下图所示：

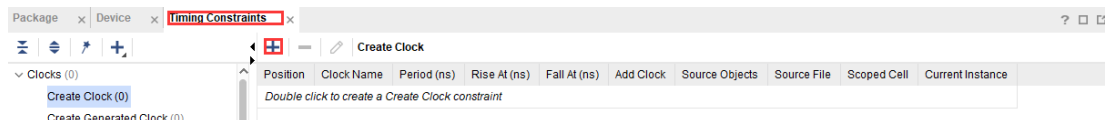


图 4-9 添加时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

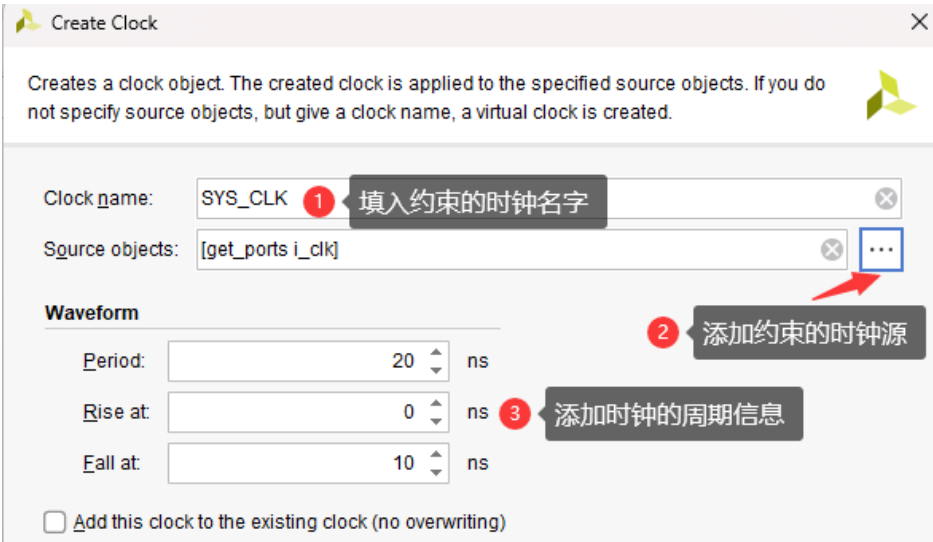


图 4-10 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} -add [get_ports i_clk]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets i_clk_IBUF_BUFG]
```

图 4-11 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，那么接下来步骤则是对管脚进行约束。

7.添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，那么接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

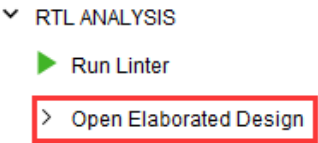


图 4-12 打开管脚约束

(2) 需要注意的是，板卡上只有两个 PL 按键，因此整个系统的复位按键是随便挂载的 40 Pin 拓展口的管脚，如下图所示：

IO Ports												
Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination
L_key (2)												
L_key[1]	IN		V14	✓	33	LVCMOS33*	3.300				NONE	NONE
L_key[0]	IN		W18	✓	33	LVCMOS33*	3.300				NONE	NONE
o_led (2)												
o_led[1]	OUT		V13	✓	33	LVCMOS33*	3.300		12	✓	NONE	FP_VTT_50
o_led[0]	OUT		V15	✓	33	LVCMOS33*	3.300		12	✓	NONE	FP_VTT_50
Scalar ports (2)												
i_clk	IN		W17	✓	33	LVCMOS33*	3.300				NONE	NONE
LrstLn	IN		W12	✓	13	LVCMOS33*	3.300				NONE	NONE

图 4-13 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
# 主时钟及复位
set_property PACKAGE_PIN W17 [get_ports i_clk]
set_property IOSTANDARD LVCMOS33 [get_ports i_clk]

set_property PACKAGE_PIN W12 [get_ports i_rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports i_rst_n]

# 输入按键
set_property PACKAGE_PIN V14 [get_ports {i_key[1]}]
set_property PACKAGE_PIN W18 [get_ports {i_key[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {i_key[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {i_key[0]}]

# 输出LED
set_property PACKAGE_PIN V15 [get_ports {o_led[0]}]
set_property PACKAGE_PIN V13 [get_ports {o_led[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {o_led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_led[1]}]
```

图 4-14 约束文件中写入结果

8.上板验证

对整个 FPGA 约束完成后，可以生成 Bit 文件进行上板验证。Bit 文件生成后，可以看下时序是否有错误，可以发现此次设计时序没有错误，如下图所示：

Timing	Setup Hold Pulse Width
Worst Negative Slack (WNS):	14.005 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	3663
Implemented Timing Report	

图 4-15 时序结果

(1) 将 Bit 下载至 FPGA，随后打开 ILA，将触发信号设置为 Flag 的上升沿，并在不按按键的情况下进行触发，触发结果如下图所示：

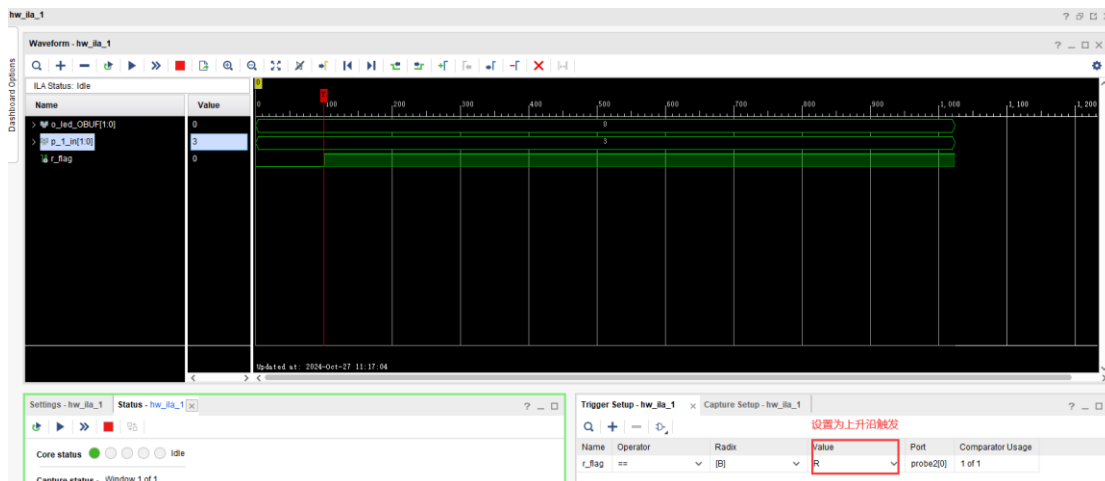


图 4-16 按键未按下触发结果

图 4-16 可以看出，当 Flag 为上升沿时，输入按键信号为 3 时，LED 输出为 0，在 LED 章节中分析过，当 LED 管脚输出 0 的时候，LED 灯不亮，因此符合设计的。

(2) 将触发信号设置为 Flag 的上升沿，按键值设置为 2'b01 进行触发，触发结果如下图所示：

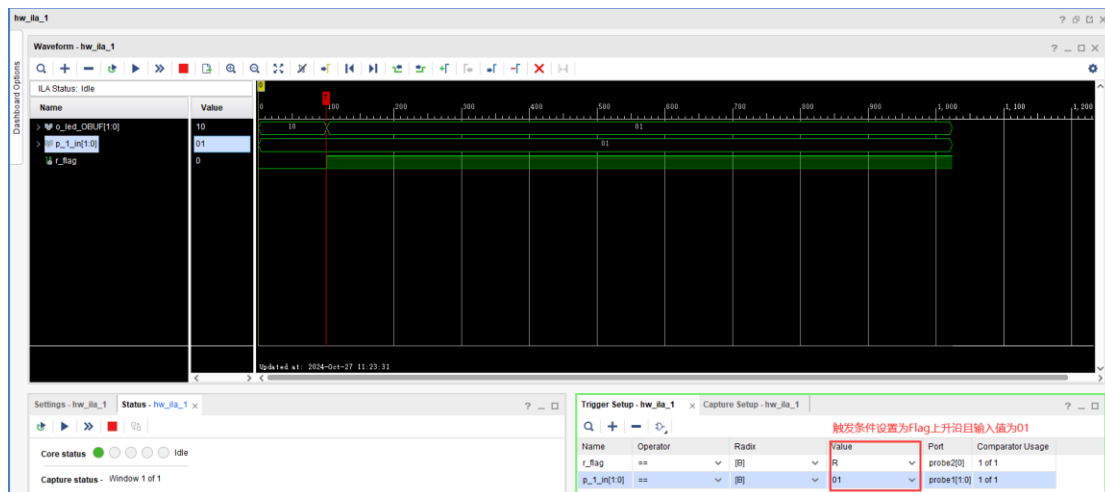


图 4-17 按键设置为 2'b01 且 Flag 为上升的触发结果

图 4-17 可以看出，当 Flag 为上升沿时，输入按键信号为 2'b01 时，LED 输出为 2'b01，一直按着按键，LED 灯呈现一种流水灯的效果。

(3) 将触发信号设置为 Flag 的上升沿，按键值设置为 2'b1 进行触发，触发结果如下图所示：

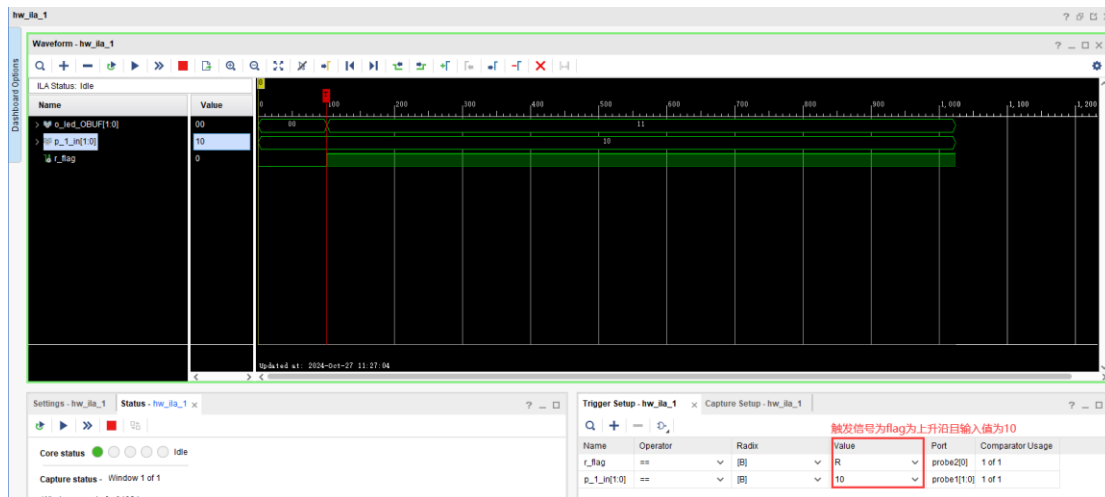


图 4-18 按键设置为 2'b10 且 Flag 为上升的触发结果

图 4-18 可以看出，当 Flag 为上升沿时，输入按键信号为 2'b10 时，LED 输出为 2'b11，一直按着按键，两个 LED 灯同时亮灭。

到此整个按键控制 LED 的实验已经完成，是当时构想的实验要求的。

9.实验总结

本章掌握了是按键电路的工作原理，即开发板上电后按键未被按下时保持高电平 1，按键被按下时保持低电平 0，同时需要掌握如何在仿真文件中添加按键的不同输入状态来模拟使用按键的不同情况。本章我还使用了新语法 case 语句的使用，case 语句的用法总结如下：

case (控制表达式)

分支表达式 1: 执行语句;

分支表达式 2: 执行语句;

...

...

...

default: 执行语句;

endcase

case 是从上往下执行的，按照这个顺序对各个分支的表达式进行判断，如果这一分支表达式等于控制表达式的值，就执行其对应操作，然后跳出 case 语句；如果都不相等时，执行 default 操作。

五. 呼吸灯实验

呼吸灯最早由苹果公司发明并应用于笔记本睡眠提示上，其一经展出，立刻吸引众多科技厂商争相效仿，并广泛用于各种电子产品中，尤其是智能手机。呼吸灯其实是在微处理器的控制下，由暗渐亮、然后再由亮渐暗，模仿人呼吸方式的 LED 灯。

1.呼吸灯实现原理

呼吸灯实际展示的效果就是一个 LED 灯的亮度由亮到暗，再由暗到亮的变化过程，并且该过程是循环往复的，像呼吸一样那么有节奏。呼吸灯是采用 PWM 驱动的方式，在固定的频率下，通过调整占空比来控制 LED 灯亮度的变化。

PWM (Pulse Width Modulation)，即脉冲宽度调制，它是利用微处理器输出的 PWM 信号对模拟电路进行控制的一种非常有效的技术，广泛应用于测量、通信、功率控制等领域。

在由计数器产生固定周期的 PWM 信号下，如果其占空比为 0，则 LED 灯不亮；如果其占空比为 100%，则 LED 灯最亮。将占空比从 0 到 100%，再从 100%到 0 不断变化，就可以实现 LED 灯的“呼吸”效果。

PWM 占空比调节示意图如下图所示：

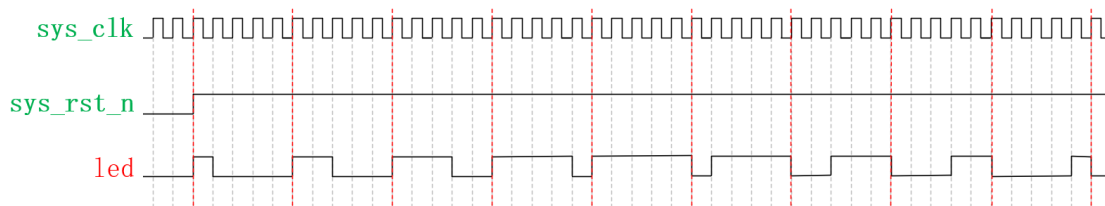


图 5-1 呼吸灯 PWM 改变占空比示意图

由上图可知，LED 高电平的时间由短渐渐变长，再由长渐渐变短，如果 LED 灯是高电平点亮，则 LED 灯会呈现出亮度由暗到亮，再由亮到暗的过程。

2.呼吸灯硬件设计

呼吸灯主要使用的是板载的 LED，关于 LED 驱动电路在第三章中已经进行详细介绍，在这不对 LED 驱动电路详细介绍。如果有不明白的部分参考第三章 PL 点亮 LED 灯章节中的硬件介绍。

3.实验任务

本节实验任务是使用开发板上的 PL LED，实现呼吸灯的效果，即由暗渐亮，然后再由亮渐暗。

4.PWM 驱动程序设计

LED 灯的亮度由亮到暗变化，就是 LED 灯信号的高电平占空比开始递减，如图 5-2 所

示；LED 灯的亮度由暗到亮变化，就是 LED 灯信号的高电平占空比开始递增，如图 5-3 所示；因此，需要定义一个 LED 灯高电平占空比递增/递减信号 inc_dec_flag。

inc_dec_flag=1'b1, 高电平占空比递减，灯由亮变暗

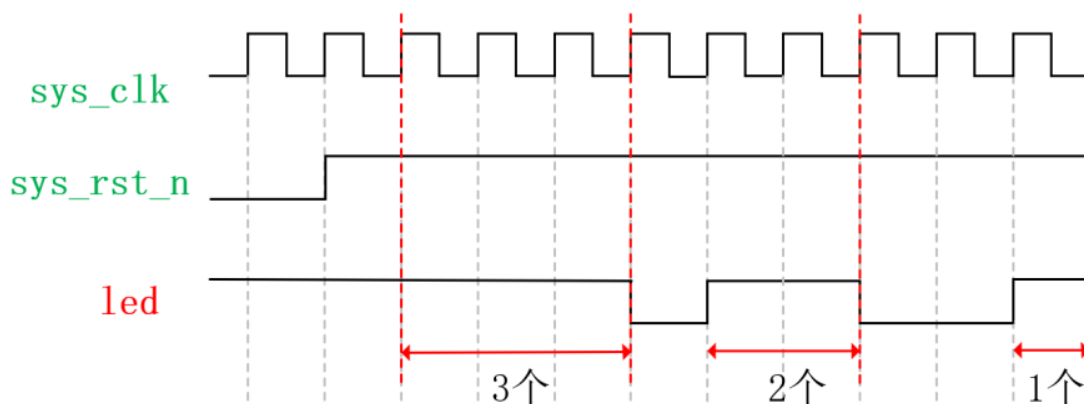


图 5-2 LED 灯的亮度由亮变暗的过程

inc_dec_flag=1'b0, 高电平占空比递增，灯由暗变亮

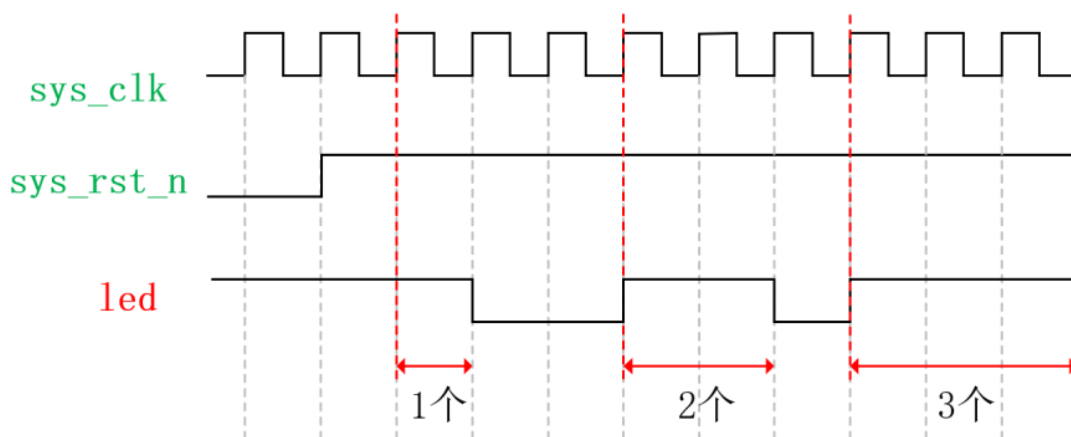


图 5-3 LED 灯的亮度由暗变亮的过程

方便大家理解呼吸灯的实现过程，将呼吸灯从亮到暗在从暗到亮这两个组成部分开说，这里我们先详细说明呼吸灯由暗到亮的渐变过程（由亮到暗的过程与由亮到暗的过程刚好相反）。

本实验中将由暗到亮的渐变过程定义为 2s，接下来将这 2s 分成 1000 份，就是由暗到亮的渐变过程中有 1000 个亮度过渡，每一份过渡时间为 2ms；接下来我们再将每一份过渡时间 2ms 均分为 1000 份，每份时间为 2us，我们以 2us 为单位递减 2ms 中的高电平的占空比，随着高电平占空比的不断减少，led 灯的亮度不断变暗。同时在每次到达 2S 的同时将 Flag 标志进行翻转，那么接下来则进行代码编写。

创建工程的过程再第三章中有详细的教程，因此不再详细描述，若有遗忘的步骤的可以翻看第三章创建文件的过程。

(1) 首先产生一个 2us 的计数器，在 2us 的时间中去调整高电平的时间，代码如下：

```
always @(posedge i_clk or negedge i_rst_n) begin
    if (~i_rst_n)
        r_cnt_2us <= 'd0;
    else if (r_cnt_2us == CNT_2us)
```

```

        r_cnt_2us <= 'd0;
    else
        r_cnt_2us <= r_cnt_2us;
    end

```

(2) 其次产生一个 2ms 的计数器，2ms 的计数器是整个 PWM 方波的周期，代码实现如下：

```

//产生 2ms 计数器，PWM 的周期
always @(posedge i_clk or negedge i_rst_n) begin
    if (~i_rst_n)
        r_cnt_2ms <= 'd0;
    else if (r_cnt_2us == CNT_2us && r_cnt_2ms == CNT_2ms)
        r_cnt_2ms <= 'd0;
    else if (r_cnt_2us == CNT_2us)
        r_cnt_2ms <= r_cnt_2ms + 'd1;
    else
        r_cnt_2ms <= r_cnt_2ms;
    end

```

(3) 再者产生一个 2s 的计数器，2s 的计数器是 LED 灯从亮到暗或者从暗到亮的周期，代码实现如下：

```

//产生 2s 的计数器，LED 的呼吸闪烁的周期
always @(posedge i_clk or negedge i_rst_n) begin
    if (~i_rst_n)
        r_cnt_2s <= 'd0;
    else if (r_cnt_2us == CNT_2us && r_cnt_2ms == CNT_2ms
        && r_cnt_2s == CNT_2s)
        r_cnt_2s <= 'd0;
    else if (r_cnt_2us == CNT_2us && r_cnt_2ms == CNT_2ms)
        r_cnt_2s <= r_cnt_2s + 'd1;
    else
        r_cnt_2s <= r_cnt_2s;
    end

```

(4) 最后产生一个 2S 翻转的 Flag 信号，代码如下：

```

// 每两秒翻转一次的 Flag
always @(posedge i_clk or negedge i_rst_n) begin
    if (~i_rst_n)
        r_flag <= 'd0;
    else if (r_cnt_2us == CNT_2us && r_cnt_2ms == CNT_2ms
        && r_cnt_2s == CNT_2s)
        r_flag <= ~r_flag;
    else
        r_flag <= r_flag;
    end

```

(5) 最后依据上述计数器实现 LED 灯的呼吸效果，代码实现如下：

```
// 驱动 LED
always @(posedge i_clk or negedge i_rst_n) begin
    if (~i_rst_n)
        ro_led <= 'd0;
    else if ((r_flag && r_cnt_2ms >= r_cnt_2s)
        || (r_flag == 'd0 && r_cnt_2ms <= r_cnt_2s))
        ro_led <= 'd1;
    else
        ro_led <= 'd0;
end
```

5.PWM 驱动仿真

关于如何创建仿真文件的步骤在第三章中有详细教程，过程不再赘述，直接进行仿真代码的编写，并对上述驱动进行仿真，观察仿真是否符合设计要求。

(1) 编写仿真代码，仿真代码中产生了 FPGA 的测试时钟、复位、将三个 CNT 的值缩小至合适的值，主要的目的是加快仿真时间，能快速观测到仿真效果。

```
module SIM_TB_Breath_LED();

    reg clk,rst_n; //定义时钟及复位

    // 初始化时钟及复位
    initial begin
        clk    <= 'd0;
        rst_n <= 'd0;
        #100
        rst_n <= 'd1;
    end

    // 产生测试时钟
    always begin
        clk <= 'd0;
        #10;
        clk <= 'd1;
        #10;
    end

    Breath_LED
    #(
        .CNT_2us      ('d9 ) ,
```

```

        .CNT_2ms      ('d19)  ,
        .CNT_2s       ('d29)
    )Breath_LED_U0
    (
        .i_clk        (clk      ),
        .i_rst_n       (rst_n    ),

        .o_led         ()
    );

```

上述仿真主要是产生一个系统 50Mhz 的时钟以及按键复位，同时将三个计数器的值分别修改为 9, 19, 29，这三个值是随便设定的，只是为了快速观测 LED 状态。

(2) LED 从暗变亮的仿真结果如下图所示：

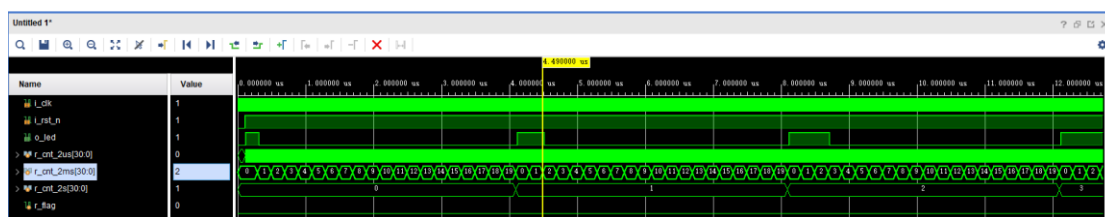


图 5-4 LED 灯的由暗变亮的过程

从图 5-4 中可以看出三个计数器也在正常增加，而 LED 的输出高电平时间也在不断拉长，符合 LED 在不断变亮的过程。

LED 由亮变暗的仿真过程如下图所示：

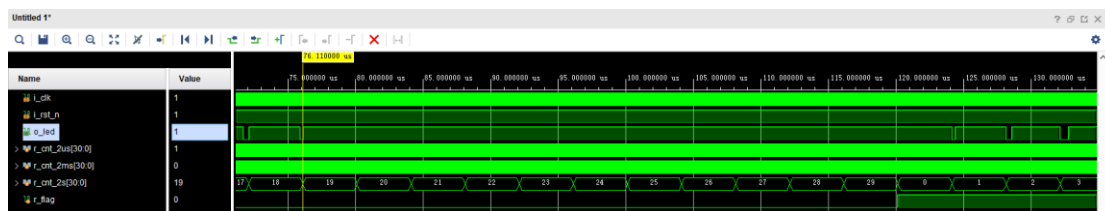


图 5-5 LED 灯的由亮变暗的过程

从图 5-5 中可以看出三个计数器也在正常增加，而 LED 的输出高电平时间也在不断缩短，符合 LED 在不断变暗的过程。

从图 5-4 以及图 5-5 可以看出，整个 LED 是符合设计要求的。

6. 添加内部逻辑分析仪

(1) 在 PWM 驱动 LED 的实验中，可以抓取三个计数器、Flag 以及输出 LED 信号，添加信号标识如下图所示：

```

    (* mark_debug = "true" *)
    output      wire      o_led
);

(* mark_debug = "true" *)
reg    [30:0]  r_cnt_2us   ;
(* mark_debug = "true" *)
reg    [30:0]  r_cnt_2ms   ;
(* mark_debug = "true" *)
reg    [30:0]  r_cnt_2s    ; //每隔25对呼吸灯的亮暗顺序发生修改
(* mark_debug = "true" *)
reg    r_flag    ;

```

图 5-6 添加 Debug 信号

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。

Set Up Debug

ILA Core Options
Choose features for the ILA debug cores.

Sample of data depth: 4096 设置深度为4096

Input pipe stages: 0

Trigger and Storage Settings

☐ Capture control

☐ Advanced trigger

? < Back Next > Finish Cancel

图 5-6 设置 Debug 信号的深度

(3) 查看是否写入约束文件

```

create_debug_core u_ila_0 ila
set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
set_property port_width 1 [get_debug_ports u_ila_0/clk]
connect_debug_port u_ila_0/clk [get_nets [list i_clk_IBUF_BUF]]
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
set_property port_width 31 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets [list {r_cnt_2ms[0]} {r_cnt_2ms[1]} {r_cnt_2ms[2]} {r_cnt_2ms[3]} {r_cnt_2ms[4]} {r_cnt_2ms[5]} {r_cnt_2ms[6]}]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
set_property port_width 31 [get_debug_ports u_ila_0/probe1]
connect_debug_port u_ila_0/probe1 [get_nets [list {r_cnt_2us[0]} {r_cnt_2us[1]} {r_cnt_2us[2]} {r_cnt_2us[3]} {r_cnt_2us[4]} {r_cnt_2us[5]} {r_cnt_2us[6]}]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
set_property port_width 31 [get_debug_ports u_ila_0/probe2]
connect_debug_port u_ila_0/probe2 [get_nets [list {r_cnt_2s[0]} {r_cnt_2s[1]} {r_cnt_2s[2]} {r_cnt_2s[3]} {r_cnt_2s[4]} {r_cnt_2s[5]} {r_cnt_2s[6]}]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe3]
set_property port_width 1 [get_debug_ports u_ila_0/probe3]
connect_debug_port u_ila_0/probe3 [get_nets [list o_led_OBUF]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe4]
set_property port_width 1 [get_debug_ports u_ila_0/probe4]
connect_debug_port u_ila_0/probe4 [get_nets [list r_flag]]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets i_clk_IBUF_BUF]

```

图 5-7 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

7. 添加时序约束

由于呼吸灯的工程比较简单，因此只需要约束主时钟，虽然 Vivado 在生成 Bit 的过程中能自动约束时钟，进行时序约束的过程是为了一个设计的完整性。

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按键就可以添加时钟约束了，如下图所示：



图 5-8 添加时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

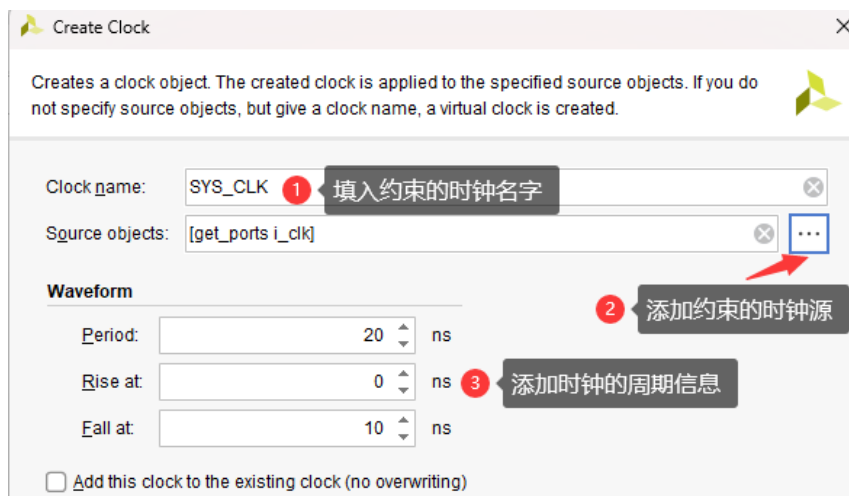


图 5-9 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} [get_ports i_clk]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets i_clk_IBUF_BUFG]
```

图 5-10 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，那接下来则是对管脚进行约束。。

8. 添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

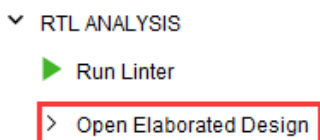


图 5-11 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
▼ All ports (3)													
▼ Scalar ports (3)													
i_clk	IN		W17	☒	33	LVCNMOS33*	3.300				NONE	NONE	
i_rst_n	IN		W18	☒	33	LVCNMOS33*	3.300				NONE	NONE	
o_led	OUT		V15	☒	33	LVCNMOS33*	3.300		12			NONE	FP_VTT_50

图 5-12 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
set_property IOSTANDARD LVCMOS33 [get_ports i_clk]
set_property IOSTANDARD LVCMOS33 [get_ports i_rst_n]

set_property PACKAGE_PIN W17 [get_ports i_clk]
set_property PACKAGE_PIN W18 [get_ports i_rst_n]

set_property PACKAGE_PIN V15 [get_ports o_led]
set_property IOSTANDARD LVCMOS33 [get_ports o_led]
```

图 5-13 约束文件中写入管脚约束

9. 上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证。Bit 文件生成后，可

以看下时序是否有错误，可以发现此次设计时序没有错误，如下图所示：

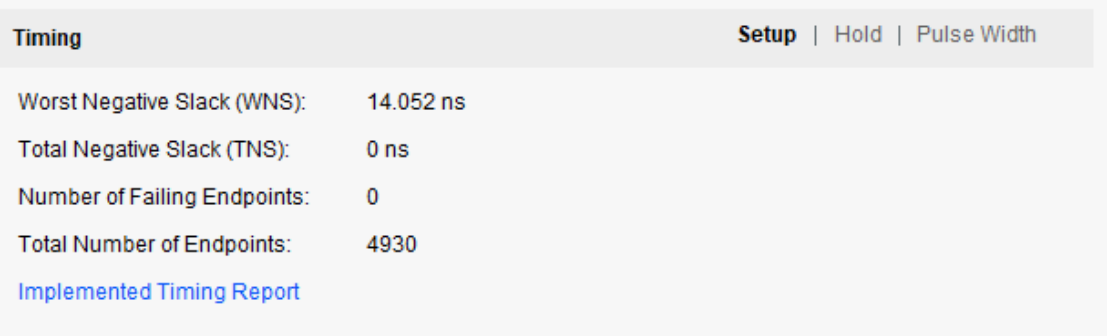


图 5-14 时序结果

(1) 将 Bit 下载至 FPGA，随后打开 ILA，将触发信号设置为 cnt_2S 为 0，随后触发，触发结果如下图所示：

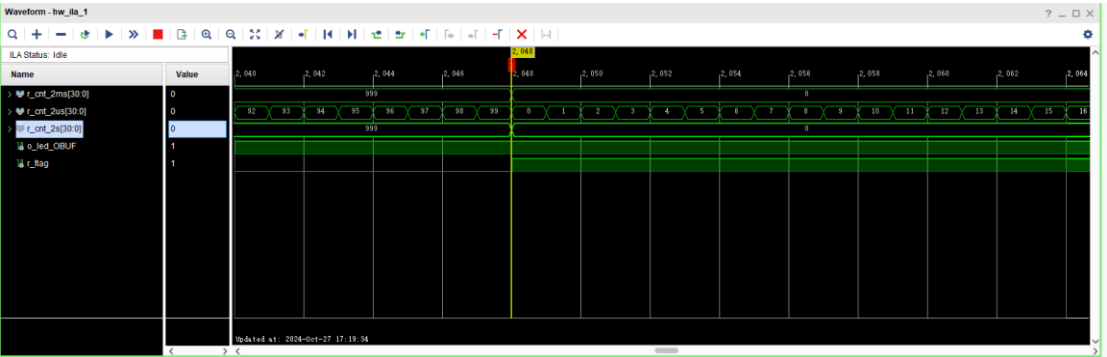


图 5-15 触发波形

将波形放大之后就可以看到 cnt_2s 的具体值，如下图所示：
此时就可以看到，在当 cnt_2s 计数器计到 2s 时，呼吸灯的呼吸状态就会发生跳变，实际调试情况与代码描述的一致。

(2) ILA 调试界面还有一个“Settings”界面，如下图所示：

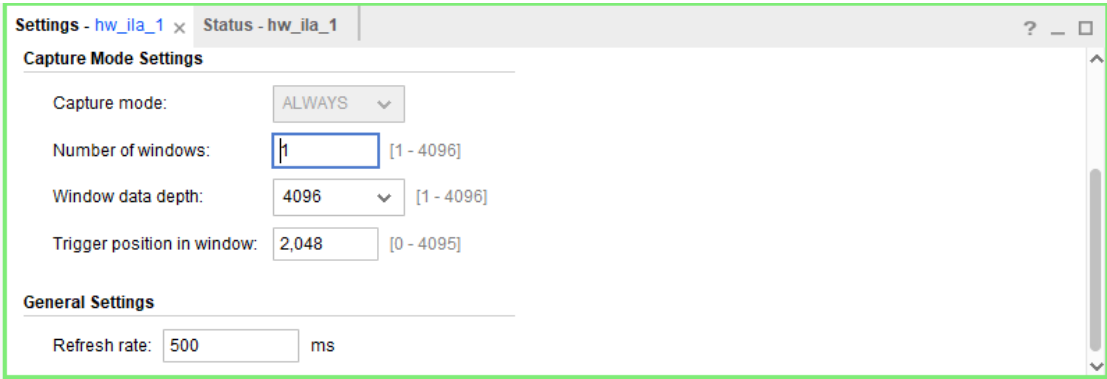


图 5-16 ILA 设置界面

Capture Mode Settings: 捕获模式设置。
Capture Mode: 这里保持默认的常用捕获模式并且不能修改。
Number of windows: 窗口数，ILA 平台会根据这里设置的数值将波形窗口进行均分，该窗口数平常使用时一般默认为 1，其捕捉结果如图 5-15 所示。
Window data depth: 每个窗口的数据深度，窗口打开数据默认深度是我们设置的采样深度 4096，窗口数是 1，那该窗口的数据深度可以设置的范围是 1~4096。

Trigger position in window: 窗口里面的触发位置设置。该位置默认是采样深度的一半，但是可以设置的范围是 0~4095。

(3) 下面展示了一个将窗口数设置为 4 的界面截图：

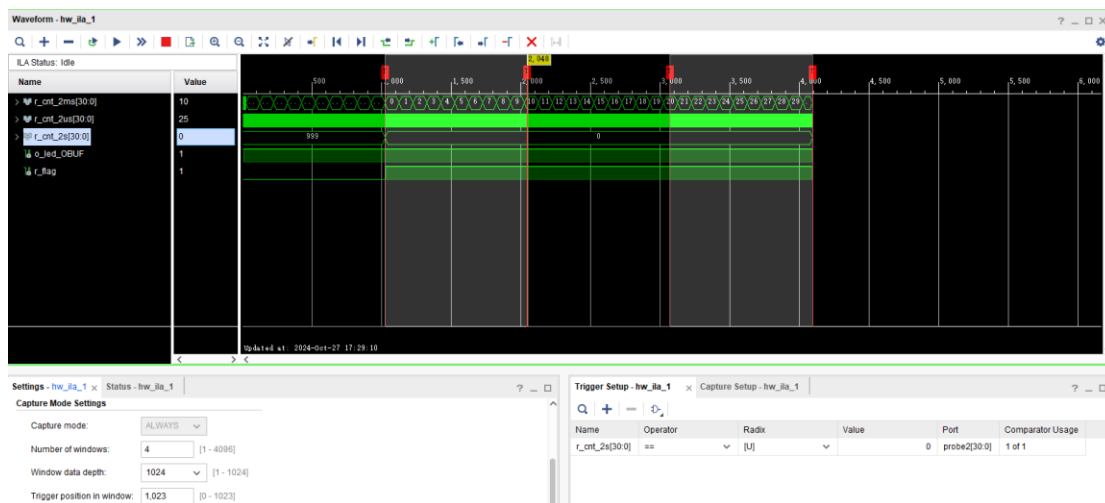


图 5-17 窗口数设置为 4 的界面

如上图所示, 因为我们设置的采样深度是 4096, 将观察的窗口数设置为 4, $4096/4=1024$; 所以每个窗口的数据深度 (Window data depth) 是 1~1024。此时的采样位置为可设置的范围是 0~1023。

10. 实验总结

本章实现的呼吸灯, 从工作原理上分析并不复杂, 但是对于没有掌握 PWM 控制原理的初学者来说, 自己从头设计思路是很难实现的, 所以需要把复杂的问题简单化, 如通过本章的学习我们知道对于某些特定的功能都有特定的原理可以实现该功能, 像呼吸灯、舵机控制等常用的应用就是使用 PWM 控制原理来实现的。

通过本章的学习, 我们知道可以在已有计数器的基础上为更长时间的计数器计时, 而不需要从头开始计数, 这样我们在定义计数器的计时上限时可以节省寄存器的数据位。需要注意的是在使用计数器时一定要明确各个判断条件, 例如开始计时条件, 清零条件等。

六. IP 核之 PLL 实验

PLL 的英文全称是 Phase Locked Loop, 即锁相环, 是一种时钟反馈电路, 具有时钟倍频、分频、相位偏移、可编程占空比和优化抖动等功能, 为了方便我们使用这些功能, Xilinx 提供了 PLL IP 核。本章我们将通过一个简单的例程来向大家介绍一下 PLL IP 核的使用方法。

1.PLL IP 核简介

锁相环作为一种反馈控制电路, 其特点是利用外部输入的参考信号来控制环路内部震荡信号的频率和相位。因为锁相环可以实现输出信号频率对输入信号频率的自动跟踪, 所以锁相环通常用于闭环跟踪电路。锁相环在工作过程中, 当输出信号的频率与输入信号的频率相等时, 输出电压与输入电压保持固定的相位差值, 即输出电压与输入电压的相位被锁住, 这就是锁相环名称的由来。

锁相环拥有强大的性能, 可以对输入到 FPGA 的时钟信号进行任意分频、倍频、相位调整、占空比调整, 从而输出一个期望时钟; 除此之外, 在一些复杂的工程中, 哪怕我们不需要修改任何时钟参数, 也常常会使用 PLL 来优化时钟抖动, 以此得到一个更为稳定的时钟信号。正是因为 PLL 的这些性能都是我们在实际设计中所需要的, 并且是通过编写代码无法实现的, 所以 PLL IP 核才会成为程序设计中最高级 IP 核之一。需要注意的是 Xilinx 中的 PLL 是模拟锁相环, 其优点是输出的稳定度高、锁定时间较短, 相位连续可调; 缺点是在极端环境 (例如极端高/低温环境, 高磁场强度等) 下容易失锁。

Xilinx7 系列器件中的时钟资源包含了时钟管理单元 CMT (全称 Clock Management Tile, 即时钟管理单元), 每个 CMT 由一个 MMCM (全称 Mixed-Mode Clock Manager, 即混合模式时钟管理) 和一个 PLL (全称 Phase Locked Loop, 即锁相环) 组成, xc7z020 芯片内部有 4 个 CMT, 接下来我们讲解一下 MMCM 和 PLL 各自的含义以及两者的区别。

(1) PLL: 为锁相回路或锁相环, 用来统一整合时钟信号, 使高频器件正常工作, 如内存的存取数据等。PLL 用于振荡器中的反馈技术。

(2) MMCM (混合模式时钟管理): 是基于 PLL 的新型混合模式时钟管理器, 实现了最低的抖动和抖动滤波, 为高性能的 FPGA 设计提供更高性能的时钟管理功能。

(3) MMCM 是一个 PLL 上加入 DCM 的一部分以进行精细的相移, 也就是说 MMCM 在 PLL 的基础上加上了相位动态调整功能, 又因为 PLL 是模拟电路, 而动态调相是数字电路, 所以 MMCM 被称为混合模式, MMCM 相对 PLL 的优势就是相位可以动态调整, 但 PLL 占用的面积更小, 而在大部分的设计当中大家使用 MMCM 或者 PLL 来对系统时钟进行分频、倍频和相位偏移都是完全可以的。

在 FPGA 开发过程中, 不需要太了解整个 PLL 的工作原理, 只需要会使用 PLL 即可。FPGA 芯片内部都集成了 PLL, 其他厂商可能不叫 PLL, 但是也有类似的功能模块, 通过 PLL 可以倍频分频, 产生其他很多时钟。本实验通过调用 PLL IP core 来学习 PLL 的使用、vivado 的 IP core 使用方法。对于时钟的使用, 通常可以看看 Xilinx 提供的文档"7 Series FPGAs Clocking Resources User Guide"。

由于采用 IP 核生成 PLL IP 核, 不需要将信号输出至外部管脚, 因此本章实验不需要使用板载外设, 直接使用内部寄存器即可验证是否满足设计要求。

2.实验任务

本节实验的任务是使用开发板输出 4 个不同频率或相位的时钟，四个时钟分别为一个倍频时钟（100MHz），一个倍频后相位偏移 180 度的时钟（100MHz），一个与系统时钟相同的时钟（50MHz）和一个分频时钟（20MHz），并在 Vivado 中进行仿真以验证结果，最后生成比特流文件并下载到开发板上，使用 ILA 来抓取时钟的频率是否正确。

3.程序设计

(1) 创建一个名为“PLL”的空工程，然后点击 Vivado 软件左侧“Flow Navigator”栏中的“IP Catalog”，如下图所示：

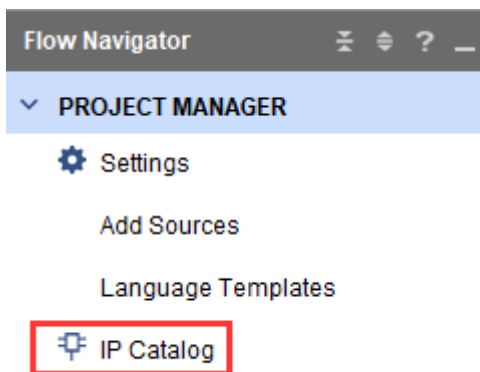


图 6-1 点击“IP Catalog”

(2) 点击“IP Catalog”后弹出的“IP Catalog”窗口后，并输入“clock”关键字，可以看到 Vivado 已经自动查找出了关键字匹配的 IP 核名称，如下图所示：

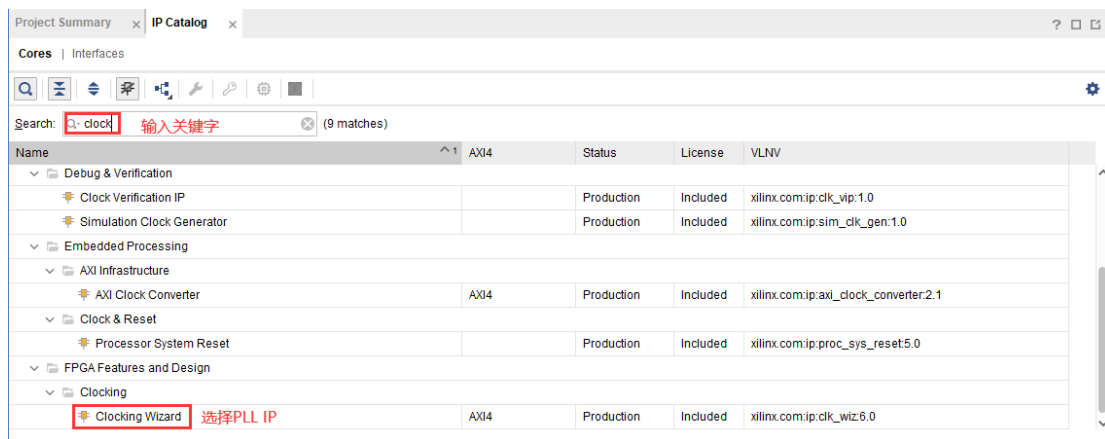


图 6-2 搜索栏中输入关键字

(3) 双击“FPGA Features and Design”→“Clocking”下的“Clocking Wizard”，弹出“Customize IP”窗口，如下图所示：



图 6-3 “Customize IP”窗口

最上面的“Component Name”一栏可以设置该 IP 元件的名称，这里保持默认命名，当然也可以命名为其它方便自己一眼看出其功能的名称。

需要说明的是，这里以 PLL IP 核为例，当大家是第一次使用某个 IP 核时，如果想要了解这个 IP 核的使用和相关参数定义的话，可以点击 Documentation 下的 Product Guide 来打开这个 IP 核的产品指南。因为本章主要讲解 PLL IP 核的使用，很多我们不需要进行修改的参数在文档中可能就一句话带过了，所以如果大家想详细的了解各个配置参数的含义的话，就可以打开这篇产品指南来进行深入学习。

(4) 接下来就是配置 IP 核的时钟参数了，第一个“Clocking Options”选项卡的参数配置如下图所示：

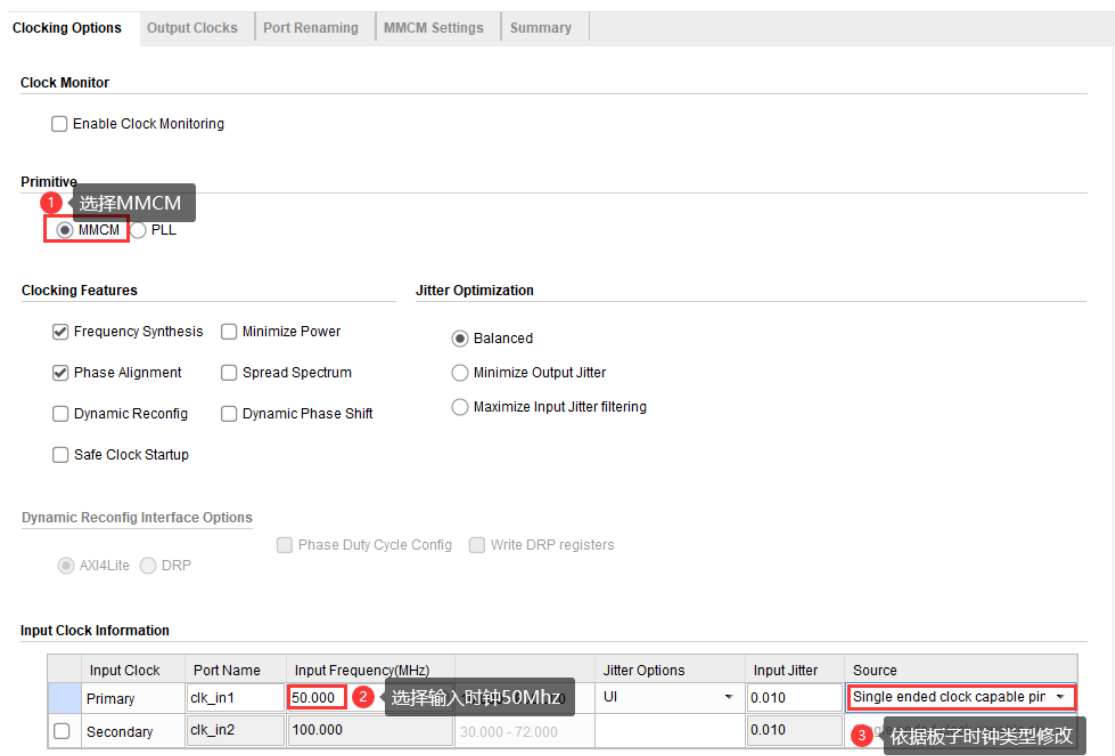


图 6-4 “Clocking Options”选项卡的设置

一般使用 PLL 只需要关注图 6-4 中的几个参数，输入频率依据板卡上的晶振频率进行修改，输入类型也需要依据晶振类型进行修改，如果输入的时钟是差分时钟提供的晶振，则需要将上图中的 3 修改为差分晶振，开发板板载的晶振为单端 50Mhz 的晶振，因此只需按图 6-4 进行配置 PLL 即可。若想了解 PLL IP 的具体功能，建议翻看 Xilinx 的手册。

(5) 接下来切换至“Output Clocks”选项卡，该选项卡中的参数配置如下图所示：

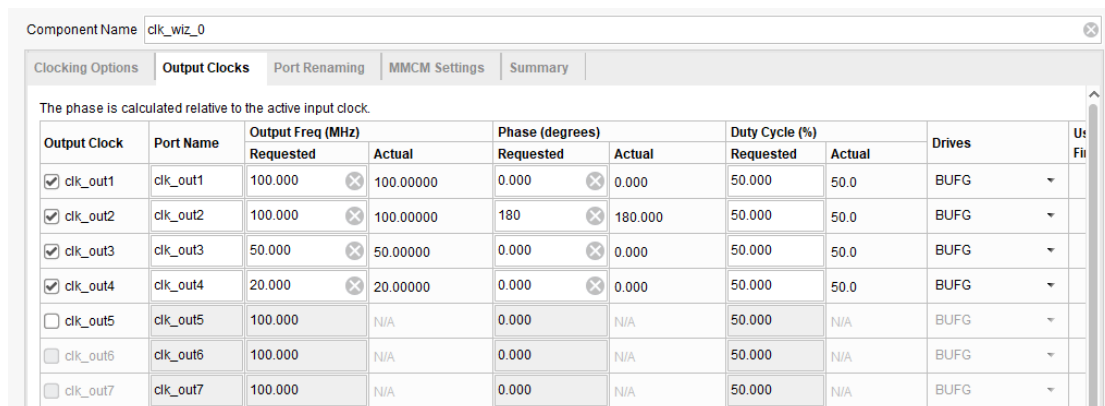


图 6-5 “Output Clocks”选项卡的设置

在“Output Clock”选项卡中，图 6-5 中各个参数的含义如下：

The phase is calculated relative to the active input clock: 用于设置输出时钟的路数（一个 PLL IP 核最多可输出六路不同频率的时钟信号）及参数，其中：

第一列“**OutputClock**”为设置输出时钟的路数，因为我们需要输出四路时钟，所以勾选前 4 个时钟。

第二列“**PortName**”为设置时钟的名字，这里我们可以保持默认的命名。

第三列“**Output Freq(MHz)**”为设置输出时钟的频率，这里我们要对“Requested（即理想值）”进行设置，我们将四路时钟的输出频率分别设为 100、100、50 和 20，设置完理想值后，我们就可以在“Actual”下看到其对应的实际输出频率。需要注意的是 PLLIP 核的时钟输出范围为 6.25MHz~800MHz，但这个范围是一个整体范围，根据驱动器类型的选择不同，其所支持的最大输出频率也会有所差异。

第四列“**Phase (degrees)**”为设置时钟的相位偏移，同样的我们只需要设置理想值，这里我们将第二路 100MHz 的时钟输出信号的相位偏移设置为 180，其余三路信号不做相位偏移处理。

第五列“**Duty cycle**”为占空比，正常情况下如果没有特殊要求的话，占空比一般都是设置为 50%，所以这里我们保持默认的设置即可。

第六列“**Drives**”为驱动器类型，有五种驱动器类型可选：

BUFG 是全局缓冲器，如果时钟信号要走全局时钟网络，必须通过 BUFG 来驱动，BUFG 可以驱动所有的 CLB，RAM，IOB。本次实验我们保持默认选项 BUFG。

BUFH 是区域水平缓冲器，BUFH 可以驱动其水平区域内的所有 CLB，RAM，IOB。

BUFGCE 是带有时钟使能端的全局缓冲器，它有一个输入端 I、一个使能端 CE 和一个输出端 O。只有当 BUFGCE 的使能端 CE 有效(高电平)时，BUFGCE 才有输出。

BUFHCE 是带有时钟使能端的区域水平缓冲器，用法与 BUFGCE 类似。

No buffer 即无缓冲器，当输出时钟无需挂在全局时钟网络上时，可以选择无缓冲区。

第七列“**Max Freqof buffer**”为缓冲器的最大频率，例如我们选取的 BUFG 缓冲器支持的最大输出频率为 628.141MHz。

(6) 其他的选项卡没有特别需要修改的，直接看最后一列“Summary”，在该选项卡中有对该 PLL IP 的总结，如下图所示：

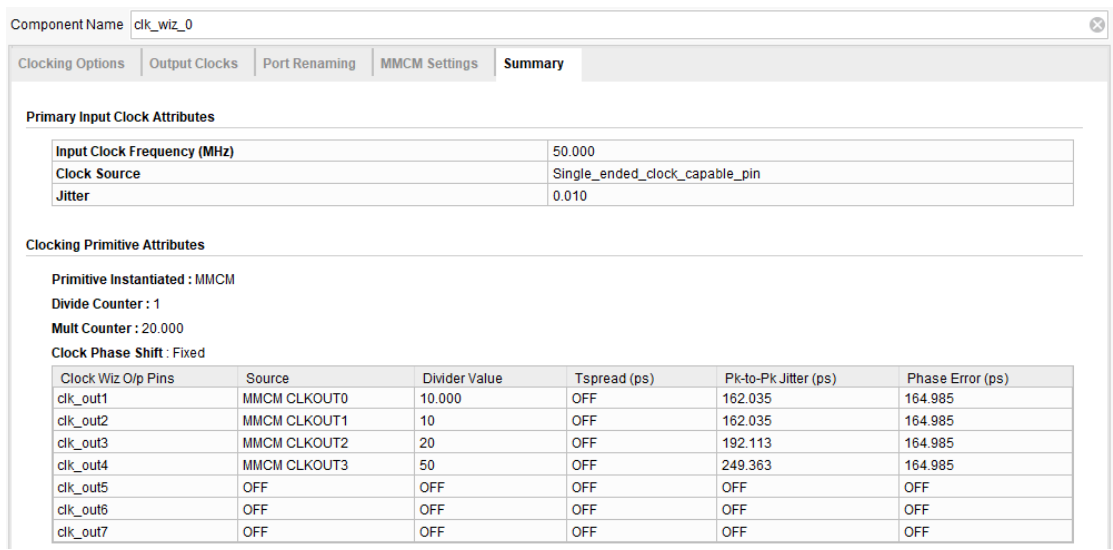


图 6-6 “Summary”选项卡的设置

(7) 若没问题的话，直接点击“OK”，并点击“Generate”生成 IP，如下图所示：

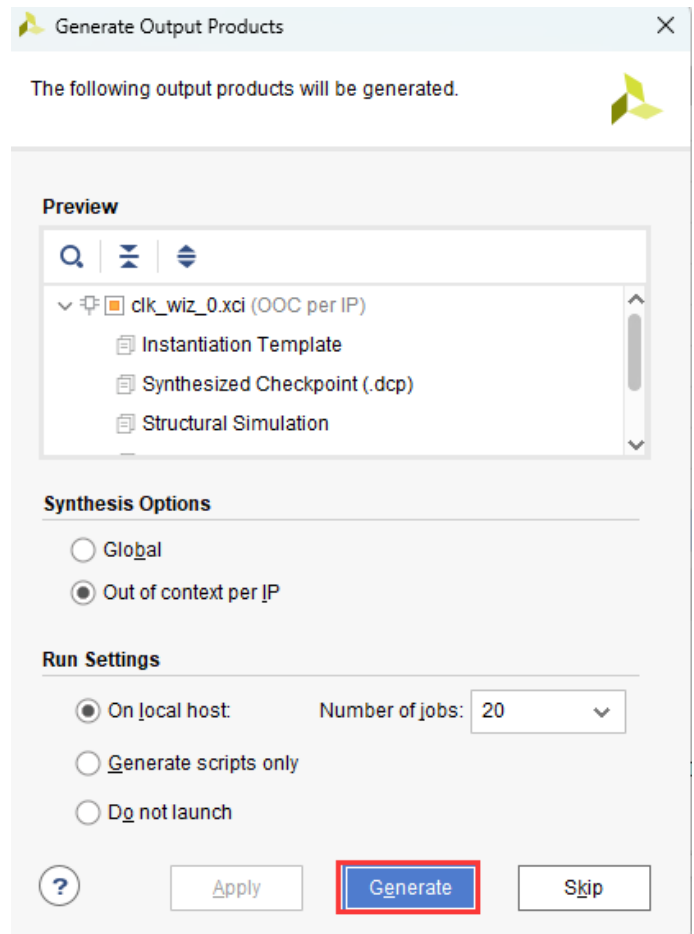


图 6-7 生成 IP

经过等待时间，IP 生成完成，我们就可以开始编写代码来调用我们设置好的 IP 核了。本次实验的目的是通过 PLL IP 核输出四路不同频率或相位的时钟，首先想要输出四路不同频率或相位的时钟，就需要输入一个基准时钟，因此实验需要用到系统时钟。

开发板的系统时钟为 50MHz，即一个时钟周期为 20ns。100MHz 的时钟一个时钟周期

为 10ns；100MHz 相位偏移 180°的时钟就相当于将 100MHz 的时钟的高低电平变化做了一个反相处理；50MHz 的时钟和系统时钟相同；25MHz 的时钟一个时钟周期为 40ns。需要注意的是，PLL IP 核会输出一个时钟锁定信号 (locked)，所有通过 PLL IP 核产生的时钟都是在 locked 为高电平（即时钟已锁定）时才会输出稳定的时钟信号。

这里需要说明一点，在时钟锁定信号 (locked) 拉高之前，时钟信号是不稳定，既可能是高电平，也可能是低电平，还可能是不稳定的时钟信号（用示波器或频谱仪可以看到是飘忽不定的）。

创建工程的过程再第三章中有详细的教程，因此不再详细描述，若有遗忘的步骤的可以翻看第三章创建文件的过程。

(1) 首先，在顶层文件中例化开始产生的 IP 核，代码如下

```
module PLL
(
    input    wire    i_clk    ,
    input    wire    i_rst_n

);

clk_wiz_0 clk_wiz_0_U0
(
    // Clock out ports
    .clk_out1    (),          // output clk_out1
    .clk_out2    (),          // output clk_out2
    .clk_out3    (),          // output clk_out3
    .clk_out4    (),          // output clk_out4

    .locked      (),          // output locked

    .reset       (),
    .clk_in1     ()           // input clk_in1
);
```

(2) 其次则将时钟参数填入例化 IP 中，代码如下

```
wire    w_locked    ; //指示 IP 是否锁定

wire    w_clk_100M      ;
wire    w_clk_100M_180edge;
wire    w_clk_50M       ;
wire    w_clk_20M       ;

clk_wiz_0 clk_wiz_0_U0
(
    .clk_out1    (w_clk_100M          ),// output clk_out1
    .clk_out2    (w_clk_100M_180edge ),// output clk_out2
    .clk_out3    (w_clk_50M           ),// output clk_out3
```

```

    .clk_out4    (w_clk_20M          ),// output clk_out4
    .locked      (w_locked           ), // output Locked
    .reset       ( i_rst_n           ),
    .clk_in1     ( i_clk              ) // input clk_in1
);

```

整个代码逻辑十分简单，只需要将 50Mhz 的系统时钟填入 IP，等待复位结束，即可产生相应时钟信号。

4.PLL 仿真设计及结果

(1) 对于 PLL 时钟的仿真十分的简单，只需要产生一个 50Mhz 的时钟以及复位信号，即可输出时钟，仿真代码如下：

```

module SIM_TB_PLL();

reg          clk,rst;

//初始化时钟与复位信号
initial begin
    clk <= 'd0;
    rst <= 'd0;
    #100
    rst <= 'd1;
end

//产生 50Mhz 的时钟
always begin
    clk <= 'd0;
    #10;
    clk <= 'd1;
    #10;
end

//例化 IP
PLL PLL_U0
(
    .i_clk   (clk),
    .i_rst_n (rst)
);

```

(2) 随后启动仿真，观察输出信号，信号如下图所示：

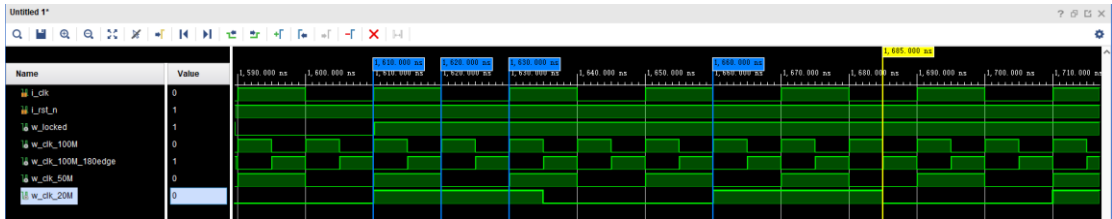


图 6-8 仿真输出结果

从图 6-8 可以看出, PLL 在 1610ns 时锁定, 也就是说在 1610ns 后输出的时钟是稳定的, 那么观测第一组 100Mhz 的时钟信号, 可以看出时钟上升沿在 1610ns, 下一个时钟上升沿在 1620ns, 两者之差为 10ns, 对差值取反计算出频率为 100Mhz, 同理可以计算出其他的信号频率分别为 100Mhz, 50Mhz 以及 20Mhz。

5. 添加内部逻辑分析仪

由于需要满足奈奎斯特采样定律, 即: 采样频率必须大于被采样信号频率的两倍, 驱动内部逻辑分析仪的时钟为 PLL 产生的 100M, 因此只能对 20Mhz 的时钟进行信号抓取, 设置 Debug 如下图所示:

```
module PLL
(
    input    wire    i_clk    ,
    input    wire    i_rst_n
);

(* mark_debug = "true" *)
wire    w_locked    ; // 指示IP是否锁定

wire    w_clk_100M    ;
wire    w_clk_100M_180edge;
wire    w_clk_50M    ;
(* mark_debug = "true" *)
wire    w_clk_20M    ;
```

图 6-9 设置 Debug

Debug 设置完后, 直接进行综合, 并添加 ILA 抓取信号的深度。该步骤与第四章第五章是一致的, 时钟设置如下图所示:

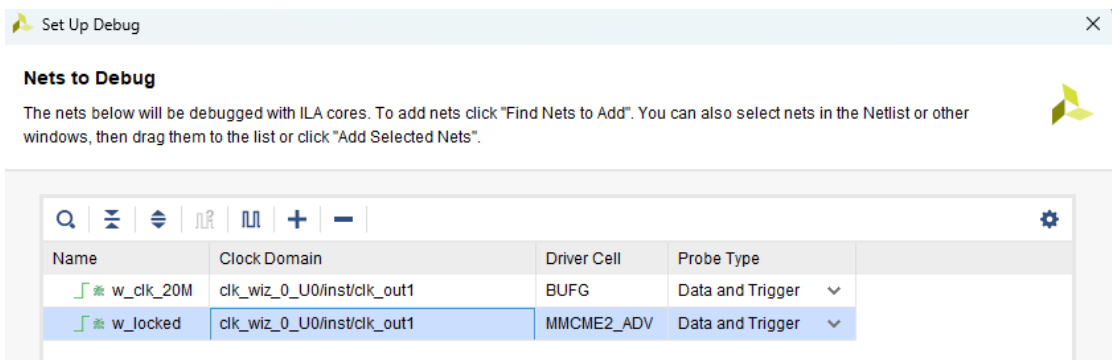


图 6-10 设置 PLL OUT1 为采样时钟

6. 添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按钮就可以添加时钟约束了，如下图所示：



图 6-11 添加时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

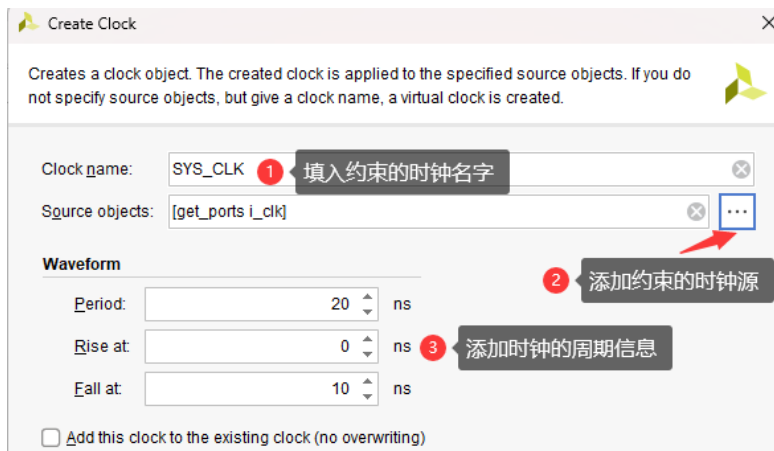


图 6-12 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} -add [get_ports i_clk]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets u_ila_0_clk_out1]
```

图 6-13 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

7. 添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

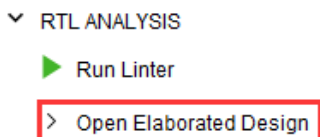


图 6-14 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
All ports (2)													
Scalar ports (2)													
i_clk	IN		W17	☒	33	LVC MOS33*	3.300				NONE	NONE	
i_rst_n	IN		W18	☒	33	LVC MOS33*	3.300				NONE	NONE	

图 6-15 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
set_property IOSTANDARD LVC MOS33 [get_ports i_clk]
set_property IOSTANDARD LVC MOS33 [get_ports i_rst_n]
set_property PACKAGE_PIN W17 [get_ports i_clk]
set_property PACKAGE_PIN W18 [get_ports i_rst_n]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clock [get_nets u_ila_0_clk_out1]
```

图 6-16 约束文件中写入管脚约束

8.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证。Bit 文件生成后，可以看下时序是否有错误，可以发现此次设计时序没有错误，如下图所示：

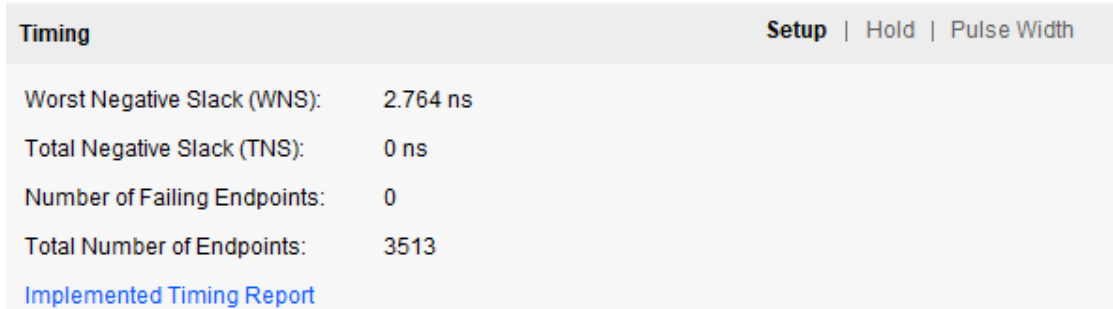


图 6-17 时序结果

将 Bit 下载至 FPGA，随后打开 ILA，随后直接触发，触发结果如下图所示：

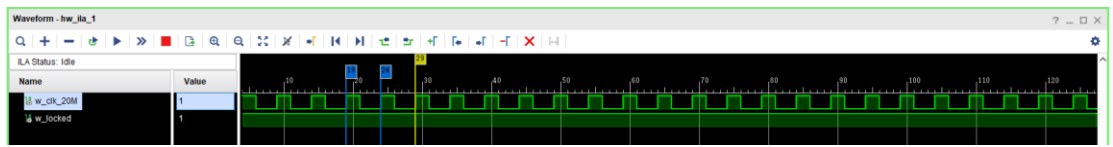


图 6-18 触发波形结果

从图 6-18 可以看出，两个脉冲之间的间隔时间为 5，当前采样时钟为 100Mhz，换算时间为 10ns，那么 $5 \times 10\text{ns} = 50\text{ns}$ ，转换为频率则为 20Mhz，从结果上来看是符合设计要求的。由于笔者手中没有示波器，只能采用 ILA 的方式验证 20Mhz 的输出，如果要将该信号通过示波器验证，可将该信号绑定在 40Pin 拓展口中的任意管脚，通过示波器测量该信号频率。

9.实验总结

本章节我主要讲解了 Vivado 软件中 PLL IP 核的调用、配置和使用方法，读者要牢记于心，这对后面章节的学习至关重要，能大大提高大家的开发效率。

PLL IP 核在未来的工程项目中会频繁运用，可以在多多熟悉该 IP 核的使用，也可以将本章实验中没有勾选的选项进行勾选后看看会有什么不同的输出效果。

七. IP 核之单端口 RAM 实验

1.RAM 简介

在了解 RAM IP 核之前,存储器的大致分为:随机存储器(RAM)以及只读存储器(ROM),随机存储器包括静态 RAM 和动态 RAM。静态 RAM 只要有供电,它保存的数据就不会丢失;而动态 RAM 在供电的情况下,还需要根据其要求的时间来对存储的数据进行刷新,才能保持存储的数据不会丢失。FPGA 内部中的 RAM 是以静态存储器的形式存在,一般包括单端口 RAM(Single Port RAM,缩写为 SPRAM)、简单双端口 RAM(Simple Dual Port RAM,缩写为 SDP RAM,也叫伪双端口 RAM)和真双端口 RAM(True Dual Port RAM,缩写为 TDPRAM)。静态 RAM 的特点是存储容量相对不是很大,但是读写速度非常高,其在 FPGA 或者 ASIC 设计中都属于非常重要的器件,可以说查找表、寄存器、组合逻辑和静态 RAM 构成了整个数字电路体系,足见静态 RAM 的重要性,本次实验使用的 RAM 属于静态 RAM,重点看下几种静态 RAM 的特性与区别:

表 7-1 静态 RAM 的区别

RAM 类型	特性说明	读写口数量	地址口数量
单端口 RAM	1、允许通过一个端口对存储进行读写访问。 2、只有一组数据总线、地址总线、时钟信号以及读写使能信号。	1	1
伪双端口 RAM	1、提供A和B两个端口,端口A只能进行写访问,端口B只能进行读访问。 2、有两组数据总线、地址总线、时钟信号以及读写使能信号。	1	2
真双端口RAM	1、提供A和B两个端口,两个端口均可对存储进行读写访问。 2、有两组数据总线、地址总线、时钟信号以及读写使能信号。	2	2

从上表可以看出,单端口 RAM 只有一个端口进行读写,即读/写只能通过这一个端口来进行。对于伪双端口 RAM 而言,其也有两个端口可以用于读写,但是其中一个端口只能读不能写,另一个端口只能写不能读;对于真双端口 RAM 而言,其有两个端口可以用于读写,且两个端口都可以进行读或写;

在 FPGA 设计中 RAM 主要用于存放程序和数据,在 RAM 的实际使用过程中,一般将 RAM 作为关键数据的临时存储。接下来就简单介绍一下 Vivado 中的 RAM IP 核。

Vivado 软件自带的 Block Memory GeneratorIP 核(缩写为 BMG,中文名为块 RAM 生成器),可以用来配置生成 RAM 或者 ROM。RAM 是一种随机存取存储器,不仅可以读出存储的数据,同时还支持对存储的数据进行修改,而 ROM 是一种只读存储器,也就是说,在使用 ROM 之前需要将数据提前写入 ROM 中,在工作时只能读出数据,而不能写入数据。需要注意的是,配置生成的 RAM 或者 ROM 使用的都是 FPGA 内部的 BRAM 资源(Block RAM,即块随机存储器,是 FPGA 厂商在逻辑资源之外,给 FPGA 加入的专用 RAM 块资源),只不过配置成 ROM 时只用到了嵌入式 BRAM 的读数据端口。本章将带领大家如何将

BMG IP 核配置成 RAM。

2.硬件设计

本章的实验主要是通过 FPGA 内部资源进行代码编写与驱动，不涉及外设的驱动，只需要用到 FPGA 的时钟以及按键复位信号，并没有新的外设硬件设计。

3.实验任务

本章实验任务是将 Xilinx BMG IP 核配置成一个单端口的 RAM 并对其进行读写操作，然后通过仿真观察波形是否正确，最后将设计下载到 FPGA 开发板中，并通过在线调试工具对实验结果进行验证。

4.程序设计

4.1 新建单端口 RAM IP

(1) 建一个名为“Single_Port_RAM”的空工程，然后点击 Vivado 软件左侧“Flow Navigator”栏中的“IP Catalog”，如下图所示：

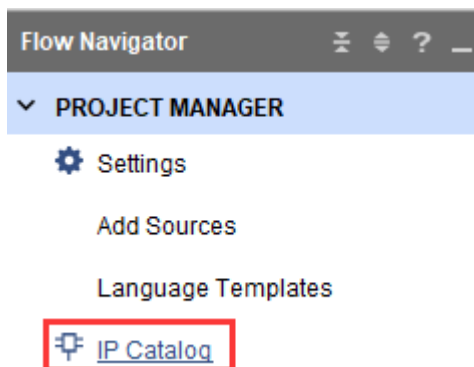


图 7-1 点击“IP Catalog”

(2) 在“IP Catalog”窗口的搜索栏中输入“Block Memory”关键字后，出现唯一匹配的“Block Memory Generator”，如下图所示（图中出现的两个 IP 核为同一个 BMG IP 核）：

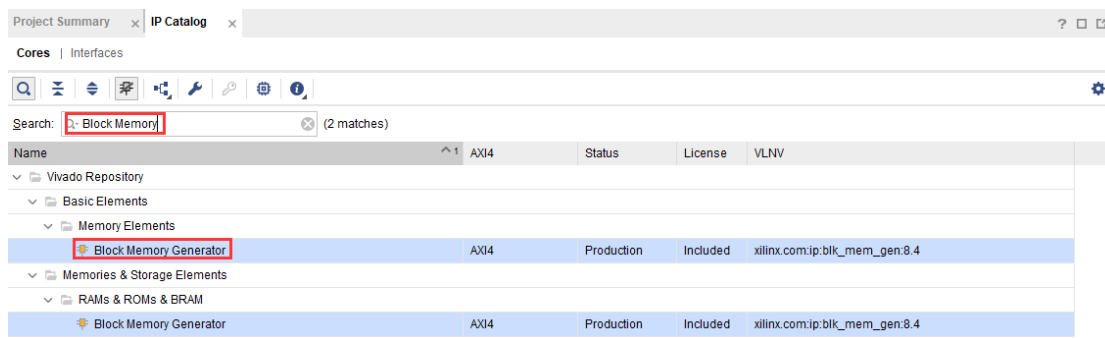


图 7-2 搜索栏中输入“Block Memory”关键字

(3) 双击“Block Memory Generator”后弹出 IP 核的配置界面，接着我们就可以对 BMG IP 核进行配置了，“Basic”选项卡配置界面如下图所示。

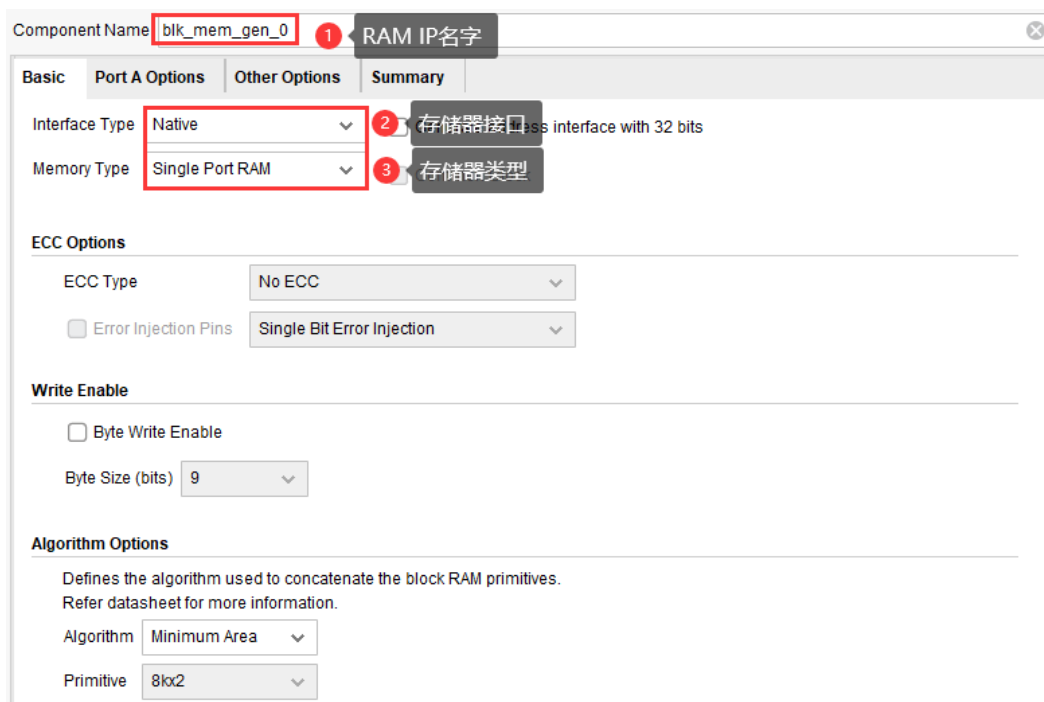


图 7-3 “Basic”选项卡配置界面

最上面的“Component Name”一栏可以设置该 IP 元件的名称，这里保持默认命名，当然也可以命名为其它方便自己一眼看出其功能的名称。

接着目光回到“Basic”选项卡上，上图中标 2, 3 的接口是最常用的接口，其含义如下：

a.“Interface Type（接口模式）”：有两种接口模式可选，分别为 Native（常规）接口和 AXI4 接口。AXI4 模式一般是在处理器中的数据需要和 BRAM 交互（如 ZYNQ 中的 PS 和 PL 交互）时才会使用，当不需要与处理器数据进行交互时，PL 端一般采用 Native 模式，所以本次实验我们选择 Native 模式。

b.“Memory Type（存储类型）”：有五种类型可选，分别为“Single Port RAM（单端口 RAM）”、“Simple Dual Port RAM（伪双端口 RAM）”、“True Dual Port RAM（真双端口 RAM）”、“Single Port ROM（单端口 ROM）”和“Dual Port ROM（双端口 ROM）”，因为本章的试验任务是学习单端口 RAM，所以我们选择“Single Port RAM”

对于其他的选项如果有兴趣，可以通过 Xilinx 提供的 IP 手册，查看其用法。

(4) 接下来对“Port A Options”选项卡进行配置，如下图所示：

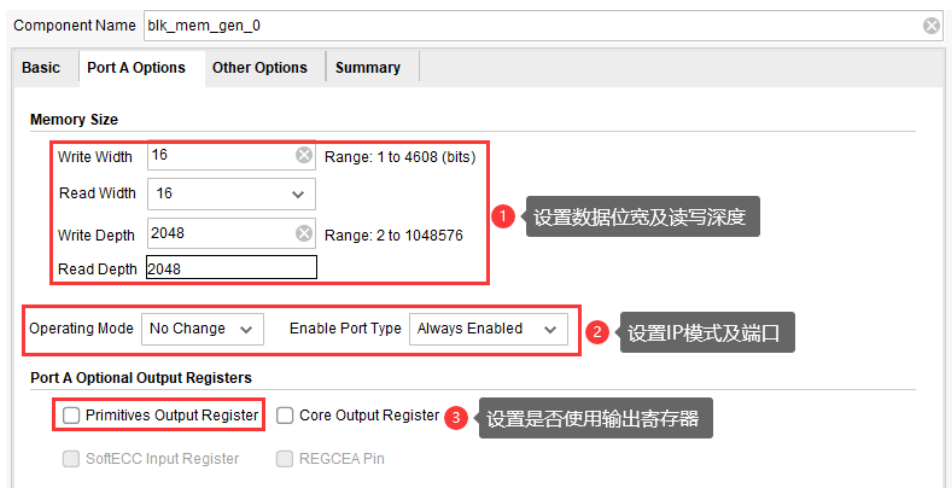


图 7-4 “Port A Options”选项卡配置界面

“Port A Options”选项卡中，重点关注图 7-4 中标 1 的内容，该内容主要用于指定端口读写的宽度与深度，读写的位宽，单位是 bit，本次实验设置成 16 位，可设置的位宽范围为：1~4608；读数据的位宽与写数据的位宽存在倍数关系，通常可以在选项卡中能看到，这次实验采用同时钟频率，因此读位宽也为 16 bit。

写深度，这里设置成 2048，即 RAM 所能访问的地址范围为 0~2047。这里有一点需要大家注意，那就是写深度和写位宽的乘积不要超过器件本身的 ram 资源的大小，否则软件会报错，由于读位宽一致，因此读深度与写深度一致，都为 2048。

Operating Mode: RAM 运作模式。该选项决定了我们在进行写操作期间，DOUT（读数据线）上的数据变化，有三种模式可选，分别为：

- Write First（写优先模式）：当对某个地址进行写操作时，则会将写入的数据传递到读数据线上。

- Read First（读优先模式）：当对某个地址进行写操作时，则会将上次写入该地址的数据递到读数据线上。

- No Change（不变模式）：在该模式下，进行写操作时，读数据线上的数据保持不变。这里选择 No Change 模式。

Enable Port Type: 使能端口类型。有两种可选，分别为 Use ENA pin（添加使能端口 A 信号）和 Always Enabled（取消使能信号，端口 A 一直处于使能状态），这里选择 Always Enabled。

(5) 对于“Other Options”选项卡，用于设置 RAM 的初始值等信息，本次实验不需要设置。对其中较为重要的接口进行介绍。

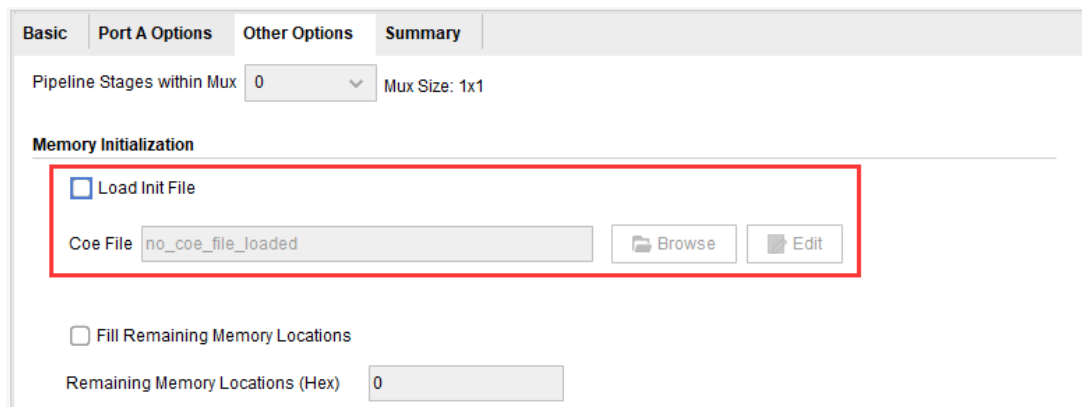


图 7-5 “Other Options”选项卡配置界面

图 7-5 中“Memory Initialization (初始化文件)”，选择是否使用本地初始化文件 (.coe 文件) 来对存储空间进行初始化，这个初始化文件是可以利用 MATLAB 或者其他上位机小程序，生成 coe 文件初始化 IP 设置，通过该初始化文件进行配置 IP，能够直接读出 IP 中的数据，该端口在 ROM IP 中会进行详细介绍。

(6) 设置完成后，点击 OK，并在弹出界面中点击“Generate”即可生成 IP。

4.2 RAM 控制程序设计

本次实验的目的是将 Xilinx BMG IP 核配置成一个单端口的 RAM 并对其进行读写操作。由于单端口 RAM 的读写是共用一组信号总线的，所以这里将读写操作需要通过计数器将状态进行切换，同时 RAM 的数据并不需要输出到 IO 引脚上，所以本实验不需要输出端口。

(1) 因为本次实验除了调用 BMG IP 核外还需要例化一个读写模块，所以我们需要创建一个顶层模块来例化 IP 核与读写模块。首先定义一个计数器，用于判断 RAM 中写入的数据是否写满。每次写满的时候将计数器清 0。

```
//RAM IP 一直拉高 en 信号
//写满整个 RAM，再去读，
always @(posedge i_clk or posedge i_rst_n) begin
    if (~i_rst_n)
        r_wr_en_cnt <= 'd0;
    else if (r_wr_en_cnt == 'd2047)
        r_wr_en_cnt <= 'd0;
    else
        r_wr_en_cnt <= r_wr_en_cnt + 'd1;
end
```

(2) 每当 RAM 读写完 2048 个数据后，将读写状态进行切换。

```
//计数器从 0 开始计数值 2047 为写，拉高；下一个 2048 周期拉低为读
always @(posedge i_clk or posedge i_rst_n) begin
    if (~i_rst_n)
        r_we <= 'd0;
    else if (r_wr_en_cnt == 'd2047)
        r_we <= ~r_we;
    else
        r_we <= r_we;
end
```

(3) 随后对 RAM 的读写地址进行控制，由于单口 RAM 的读写地址是共用的，所以是通过计数器的状态进行控制，每次计数器计满的同时会对地址进行重新累加，读写状态进行切换。

```
//控制读写地址
always @(posedge i_clk or posedge i_rst_n) begin
    if (~i_rst_n)
        r_addr <= 'd0;
    else if (r_wr_en_cnt == 'd2047)
        r_addr <= 'd0;
```

```

    else
        r_addr <= r_addr + 'd1;
end

```

(4) 控制写入 RAM 的数据, 只有再写入有效的同时自增, 为了对上述计数器进行区分, 每次自增都增加 2。

```

//控制写入的数据
always @(posedge i_clk or posedge i_rst_n) begin
    if (~i_rst_n)
        r_data_i <= 'd0;
    else if (r_wr_en_cnt == 'd2047)
        r_data_i <= 'd0;
    else if (r_we == 'd1)
        r_data_i <= r_data_i + 'd2;
    else
        r_data_i <= r_data_i;
end

```

(5) 最后则是例化 IP

```

blk_mem_gen_0 blk_mem_gen_0_u0
(
    .clka      (i_clk      ),    // input wire clka
    .wea       (r_we       ),    // input wire [0 : 0] wea
    .addra     (r_addr     ),    // input wire [10 : 0] addra
    .dina      (r_data_i   ),    // input wire [15 : 0] dina
    .douta     (w_data_o   )     // output wire [15 : 0] douta
);

```

5.RAM 仿真设计及结果

(1) 对于 RAM 的仿真十分的简单, 只需要产生一个 50Mhz 的时钟以及复位信号, 即可输出时钟, 仿真代码如下:

```

module TB_SIM_RAM();

reg clk,rst_n;

initial begin
    clk      <= 'd0;
    rst_n    <= 'd0;
    #100;
    rst_n    <= 'd1;
end

```

```

always begin
    clk <= 'd0;
    #10;
    clk <= 'd1;
    #10;
end

RAM_RW_TOP RAM_RW_TOP_U0
(
    .i_clk      (clk),
    .i_rst_n    (rst_n)
);

endmodule

```

(2) 随后启动仿真，观察输出信号，信号如下图所示：

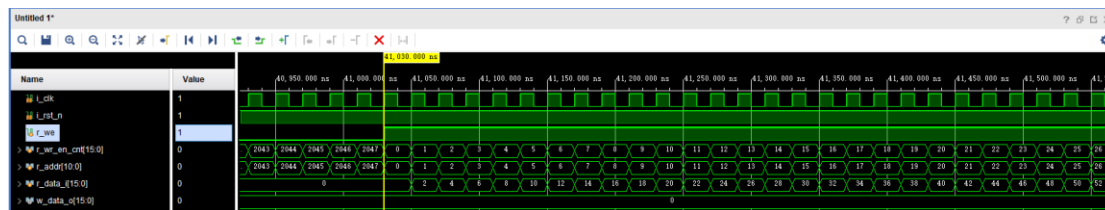


图 7-6 RAM 写数据

从图 7-6 中可以看出当 we 信号拉高时，地址信号在自增，与此同时写入的数据以 2 的步进进行自增。

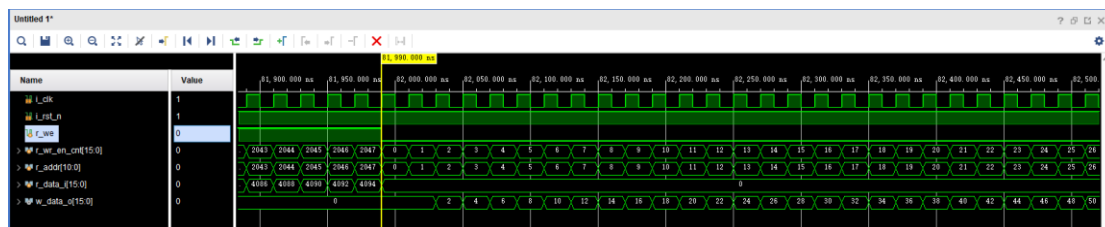


图 7-7 RAM 读数据

从图 7-7 中可以看出当 we 信号拉低时，地址信号在自增，与此同时读出的数据以 2 的步进进行自增。

整体的读写结果如下图所示：

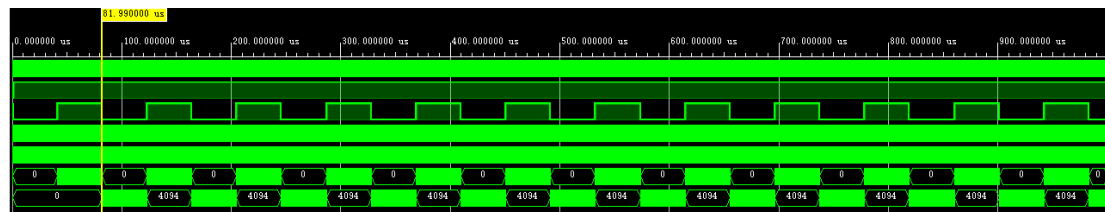


图 7-8 RAM 读写数据图

从图 7-8 中可以 RAM 读写是循环进行的，基本符合前文设置的实验任务。

6. 添加内部逻辑分析仪

(1) 在单口 RAM 的实验中，可以抓取读写数据的计数器、RAM 控制信号及 RAM 数据信号，添加信号标识如下图所示：

```
(* mark_debug = "true" *)
reg      [15:0]  r_wr_en_cnt ;    //读写数据计数器
(* mark_debug = "true" *)
reg      r_we      ;    //控制RAM读还是写
(* mark_debug = "true" *)
reg      [10:0]  r_addr      ;    //控制RAM读写地址
(* mark_debug = "true" *)
reg      [15:0]  r_data_i    ;    //控制RAM写的数据
(* mark_debug = "true" *)
wire      [15:0]  w_data_o    ;    //RAM读出的数据
```

图 7-9 添加 Debug 信号

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。

Set Up Debug

ILA Core Options
Choose features for the ILA debug cores.

Sample of data depth: 4096 写入数据深度为2048，设置触发深度为4096，能一次抓完RAM数据

Input pipe stages: 0

Trigger and Storage Settings

☐ Capture control

☐ Advanced trigger

? < Back Next > Finish Cancel

图 7-10 设置 Debug 信号的深度

(3) 查看是否写入约束文件

```

1 create_debug_core u_ila_0 ila
2 set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
3 set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
4 set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
5 set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
6 set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
7 set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
8 set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
9 set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
10 set_property port_width 1 [get_debug_ports u_ila_0/clk]
11 connect_debug_port u_ila_0/clk [get_nets [list i_clk_IBUF_BUF]]
12 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
13 set_property port_width 16 [get_debug_ports u_ila_0/probe0]
14 connect_debug_port u_ila_0/probe0 [get_nets [list {r_data_i[0]} {r_data_i[1]} {r_data_i[2]} {r_data_i[3]} {r_data_i[4]} {r_data_i[5]} {r_data_i[6]} {r_data_i[7]} {r_data_i[8]} {r_data_i[9]} {r_data_i[10]} {r_data_i[11]} {r_data_i[12]} {r_data_i[13]} {r_data_i[14]} {r_data_i[15]}]]
15 create_debug_port u_ila_0 probe0
16 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
17 set_property port_width 11 [get_debug_ports u_ila_0/probe1]
18 connect_debug_port u_ila_0/probe1 [get_nets [list {r_addr[0]} {r_addr[1]} {r_addr[2]} {r_addr[3]} {r_addr[4]} {r_addr[5]} {r_addr[6]} {r_addr[7]} {r_addr[8]} {r_addr[9]} {r_addr[10]}]]
19 create_debug_port u_ila_0 probe1
20 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
21 set_property port_width 16 [get_debug_ports u_ila_0/probe2]
22 connect_debug_port u_ila_0/probe2 [get_nets [list {r_wr_en_cnt[0]} {r_wr_en_cnt[1]} {r_wr_en_cnt[2]} {r_wr_en_cnt[3]} {r_wr_en_cnt[4]} {r_wr_en_cnt[5]} {r_wr_en_cnt[6]} {r_wr_en_cnt[7]} {r_wr_en_cnt[8]} {r_wr_en_cnt[9]} {r_wr_en_cnt[10]} {r_wr_en_cnt[11]} {r_wr_en_cnt[12]} {r_wr_en_cnt[13]} {r_wr_en_cnt[14]} {r_wr_en_cnt[15]}]]
23 create_debug_port u_ila_0 probe2
24 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe3]
25 set_property port_width 16 [get_debug_ports u_ila_0/probe3]
26 connect_debug_port u_ila_0/probe3 [get_nets [list {w_data_o[0]} {w_data_o[1]} {w_data_o[2]} {w_data_o[3]} {w_data_o[4]} {w_data_o[5]} {w_data_o[6]} {w_data_o[7]} {w_data_o[8]} {w_data_o[9]} {w_data_o[10]} {w_data_o[11]} {w_data_o[12]} {w_data_o[13]} {w_data_o[14]} {w_data_o[15]}]]
27 create_debug_port u_ila_0 probe3
28 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe4]
29 set_property port_width 1 [get_debug_ports u_ila_0/probe4]
30 connect_debug_port u_ila_0/probe4 [get_nets [list r_we]]
31 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
32 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
33 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
34 connect_debug_port dbg_hub/clk [get_nets i_clk_IBUF_BUF]

```

Debug 信号添加完毕，随后可以进行时序约束

Debug 信号添加完毕，随后可以进行时序约束

7.添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按键就可以添加时钟约束了，如下图所示：

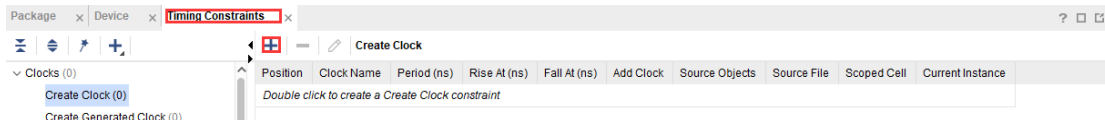


图 7-12 添加时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

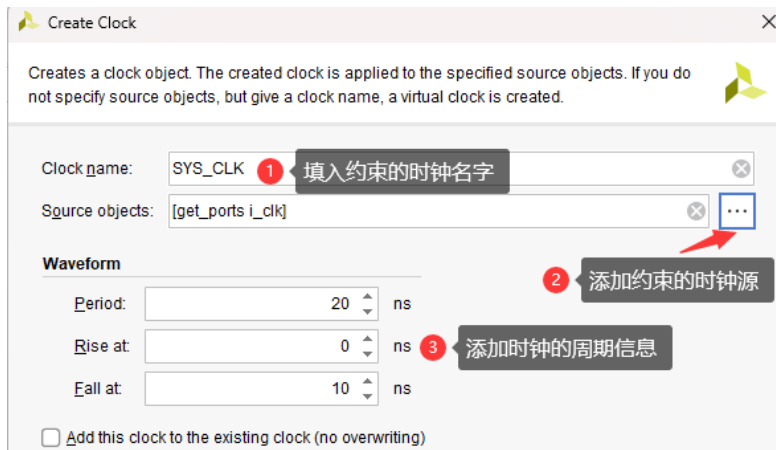


图 7-13 约束主时钟

(3) 保存后, 查看约束文件是否写入时钟约束, 如下图所示:

```
create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} [get_ports i_clk]
```

图 7-14 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

8. 添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

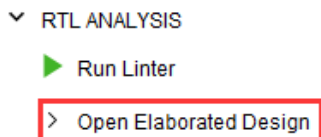


图 7-15 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
BRAM_PORTA_25633 (1)	IN			☒	33	LVCMO533*	3.300				NONE	NONE	
Scalar ports (1)	IN		W17	☒	33	LVCMO533*	3.300				NONE	NONE	
Scalar ports (1)	IN		W18	☒	33	LVCMO533*	3.300				NONE	NONE	

图 7-16 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
set_property IOSTANDARD LVCMOS33 [get_ports i_clk]
set_property IOSTANDARD LVCMOS33 [get_ports i_rst_n]
set_property PACKAGE_PIN W17 [get_ports i_clk]
set_property PACKAGE_PIN W18 [get_ports i_rst_n]
```

图 7-17 约束文件中写入管脚约束

9. 上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证。Bit 文件生成后，可以看下时序是否有错误，可以发现此次设计时序没有错误，如下图所示：

Timing	Setup Hold Pulse Width
Worst Negative Slack (WNS):	13.615 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	4510
Implemented Timing Report	

图 7-18 时序结果

将 Bit 下载至 FPGA，随后打开 ILA，随后设置 r_we 上升沿触发，观测 RAM 输入写数据，触发结果如下图所示：

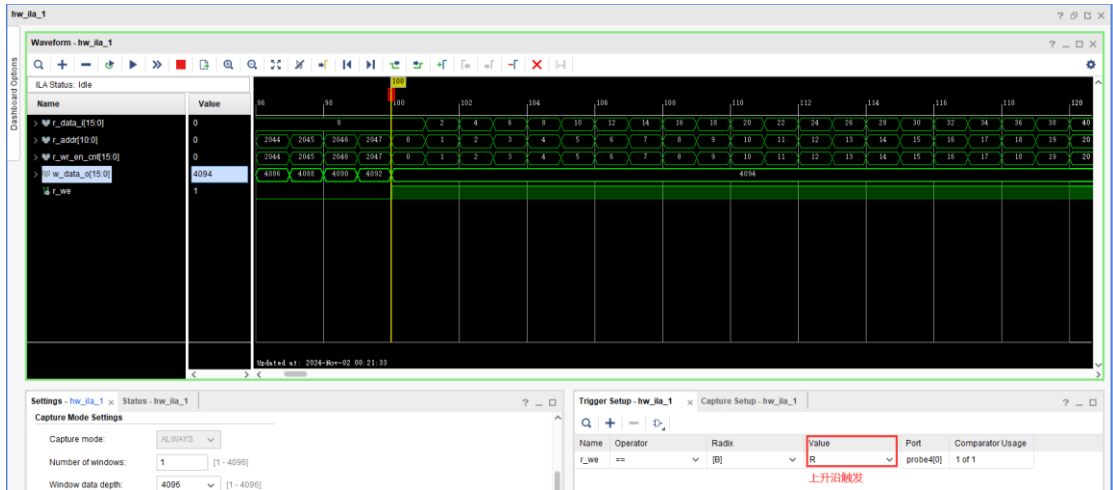


图 7-19 读写信号上升沿触发波形

从图 7-19 可以看出，当 r_we 信号为高，RAM 地址在自增同时也在写入数据至 RAM 中，RAM 并没有数据读出。

设置 r_we 下降沿触发，观测 RAM 读数据，触发结果如下图所示：

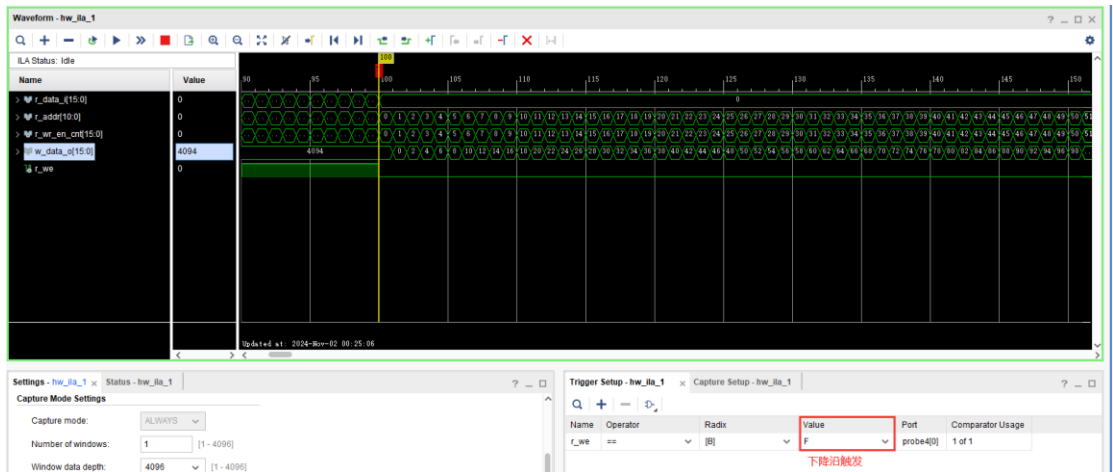


图 7-20 读写信号下降沿触发波形

从图 7-20 可以看出，当 r_we 信号为低，RAM 地址在自增的同时也从 RAM 中读出数据，RAM 并没有写入数据。

10.实验总结

本章节重点讲解了 Vivado 软件中如何将 BGM IP 核配置成单端口 RAM 的方法，并调用其进行读写操作，读者要牢记于心，这对后面章节的学习至关重要，能大大提高大家的开发效率，同时也更熟悉对 vivado 开发流程。

八. IP 核之双端口 RAM 实验

关于 RAM IP 的介绍在第 7 章中进行过详细介绍，因此关于 RAM IP 本章不浪费篇章进行重复的介绍，采用双端口的 RAM 可以将读写分离，以方便后续的 FPGA 代码开发，接下来则对双端口 RAM 进行 IP 创建以及代码编写。

1. 实验任务

本章实验任务是将 Xilinx BMG IP 核配置成一个简单双端口的 RAM，以 50M 时钟对端口 A 写入 16 bit 位宽，深度为 2048 的数据，而端口 B 则以 75M 时钟读出 8 bit 位宽，深度为 4096 的数据，由于读写时钟不一致需要借助状态机进行控制，在这一章节中会对状态机进行简单的介绍，随后通过仿真观察波形是否正确，最后将设计下载到 FPGA 开发板中，并在线调试工具对实验结果进行验证。

2. 状态机简介

FPGA 不同于 CPU 的一点特点就是 CPU 是顺序执行的，而 FPGA 是同步执行（并行）的。那么 FPGA 如何处理明显具有时间上先后顺序的事件呢？这个时候我们就需要使用到有限状态机了。

状态机简称为 FSM (Finite State Machine)，也称为同步有限状态机，一般简称为状态机，之所以说“同步”是因为状态机中所有的状态跳转都是在时钟的作用下进行的，而“有限”则是说状态的个数是有限的。

状态机的每一个状态代表一个事件，从执行当前事件到执行另一事件称之为状态的跳转或状态的转移，需要做的就是执行该事件然后跳转到一下时间，这样整个系统就“活”了，可以正常的运转起来了。状态机通过控制各个状态的跳转来控制流程，使得整个代码看上去更加清晰易懂，在控制复杂流程的时候，状态机优势明显。

状态机可以分为一段式、二段式和三段式状态机：

a.一段式状态机：整个状态机写到一个 always 模块里面，在该模块中既描述状态转移，又描述状态的输入和输出。

b.二段式状态机：用两个 always 模块来描述状态机，其中一个 always 模块采用同步时序描述状态转移；另一个模块采用组合逻辑判断状态转移条件，描述状态转移规律以及输出。不同于一段式状态机的是，它需要定义两个状态，现态和次态，然后通过现态和次态的转换来实现时序逻辑。

c.三段式状态机：在两个 always 模块描述方法的基础上，使用三个 always 模块，一个 always 模块采用同步时序描述状态转移，一个 always 采用组合逻辑判断状态转移条件，描述状态转移规律，另一个 always 模块描述状态输出。

上述只是对状态机的概念进行一个简单介绍，在接下来代码的设计中会对状态机的设计思想进行介绍，用状态机去控制双端口在不同时钟下的 RAM 读写控制。

3.程序设计

3.1 新建双端口 RAM IP

(1) 建一个名为“Single_Port_RAM”的空工程, 然后点击 Vivado 软件左侧“Flow Navigator”栏中的“IP Catalog”, 如下图所示:

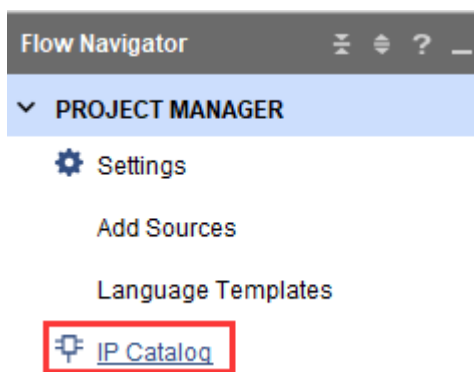


图 8-1 点击“IP Catalog”

(2) 在“IP Catalog”窗口的搜索栏中输入“Block Memory”关键字后, 出现唯一匹配的“Block Memory Generator”, 如下图所示 (图中出现的两个 IP 核为同一个 BMG IP 核):

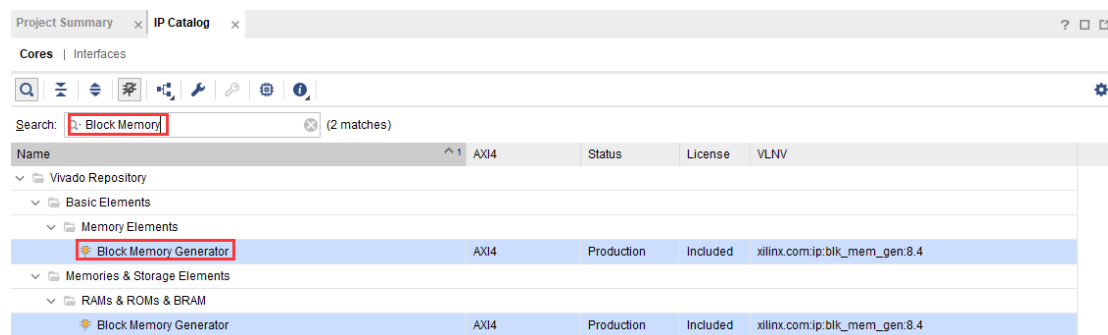


图 8-2 搜索栏中输入“Block Memory”关键字

(3) 双击“Block Memory Generator”后弹出 IP 核的配置界面, 接着我们就可以对 BMG IP 核进行配置了, “Basic”选项卡配置界面如下图所示。

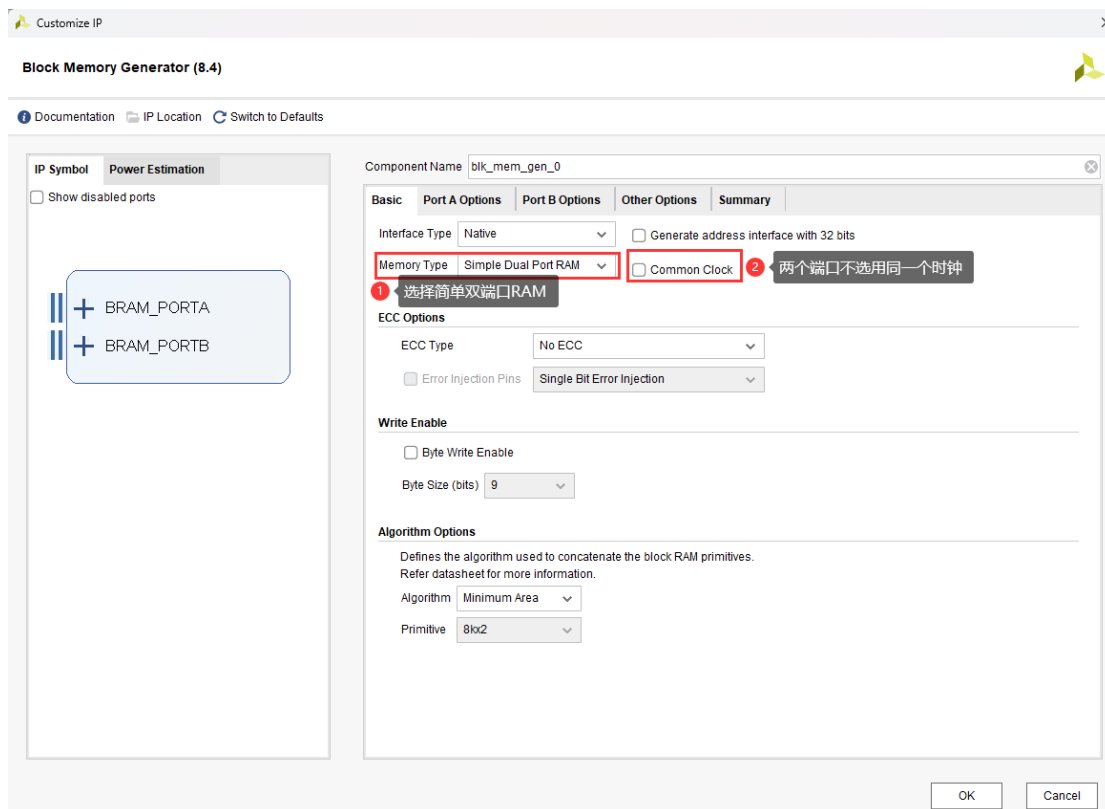


图 8-3 “Basic”选项卡配置界面

从图 8-2 中可以看出，此次配置的 RAM 类型为简单的双端口 RAM，同时没有勾选 Common Clock，没有勾选 Common Clock 的是可以采用不同的时钟进行读写，这样可以将第 6 章中添加的 PLL 教程与 RAM 进行联合使用

(4) 接下来对“Port A Options”选项卡进行配置，如下图所示：

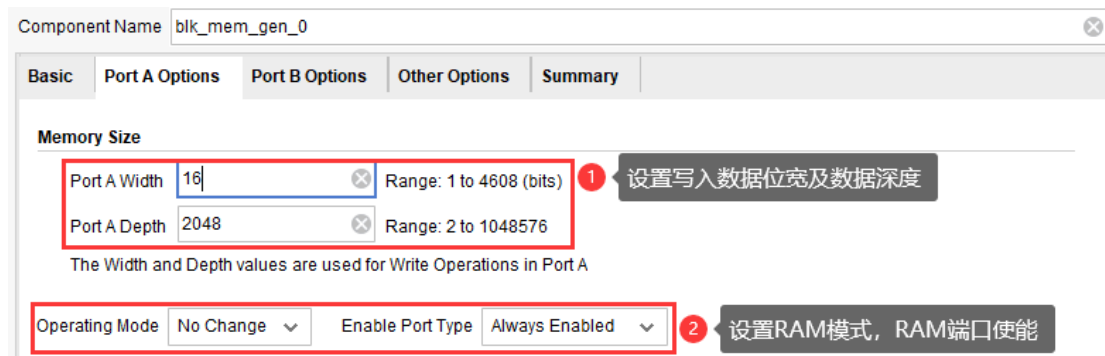


图 8-4 “Port A Options”选项卡配置界面

“Port A Options”选项卡中，重点关注图 7-4 中标 1 的内容，该内容主要用于指定端口读写的宽度与深度，读写的位宽，单位是 bit，本次实验设置成 16 位，可设置的位宽范围为：1~4608；

(5) 接下来对“Port B Options”选项卡进行配置，如下图所示：

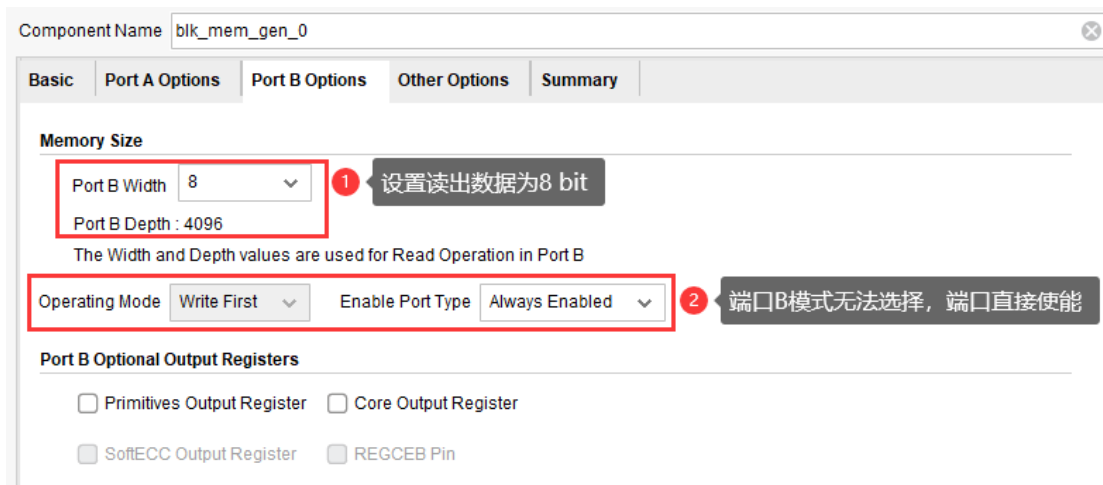


图 8-5 “Port B Options”选项卡配置界面

读数据的位宽与写数据的位宽存在倍数关系，通常可以在“Port B Options”选项卡中能看到，这次实验采用不同的时钟进行读写，因此读端口的位宽设置为 8 bit，而端口 A 的写入深度为 2048，由于 Port B 的位宽设置为 8 bit，端口 B 的深度会依据位宽的设置自动进行匹配，如图 5-8 中端口设置为 8 bit，深度则为 4096。

操作模式有一点需要注意，因为此次设计的 RAM 为简单双端口 RAM，该 RAM 类型为 Port A 写数据，Port B 读数据，因此在图 8-5 中可以发现无法设置端口操作模式。

(6) 对于“Other Options”选项卡，用于设置 RAM 的初始值等信息，本次实验不需要设置。对其中较为重要的接口进行介绍。

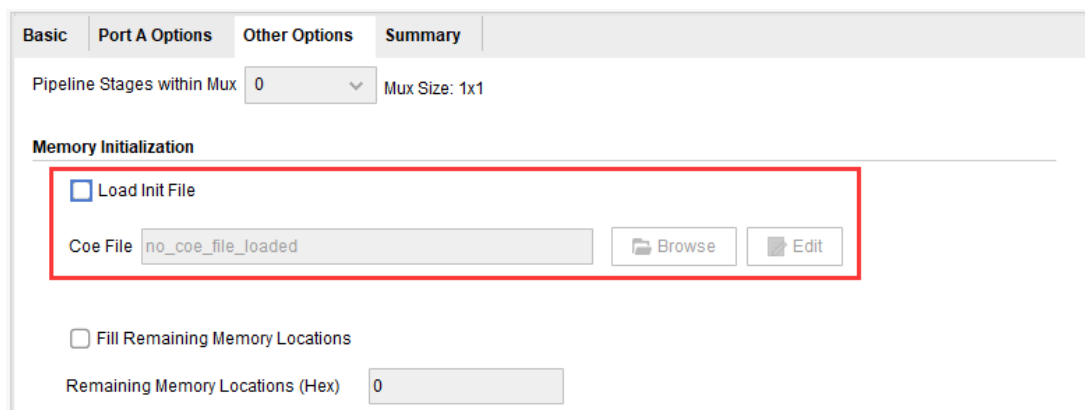


图 8-6 “Other Options”选项卡配置界面

图 8-6 中“Memory Initialization（初始化文件）”，选择是否使用本地初始化文件（.coe 文件）来对存储空间进行初始化，这个初始化文件是可以利用 MATLAB 或者其他上位机小程序，生成 coe 文件初始化 IP 设置，通过该初始化文件进行配置 IP，能够直接读出 IP 中的数据，该端口在 ROM IP 中会进行详细介绍。

(7) 设置完成后，点击 OK，并在弹出界面中点击“Generate”即可生成 IP。

3.2 双端口 RAM 读写设计时序

在第 2 节中简单介绍了状态机的定义，但关于状态机的使用却没有进行详细介绍，在本章则以时序的方式，从头开始介绍状态机的详细设计。

(1) 首先，在设计状态机的时候一定有一个全局思想，因此在一个系统刚启动时会进入 IDLE 状态，以该双端口读写实验为例，当系统上电后，并复位完成后则进入 IDLE 状态。

(2) 系统在 IDLE 状态下, 在 PortA 的时序中直接跳转至写 RAM 的状态, 那么什么时候跳出写 RAM 状态呢? 当然是 RAM 写满的时候, 并且收到读模块中的读信号, 跳转至 RAM 读状态。

(3) 系统跳转至 RAM 读状态后, 就得考虑什么时候跳出读, 进入 IDLE 状态, 这当然也是 RAM 读空的状态跳转至 IDLE 状态, 则可跳转至 IDLE 状态。整个时序图如下所示:

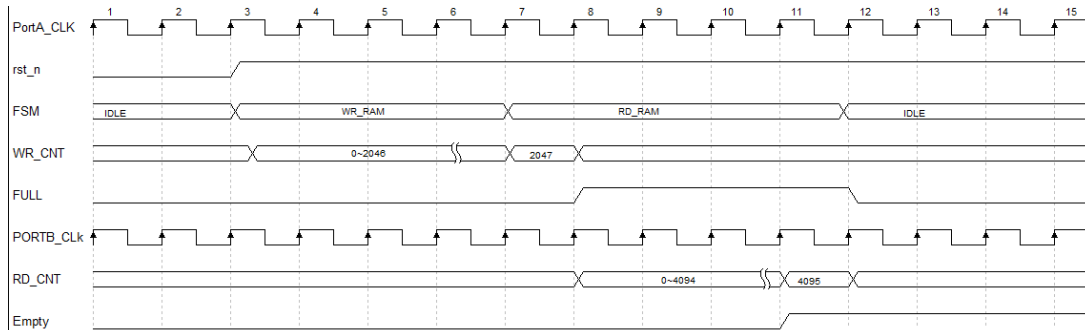


图 8-7 状态机读写时序图

(4) 图 8-7 中的时序图是十分基础的状态机跳转时序, 其中由于涉及到不同的时钟读写, 因此需要注意 FULL 与 Empty 信号的跨时钟处理, 接下来则开始编写异步时钟的 RAM 读写代码。

3.3 双端口 RAM 读写模块设计

由于双端口 RAM 是异步时钟的设计, 因此需要将读写状态机分开设计, 因此本章代码设计是采用模块化设计。主要分为时钟模块, 写模块以及读模块, 由于代码量稍多, 不浪费大量篇幅粘贴代码, 以下的代码只涉及关键逻辑的代码编写, 主要是为了理清代码思路。

(1) RAM 写数据模块的设计, 首先确定在写数据模块中的状态机只有初始 IDLE 状态与 RAM_WR 状态, 因此状态机设计只有两个状态, 代码设计如下所示:

//三段式状态机: 将次态赋值给现态

```
always @(posedge i_clk_50m or posedge i_50m_rst) begin
    if (i_50m_rst)
        r_st_current <= P_ST_IDLE;
    else
        r_st_current <= r_st_next;
end
```

//三段式状态机: 依据现态编写次态跳转

```
always @(*) begin
    case (r_st_current)
        P_ST_IDLE:
            begin
                if(w_empty_pos)//当检测至RAM 为空的上升沿时, 则跳转
                    r_st_next <= P_ST_WR_RAM;
                else
                    r_st_next <= P_ST_IDLE;
            end
    end
```

```

P_ST_WR_RAM: //检测至 RAM 满信号上升沿则可跳转至 IDLE 状态
begin
    if(r_ram_full)
        r_st_next <= P_ST_IDLE;
    else
        r_st_next <= P_ST_WR_RAM;
    end
default: r_st_next <= P_ST_IDLE;
endcase
end

```

上述代码很好理解，当系统卡在 IDLE 状态下时，检测到 RAM 读模块中读空了，则开始写，循环进行跳转。

(2) 当状态机处于写状态中，则拉高 we 信号、写入地址、写入数据，代码如下：

```

//控制 RAM 写地址
always@(posedge i_clk_50m or posedge i_50m_rst) begin
    if (i_50m_rst)
        r_ram_addr <= 'd0;
    else if (r_ram_full)
        r_ram_addr <= 'd0;
    else if (r_st_current == P_ST_WR_RAM && r_ram_we)
        r_ram_addr <= r_ram_addr + 'd1;
    else
        r_ram_addr <= r_ram_addr;
end

```

```

//控制 RAM 写数据
always@(posedge i_clk_50m or posedge i_50m_rst) begin
    if (i_50m_rst)
        r_ram_data <= 'd0;
    else if (r_ram_full)
        r_ram_data <= 'd0;
    else if (r_st_current == P_ST_WR_RAM && r_ram_we)
        r_ram_data <= r_ram_data + 'd2;
    else
        r_ram_data <= r_ram_data;
end

```

```

//对写入数据进行计数
always@(posedge i_clk_50m or posedge i_50m_rst) begin
    if (i_50m_rst)
        r_ram_wr_cnt <= 'd0;
    else if (r_ram_full)

```

```

        r_ram_wr_cnt <= 'd0;
    else if (r_st_current == P_ST_WR_RAM && r_ram_we)
        r_ram_wr_cnt <= r_ram_wr_cnt + 'd1;
    else
        r_ram_wr_cnt <= r_ram_wr_cnt;
end

```

上述代码则是控制在写 RAM 状态下，将数据，地址等信息写入 RAM 中，地址的改变是当 RAM 写满则清 0，数据在写 RAM 的状态下随着时钟递增。

(3) 当 RAM 写满时，则需要拉高 FULL 信号，用于读模块的判断，此时需要注意，FULL 信号所处时钟域为 50Mhz，而读模块时钟为 75Mhz，因此需要将该数据进行跨时钟处理。单 bit 数据从慢时钟域同步至快时钟域只需要在快时钟域下打两拍则可完成单 bit 数据的同步。

// 写满 2048 个数据则拉高一个周期

```

always@(posedge i_clk_50m or posedge i_50m_rst) begin
    if (i_50m_rst)
        r_ram_full <= 'd0;
    else if (r_st_current == P_ST_WR_RAM && r_ram_we && r_ram_wr_cnt == 'd2046)
        r_ram_full <= 'd1;
    else
        r_ram_full <= 'd0;
end

```

(4) 上述完成 RAM 写模块的控制，那么接下来则是对读模块的控制，整体代码的编写是一致的。首先确定状态机只有两个状态，一个初始状态，一个读 RAM 状态，从初始状态跳转至读 RAM 状态则是通过判断 RAM 写满的信号，因为涉及跨时钟处理，优先将跨时钟信号处理，代码如下：

//full 信号由慢时钟进快时钟直接打两拍进行信号同步

```

always @(posedge i_clk_75m or posedge i_75m_rst) begin
    if (i_75m_rst)
        r_ram_full <= 'd0;
    else
        r_ram_full <= {r_ram_full[1:0], i_ram_full};
end

```

//三段式状态机：将次态赋值给现态

```

always @(posedge i_clk_75m or posedge i_75m_rst) begin
    if (i_75m_rst)
        r_st_current <= P_ST_IDLE;
    else
        r_st_current <= r_st_next;
end

```

//三段式状态机：依据现态编写次态跳转

```
always @(*) begin
    case (r_st_current)
        P_ST_IDLE:
            begin
                if(w_ram_full_pos)//当检测至RAM 为满的上升沿时，则跳转
                    r_st_next <= P_ST_RD_RAM;
                else
                    r_st_next <= P_ST_IDLE;
            end
        P_ST_RD_RAM: //检测至RAM 空信号上升沿则可跳转至IDLE 状态
            begin
                if(w_ram_empty_pos)
                    r_st_next <= P_ST_IDLE;
                else
                    r_st_next <= P_ST_RD_RAM;
            end
        default: r_st_next <= P_ST_IDLE;
    endcase
end
```

(5) 当跳转至读 RAM 状态中则需要产生读地址，代码如下：

//控制读RAM 地址

```
always @(posedge i_clk_75m or posedge i_75m_rst) begin
    if (i_75m_rst)
        r_ram_addr <= 'd0;
    else if (r_ram_addr == 'd4094)
        r_ram_addr <= 'd0;
    else if (r_st_current == P_ST_RD_RAM)
        r_ram_addr <= r_ram_addr + 'd1;
    else
        r_ram_addr <= r_ram_addr;
end
```

(6) 当读完 RAM 中数据读完后需要产生一个空信号，由于采用寄存器赋值，因此空信号需要提前一拍拉高，这样空信号会与最后一个读地址对齐。同时由于从快时钟域同步信号至慢时钟域的原因，需要将 empty 信号延展，以方便慢时钟域实现信号采集。代码如下：

//控制输出空信号

```
always @(posedge i_clk_75m or posedge i_75m_rst) begin
    if (i_75m_rst)
        r_ram_empty <= 'd1;
    else if (r_st_current == P_ST_RD_RAM && r_ram_rd_cnt == 'd4093)
        r_ram_empty <= 'd1;
```

```

else if (r_st_current == P_ST_RD_RAM)
    r_ram_empty <= 'd0;
else
    r_ram_empty <= r_ram_empty;
end
//empty 信号需要从快时钟进满时钟所以将其延展20个时钟周期，方便慢时钟采集
always @(posedge i_clk_75m or posedge i_75m_rst) begin
    if (i_75m_rst)
        r_ram_empty_cnt <= 'd0;
    else if (r_ram_empty_cnt == 'd19)
        r_ram_empty_cnt <= 'd0;
    else if (w_ram_empty_pos || r_ram_empty_cnt)
        r_ram_empty_cnt <= r_ram_empty_cnt + 'd1;
    else
        r_ram_empty_cnt <= r_ram_empty_cnt;
end

//延展empty 信号
always @(posedge i_clk_75m or posedge i_75m_rst) begin
    if (i_75m_rst)
        r_ram_empty_slow <= 'd1;
    else if (r_st_current == P_ST_RD_RAM && r_st_cnt == 'd0)
        r_ram_empty_slow <= 'd0;
    else if (r_st_current == P_ST_RD_RAM && w_ram_empty_pos)
        r_ram_empty_slow <= 'd1;
    else
        r_ram_empty_slow <= r_ram_empty_slow;
end

```

(7) 上述基本完成了 RAM 的读写控制，而此次采用的不同时钟的控制信号，因此时钟也要进行相应的处理，将不同的时钟情况下产生不同的复位信号，PLL 锁定后，在产生复位信号，代码如下：

```

always@(posedge w_clk_50m) begin
    if (~w_pll_lock) begin
        r_50M_rst_cnt <= 'd0;
        r_rst_50m <= 'd1;
    end else if (r_50M_rst_cnt[9] == 1'b1) begin
        r_50M_rst_cnt <= r_50M_rst_cnt;
        r_rst_50m <= 'd0;
    end else begin
        r_50M_rst_cnt <= r_50M_rst_cnt + 'd1;
        r_rst_50m <= r_rst_50m;
    end
end

```

```

end

always@(posedge w_clk_75m) begin
    if (~w_pll_lock) begin
        r_75M_rst_cnt <= 'd0;
        r_rst_75m     <= 'd1;
    end else if (r_75M_rst_cnt[9] == 1'b1) begin
        r_75M_rst_cnt <= r_75M_rst_cnt;
        r_rst_75m     <= 'd0;
    end else begin
        r_75M_rst_cnt <= r_75M_rst_cnt + 'd1;
        r_rst_75m     <= r_rst_50m;
    end
end
end

```

4.RAM 仿真设计及结果

(1) 对于 RAM 的仿真十分的简单，只需要产生一个 50Mhz 的时钟以及复位信号，即可输出时钟，仿真代码如下：

```

module TB_SIM_RAM();

reg clk,rst_n;

initial begin
    clk    <= 'd0;
    rst_n  <= 'd0;
    #100;
    rst_n  <= 'd1;
end

always begin
    clk <= 'd0;
    #10;
    clk <= 'd1;
    #10;
end

RAM_RW_TOP RAM_RW_TOP_U0
(
    .i_clk      (clk),
    .i_rst_n    (rst_n)

```

);

endmodule

(2) 因为涉及到三个模块，仿真是需要每个模块都需要看下仿真结果，首先查看时钟以及复位：

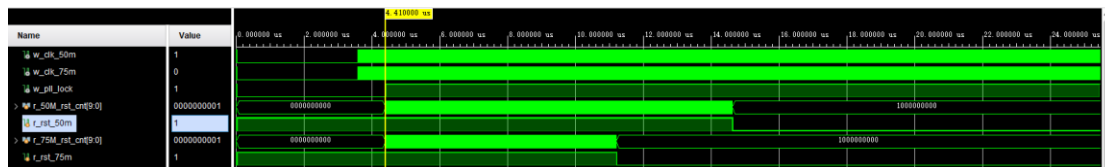


图 8-8 时钟及复位

从图 8-8 中可以看出，从 PLL 中产生了 50M 与 75M 时钟，于此同时产生了两个时钟的复位信号，从仿真中可以看时钟与复位信号是没有问题的。

(3) 随后，查看 RAM 写模块的仿真。

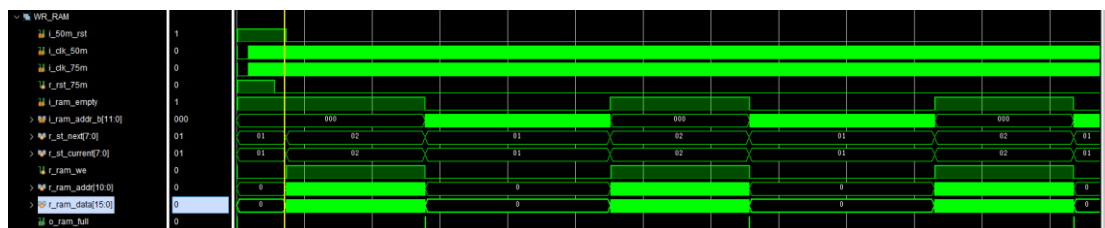


图 8-9 写 RAM 控制

图 8-9 中粗略看了下，状态机在正常跳转，同时写有效、写地址、写数据及满信号都产生了，但数据都挤在一起，并不能很详细的看出来，因此放大仿真波形，仿真结果如下：

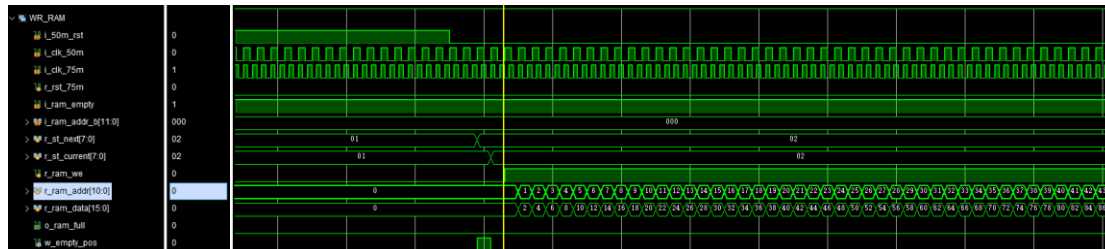


图 8-10 写 RAM 详细结果

(4) 最后查看读控制，波形如下图所示：

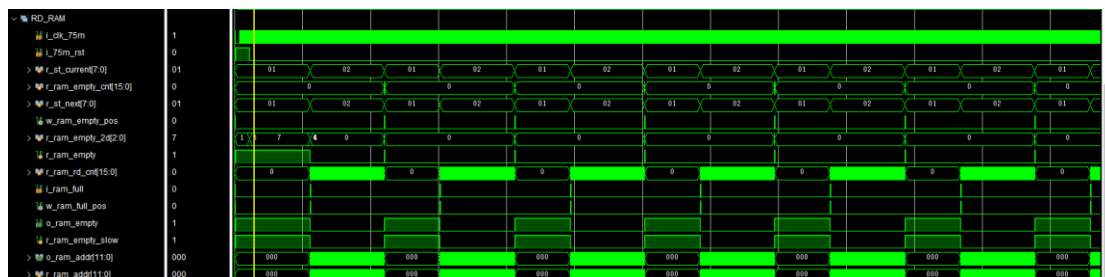


图 8-11 读 RAM 控制

图 8-11 中粗略看了下，状态机在正常跳转，同时读地址、空信号都产生了，但数据都挤在一起，并不能很详细的看出来，因此放大仿真波形，仿真结果如下：



图 8-12 读 RAM 控制

从图 8-12 中可以看出，读地址正常增加，数据结果也在正常输出。

5. 添加内部逻辑分析仪

(1) 在简单双端口 RAM 的实验中，可以抓取读写数据的计数器、RAM 控制信号及 RAM 数据信号，添加信号标识如下图所示：

```
module WR_RAM
(
    input    wire    i_clk_50m    ,
    input    wire    i_50m_rst    ,
    output   wire    o_ram_full    , //50m时钟下RAM满信号，输出至读模块

    input    wire    i_clk_75m    ,
    (* mark_debug = "true" *)
    input    wire    i_ram_empty   , //75m时钟下RAM空信号，读模块输入
    (* mark_debug = "true" *)
    input    wire    [11:0] i_ram_addr_b    ,
    (* mark_debug = "true" *)
    output   wire    [7:0] o_ram_data
);

wire        w_empty_pos    ; //空信号上升沿

(* mark_debug = "true" *)
reg    [7:0]    r_st_current    ;
reg    [7:0]    r_st_next      ;

(* mark_debug = "true" *)
reg    r_ram_we    ; //RAM写有效
(* mark_debug = "true" *)
reg    [10:0]    r_ram_addr    ; //RAM写地址
(* mark_debug = "true" *)
reg    [15:0]    r_ram_data    ; //RAM写数据
(* mark_debug = "true" *)
reg    r_ram_full    ;
reg    [15:0]    r_ram_wr_cnt    ; //RAM写数据计数器
reg    [2:0]    ram_empty    ;
```

图 8-12 添加 Debug 信号

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。

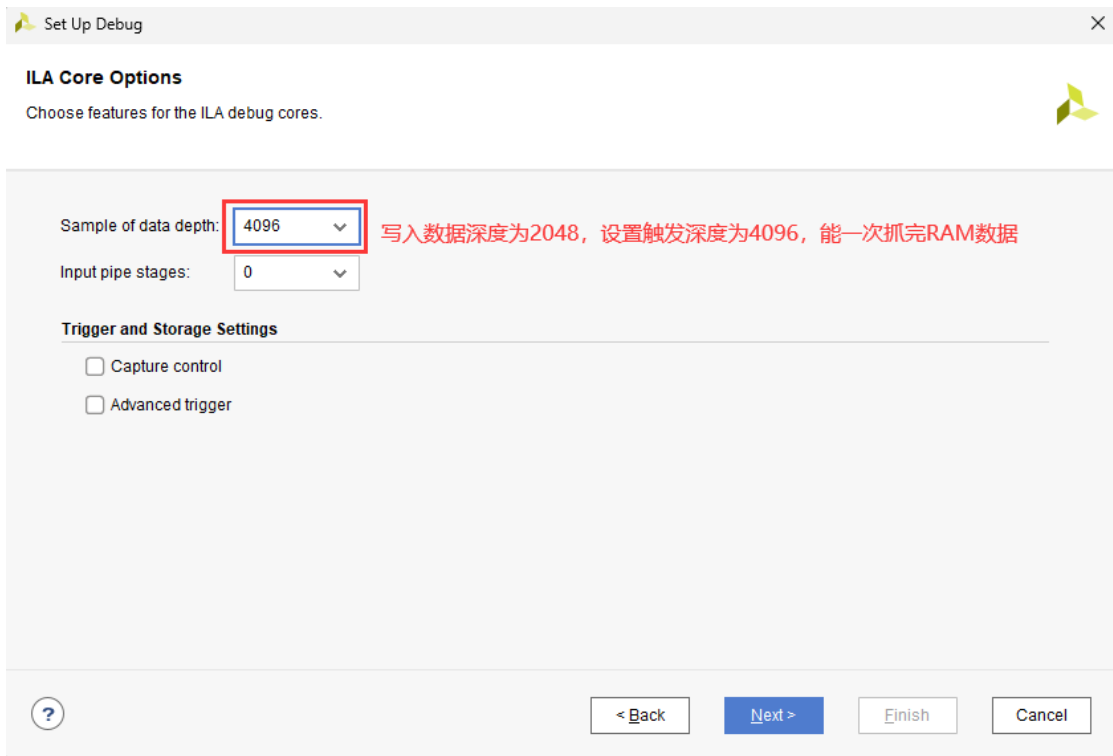


图 8-13 设置 Debug 信号的深度

(3) 查看是否写入约束文件

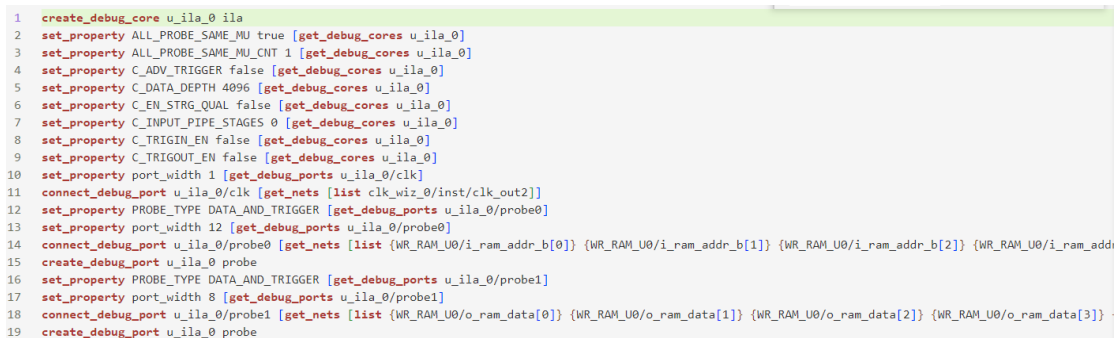


图 8-14 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

6. 添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按键就可以添加时钟约束了，如下图所示：

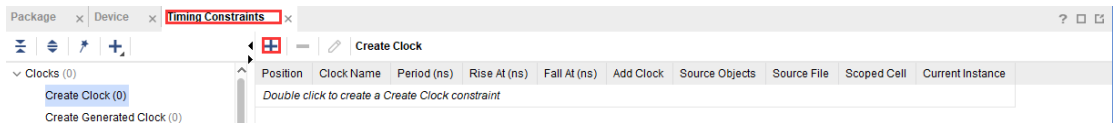


图 8-15 添加时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

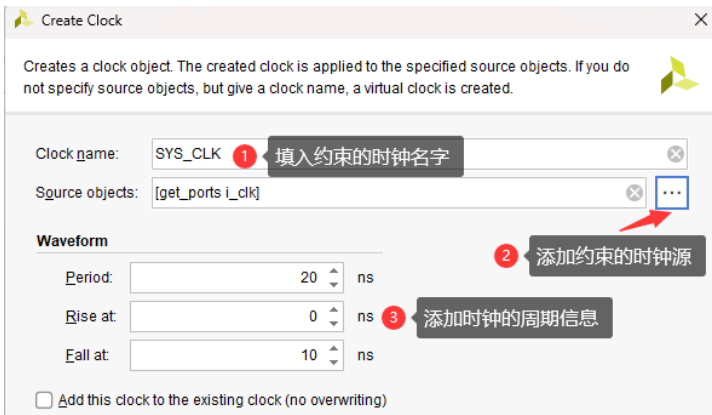


图 8-16 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
1 create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} -add [get_ports i_clk]
2 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
3 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
4 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
5 connect_debug_port dbg_hub/clock [get_nets w_clk_50m]
```

图 8-17 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

7.添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

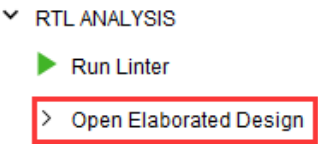


图 8-18 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
BRAM_PORTA_25633 (1)	IN			☒	33	LVCMOS33*	3.300				NONE	NONE	
Scalar ports (1)	IN		W17	☒	33	LVCMOS33*	3.300				NONE	NONE	
Scalar ports (1)	IN		W18	☒	33	LVCMOS33*	3.300				NONE	NONE	

图 8-19 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
set_property IOSTANDARD LVCMOS33 [get_ports i_clk]
set_property IOSTANDARD LVCMOS33 [get_ports i_rst_n]
set_property PACKAGE_PIN W17 [get_ports i_clk]
set_property PACKAGE_PIN W18 [get_ports i_rst_n]
```

图 8-20 约束文件中写入管脚约束

8.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，此次工程中有跨时钟处理，因此在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

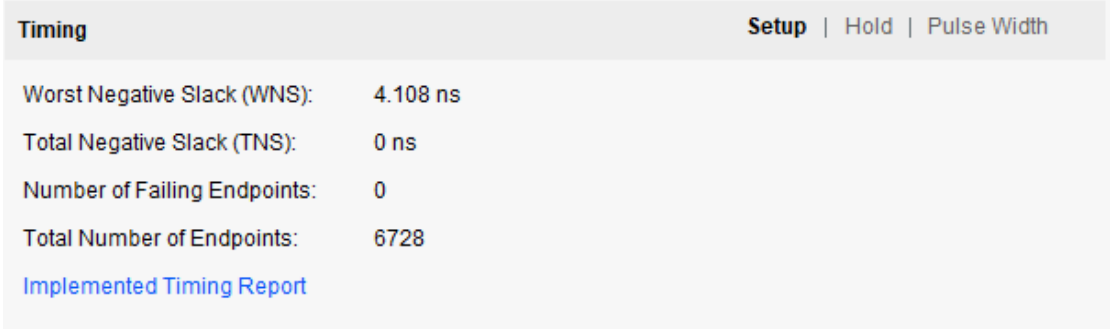


图 8-21 时序结果

将 Bit 下载至 FPGA，随后打开 ILA，随后设置 ila 为 r_ram_we 上升沿触发，观测 RAM 输入写数据，触发结果如下图所示：

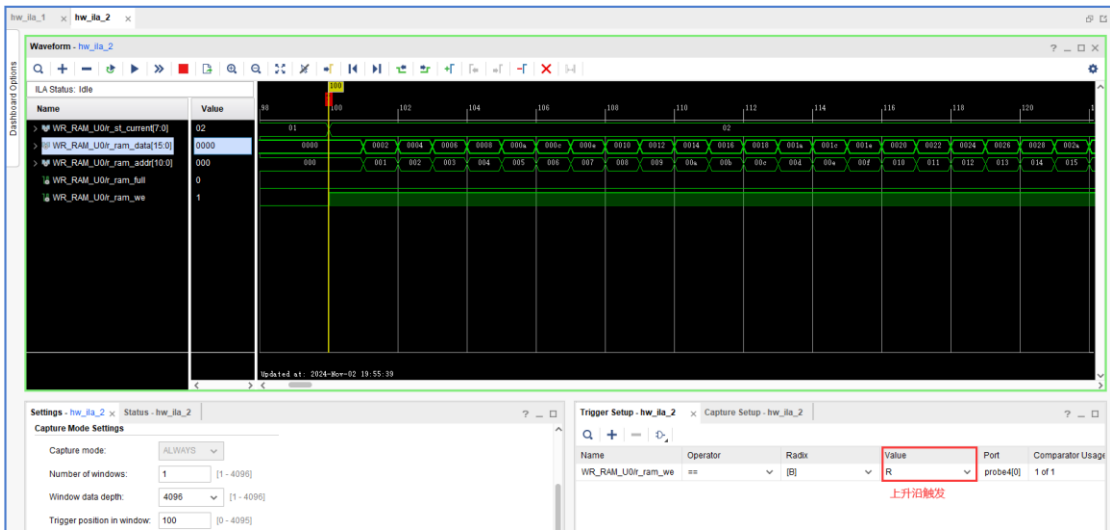


图 8-22 读写有效信号上升沿触发波形

从图 8-22 可以看出，当 r_ram_we 信号为高，RAM 地址在自增同时也在写入数据至 RAM 中，RAM 并没有数据读出。

随后设置 empty 上升沿触发，观测 RAM 读数据，触发结果如下图所示：

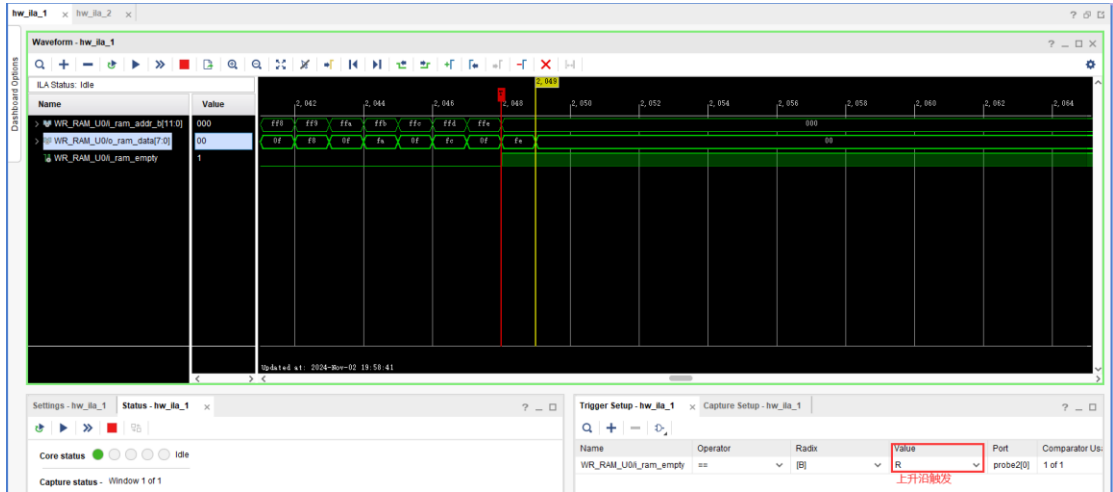


图 8-23 空信号上升沿触发波形

从图 8-23 可以看出，当 empty 信号为高时，可以在 empty 前观测到读出 RAM 中最后的数据，最后几个数据与仿真中是一致的，可以多次观测 RAM 读出的数据。

九. IP 核之 FIFO 实验

FIFO (First In First Out, 即先入先出), 是一种数据缓存器, 用来实现数据先入先出的读写方式。在 FPGA 或者 ASIC 中使用到的 FIFO 一般指的是对数据的存储具有先入先出特性的缓存器, 常被用于多比特数据跨时钟域的转换、读写数据带宽不同步等场合。

1.FIFO IP 简介

FIFO 本质上是由 RAM 加读写控制逻辑构成的一种先进先出的数据缓冲器, 其与普通存储器 RAM 的区别在于 FIFO 没有外部读写地址线, 使用起来非常简单, 但 FIFO 只能顺序写入数据, 并按顺序读出数据, 其数据地址由内部读写指针自动加 1 完成, 不能像普通存储器那样可以由地址线决定读取或写入某个指定的地址, 不过也正是因为这个特性, 使得 FIFO 在使用时并不存在像 RAM 那样的读写冲突问题。

根据 FIFO 工作的时钟域, 可以将 FIFO 分为同步 FIFO 和异步 FIFO。同步 FIFO 是指读时钟和写时钟为同一个时钟, 在时钟沿来临时同时发生读写操作, 常用于两边数据处理带宽不一致的临时缓冲。异步 FIFO 是指读写时钟不一致, 读写时钟是互相独立的, 一般用于数据信号跨时钟域处理。

由于 FIFO 是使用最广泛的 IP, 对 FIFO 常见参数得有一个简单的了解:

- a. FIFO 的宽度: FIFO 一次读写操作的数据位宽 N。
- b. FIFO 的深度: FIFO 可以存储多少个宽度为 N 位的数据。
- c. 将空标志: `almost_empty`, FIFO 即将被读空。
- d. 空标志: `empty`, FIFO 已空时由 FIFO 的状态电路送出的一个信号, 以阻止 FIFO 的读操作继续从 FIFO 中读出数据而造成无效数据的读出。
- e. 将满标志: `almost_full`, FIFO 即将被写满。
- f. 满标志: `full`, FIFO 已满时由 FIFO 的状态电路送出的一个信号, 以阻止 FIFO 的写操作继续向 FIFO 中写数据而造成溢出。
- g. 写时钟: 写 FIFO 时所遵循的时钟, 在每个时钟的上升沿触发。
- h. 读时钟: 读 FIFO 时所遵循的时钟, 在每个时钟的上升沿触发。
- i. 可配置满阈值: 影响可配置满信号于何时有效, 其可配置范围一般为 3~写深度-3。
- j. 可配置满信号: `prog_full`, 表示 FIFO 中存储的数据量达到可配置满阈值中配置的数值。
- k. 可配置空阈值: 影响可配置空信号于何时有效, 其可配置范围一般为 2~读深度-3。
- l. 可配置空信号: `prog_empty`, 表示 FIFO 中剩余的数据量已经减少到可配置空阈值中配置的数值。

这里还有两点需要特别注意:

1、“almost_empty”和“almost_full”这两个信号分别被看作“empty”和“full”的警告信号, 他们相对于真正的空 (empty) 和满 (full) 都会提前一个时钟周期拉高。

2、FIFO 中, 先写入的数据被置于高位, 后写入的数据被置于低位, 由于其先入先出的特性, 所以读出的数据也是高位在前, 低位在后。这一点在读写数据位宽不对等时尤为重要, 例如我们写数据位宽为 8, 读数据位宽为 2, 当写入的数据为 11000111 时, 读出的数据依次为 11、00、01、11, 如下图所示:

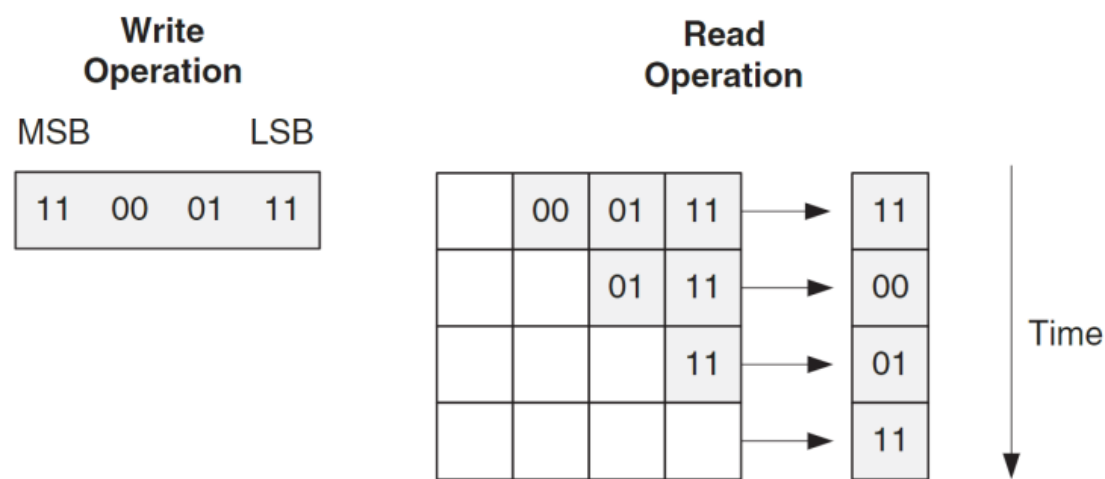


图 9-1 写位宽大于读位宽时的数据排序

读位宽大于写位宽时，原理是相同的，其示意图如下：

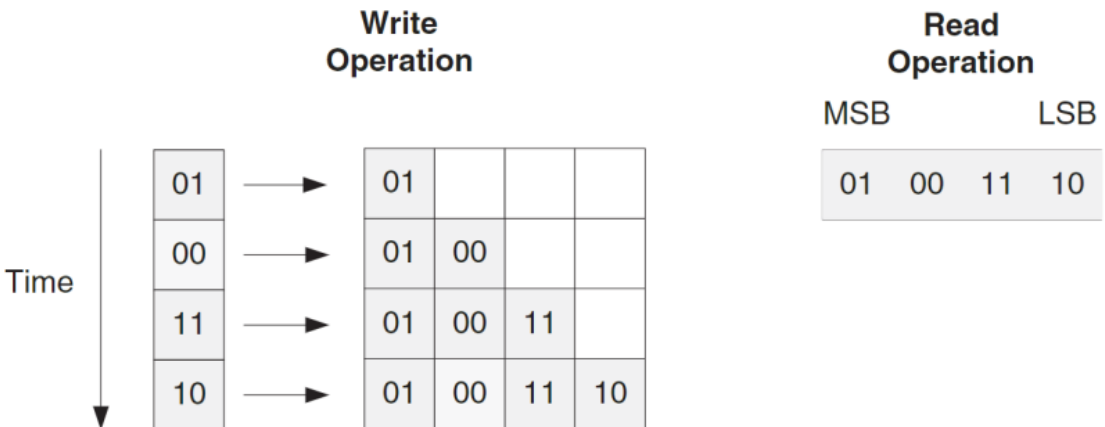


图 9-2 读位宽大于写位宽时的数据排序

对于 FIFO 的基本知识先了解这些就足够了，在逻辑设计的时候，尤其是在 FPGA 设计中，FIFO 都有哪些实现方法：

- a. 根据需求自己设计并编写 FIFO 的控制逻辑，一般对 FIFO 的功能有特殊需求时，可以使用此种方法。
- b. 使用第三方提供的开源 IP 核，一般是以源码的形式提供的，能够快速应用到用户系统中，当对 FIFO 有特殊需求时，可以在此源码的基础上自行修改，以适应自己的设计需求。
- c. 调用官方开发软件中免费提供的 FIFOIP 核，我们可以通过软件中的图形化界面对 FIFO 的各项参数和结构进行配置，生成的 FIFOIP 核针对旗下的不同系列器件，还可以实现结构上的优化，且其可配置的功能足以实现大部分的设计需求，因此在实际应用中推荐大家使用 IP 核来实现 FIFO 设计。

Vivado 软件中用于实现 FIFO 设计的 FIFO Generator IP 核（FIFO 发生器），其信号框图如下图所示：

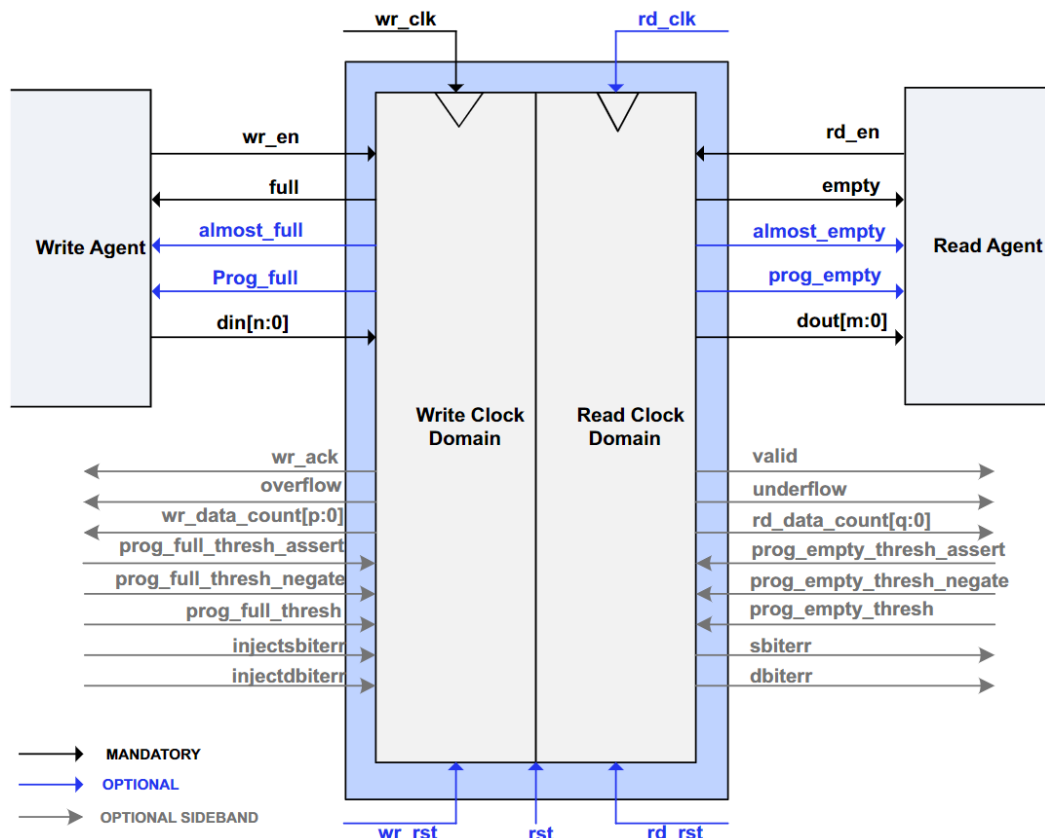


图 9-3 常规接口的 FIFO 信号框图

首先说明下,上图中黑色箭头表示此信号为必要信号;蓝色箭头表示此信号为可选信号;灰色箭头表示此信号为可选的边带信号。

从图 9-3 中可以看出,当被配置为同步 FIFO 时,只使用 `wr_clk`,所有的输入输出信号都同步于 `wr_clk` 信号。而当被配置为异步 FIFO 时,写端口和读端口分别有独立的时钟,所有与写相关的信号都是同步于写时钟 `wr_clk`,所有与读相关的信号都是同步于读时钟 `rd_clk`。

对于异步 FIFO 的使用需要特别注意的点是:满与空的信号是基于各自的时钟域,因此在使用时需要特别注意跨时钟问题,否则十分容易出现跨时钟问题,尽量做到读写时钟域的隔离,不要采用写时钟域信号控制读信号,或者采用读时钟域控制写信号。

2.实验任务

本节的实验任务是使用 Vivado 生成一个异步 FIFO,并实现以下功能:当 FIFO 为空时,向 FIFO 中写入数据,写入 FIFO 的时钟为 100Mhz,直至将 FIFO 写满后停止写操作;当 FIFO 为满时,从 FIFO 中读出数据,读 FIFO 的时钟为 75Mhz,直到 FIFO 被读空后停止读操作,以此向大家详细介绍一下 FIFO IP 核的使用方法。

同步 FIFO 的使用与异步 FIFO 的使用是一致的,且满空信号不存在跨时钟问题,因此以使用时遇到问题较多的异步 FIFO 进行本次实验

3.程序设计

3.1 新建异步 FIFO IP 核

(1) 首先创建一个空白工程，然后点击 Vivado 软件左侧“Flow Navigator”栏中的“IP Catalog”，在弹出的“IP Catalog”窗口的搜索栏中输入“fifo”关键字后，找到“FIFO Generator”如下图所示：

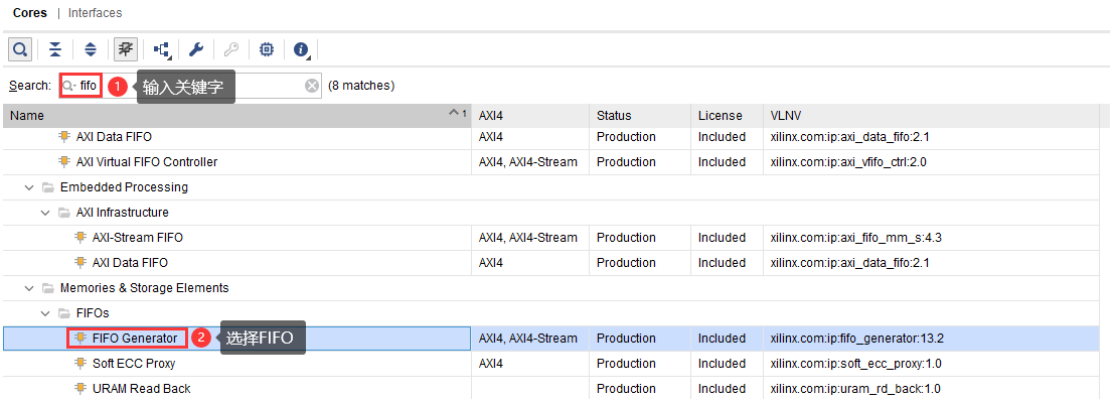


图 9-4 搜索栏中输入“fifo”关键字

(2) 双击“FIFO Generator”后弹出 IP 核的配置界面，接着就可以对 FIFO IP 核进行配置了，“Basic”选项卡配置界面如下图所示。

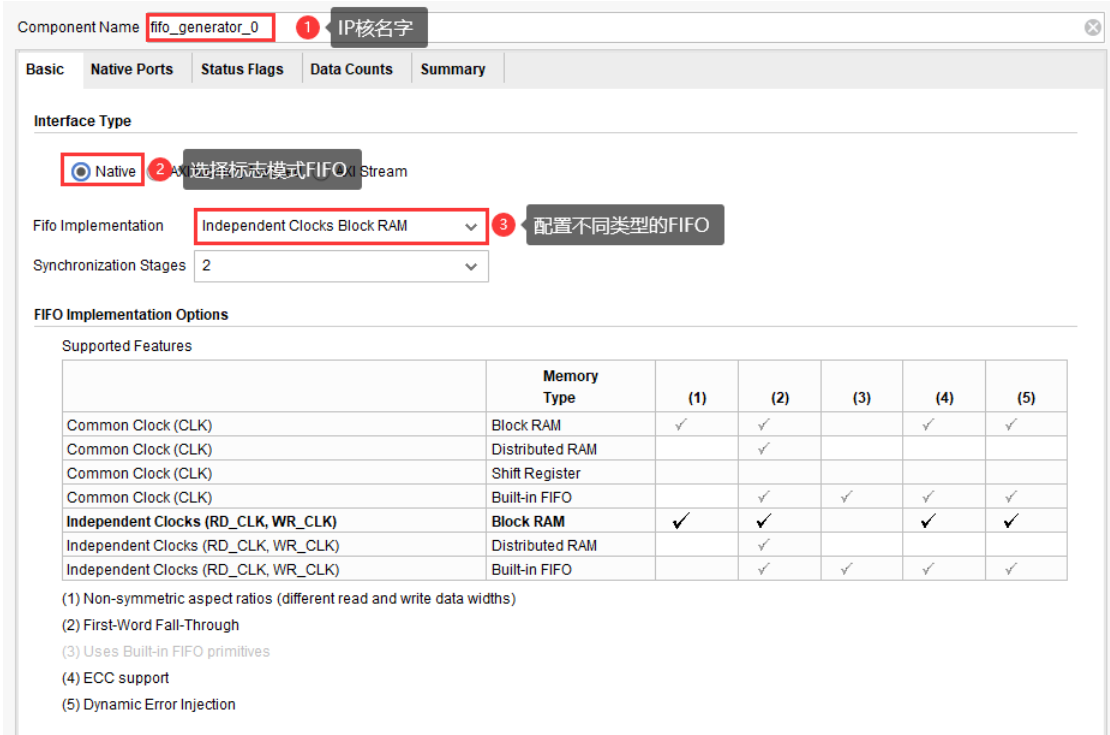


图 9-5 “Basic”选项卡配置界面

最上面的“Component Name”一栏可以设置该 IP 元件的名称，这里我们保持默认命名，当然也可以命名为其它方便自己一眼看出其功能的名称。

接着目光回到“Basic”选项卡上，该选项卡下各参数（重点关注 (b) 和 (c) 中的内容）含义如下：

a.“Interface Type（接口模式）”：有三种接口模式可选，分别为 Native（常规）接口、AXI Memory Mapped（内存映射）接口和 AXI Stream（流）接口。其中 AXI Memory Mapped 接口一般用于与 PS 端进行数据交互；AXI Stream 接口一般应用于高速信号处理场景中，例如光口通信；通常情况下我们一般采用 Native 模式，所以本次实验我们选择 Native 模式。

b.“Fifo Implementation（FIFO 实现）”：用于设置用什么资源来实现什么样的 FIFO。可配置用于实现 FIFO 的资源有四种，分别为 Block RAM（块 RAM）、Distributed RAM（分布式 RAM）、Shift Register（移位寄存器）和 Builtin FIFO（内置 FIFO），其中移位寄存器仅可用于实现同步 FIFO。可配置的 FIFO 类型有两类，分别为 CommonClocks（公共时钟，即同步 FIFO）和 IndependentClocks（独立时钟，即异步 FIFO）。资源与种类两两组合之下便有了七种不同的选择。需要说明的是 BRAM 和分布式 RAM 是创建 FIFO 时最常选用的存储资源类型，一般来说，FIFO 容量超过 1024 个字节就需要考虑使用 BRAM 了，没超过 1024 字节选择分布式 RAM。当然，如果芯片 BRAM 资源很富余的话，全部采用 BRAM 也是可以的，后两种基本用不到。本次实验选择“Independent Clocks Block RAM”，即使用 BRAM 资源来实现一个异步 FIFO。

c.“synchronization Stages（同步阶段）”：定义跨交叉时钟域逻辑的同步器级数，即设置读写状态信号的输出延迟。保持默认设置 2 即可。

d.“FIFO Implementation Options（FIFO 实现方案）”：此处的表格将实现 FIFO 的七种方案的特征都一一列出了，当我们不清楚自己的 FIFO 设计该使用哪种方案实现时，可在 Vivado 中查看该表格。

(3) 接下来我们对“NativePorts”选项卡进行配置，如下图所示：

图 9-6 “Native”选项卡配置界面

在图 9-6 中需要注意的有两个点分别对应图中标 1 的“Read Mode”与标 3 处的

“Initialization”，其分别为以下意义：

“Read Mode（读取模式）”：有“Standard FIFO（标准 FIFO）”和“First Word Fall Through（首字直通，简称 FWFT 模式，即预读模式）”两种可选，需要注意的是标准模式的数据输出会比读使能延迟一拍，预读模式的数据与读使能同时输出，这里选择默认的标准模式。

“Initialization（初始化）”：其中最重要的是 Full Flags Reset Value（满信号的重置值）：用于设置复位时三个满信号（满信号，将满信号，设置满信号）的状态是高电平还是低电平。若采用默认的信号 1，会存在 FIFO 读处 FULL 信号为 1，有些控制条件采用的是 FULL 为 1 开始读，那么此时存在一种可能是不符合 FIFO 的复位设计，因此在复位时，将初始值设定为 0。

（4）接下来对“Status Flags”选项卡进行配置，如下图所示

Component Name

Basic

Native Ports

Status Flags

Data Counts

Summary

Optional Flags

☐ Almost Full Flag ☐ Almost Empty Flag

Handshaking Options

Write Port Handshaking

☒ Write Acknowledge Active High ☐ Overflow Active High

Read Port Handshaking

☒ Valid Flag Active High ☐ Underflow Flag Active High

Programmable Flags

Programmable Full Type	No Programmable Full Threshold	
Full Threshold Assert Value	1021	[4 - 1021]
Full Threshold Negate Value	1020	[3 - 1020]
Programmable Empty Type	No Programmable Empty Threshold	
Empty Threshold Assert Value	2	[2 - 2042]
Empty Threshold Negate Value	3	[3 - 2043]

图 9-7 “Status Flags”选项卡配置界面

勾选 valid flag，是在 FIFO 输出时，拉高表示数据有效。勾选 Write Acknowledge，拉高表示数据写入有效。

（5）其他选项卡中不需要进行配置，Data Counts 中计数器在异步 FIFO 的使用中是不准确的，在实际的使用中是不使用的。直接点开“Summary”选项卡，选项卡如下图所示：

Basic

Native Ports

Status Flags

Data Counts

Summary

WARNING : The use of Asynchronous Reset can lead to BRAM data corruption(AR 42571). It is recommended to Enable Safety Circuit

Block RAM resource(s) (18K BRAMs): 1

Block RAM resource(s) (36K BRAMs): 0

Clocking Scheme	Independent Clocks
Memory Type	Block RAM
Model Generated	Behavioral Model
Write Width	16
Write Depth	1023
Read Width	8
Read Depth	2046
Almost Full/Empty Flags	Not Selected/Not Selected
Programmable Full/Empty Flags	Not Selected/Not Selected
Data Count Outputs	Not Selected
Handshaking	Selected
Read Mode / Reset	Standard FIFO / Asynchronous
Read Latency (From Rising Edge of Read Clock)	1

图 9-8 “Summary”选项卡界面

“Summary”选项卡主要对 FIFO 配置进行总结，在该界面中可以看出，当前 FIFO 写深度为 1023，读深度为 2046，读数据延迟一个时钟周期。

3.2 异步 FIFO 时序

同步 FIFO 与异步 FIFO 的读写时序是十分类似的，如下图所示：

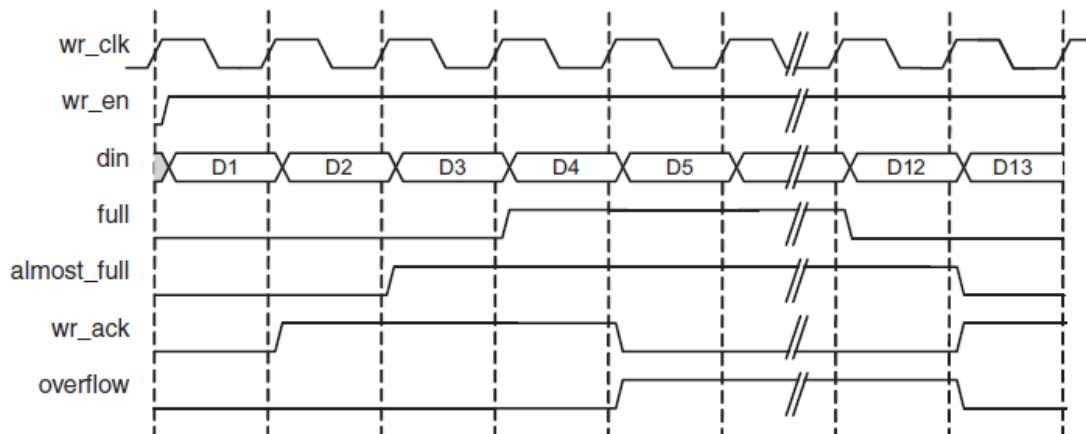


图 9-10 FIFO 写操作时序

从图 9-10 中可以看出，当 wr_en 为高的同时，数据会立刻写入 FIFO 中，随后下一个周期拉高 wr_ack 表示数据已经写入 FIFO 中，当 full 信号拉高时，wr_en 为高时，数据也不会写入 FIFO，同时 overflow 信号拉高，表示当前数据已经溢出。

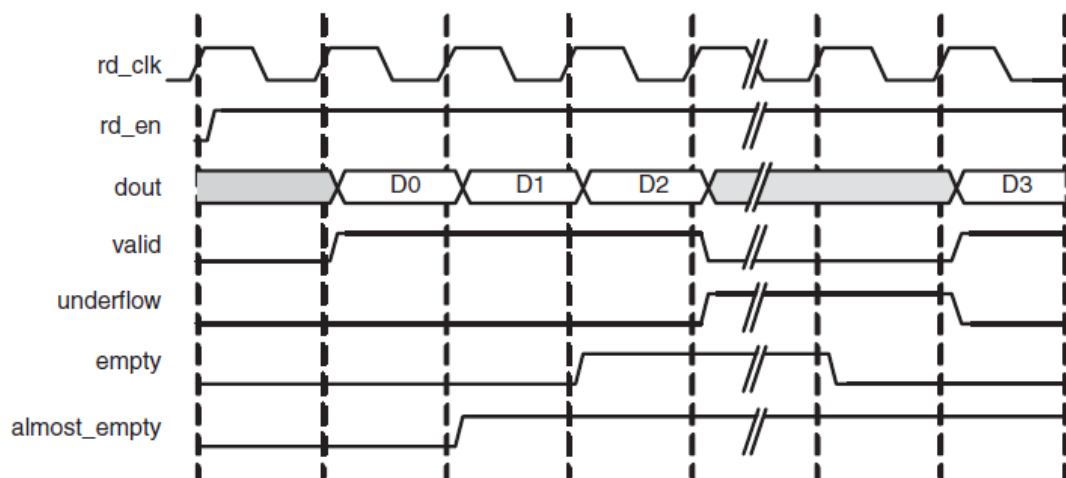


图 9-11 Standard FIFO 读操作时序

从图 9-11 中可以看出，当 rd_en 为高的同时，数据在下一个 rd_clk 的上升沿将数据从 FIFO 中读出，随后下一个周期拉高 valid 表示当前读出的数据有效，当 empty 信号拉高时，rd_en 为高时，数据也不会从 FIFO 中读出，同时 underflow 信号拉高，表示当前数据已经溢出。

下图是关于 FWFT 模式下读出数据的时序图，如下图所示：

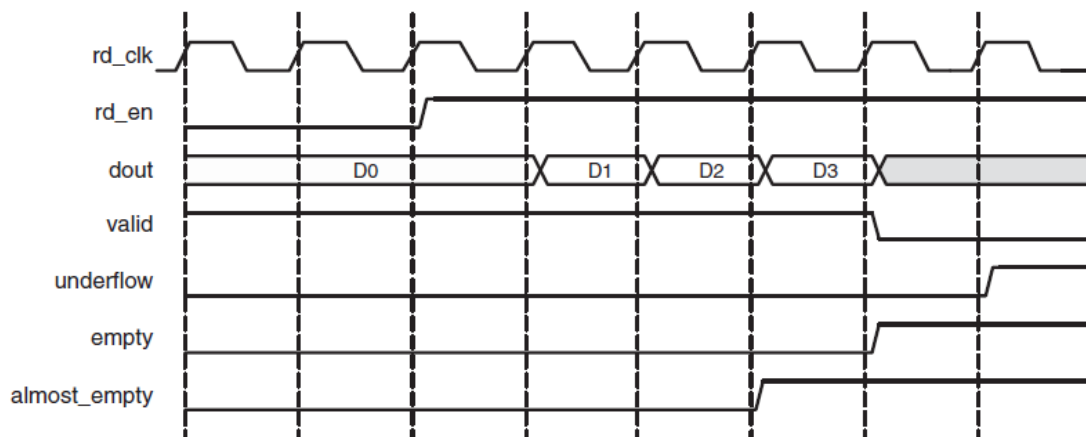


图 9-12 FWFT 模式 FIFO 读操作时序

FWFT FIFO 读操作与 Standard FIFO 读操作是一致的，信号含义也是一致的，只是会在拉高 rd_en 的同时会将数据输出，没有读潜伏期。

除此之外，大家还需要注意下 FIFO 的复位信号，其时序图如下所示：

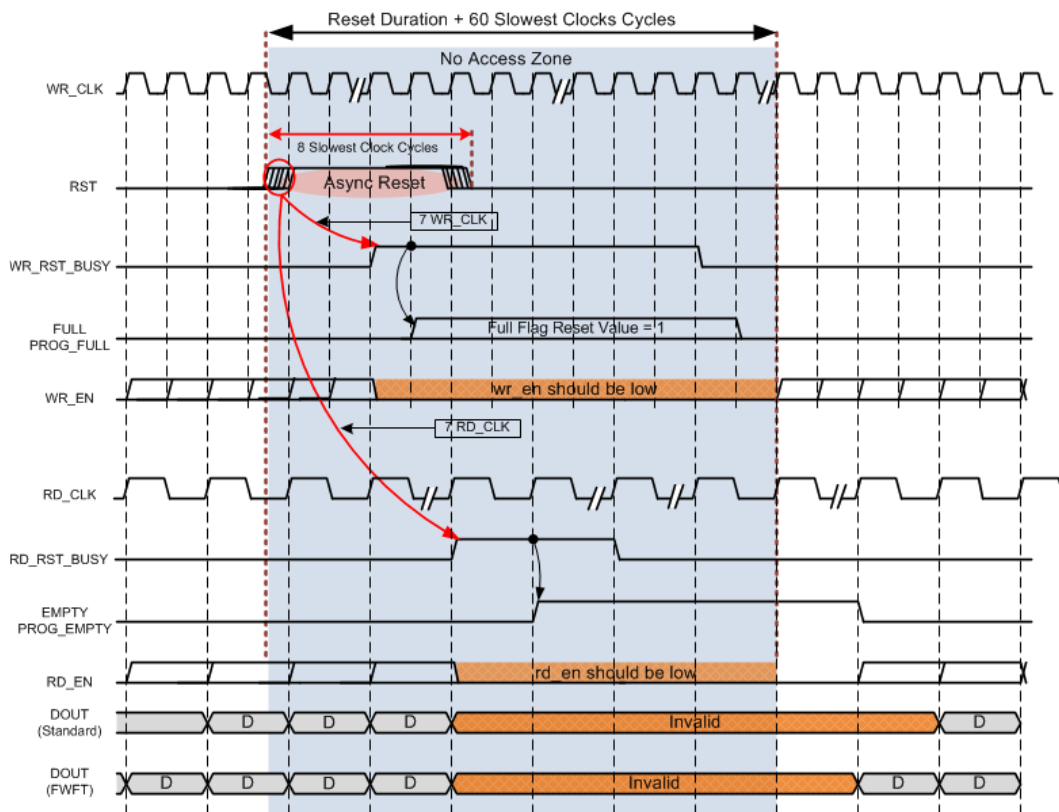


图 9-12 安全电路下的异步 FIFO 复位时序图

图 9-12 总结出以下几点注意事项：

- Xilinx 建议在安全电路下的异步 FIFO 的复位信号 (RST) 至少要保持八个时钟周期 (以慢时钟为准) 的有效。
- 在 RST 上升沿时期，经过 7 个 WR_CLK (写时钟) 周期后 WR_RST_BUSY (写复位忙) 信号拉高，FULL (满) 信号拉高，此时的 WR_EN (写使能) 信号应该拉低。
- 在 RST 上升沿时期，经过 7 个 RD_CLK (读时钟) 周期后 RD_RST_BUSY (读复位忙) 信号拉高，EMPTY (空) 信号拉高，此时的 RD_EN (读使能) 信号应该拉低，而且此

时的数据输出端口均为无效。

d.图中建议至少在复位结束后, 经过 60 个时钟周期(以慢时钟为准)后再对 FIFO 进行写操作, 在实际应用中我们以读写复位的忙信号来判断是否对 FIFO 进行写操作也是可以的。

所以大家对 FIFO 进行复位时, 一定要注意避免在复位结束时就立马对 FIFO 进行写操作, 因为此时的 FIFO 仍然处于复位状态。

3.3 FIFO 读写模块设计

由于是异步 FIFO 的设计, 因此需要将读写状态机分开设计, 因此本章代码设计是采用模块化设计。主要分为时钟模块, 写模块以及读模块, 由于代码量稍多, 不浪费大量篇幅粘贴代码, 以下的代码只涉及关键逻辑的代码编写, 主要是为了理清代码思路。

(1) 异步 FIFO 写数据模块的设计, 首先确定在写数据模块中的状态机只有初始 IDLE 状态与 FIFO_WR 状态, 因此状态机设计只有两个状态, 代码设计如下所示:

//三段式状态机: 第一段跳转

```
always @(posedge i_clk_100m or posedge i_rst_100m) begin
    if (i_rst_100m)
        r_st_current <= P_ST_ILDE;
    else
        r_st_current <= r_st_next;
end
```

//三段式状态机跳转控制

```
always @(*) begin
    case (r_st_current)
        P_ST_ILDE:
            begin
                if (r_empty_2d[2] == 'd1)
                    if (r_rst_cnt == 'd800)
                        r_st_next <= P_ST_WR_FIFO;
                    else
                        r_st_next <= P_ST_ILDE;
                else
                    r_st_next <= P_ST_ILDE;
            end
        P_ST_WR_FIFO :
            begin
                if (w_fifo_full_pos)
                    r_st_next <= P_ST_ILDE;
                else
                    r_st_next <= P_ST_WR_FIFO;
            end
        default: r_st_next <= P_ST_ILDE;
    endcase
end
```

其中状态机在 IDLE 状态跳转至写 FIFO 状态的判断，是在 FIFO 复位完成后，并延迟了 800 个时钟周期，该周期是为了符合 Xilinx 手册中对异步 FIFO 的复位要求，满足复位要求后，则进入写状态，直到 FIFO 中被写满跳入 IDLE 状态，等待 FIFO 输出的空信号，并再次进行写 FIFO 状态。

(2) 当状态机在写 FIFO 状态中，产生写 FIFO 的有效信号，于此同时产生写入 FIFO 的自增数据，代码如下：

```
//写FIFO的状态下，产生写入数据有效
always @(posedge i_clk_100m or posedge i_rst_100m) begin
    if (i_rst_100m)
        r_fifo_wr_en <= 'd0;
    else if (w_fifo_full_pos)
        r_fifo_wr_en <= 'd0;
    else if (r_st_current == P_ST_WR_FIFO && r_empty_2d[2])
        r_fifo_wr_en <= 'd1;
    else
        r_fifo_wr_en <= r_fifo_wr_en;
end

//写FIFO的状态下，产生写数据
always @(posedge i_clk_100m or posedge i_rst_100m) begin
    if (i_rst_100m)
        r_fifo_data <= 'd0;
    else if (w_fifo_full_pos)
        r_fifo_data <= 'd0;
    else if (r_st_current == P_ST_WR_FIFO && r_fifo_wr_en)
        r_fifo_data <= r_fifo_data + 'd1;
    else
        r_fifo_data <= r_fifo_data;
end
```

从上述代码中可以看出，FIFO 产生的有效数据是会在 FIFO Full 信号拉高后多产生一个周期的数据及数据有效信号，但此时 Full 信号已经拉高了，所以该数据不会被写入 FIFO，可以在仿真中观测该情况。

(3) 上述两步完成 FIFO 写模块的控制，那么接下来则是对 FIFO 读模块的控制，整体代码的编写是一致的。首先确定状态机只有两个状态，一个初始状态，一个读 FIFO 的状态，从初始状态跳转至读 FIFO 状态则是通过判断 FIFO 写满的信号，FIFO 满信号是在写时钟下，涉及快时钟域信号同步至慢时钟，而这部分在写模块中已经处理，因此在读模块可直接使用，在这不做过多解释，代码如下：

```
//三段式状态机：第一段跳转
always @(posedge i_clk_75m or posedge i_rst_75m) begin
    if (i_rst_75m)
        r_st_current <= P_ST_ILDE;
    else
```

```

        r_st_current <= r_st_next;
end

//三段式状态机跳转控制
always @(*) begin
    case (r_st_current)
        P_ST_ILDE :
            begin
                if (r_rd_cnt == 'd99)
                    r_st_next <= P_ST_RD_FIFO;
                else
                    r_st_next <= P_ST_ILDE;
            end
        P_ST_RD_FIFO :
            begin
                if (w_fifo_empty_pos)
                    r_st_next <= P_ST_ILDE;
                else
                    r_st_next <= P_ST_RD_FIFO;
            end
        default: r_st_next <= P_ST_ILDE;
    endcase
end

```

其中状态机在 IDLE 状态跳转至读 FIFO 状态的判断，是检测到 FIFO Full 信号后，延迟了 100 个时钟周期，该周期是为了符合 Xilinx 手册中对异步 FIFO 的写入后，满足读时间的延迟，随后进入读 FIFO 状态，直到 FIFO 中被读空跳入 IDLE 状态，等待 FIFO 输出的满信号，并再次进行读 FIFO 状态。

(4) 当状态机在读 FIFO 状态中，产生读 FIFO 的有效信号，代码如下：

```

//产生读FIFO 信号
always @(posedge i_clk_75m or posedge i_rst_75m) begin
    if (i_rst_75m)
        r_fifo_rd_en <= 'd0;
    else if (w_fifo_empty_pos)
        r_fifo_rd_en <= 'd0;
    else if (r_st_current == P_ST_RD_FIFO && r_st_cnt == 'd0)
        r_fifo_rd_en <= 'd1;
    else
        r_fifo_rd_en <= r_fifo_rd_en;
end

```

FIFO 读的逻辑控制十分的简单，当进入读状态中，就可以拉高 FIFO 读的有效信号，直到 FIFO 输出空信号时，则可跳出读状态，从而进入 IDLE 状态，等待 FIFO 输出满信号再次进入读状态。需要注意的是，因为采用空信号的上升沿作为结束读的控制条件，因此会存

在多读一个时钟的情况，但此时 FIFO 已经读空，不会从 FIFO 中读出错误数据，可以在仿真中观测该现象。

4.FIFO 仿真设计及结果

(1) 对于 RAM 的仿真十分的简单，只需要产生一个 50Mhz 的时钟以及复位信号，即可输出时钟，仿真代码如下：

```
module TB_SIM_FIFO();

reg clk,rst_n;

initial begin
    clk    <= 'd0;
    rst_n  <= 'd0;
    #100;
    rst_n  <= 'd1;
end

always begin
    clk <= 'd0;
    #10;
    clk <= 'd1;
    #10;
end

TOP_FIFO_WR_RD TOP_FIFO_WR_RD_u0
(
    .i_clk      (clk),
    .i_rst_n    (rst_n)
);

endmodule
```

(2) 因为涉及到三个模块，仿真是需要每个模块都需要看下仿真结果，首先查看时钟及复位：

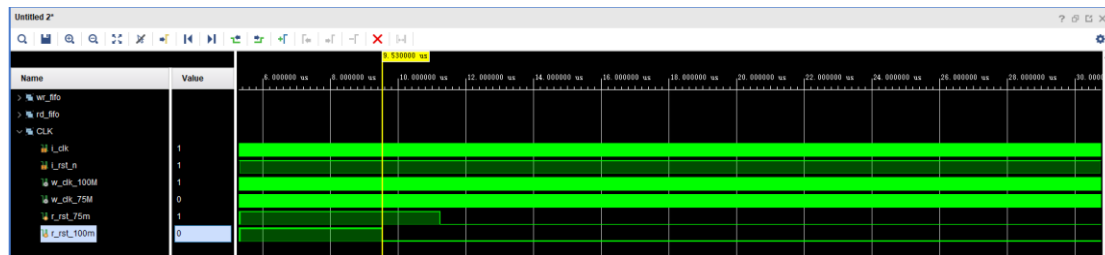


图 9-13 时钟及复位

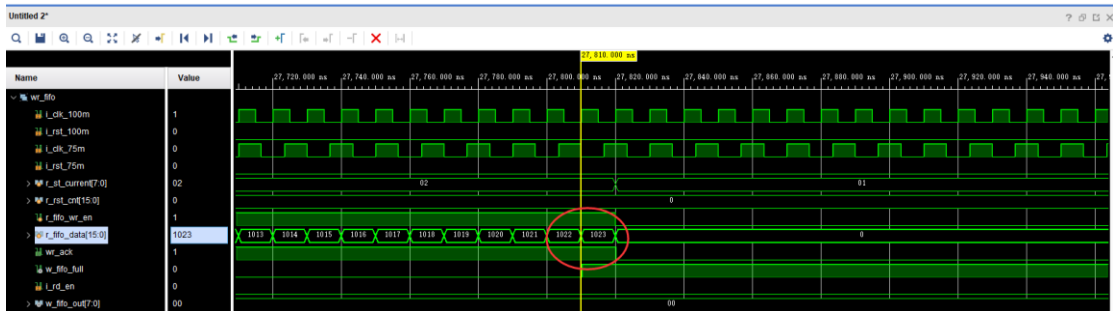
(3) 随后查看 FIFO 写模块的仿真结果

[illegible]

从图 9-14 中可以看出，在 75M 时钟域下复位 FIFO 的时候，写 FIFO 有效被拉低为 0，

在代码编写时提过，当 FULL 拉高时，会多产生一个写有效信号以及测试数据，但此时

FULL 已经拉高，所以在 IP 设置时的 ACK 信号则产生了作用，这个信号是指示当前写入的数据是否有效，将图 9-14 放大后，波形如下图所示：



由于 ACK 信号会晚于 WR_EN 信号一个时钟周期，因此图 9-15 中指示的是 1022 被写

入 FIFO 中，而 1023 没有写入 FIFO。

（一）数据窗口

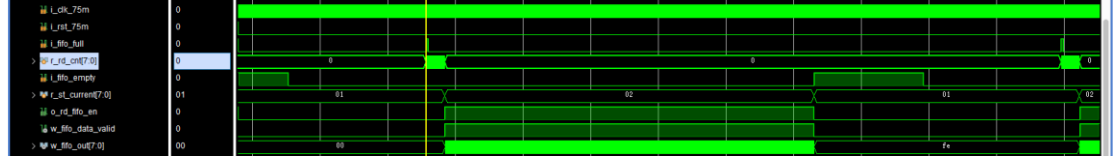


图 9-16 中可以看出, 当 FIFO 检测到 FULL 信号后, 延迟了 100 个时钟周期, 跳转至读

FIFO 状态，当进入读 FIFO 状态后，拉高读使能，并将该信号连接至 FIFO 上，随后延迟一个时钟输出数据，当 FIFO 读空后，状态机跳转至 IDLE 状态，等到下次 FIFO 写满，在进入读状态。

在代码编写时提过，当 Empty 拉高时，会多产生一个读有效信号，但此时 Empty 已经拉高，所以在 IP 设置时的 Valid 信号则产生了作用，这个信号用于标识当前读出的数据是否有效有效，将图 9-16 放大后，波形如下图所示：

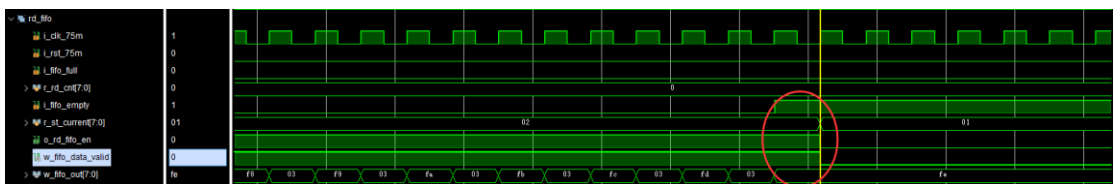


图 9-17 FIFO 最后读出数据控制

由于 Standard FIFO 在输入 RD_EN 信号时，数据会晚与 RD_EN 一个时钟周期，因此 valid 信号是与数据对齐的，因此图 9-17 中指示的是最后一个数据为 8'hfe。另外需要注意的是，该 FIFO 是将 16 bit 的数据转换为 8 bit 数据，也就是说是将一个 16 bit 的数据拆分为 8 bit 数据，将前一个字节的数：8'h03 与后一个字节：8'hfe 进行拼接，该数据为 16'd1022。这符合上述中写入的数据，同时也验证了当时介绍的 FIFO 特性，会将大位宽转化为小位宽数据，同时高位在前，低位在后。

5.添加内部逻辑分析仪

(1) 在 FIFO 的实验中，主要分析 FIFO 的控制信号以及读出的数据，添加信号标识如下图所示：

```
(* mark_debug = "true" *)
wire [7:0] w_fifo_out ;
(* mark_debug = "true" *)
wire w_fifo_full ;
(* mark_debug = "true" *)
wire w_fifo_empty ;
(* mark_debug = "true" *)
wire w_fifo_data_valid ;
wire w_fifo_full_pos ;
(* mark_debug = "true" *)
wire w_fifo_wr_ack ;

reg [7:0] r_st_current ;
reg [7:0] r_st_next ;
reg [15:0] r_st_cnt ;

(* mark_debug = "true" *)
reg r_fifo_wr_en ;
(* mark_debug = "true" *)
reg [15:0] r_fifo_data ;
```

图 9-17 添加 Debug 信号

(2) 添加完 Debug 信号后, 会对当前工程进行综合并设置需要抓取信号的深度。

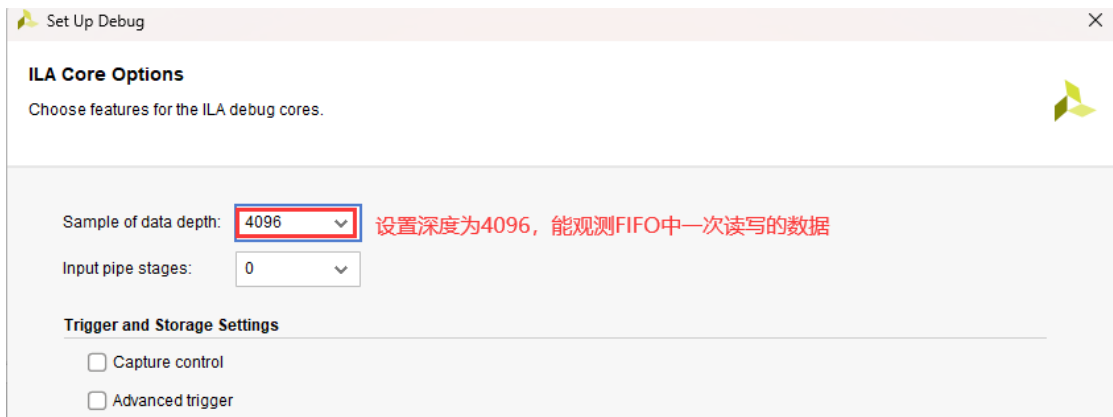


图 9-18 设置 Debug 信号的深度

(3) 查看是否写入约束文件

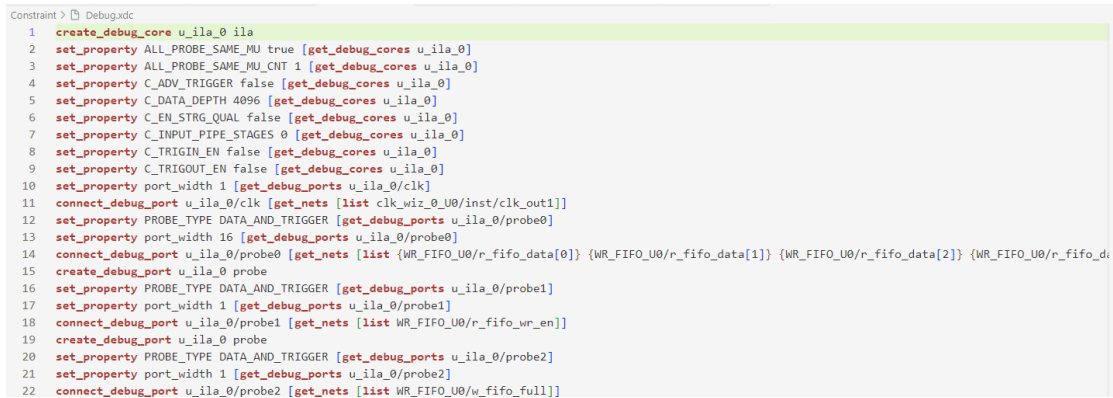


图 9-19 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

6. 添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按键就可以添加时钟约束了，如下图所示：

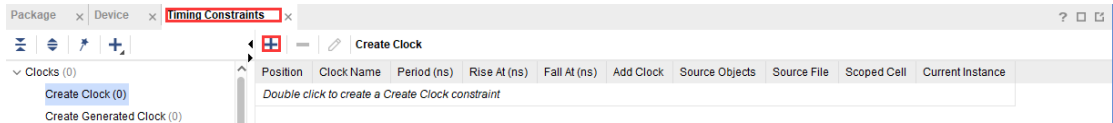


图 9-20 添加时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

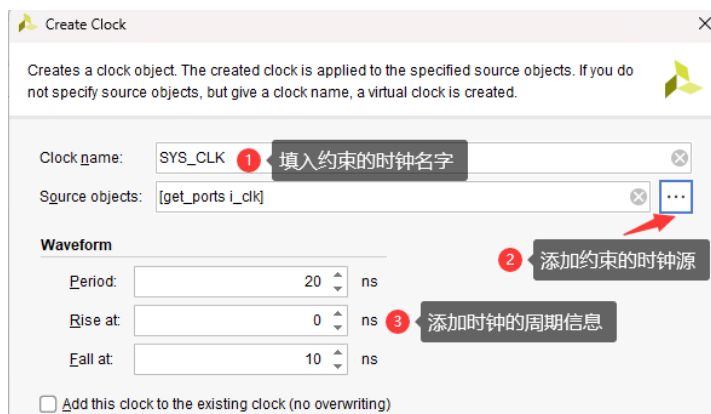


图 9-21 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} -add [get_ports i_clk]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clock [get_nets w_clk_75M]
```

图 9-22 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

7. 添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

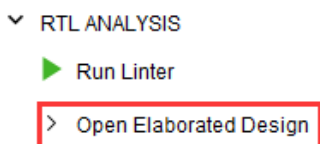


图 9-23 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	IO Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	RL_TERM
All ports (2)													
Scalar ports (2)													
i_clk	IN		W17	✓	33	LVCMOS33*	3.300				NONE	NONE	
i_rst_n	IN		W18	✓	33	LVCMOS33*	3.300				NONE	NONE	

图 9-24 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
set_property IOSTANDARD LVCMOS33 [get_ports i_clk]
set_property IOSTANDARD LVCMOS33 [get_ports i_rst_n]
set_property PACKAGE_PIN W17 [get_ports i_clk]
set_property PACKAGE_PIN W18 [get_ports i_rst_n]
```

图 9-25 约束文件中写入管脚约束

8.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，此次工程中有跨时钟处理，因此在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

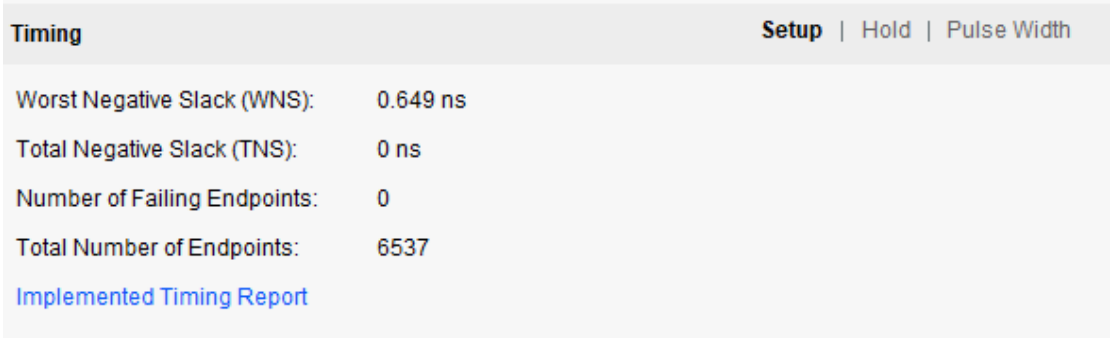


图 9-26 时序结果

将 Bit 下载至 FPGA，随后打开 ILA，随后设置 ila 为 r_fifo_wr_en 上升沿触发，观测 FIFO 写数据，触发结果如下图所示：

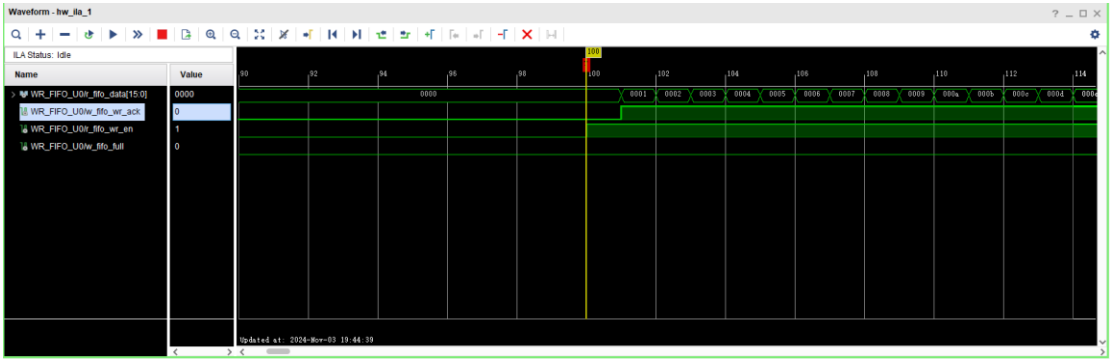
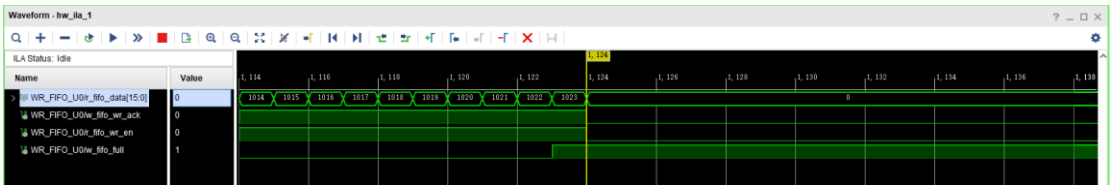


图 9-27 读写有效信号上升沿触发波形



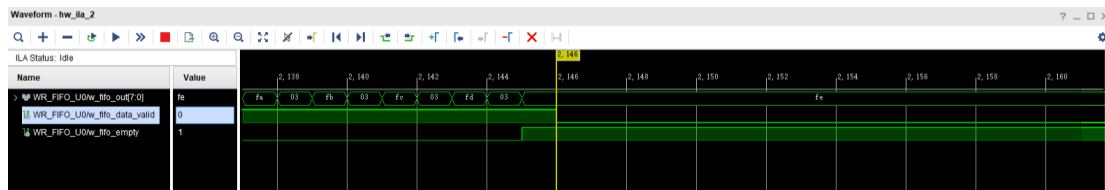


图 9-29 w_fifo_data_valid 信号下降沿触发波形

从图 9-28 可以看出，当 w_fifo_data_valid 信号为高，读出 FIFO 中的数据，同时，从图 9-29 中可以看出，当 Empty 信号拉高时，w_fifo_data_valid 指示最后一个数据为 8'hfe，与仿真结果是一致的，同时将数据进行拼接可以看出最后读出的数据为 1023，这说明异步 FIFO 的实验是成功的。

9.实验总结

本章节主要讲解了 FIFO 的概念和使用场景，并介绍了 Vivado 软件中如何对 FIFO GeneratorIP 核进行配置和调用（例化），主要是采用异步 FIFO 进行位宽转换实验，但此次 FIFO 采用的是 Standard FIFO 模式，大家可以自己尝试下 FWFT FIFO 模式，观察最后输出的结果是否一致。

十. UART 串口通信实验

UART 的英文全称是 Universal Asynchronous Receiver/Transmitter，即通用异步收发器，串口是串行接口的简称，两者组合起来就是通用异步串行通信接口，它包括了 RS232、RS499、RS423、RS422 和 RS485 等接口标准规范和总线标准规范，因此串口广泛应用于嵌入式、工业控制等领域。鉴于串口通信在工业控制领域中广泛使用，虽然在开发板 PL 端没有集成 UART 模块，但可以使用 40 Pin 拓展口外接 USB 转串口模块完成此次实验。

1. UART 串口简介

通信一般分为串行通信 (serial communication) 和并行通信 (parallel communication)。首先需要了解下并行通信的定义，并行通信是指多比特数据同时通过并行线进行传送，一般以字或字节为单位并行进行传输。这种传输方式使用的数据线较多，在通信距离较近的情况下是没有问题的，但当通信速度提升后，会存在数据无法同时到达接收端的情况，因此并行通信一般适合近距离且速率较低的通信，通常传输距离小于 30 米。

串行通信是指数据在一条数据线上，一比特接一比特按顺序进行数据传送，这一点与并行通信是不同的。这里以传输一个字节 (8 bit) 数据为例，在并行通信中，一个字节的的数据是在 8 条并行传输线上同时由源地传送到目的地；而在串行通信中，因为数据是在一条传输线上一位接一位地顺序传送的，所以一个字节的的数据要分 8 次进行传送。若以 T 为一个时间单位的话，那么并行通信发送一个字节的的数据只需要 1T 的时间，而串行通信需要 8T 的时间，由此可以总结出串行通信的特点：一是节省传输线，大大降低了使用成本，二是数据传送速度慢，这一点在大位宽的数据传输上尤为明显。综上所述，串行通信主要应用于长距离、低速率的通信场合，而串口通信则属于串行通信。

串行通信一般有 2 种通信方式：同步串行通信 (synchronized serial communication) 和异步串行通信 (asynchronous serial communication)。同步串行通信需要通信双方在同一时钟的控制下同步传输数据；异步串行通信是指具有不规则数据段传送特性的串行数据传输。在常见的通信总线协议中，I2C、SPI 属于同步通信而 UART 属于异步通信。同步通信的通信双方必须先建立同步，即双方的时钟要调整到同一个频率，收发双方不停地发送和接收连续的同步比特流。异步通信在发送字符时，发送端可以在任意时刻开始发送字符，所以，在 UART 通信中，数据起始位和停止位是必不可少的。

UART 是一种采用异步串行通信方式的通用异步收发传输器 (universal asynchronous receiver-transmitter)，它在发送数据时将并行数据转换成串行数据来传输，在接收数据时将接收到的串行数据转换成并行数据。

UART 串口通信需要两根信号线来实现，一根用于串口发送，另外一根负责串口接收，如下图所示。对于 PC 来说它的 TX 要和对于 FPGA 来说的 RX 连接，同样 PC 的 RX 要和 FPGA 的 TX 连接，如果是两个 TX 或者两个 RX 连接那数据就不能正常被发送出去或者接收到，因为开发板上并没有板载 UART 模块，需要自己外接 USB 转串口模块，因此特别需要注意，将 FPGA 端的 RX 接 USB 转串口模块的 TX 端，TX 接 USB 转串口模块的 RX 端。

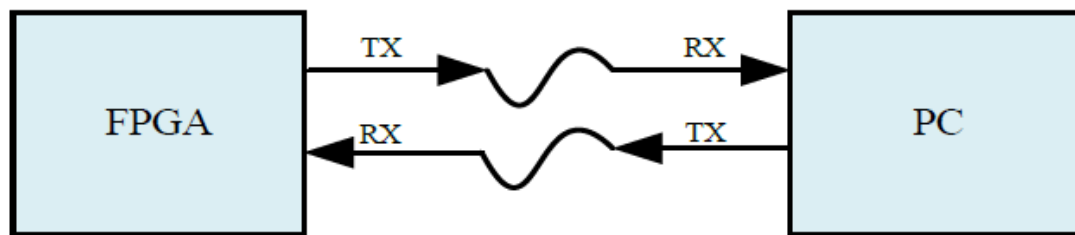


图 10-1 串口与电脑 PC 的连接

UART 在发送或接收过程中的一帧数据由 4 部分组成，起始位、数据位、奇偶校验位和停止位，如下图所示。

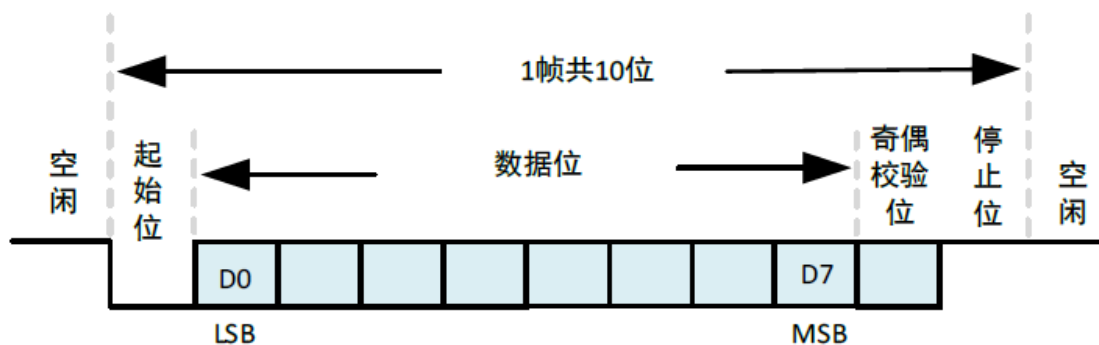


图 10-2 异步串行通信数据格式

起始位：当 UART 不传输数据时，总线通常保持高电压电平。若要开始数据传输，发送 UART 会将传输线从高电平拉到低电平并保持 1 个波特率周期。当接收 UART 检测到从高到低电压变换时，便开始以波特率对应的频率读取数据帧中的位。

数据帧：数据帧中包含所传输的实际数据。如果使用奇偶校验位，数据帧长度可以是 5 位到 8 位。如果不使用奇偶校验位，数据帧长度可以是 9 位。在大多数情况下，数据以最低有效位优先方式发送。

奇偶校验：通过奇偶校验位去判断 UART 在传输期间是否有数据发生改变。这种数据改变通常由电磁辐射、不一致的波特率或长距离数据传输导致的。当 UART 接收完数据帧后，检查总数是偶数还是奇数。如果奇偶校验位为 0（偶数奇偶校验），则数据帧中的 1 或逻辑高位总计应为偶数。如果奇偶校验位为 1（奇数奇偶校验），则数据帧中的 1 或逻辑高位总计应为奇数。当奇偶校验位与数据匹配时，UART 认为传输未出错。但是，如果奇偶校验位为 0，而总和为奇数，或者奇偶校验位为 1，而总和为偶数，则 UART 认为数据帧中的位已改变。

停止位：为了表示数据包结束，发送 UART 将数据传输线从低电平转变为高电平并保持 1 到 2 位时间。

UART 通信过程中的数据格式及传输速率是可设置的，为了正确的通信，收发双方约定并遵循同样的设置。数据位可选择为 5、6、7、8 位，其中 8 位数据位是最常用的，在实际应用中一般都选择 8 位数据位；校验位可选择奇校验、偶校验或者无校验位；停止位可选择 1 位（默认），1.5 或 2 位。串口通信的速率用波特率表示，它表示每秒传输二进制数据的位数，单位是 bps（位/秒），常用的波特率有 9600、19200、38400、57600 以及 115200 等。

那么什么是波特率呢？波特率：即每秒传输的位数(bit)。一般选波特率都会有 9600，19200，115200 等选项，意思就是每秒传输这么多个比特位数(bit)。在信息传输通道中，携带数据信息的信号单元叫作码元（因为串口是 1bit 进行传输的，所以其码元就代表一个二进制数），每秒通过信号传输的码元数称为码元的传输速率，简称“波特率”，常用符号“Baud”

表示，其单位为“波特每秒”（Bps）。串口常见的波特率有 4800、9600、115200 等，此次实验选用 115200 的波特率进行代码编写。

通信信道每秒传输的信息量称为位传输速率，简称“比特率”，其单位为“每秒比特数”（bps）。比特率可由波特率计算得出，公式为比特率=波特率×单个调制状态对应的二进制位数。如果使用的是 115200 的波特率，其串口的比特率为 $115200\text{Bps} \times 1\text{bit} = 115200\text{bps}$ ，由计算得串口发送或者接收 1bit 数据的时间为一个波特，即 $1/115200\text{s}$ 。

在设置好数据格式及传输速率之后，UART 负责完成数据的串并转换，而信号的传输则由外部驱动电路实现。电信号的传输过程有着不同的电平标准和接口规范，针对异步串行通信的接口标准有 RS232、RS422、RS485 等，它们定义了接口不同的电气特性，如 RS-232 是单端输入输出，而 RS-422/485 为差分输入输出等。

关于 UART 的介绍，只介绍到这，若读者想具体了解 UART、RS485 等常用接口的标准可自行了解。

2.实验任务

本节实验任务是上位机通过串口调试助手发送数据给开发板，开发板 PL 端通过 USB_UART 串口模块接收数据并将接收到的数据发送给上位机，完成串口数据环回。

3.程序设计

首先分析下本次设计中都需要哪些功能模块。由实验任务可知，PL 端是通过 USB 转串口模块来接收上位机发出的数据，所以 FPGA 内需要一个串口接收模块（uart_rx），该模块用于将串口接收端口（uart_rxd）上的串行数据解析回并行数据，并将解析完成的并行数据（uart_rx_data）作为模块的输出信号来供其它模块使用。需要注意的是，数据并不是一直有效的，所以为了告诉其它模块何时输出的 uart_rx_data 数据是有效的，还需要输出一个用于表示串口数据接收完成的信号，该信号可以说明当前的输出的 uart_rx_data 数据是完整且有效的，将该信号命名为 uart_rx_done。

还需要一个能将接收的数据发给上位机的串口发送模块（uart_tx），该模块用于将 uart_rx 模块解析完成的并行数据（uart_rx_data）转成串行数据，并通过串口发送端口（uart_txd）发回上位机，所以需要将 uart_rx_data 数据传递给 uart_tx 模块，即 uart_tx 模块需要一个用于接收 uart_rx_data 数据的输入端口，这里将该端口命名为 uart_tx_data，且位宽与 uart_rx_data 相等。除此之外，因为 uart_tx 模块并不需要实时运行，所以需要输入一个使能信号（uart_tx_en）来控制 uart_tx 模块在什么时候工作，又因为需要在 uart_rx 模块解析完上位机发来的数据后，就将该数据发送回上位机，因此可以将 uart_rx 模块输出的 uart_rx_done 信号作为 uart_tx 模块的使能信号。值得一提的是，在一些复杂的工程项目，uart_tx 模块需要发送的数据可能是其它功能模块内部的某个数据，所以为了避免在发送数据的过程中因内部数据发生改变而造成的误码和丢码现象，也通常会让 uart_rx 模块输出一个忙信号（uart_tx_busy），以此来告诉其它功能模块不要在 uart_tx 模块运行期间（即发送数据的过程中）更新需要通过串口发送的数据。

综上，可以绘制出如下所示的系统框图：

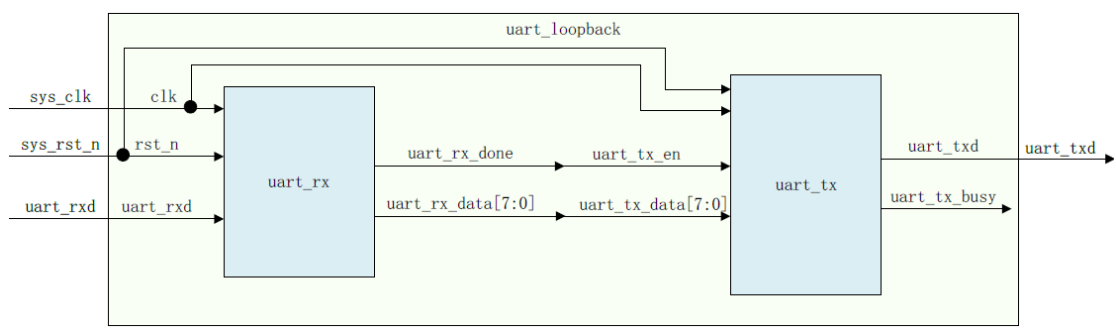


图 10-3 顶层系统框图

3.1 UART_RX 模块

首先介绍串口接收模块的设计，串口接收模块的输入信号主要有系统时钟信号、系统复位信号与串口接收端口。当一帧的数据接收完成后，那么要告诉下级模块已经将一帧数据接收完成了，所以输出为接收数据有效标志和串口接收数据信号。模块接口框图如下所示：

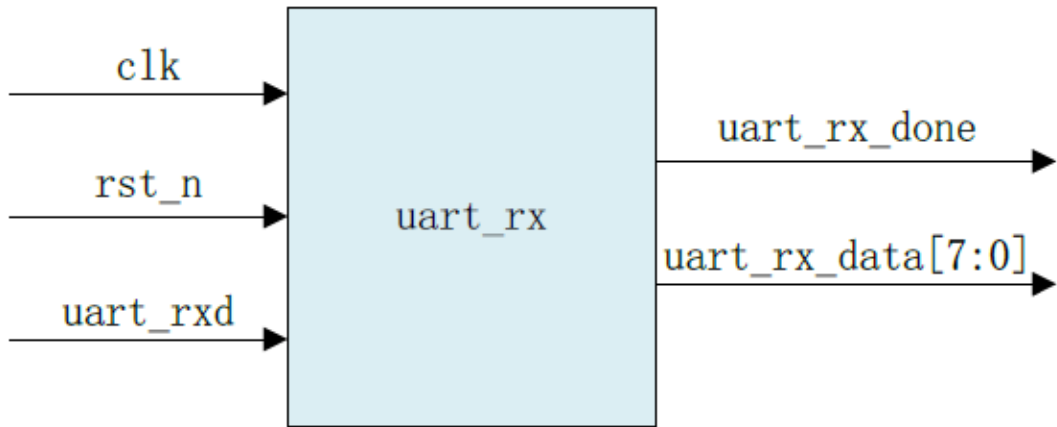


图 10-4 串口接收模块图

编写代码的前提要确定串口通信的数据格式及波特率。在这里以串口比较常用的一种模式，数据位为 8 位，停止位为 1 位，无校验位，波特率为 115200bps。则传输一帧数据的时序图如下图所示：

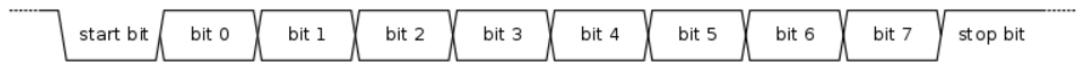


图 10-5 串口通信时序图

如果使用的是 115200 的波特率，其串口的比率为 $115200\text{Bps} \times 1\text{bit} = 115200\text{bps}$ 。由计算得串口发送或者接收 1bit 数据的时间为一个波特，即 $1/115200\text{s}$ ，开发板使用的晶振是 50MHz（周期为 20ns），115200 需要计数的个数为 $\text{cnt} = (1\text{s} \times 10^9\text{ns}) / 115200\text{bit} \times 20\text{ns} \approx 434$ 个系统时钟周期，即每位数据之间的间隔要在 50MHz 的时钟频率下计数 434 次，这样可以采用时钟分频模块，将 50Mhz 的时钟降为串口模块所需要的时钟。

需要注意的是，串口输入的数据信号需要打两拍，降低亚稳态，那么什么是亚稳态呢？亚稳态是由于违背了触发器的建立和保持时间而产生的。寄存器采样需要满足一定的建立时间（setup）和保持时间（holdup），而异步电路没有办法保证建立时间（setup）和保持时间（holdup），所以会出现亚稳态。下图为两个不同时钟的寄存器的示意图，寄存器 B 采样可能采样到寄存器 A 输出的任意状态，包括 Q 的信号跳变沿。

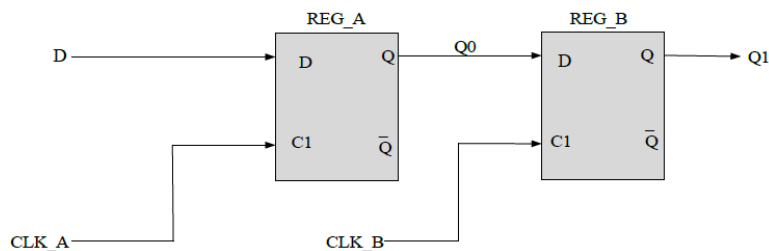


图 10-6 不同时钟的寄存器采样

但是寄存器的 D 端信号需要满足建立时间 T_{su} 和保持时间 T_h 要求, 否则就会出现 Q 端采样到不确定的值的状态, 就是俗称的亚稳态。

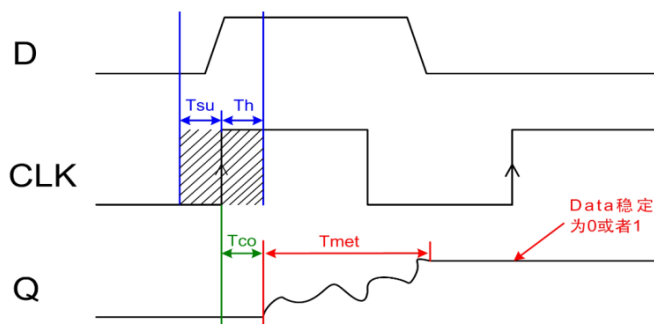


图 10-7 亚稳态时序电路

采样时采样到不同的电平状态 (因为信号跳变沿是一个信号上升的过程, 时钟在信号跳变沿采样会采样到不同的电平), 那么会导致不同决断时间, 决断时间指的是最终逻辑从不确定值到为 0 或者为 1 的持续时间, 也就是无法预测该单元的输出电平, 并且这种不确定的输出电平可以沿信号通道上的各个触发器级联式传播下去。亚稳态会导致这种严重的后果, 那么解决亚稳态有哪些方法呢? 解决亚稳态有以下几种方式:

单 bit 信号: 直接多级寄存器同步法, 一般采用 2-3 级寄存器进行同步处理, 这个 2-3 级寄存器也称作同步器, 在 ASIC 设计中, 一般都有提供专用的同步器库, 因为同步器要求多级寄存器位置靠的越近越好, 靠的越近, 亚稳态消失的概率就越大。FPGA 设计中, 直接使用 2-3 级寄存器进行同步处理即可。

多 bit 信号: 异步 FIFO 或者使用多次握手同步方法。在握手协议中, 异步的 REQ/ACK 也需要使用单 bit 同步技术进行同步处理, 异步 FIFO 也是如此。

而串口为单 bit 信号, 那么采用多级寄存器同步法, 可降低在跨时钟时发生亚稳态的概率, 那么接下来则进行代码编写:

(1) 将当前输入的信号打两拍, 进行信号同步, 如下所示:

// 针对异步信号的同步处理

```
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        uart_rxd_d0 <= 1'b0;
        uart_rxd_d1 <= 1'b0;
        uart_rxd_d2 <= 1'b0;
    end
    else begin
        uart_rxd_d0 <= uart_rxd;
        uart_rxd_d1 <= uart_rxd_d0;
```

```

        uart_rxd_d2 <= uart_rxd_d1;
    end
end

```

(2) 通过检测串口接收端 `uart_rxd` 的下降沿来捕获起始位。一旦检测到起始位，输出一个时钟周期的脉冲 `start_en`，并进入串口接收过程。串口接收状态用 `rx_flag` 来标志，`rx_flag` 为高标志着串口接收过程正在进行，此时启动系统时钟计数器 `clk_cnt` 与接收数据计数器 `rx_cnt`。

// 捕获接收端口下降沿(起始位)，得到一个时钟周期的脉冲信号

```
assign start_en = uart_rxd_d2 & (~uart_rxd_d1) & (~rx_flag);
```

// 给接收标志赋值

```
always @(posedge clk or negedge rst_n) begin
```

```
    if(!rst_n)
```

```
        rx_flag <= 1'b0;
```

```
    else if(start_en)    // 检测到起始位
```

```
        rx_flag <= 1'b1; // 接收过程中，标志信号 rx_flag 拉高
```

```
    // 在停止位一半的时候，即接收过程结束，标志信号 rx_flag 拉低
```

```
    else if((rx_cnt == 4'd9) && (baud_cnt == BAUD_CNT_MAX/2 - 1'b1))
```

```
        rx_flag <= 1'b0;
```

```
    else
```

```
        rx_flag <= rx_flag;
```

```
end
```

// 波特率的计数器赋值

```
always @(posedge clk or negedge rst_n) begin
```

```
    if(!rst_n)
```

```
        baud_cnt <= 16'd0;
```

```
    else if(rx_flag) begin    // 处于接收过程时，波特率计数器 (baud_cnt) 进行循环计数
```

```
        if(baud_cnt < BAUD_CNT_MAX - 1'b1)
```

```
            baud_cnt <= baud_cnt + 16'b1;
```

```
        else
```

```
            baud_cnt <= 16'd0; // 计数达到一个波特率周期后清零
```

```
    end
```

```
    else
```

```
        baud_cnt <= 16'd0;    // 接收过程结束时计数器清零
```

```
end
```

// 对接收数据计数器 (rx_cnt) 进行赋值

```
always @(posedge clk or negedge rst_n) begin
```

```
    if(!rst_n)
```

```
        rx_cnt <= 4'd0;
```

```

else if(rx_flag) begin                                //处于接收过程时rx_cnt 才进行计
数
    if(baud_cnt == BAUD_CNT_MAX - 1'b1) //当波特率计数器计数到一个波特
率周期时
        rx_cnt <= rx_cnt + 1'b1;                //接收数据计数器加1
    else
        rx_cnt <= rx_cnt;
    end
else
    rx_cnt <= 4'd0;                                //接收过程结束时计数器清零
end

```

上述代码中的 BAUD_CNT_MAX=CLK_FREQ/UART_BPS 意思为:BAUD_CNT_MAX 为当前波特率下, 串口传输一位所需要的系统时钟周期个数。因此 baud_cnt 从零计数到 BAUD_CNT_MAX-1 时, 串口刚好完成一位数据的传输。由于接收数据计数器 rx_cnt 在每次 baud_cnt 计数到 BAUD_CNT_MAX-1 时加 1, 因此由 rx_cnt 的值可以判断串口当前传输的是第几位数据。

(3) 随后依据接收计数器, 对接收的数据进行串并转换, 转换为并行数据, 由于串口发送形式是低位在前, 高位在后, 因此进行串转并时, 需要将接收的串行数据从低位开始拼数据, 直到 8 个时钟周期后, 拼成一个并行数据。

```

//根据 rx_cnt 来寄存 rxd 端口的数据
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        rx_data_t <= 8'b0;
    else if(rx_flag) begin                            //系统处于接收过程时
        if(baud_cnt == BAUD_CNT_MAX/2 - 1'b1) begin //判断 baud_cnt 是否
计数到数据位的中间
            case(rx_cnt)
                4'd1 : rx_data_t[0] <= uart_rxd_d2; //寄存数据的最低位
                4'd2 : rx_data_t[1] <= uart_rxd_d2;
                4'd3 : rx_data_t[2] <= uart_rxd_d2;
                4'd4 : rx_data_t[3] <= uart_rxd_d2;
                4'd5 : rx_data_t[4] <= uart_rxd_d2;
                4'd6 : rx_data_t[5] <= uart_rxd_d2;
                4'd7 : rx_data_t[6] <= uart_rxd_d2;
                4'd8 : rx_data_t[7] <= uart_rxd_d2; //寄存数据的高低位
                default : ;
            endcase
        end
    else
        rx_data_t <= rx_data_t;
    end
else

```

```

        rx_data_t <= 8'b0;
    end

```

(4) 最后当接收完成后，拉高一个 done 信号，并对数据进行赋值。

//给接收完成信号和接收到的数据赋值

```

always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        uart_rx_done <= 1'b0;
        uart_rx_data <= 8'b0;
    end
    //当接收数据计数器计数到停止位，且 baud_cnt 计数到停止位的中间时
    else if(rx_cnt == 4'd9 && baud_cnt == BAUD_CNT_MAX/2 - 1'b1) begin
        uart_rx_done <= 1'b1 ; //拉高接收完成信号
        uart_rx_data <= rx_data_t; //并对 UART 接收到的数据进行赋值
    end
    else begin
        uart_rx_done <= 1'b0;
        uart_rx_data <= uart_rx_data;
    end
end
end

```

3.2 UART_TX 模块

串口发送模块与串口接收模块异曲同工，串口接收模块是进行串转并，串口发送模块是并转串，串口发送模块的端口如下图所示：

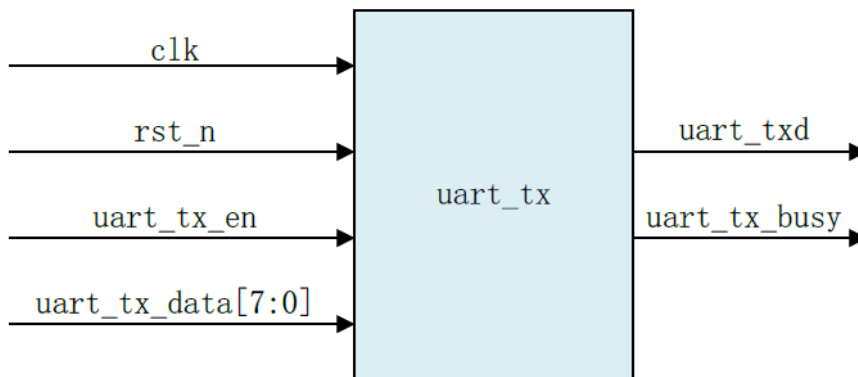


图 10-8 串口发送模块图

因此串口发送模块也需要一个 16 位的系统时钟计数器 (baud_cnt) 和 4 位的发送数据计数器 (tx_cnt)。当串口发送模块接收到串口接收模块发送过来的高电平发送使能 (uart_tx_en) 时，拉高发送忙状态标志 (uart_tx_busy) 同时寄存待发送的数据 (tx_data_t)。在整个发送过程中发送忙状态标志保持高电平，tx_cnt 对串口数据进行计数，同时 tx_data_t 的各个数据位依次通过串口发送端 uart_txd 发送出去。当 tx_cnt 计数到 9 时，串口数据发送完成，开始发送停止位。在一个波特率周期的停止位发送完成后，串口发送过程结束，uart_tx_busy 信号拉低，表明串口发送模块进入空闲状态。

(1) 首先，当发送模块接收到接收模块发送的 EN 信号，则拉高 Busy 信号，指示当前模

块正在发送数据。

```
//当uart_tx_en 为高时，寄存输入的并行数据，并拉高BUSY 信号
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        tx_data_t <= 8'b0;
        uart_tx_busy <= 1'b0;
    end
    //发送使能时，寄存要发送的数据，并拉高BUSY 信号
    else if(uart_tx_en) begin
        tx_data_t <= uart_tx_data;
        uart_tx_busy <= 1'b1;
    end
    //当计数到停止位结束时，停止发送过程
    else if(tx_cnt == 4'd9 && baud_cnt == BAUD_CNT_MAX - 1) begin
        tx_data_t <= 8'b0; //清空发送数据寄存器
        uart_tx_busy <= 1'b0; //并拉低BUSY 信号
    end
    else begin
        tx_data_t <= tx_data_t;
        uart_tx_busy <= uart_tx_busy;
    end
end
end
```

(2) 当处于发送状态时对发送数据计数器进行计数。

```
//tx_cnt 进行赋值
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        tx_cnt <= 4'd0;
    else if(uart_tx_en)
        tx_cnt <= 16'd0;
    else if(uart_tx_busy) begin //处于发送过程时tx_cnt 才进行计数
        if(baud_cnt == BAUD_CNT_MAX - 1'b1) //当波特率计数器计数到一个波特率周期时
            tx_cnt <= tx_cnt + 1'b1; //发送数据计数器加1
        else
            tx_cnt <= tx_cnt;
    end
    else
        tx_cnt <= 4'd0; //发送过程结束时计数器清零
end
```

(3) 最后，在发送状态下，依据发送计数器对串口输出进行赋值。

```
//根据tx_cnt 来给uart 发送端口赋值
```

```

always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        uart_txd <= 1'b1;
    else if(uart_tx_busy) begin
        case(tx_cnt)
            4'd0 : uart_txd <= 1'b0 ; //起始位
            4'd1 : uart_txd <= tx_data_t[0]; //数据位最低位
            4'd2 : uart_txd <= tx_data_t[1];
            4'd3 : uart_txd <= tx_data_t[2];
            4'd4 : uart_txd <= tx_data_t[3];
            4'd5 : uart_txd <= tx_data_t[4];
            4'd6 : uart_txd <= tx_data_t[5];
            4'd7 : uart_txd <= tx_data_t[6];
            4'd8 : uart_txd <= tx_data_t[7]; //数据位最高位
            4'd9 : uart_txd <= 1'b1 ; //停止位
            default : uart_txd <= 1'b1;
        endcase
    end
    else
        uart_txd <= 1'b1; //空闲时发送端口为高电平
end

```

4.UART 驱动仿真及仿真结果

(1) 对于串口的驱动，产生一个时钟，复位及自增数据，仿真代码如下：

```

//parameter define
parameter CLK_PERIOD = 20; //时钟周期为 20ns

//reg define
reg sys_clk ; //时钟信号
reg sys_rst_n; //复位信号
reg uart_rxd ; //UART 接收端口

//wire define
wire uart_txd ; //UART 发送端口

//*****
//** main code
//*****

//发送 8'h55 8'b0101_0101

```

```

initial begin
    sys_clk <= 1'b0;
    sys_rst_n <= 1'b0;
    uart_rxd <= 1'b1;
    #200
    sys_rst_n <= 1'b1;
    #1000
    uart_rxd <= 1'b0;    //起始位
    #8680
    uart_rxd <= 1'b1;    //D0
    #8680
    uart_rxd <= 1'b0;    //D1
    #8680
    uart_rxd <= 1'b1;    //D2
    #8680
    uart_rxd <= 1'b0;    //D3
    #8680
    uart_rxd <= 1'b1;    //D4
    #8680
    uart_rxd <= 1'b0;    //D5
    #8680
    uart_rxd <= 1'b1;    //D6
    #8680
    uart_rxd <= 1'b0;    //D7
    #8680
    uart_rxd <= 1'b1;    //停止位
    #8680
    uart_rxd <= 1'b1;    //空闲状态
end

//50Mhz 的时钟，周期则为1/50Mhz=20ns，所以每10ns，电平取反一次
always #(CLK_PERIOD/2) sys_clk = ~sys_clk;

//例化顶层模块
uart_top uart_top_u0(
    .sys_clk      (sys_clk ),
    .sys_rst_n    (sys_rst_n),
    .uart_rxd     (uart_rxd ),
    .uart_txd     (uart_txd )
);

```

复位拉高后，再经过 1000ns 后，将 uart_rxd 拉低，表示已经开始发送起始位了，经过 8680ns (434*20ns=8680，这个是 115200 波特率的一个波特率周期) 后，发送 bit0 位，循环

8 次后，8bit 数据已经全部发送完成了，后续就是开始发送停止位

(2) uart_rx 模块仿真结果如下所示：

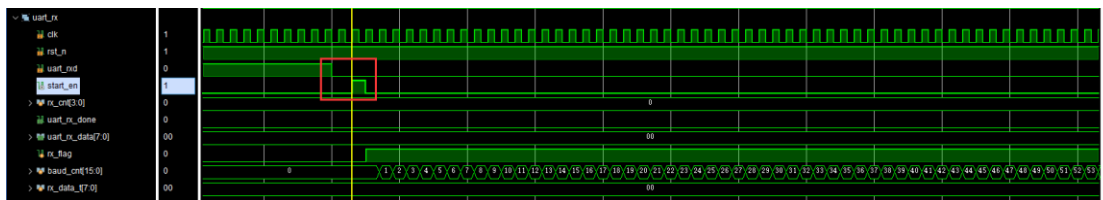


图 10-9 uart_rx 模块

从图 10-9 中可以看出，当 uart_rxd 信号拉低时，start_en 信号线拉高，表示当前开始接收数据，查看的接收数据计数器发现当接收 8 个数据后拉高 done 信号，如下图所示：

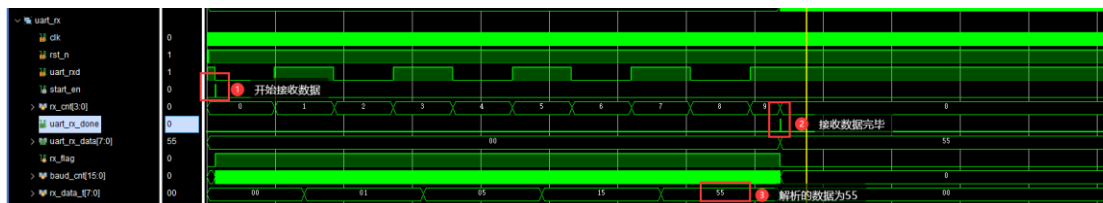


图 10-10 整体接收模块

从图 10-10 中可以看出，当 uart_rx_done 信号拉高时，表示一帧数据接收完毕，此时解析的数据为 55，与仿真数据一致。

(3) 当 uart_rx_done 信号拉高时，uart_tx 模块则开始发送，仿真结果如下所示：

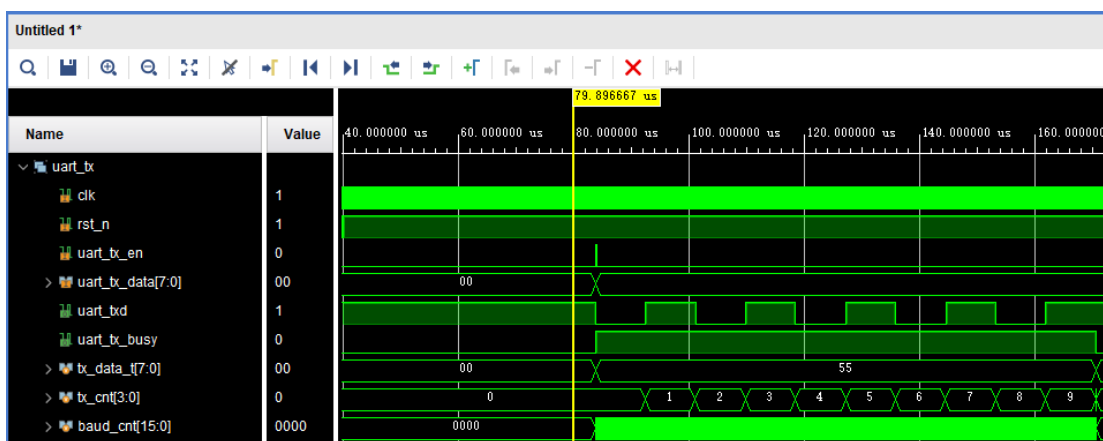


图 10-11 串口发送模块中发送数据

当 uart_tx_en 信号为高时，串口 tx 信号会拉低，表示当前开始发送数据，随后拉高 uart_tx_busy 信号，表示当前进行发送数据状态，发送 8 个数据后，则拉高 uart_txd 信号线，表示一帧数据发送结束。

发送 8'd1 的最低位为 1，随后发送 0，由于当前校验模式位奇校验，发送的数据为 8'b01，刚好为奇数个 1，那么最后校验添 0，如图 10-10 中红框处所示，随后拉高 uart_tx 表示一帧数据发送结束。

5. 添加内部逻辑分析仪

(1) 在 UART 环回的实验中，主要分析 UART 两条串行数据线上的数据，以及串并转换后的数据，添加信号标识如下图所示：

```

module uart_top
(
    input          sys_clk ,    //外部50MHz时钟
    input          sys_rst_n,   //系外部复位信号, 低有效

    //UART端口
    input          uart_rxd ,    //UART接收端口
    output         uart_txd     //UART发送端口
);

//parameter define
parameter CLK_FREQ = 50000000; //定义系统时钟频率
parameter UART_BPS = 115200 ;   //定义串口波特率

//wire define
(* mark_debug = "true" *)
wire          uart_rx_done;     //UART接收完成信号
(* mark_debug = "true" *)
wire [7:0]    uart_rx_data;     //UART接收数据

```

图 10-12 添加 Debug 信号

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。

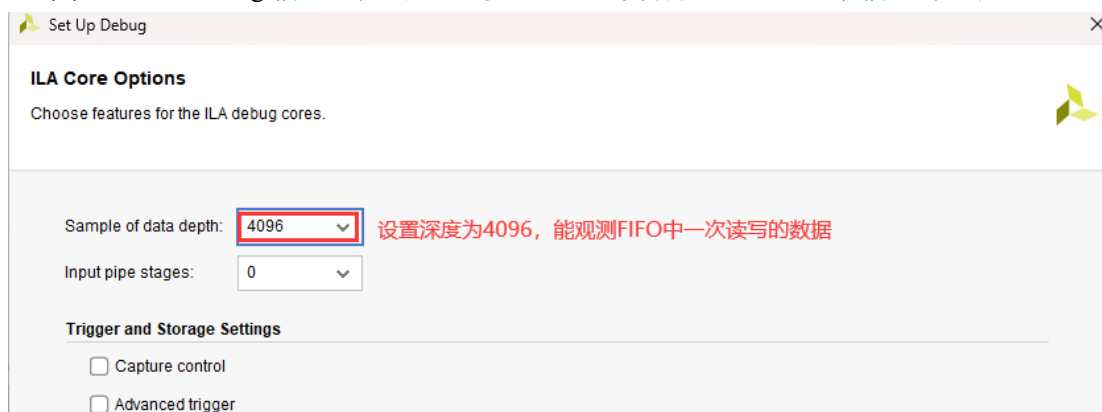


图 10-13 设置 Debug 信号的深度

(3) 查看是否写入约束文件

```
create_debug_core u_ila_0 ila
set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
set_property port_width 1 [get_debug_ports u_ila_0/clk]
connect_debug_port u_ila_0/clk [get_nets [list sys_clk_IBUF_BUF]]
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
set_property port_width 8 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets [list {uart_rx_data[0]} {uart_rx_data[1]} {uart_rx_data[2]} {uart_rx_data[3]} {uart_rx_data[4]} {uart_rx_data[5]} {uart_rx_data[6]} {uart_rx_data[7]}]]
create_debug_port u_ila_0 probe
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
set_property port_width 1 [get_debug_ports u_ila_0/probe1]
connect_debug_port u_ila_0/probe1 [get_nets [list uart_rx_done]]
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets sys_clk_IBUF_BUF]]
```

图 10-14 约束文件写入 Debug 信息
Debug 信号添加完毕，随后可以进行时序约束

6.添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按键就可以添加时钟约束了，如下图所示：



图 10-15 添加主时钟时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

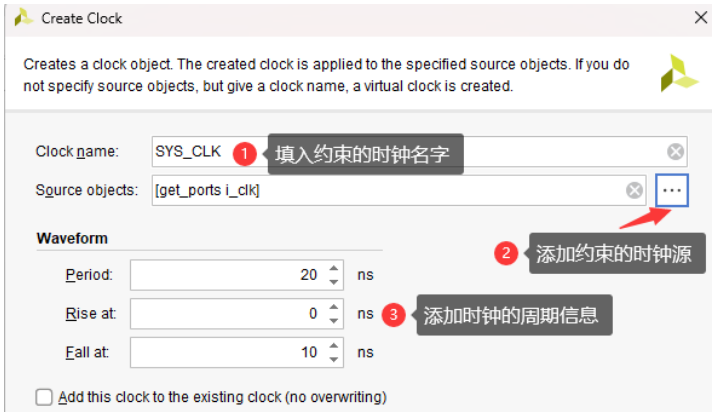


图 10-16 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
Constraint > Timing_Constraint.xdc
1 create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} [get_ports sys_clk]
2
```

图 10-17 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

7.添加管脚约束

在上述步骤中添加了逻辑分析仪、时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

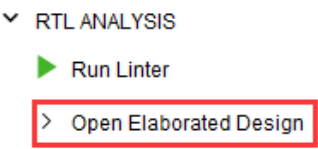


图 10-18 打开管脚约束

(2) 管脚约束，如下图所示：

The screenshot shows the 'IO Ports' window with a table of pin constraints. The table has columns: Name, Direction, Neg Diff Pair, Package Pin, Fixed, Bank, I/O Std, Vcco, Vref, Drive Strength, Slew Type, Pull Type, Off-Chip Termination, and IN_TERM.

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
▼ All ports (3)													
▼ Scalar ports (3)													
i_clk	IN		W17	☒	33	LV	3.300				NONE	NONE	
i_uart_rx	IN		V12	☒	13	LV	3.300				NONE	NONE	
o_uart_tx	OUT		W12	☒	13	LV	3.300		12		NONE	FP_VTT_50	

图 10-19 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
1  # 主时钟
2  set_property PACKAGE_PIN W17 [get_ports i_clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports i_clk]
4
5  # 串口管脚
6  set_property PACKAGE_PIN W12 [get_ports o_uart_tx]
7  set_property IOSTANDARD LVCMOS33 [get_ports o_uart_tx]
8
9  set_property PACKAGE_PIN V12 [get_ports i_uart_rx]
10 set_property IOSTANDARD LVCMOS33 [get_ports i_uart_rx]
```

图 10-20 约束文件中写入管脚约束

8.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

Timing		Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	13.949 ns			
Total Negative Slack (TNS):	0 ns			
Number of Failing Endpoints:	0			
Total Number of Endpoints:	3731			
Implemented Timing Report				

图 10-21 时序结果

由于开发板上并没有在 PL 端集成串口模块，因此，笔者采用外接 USB 转串口的方式进行此次实验的上板验证，外设连接图如下：

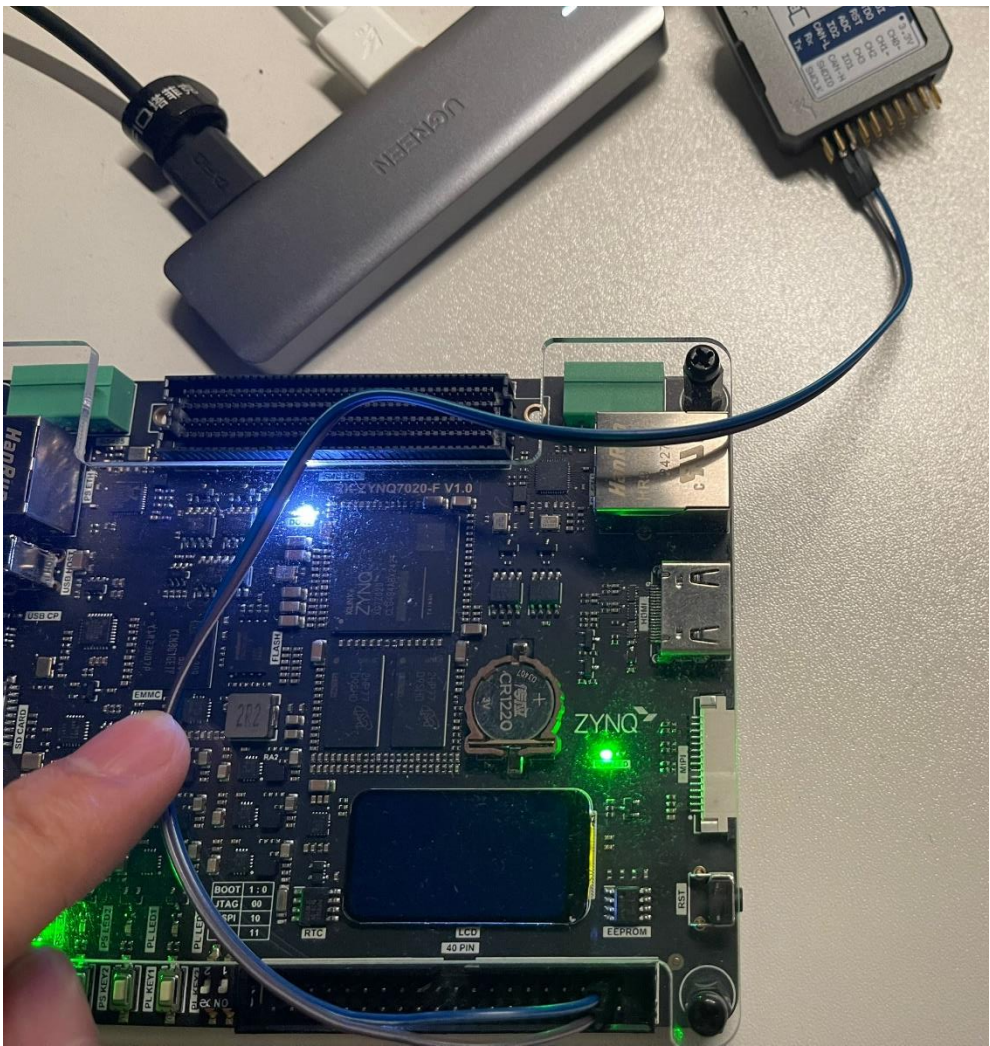


图 10-22 外设连接图

图中 USB 转串口的 RX 接口（蓝色杜邦线）连接的是 FPGA 顶层的 TX 端口，TX 接口（玫红色杜邦线）连接的是 FPGA 顶层模块的 RX 端口。

随后打开串口助手，连接至外部的串口模块的端口，随后发送“RK-ZYNQ7020-F”，查看串口助手的返回的数据效果。当设置为发送数据 8bit，1 位停止位及无校验，波特率 115200 时，串口环回效果如下：

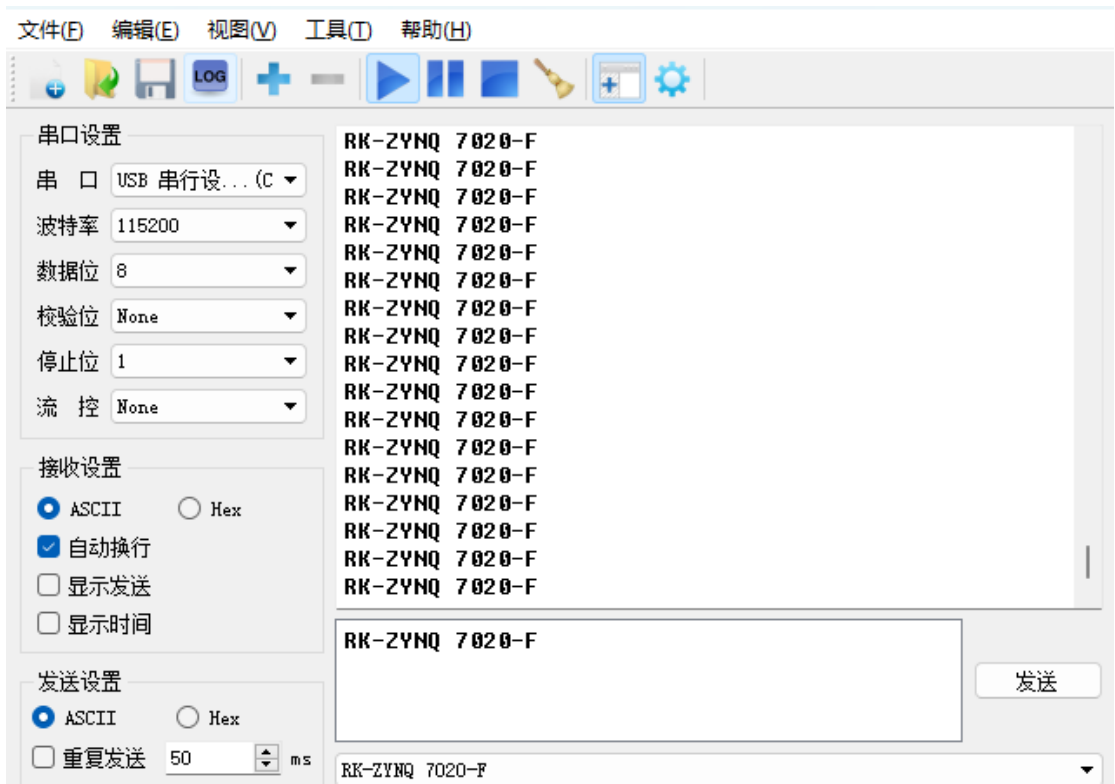


图 10-23 串口环回测试图。

9.实验总结

到这里，本章节关于串口的基本知识就讲解完毕了，通过实验，相信读者对于串口传输的基本知识和概念已经了解。通过本次实验大家需要了解与掌握以下几个知识点：

- a.了解亚稳态产生的原因以及怎么消除亚稳态；
- b.掌握串并转换，因为串并转换是接口中很常用的一种方法。

十一. RS-485 串口通信实验

RS-485 是针对 UART 串口的一种接口标准，它定义了串行通信系统中发送器和接收器的一系列电气特性。相比于 RS-232，RS-485 标准的通信系统抗干扰能力较强，可实现长距离的数据传输，同时支持多个收发器连接到同一个通信网络中。因此，RS-485 在工业控制领域以及有类似需求的系统中得到了广泛的应用。

1.RS485 简介

在“UART 串口通信实验”章节我们介绍了 UART 串口通，实际上，RS-232、RS-422 和 RS-485 也都是常用的串行通信接口标准，它们定义了接口不同的电气特性，如 RS-232 是单端输入输出，而 RS-422/485 为差分输入输出等。

在介绍 RS-485 之前，先来了解一下串口通信过程中单端传输与差分传输的差别。单端传输是指在发送或接收过程中，用信号线对地线的电压值来表示逻辑“0”和“1”。而差分传输使用两根信号线来传输一路信号，这两根信号线上传输的信号幅值相等，极性相反，用它们的差值来表示逻辑“0”和“1”，如下图所示。

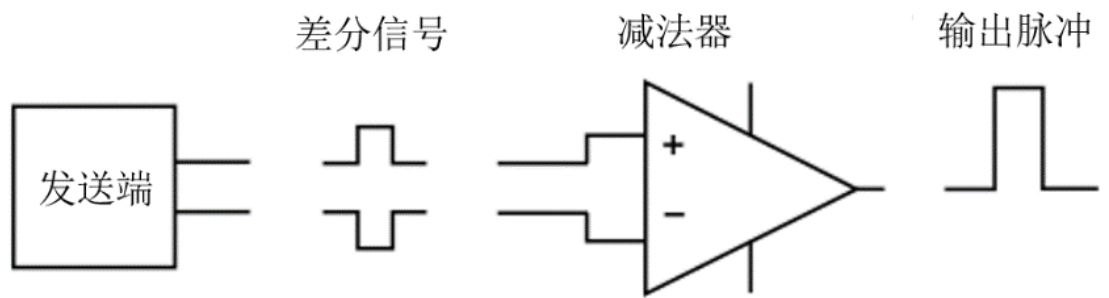


图 11-1 差分传输方式

在传输过程中，当信号线上叠加了频率、幅值和相位都相同的干扰信号时（共模干扰），对于单端传输而言，由于地线电位为 0，则传输的信号就包含了干扰信号；而在差分传输方式下，干扰可以通过两个信号线上电压的差值抵消，相当于抑制了共模干扰，如下图所示。因此相对于单端传输方式，差分传输大大提高了信号在传输过程中的抗干扰能力，但是需要多余的信号线来实现信号传输。

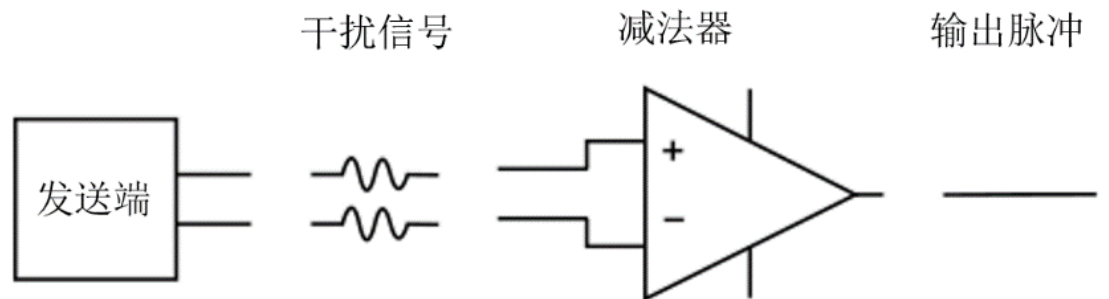


图 11-2 差分传输抑制共模干扰

RS-232 接口标准出现较早，信号采用负逻辑电平、单端传输方式工作。通过一根信号线发送，一根信号线接收，加上一根地线，RS-232 可实现全双工通信。由于单端传输方式抗干扰能力差，导致 RS-232 标准通信距离短（小于 15 米），数据传输速率低等问题。另外

RS-232 仅支持一对一通信，存在无法实现多个设备互联的缺点。

RS-422 由 RS-232 发展而来，它是为弥补 RS-232 的不足而提出的。RS-422 采用差分传输（又称平衡传输）方式，将最大传输速率提高到 10Mbps（40 英尺内，约 12.2 米）；当传输速率在 100kbps 以下时，传输距离可达 4000 英尺（约 1219 米）。由于采用差分传输方式，RS-422 需要 4 根信号线来实现全双工通信，两根用于发送、两根用于接收。RS-422 允许在一条传输总线上连接最多 10 个接收器，从而实现单个设备发送，多个设备接收的功能。

为扩展应用范围，在 RS-422 基础上又制定了 RS-485 标准。RS-485 同样采用差分传输方式，但是 RS-485 只有 2 根信号线（A 和 B），由于发送和接收共用一组信号线，因此发送和接收不能同时进行，只能实现半双工通信。RS-485 增加了多点、双向通信能力，允许多达 128 个收发器连接到同一条总线上，各设备通过使能信号控制发送和接收过程。

2.RS485 硬件设计

板载的 RS485 的通信芯片是直接连接到开发板 PL 端管脚，电路图如下图所示：

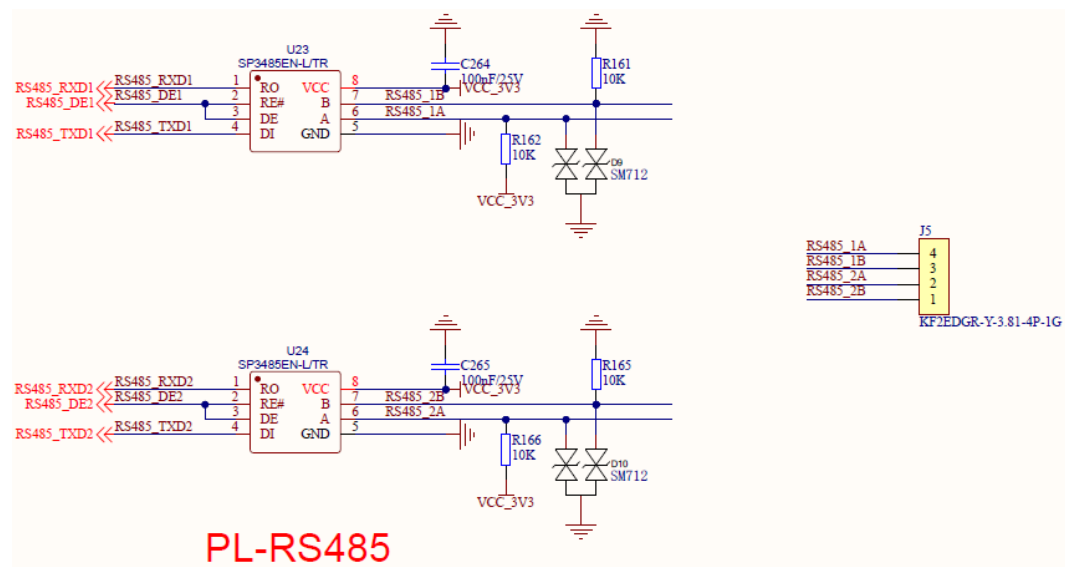


图 11-3 RS485 通信电路

来看下用于实现 RS485 的电路设计,由于 ZYNQ PL 侧串口输入输出引脚为 TTL 电平,用 3.3V 代表逻辑“1”,0V 代表逻辑“0”;而 RS485 电平标准采用差分信号的差值电压来代表逻辑“0”和“1”。因此当 FPGA 与 RS485 接口标准的设备通信时，需要使用电平转换芯片 SP3485 来实现 RS485 电平与 TTL 电平的转换。其原理图如图 11-3 所示。

SP3485 各引脚含义如下表所示：

表 11-1 SP3485 芯片引脚与功能描述

引脚号	引脚名	引脚说明
1	RO	发送的数据
2	RE	数据接收使能，低电平有效
3	DE	数据发送使能，高电平有效
4	DI	接收的数据
5	GND	地
6	A	差分传输线+（正向）
7	B	差分传输线-（反向）

SP3485 的逻辑电平由 A 和 B 的电压差值表示，当 $A-B > +0.2V$ 时，表示逻辑 1；当 $A-B < -0.2V$ 时，表示逻辑 0。在图 28.2.3 中有三个电阻，R1、R3 和 R4，可简化为如下所示的电路图：

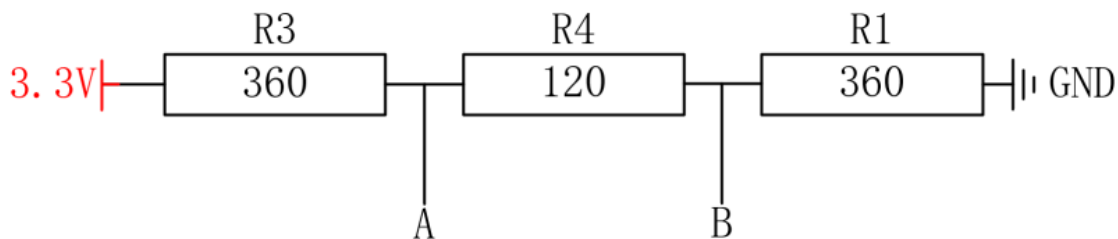


图 11-4 A 和 B 引脚的电路图

如上图所示的三个电阻在设计中一般要让 R1 和 R3 阻值相同，中间的电阻 R4 要确保占有 0.2V 以上的电压，即 A 线和 B 线之间的电压差大于 0.2V。由图 11-4 可知，三个电阻的阻值比为 3: 1: 3，即 3.3V 可以均分成 7 份，由此可计算得出 A 线的电压为 $(3.3V \times 4) / 7 = 1.88V$ ，R4 所占电压为 $3.3V / 7 = 0.47V$ ，B 线的电压为 $(3.3V \times 3) / 7 = 1.41V$ 。

由于 RS-485 为半双工通信方式，需要通过使能信号来控制发送和接收过程。在图 11-3 中，电平转换芯片 SP3485 的 2 号引脚为接收使能（低电平有效），3 号引脚为发送使能（高电平有效）。因为这两个使能信号线的有效值相反，所以可以将两个引脚连接在一起，这样只需要通过一个信号 RS485_DE 即可控制收发过程：当 RS485_DE 为高电平时，SP3485 处于发送过程；当 RS485_DE 为低电平时，SP3485 处于接收过程。

3. 实验任务

本质上 485 通信也是串口通信，因此本节的实验任务与串口通信是一致的通过上位机的串口调试助手发送数据通过 485 转换芯片发送数据至开发板，开发板 PL 端再通过 485 电平芯片将接收到的数据发送给上位机，完成数据环回。

4. 程序设计

由于 RS485 是半双工传输，那么就需要制定传输协议进行握手，设定第一个字节为 8'h55，表示一帧数据的开始，接下来是传输的数据长度信息，由于 FIFO 大小限制（256），范围为 1~255，接下来则是数据，通信格式即为：起始 8'h55+数据长度+数据。

(1) RS485 的接收部分是串口通信的接收部分，这部分代码与串口通信中的接收部分是相似的，只是将接收部分代码采用状态机进行编写，从状态机中的控制可以看出，该接收的数据格式是和串口接收的数据格式是一致的，该部分状态机控制部分如下：

```
always@(*)
begin
    case(state)
        S_IDLE:
            if(rx_negedge)
                next_state <= S_START;
            else
                next_state <= S_IDLE;
```

```

    S_START:
        if(cycle_cnt == CYCLE - 1) //one data cycle
            next_state <= S_REC_BYTE;
        else
            next_state <= S_START;
    S_REC_BYTE:
        if(cycle_cnt == CYCLE - 1 && bit_cnt == 3'd7) //receive
8bit data
            next_state <= S_STOP;
        else
            next_state <= S_REC_BYTE;
    S_STOP:
        if(cycle_cnt == CYCLE/2 - 1) //half bit cycle,to avoid
missing the next byte receiver
            next_state <= S_DATA;
        else
            next_state <= S_STOP;
    S_DATA:
        if(rx_data_ready) //data receive complete
            next_state <= S_IDLE;
        else
            next_state <= S_DATA;
    default:
        next_state <= S_IDLE;
endcase
end

```

(2) RS485 的发送部分是串口通信的发送部分，这部分代码与串口通信中的发送模块是相似的，只是将发送模块采用状态机进行编写，从状态机中的控制可以看出，该发送模块的数据格式是和串口的发送数据格式是一致的，该部分状态机控制部分如下：

```

always@(*)
begin
    case(state)
        S_IDLE:
            if(tx_data_valid == 1'b1)
                next_state <= S_START;
            else
                next_state <= S_IDLE;
        S_START:
            if(cycle_cnt == CYCLE - 1)
                next_state <= S_SEND_BYTE;
            else
                next_state <= S_START;
    endcase
end

```

```

S_SEND_BYTE:
    if(cycle_cnt == CYCLE - 1 && bit_cnt == 3'd7)
        next_state <= S_STOP;
    else
        next_state <= S_SEND_BYTE;
S_STOP:
    if(cycle_cnt == CYCLE - 1)
        next_state <= S_IDLE;
    else
        next_state <= S_STOP;
default:
    next_state <= S_IDLE;
endcase
end

```

(3) 设计的功能为初始状态下将 DE 设为 0，也就是输入，等待接收上位机发来的数据，并缓存到 FIFO 中，FIFO 大小设置为 256，然后切换 DE 为 1，也就是输出，把接收到的数据从 FIFO 中读出并发送出去。注意缓存的数据是除去起始 8'h55 和数量信息的。

当接收到数据时，会判断第一个数据是否位 8'h55，若位 8'h55 则跳转状态机。

```

RCV_HEAD:
    begin
        if (rx_data_valid == 1'b1 && rx_data == 8'h55) //when
received data is valid and data is 8'h55,it means the start of data
            state <= RCV_COUNT ;
    end

```

(4) 在 RCV_COUNT 状态时，如果数据长度小于 0，则跳转到 IDLE 状态，如果大于 0，则进入接收数据状态。

```

RCV_COUNT:
    begin
        if (rx_data_valid == 1'b1) //record data counter
            begin
                if (rx_data > 0)
                    state <= RCV_DATA ;
                else
                    state <= IDLE ;

                data_count <= rx_data ;
            end
    end

```

(5) 在 RCV_DATA 状态下，把数据写入 FIFO，并且检查数据长度，切换 RS485 的方向为输出，并跳转状态。

```

RCV_DATA:
    begin

```

```

        fifo_wren <= rx_data_valid ;
        fifo_wdata <= rx_data ;

        if (rx_data_valid == 1'b1)
        begin
            if (rx_cnt == data_count - 1)
            begin
                rx_cnt      <= 8'd0 ;
                rs485_de     <= 1'b1 ;
                state        <= WAIT ;
            end
            else
                rx_cnt <= rx_cnt + 1'b1 ;
            end
        end
    end
end

```

(6) 在切换总线状态时，为了可靠工作在 WAIT 状态下，延时 1ms 进行方向切换。

```

WAIT:
    begin
        fifo_wren <= 1'b0 ;
        if (wait_cnt >= CLK_FRE * 1000)
        begin
            wait_cnt <= 32'd0 ;
            state <= SEND_WAIT;
        end else begin
            wait_cnt <= wait_cnt + 32'd1;
        end
    end
end

```

(7) 再然后是发送 FIFO 中的数据，SEND_WAIT 状态是控制读使能信号 fifo_rden，并且判断数据是否发送完，发送完后进入 IDLE 状态。

```

SEND_WAIT:
    begin
        if (tx_data_ready == 1'b1)
        begin
            if (tx_cnt == data_count)
            begin
                tx_cnt      <= 8'd0 ;
                fifo_rden    <= 1'b0 ;
                state        <= IDLE ;
            end
            else
            begin
                fifo_rden    <= 1'b1 ;
            end
        end
    end

```

```

        state      <= SEND ;
    end
end
tx_data_valid    <= 1'b0 ;
end
SEND:
begin
    fifo_rden      <= 1'b0 ;
    tx_data_valid  <= 1'b1 ;

    if(tx_data_valid == 1'b1 && tx_data_ready == 1'b1
    && tx_cnt < data_count)
    begin
        tx_cnt <= tx_cnt + 8'd1; //Send data counter
        state <= SEND_WAIT ;
    end
end
end

```

5.RS485 驱动仿真及仿真结果

(1) 对于 RS485 的仿真代码的编写思路与串口的仿真是一致的，可以单独测试每个模块功能的完整性，如 RS485 的发送模块是否正常，在将发送模块的数据连接至接收端口，查看数据是否能进行环回。

```

module SIM_TB_RS485();

reg clk , rst ;

//定义发送8bit 数据
reg      [7:0]          i_user_tx_data ;
reg      i_user_tx_valid;
wire      o_user_tx_ready;

wire      w_user_tx_rst  ;
wire      w_user_tx_clk  ;
wire      uart_data      ;

//时钟信号
initial begin
    clk <= 'd0;
    rst <= 'd0;
    #105

```

```

    @(posedge clk)rst <= 'd1;
end

always begin
    clk <= 0;
    #10;
    clk <= 1;
    #10;
end

uart_tx#
(
    .CLK_FRE           (50),
    .BAUD_RATE         (115200)
) uart_tx_inst
(
    .clk                (clk                ),
    .rst_n              (rst                ),
    .tx_data            (i_user_tx_data     ),
    .tx_data_valid      (i_user_tx_valid    ),
    .tx_data_ready      (o_user_tx_ready    ),
    .tx_pin             (                   )
);

//产生数据
always @(posedge clk or negedge rst) begin
    if (~rst)
        i_user_tx_data <= 'd0;
    else if (i_user_tx_valid && o_user_tx_ready)
        i_user_tx_data <= i_user_tx_data + 'd1;
    else
        i_user_tx_data <= i_user_tx_data;
end

always @(posedge clk or negedge rst) begin
    if (~rst)
        i_user_tx_valid <= 'd0;
    else if (i_user_tx_valid && o_user_tx_ready)
        i_user_tx_valid <= 'd0;
    else if (o_user_tx_ready)
        i_user_tx_valid <= 'd1;
    else

```

```

        i_user_tx_valid <= i_user_tx_valid;
    end
endmodule

```

上述主要是测试了 RS485 的发送模块，发送数据是随着每个时钟自增的，可以通过数据快速测试是否符合数据发送结果。

(2) 发送部分测试结果：

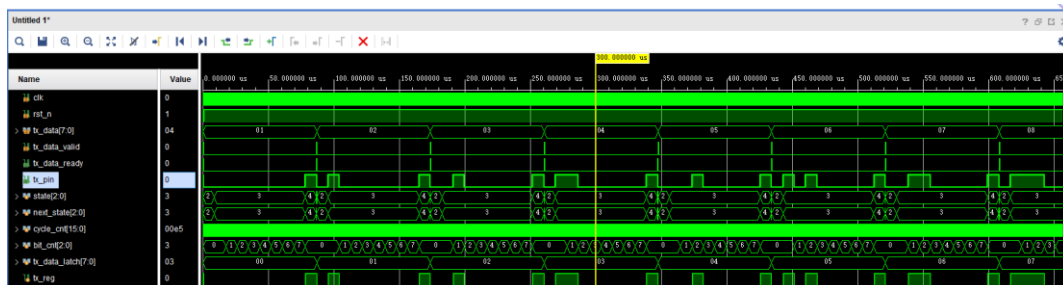


图 11-5 发送部分仿真图

从图 11-5 中可以看出，随着时间的增加，每个时钟的数据也在不断的自增，同时 tx 总线上的数据也在按串口的格式将数据发送至 rx 总线。最后在 RX 总线上进行数据解算。

由于接收部分与串口的接收部分是一致的，这边就不重复进行仿真控制，若有兴趣可自己对接收部分进行仿真测试。

6. 添加时序约束

由于 RS485 的接收数据与发送数据的间隔较长，采用内部逻辑分析仪可能存在无法一次看完结果的效果，因此在此次实验不进行内部逻辑分析仪的实验。

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按钮就可以添加时钟约束了，如下图所示：

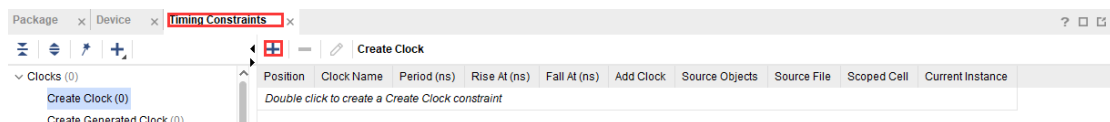


图 11-6 添加主时钟时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

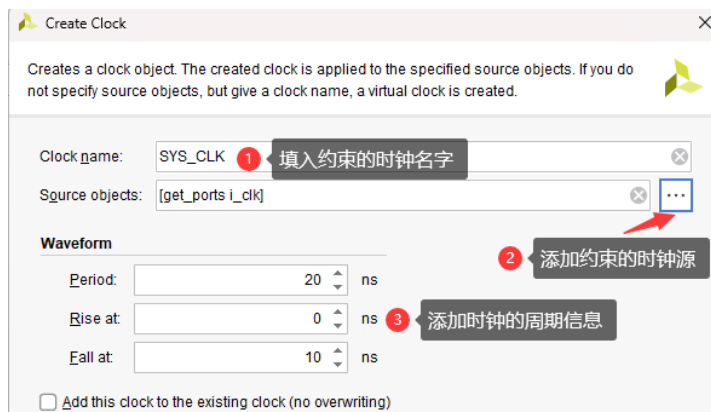


图 11-7 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
Constraint > Timing_Constraint.xdc
1 create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} [get_ports i_clk]
2
```

图 11-8 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

7.添加管脚约束

在上述步骤中添加了时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

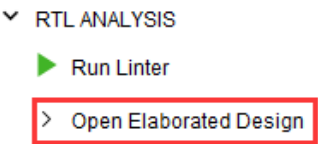


图 11-9 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	IO Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
core_clk_3205 (1)	IN			☒	33	LVCMS633*	3.300				NONE	NONE	
Scalar ports (1)													
L_clk	IN		W17	☒	33	LVCMS633*	3.300				NONE	NONE	
Scalar ports (4)													
rs485_de1	OUT		Y20	☒	33	LVCMS633*	3.300	12			NONE	FP_VTT_50	
rs485_rx1	IN		AA19	☒	33	LVCMS633*	3.300				NONE	NONE	
rs485_tx1	OUT		W22	☒	33	LVCMS633*	3.300	12			NONE	FP_VTT_50	
rst_n	IN		W18	☒	33	LVCMS633*	3.300				NONE	NONE	

图 11-10 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
1  # 主时钟
2  set_property PACKAGE_PIN W17 [get_ports i_clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports i_clk]
4
5  set_property PACKAGE_PIN W18 [get_ports rst_n]
6  set_property IOSTANDARD LVCMOS33 [get_ports rst_n]
7
8  # RS485管脚
9  set_property PACKAGE_PIN W22 [get_ports rs485_tx1]
10 set_property IOSTANDARD LVCMOS33 [get_ports rs485_tx1]
11
12 set_property PACKAGE_PIN AA19 [get_ports rs485_rx1]
13 set_property IOSTANDARD LVCMOS33 [get_ports rs485_rx1]
14
15 set_property PACKAGE_PIN Y20 [get_ports rs485_de1]
16 set_property IOSTANDARD LVCMOS33 [get_ports rs485_de1]
```

图 11-11 约束文件中写入管脚约束

8.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

Timing		Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	15.495 ns			
Total Negative Slack (TNS):	0 ns			
Number of Failing Endpoints:	0			
Total Number of Endpoints:	305			
Implemented Timing Report				

图 11-12 时序结果

此次实验使用的是绿联 USB 转 485，实验时依据手中 USB 转 485 与开发板进行接线，接线规则为板载 485 A 端接 USB 转 485 A 端，板载 485 B 端接 USB 转 485 B 端接线图如下所示：

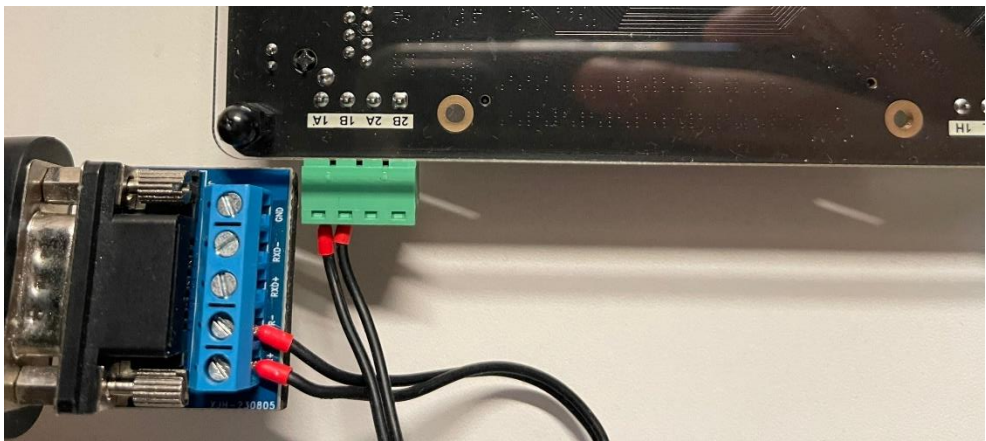


图 11-13 外设连接图

打开串口工具，设置好串口号波特率，选择 16 进制发送，发送数据以 8'h55、8'h08 开头，点击发送，即可在接收窗口看到返回的数据。实验效果如下图所示：

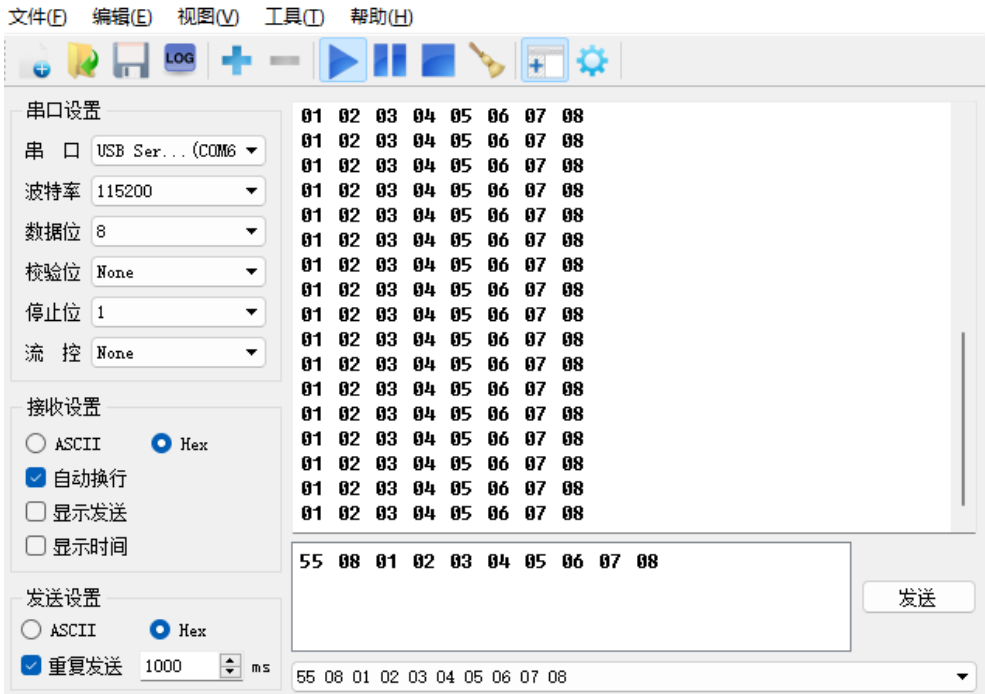


图 11-14 RS-485 环回测试图

9.实验总结

经过对 RS-485 实验学习，做出了如下总结：

a.本章节是“UART 串口通信实验”的延伸，需要注意的是，接口标准（例如 RS232、RS422 和 RS485 等）与串口协议（例如 UART）是两个不同的概念，只是在应用中常常会将两者进行组合，以实现相应的串行通信方式。

b.本章实现的功能可以理解为一个简单的串口数据编解码过程，通过校验数据位的方式，将数据发送出去。

十二. HDMI 彩条显示实验

HDMI 接口在消费类电子行业,如电脑、液晶电视、投影仪等产品中得到了广泛的应用。一些专业的视频设备如摄像机、视频切换器等也都集成了 HDMI 接口。本章将学习如何在 ZYNQ 开发板上的驱动 HDMI 接口。

1.HDMI 简介

HDMI 是新一代的多媒体接口标准,英文全称是 High-Definition Multimedia Interface,即高清多媒体接口。它能够同时传输视频和音频,简化了设备的接口和连线;同时提供了更高的数据传输带宽,可以传输无压缩的数字音频及高分辨率视频信号。HDMI 1.0 版本于 2002 年发布,最高数据传输速度为 5Gbps; HDMI 2.0 版本于 2013 年推出的,2.0 理论传输速度能达到 18Gbit/s,实际传输速度能达到 14.4Gbit/s;而 2017 年发布的 HDMI 2.1 标准的理论带宽可达 48Gbps,实际速度也能达到 42.6Gbit/s。

在 HDMI 接口出现之前,被广泛应用的是 VGA 接口。VGA 的全称是 Video Graphics Array,即视频图形阵列,是一个使用模拟信号进行视频传输的标准。VGA 接口采用 15 针插针式结构,里面传输模拟信号颜色分量、同步等信号,是很多老显卡、笔记本和投影仪所使用的接口。由于 VGA 接口传输的是模拟信号,其信号容易受到干扰,因此 VGA 在高分辨率下字体容易虚,信号线长的话,图像有拖尾现象。VGA 接口由下图所示:



图 12-1 VGA 接口

VGA 接口除信号容易受到干扰外,其体积也较大,因此 VGA 接口已逐渐退出舞台,一些显示器也不再带有 VGA 接口,在数字设备高度发展的今天,取而代之的是 HDMI 接口和 DP (Display Port) 接口等。

HDMI 向下兼容 DVI,但是 DVI (数字视频接口)只能用来传输视频,而不能同时传输音频,这是两者最主要的差别。此外,DVI 接口的尺寸明显大于 HDMI 接口,如下图所示:



图 12-2 DVI 接口 (左) 和 HDMI 接口 (右) 实物图

图 12-2 中右侧是生活中最常见的 A 型 HDMI 接口，其引脚定义如下图所示：

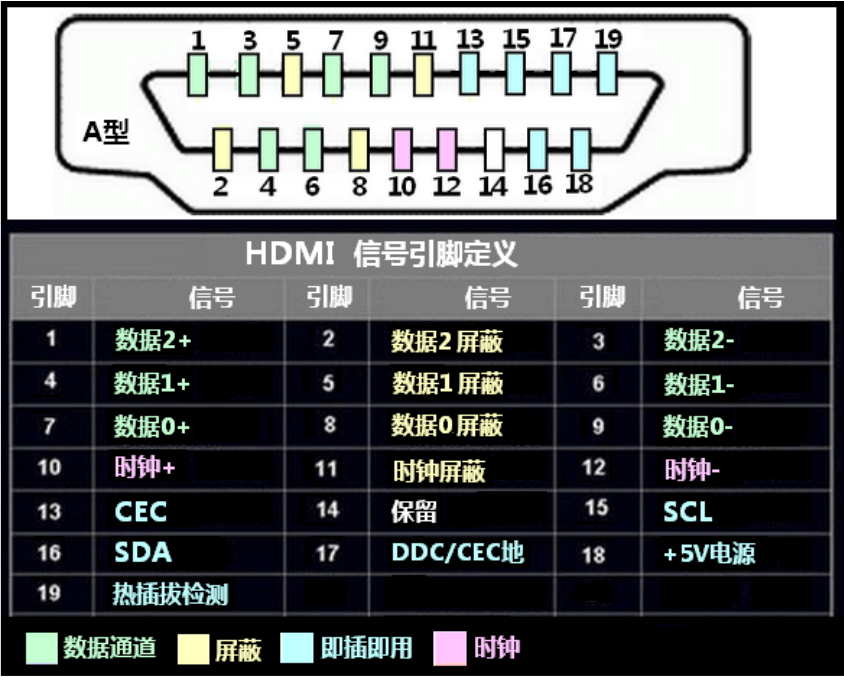


图 12-2 HDMI 接口引脚定义

DVI 和 HDMI 接口协议在物理层使用 TMDS 标准传输音视频数据。TMDS (Transition Minimized Differential Signaling, 最小化传输差分信号) 是美国 Silicon Image 公司开发的一项高速数据传输技术, 在 DVI 和 HDMI 视频接口中使用差分信号传输高速串行数据。TMDS 差分传输技术使用两个引脚 (如图 12-2 中的“数据 2+”和“数据 2-”) 来传输一路信号, 利用这两个引脚间的电压差的正负极性和大小来决定传输数据的数值 (0 或 1)。

Xilinx 在 Spartan-3A 系列之后的器件中, 加入了对 TMDS 接口标准的支持, 用于在 FPGA 内部实现 DVI 和 HDMI 接口。由于本次实验只是使用 HDMI 接口来显示图像, 不需要传输音频, 因此只需要实现 DVI 接口的驱动逻辑即可。不过在此之前还需要简单地了解一下 TMDS 视频传输协议。

图 12-3 是 TMDS 发送端和接收端的连接示意图, DVI 或 HDMI 视频传输所使用的 TMDS 连接通过四个串行通道实现。对于 DVI 来说, 其中三个通道分别用于传输视频中每个像素点的红、绿、蓝三个颜色分量 (RGB 4:4:4 格式)。HDMI 默认也是使用三个 RGB 通道, 但是它同样可以选择传输像素点的亮度和色度信息 (YCrCb 4:4:4 或 YCrCb 4:2:2 格式)。第四个通道是时钟通道, 用于传输像素时钟。独立的 TMDS 时钟通道为接收端提供接收的参考频率, 保证数据在接收端能够正确恢复。

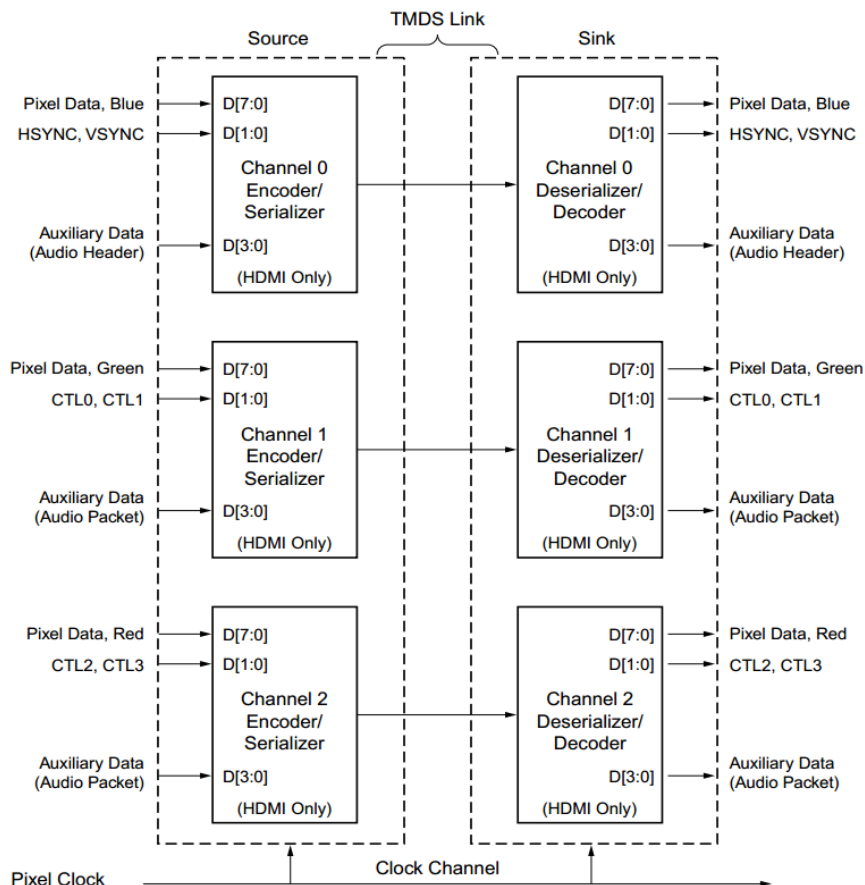


图 12-3 TMDS 连接示意图

如果每个像素点的颜色深度为 24 位，即 RGB 每个颜色分量各占 8 位，那么每个通道上的颜色数据将通过编码器来转换成一个 10 位的像素字符。然后这个 10 位的字符通过并串转换器转换成串行数据，最后由 TMDS 数据通道发送出去。这个 10:1 的并转串过程所生成的串行数据速率是实际像素时钟速率的 10 倍。

在传输视频图像的过程中，数据通道上传输的是编码后的有效像素字符。而在每一帧图像的行与行之间，以及视频中不同帧之间的时间间隔（消隐期）内，数据通道上传输的则是控制字符。每个通道上有两位控制信号的输入接口，共对应四种不同的控制字符。这些控制字符提供了视频的行同步（HSYNC）以及帧同步（VSYNC）信息，也可以用来指定所传输数据的边界（用于同步）。

对于 DVI 传输，整个视频的消隐期都用来传输控制字符。而 HDMI 传输的消隐期除了控制字符之外，还可以用于传输音频或者其他附加数据，比如字幕信息等。这就是 DVI 和 HDMI 协议之间最主要的差别。从图 31.1.4 中也可以看出这一差别，即“Auxiliary Data”接口标有“HDMI Only”，即它是 HDMI 所独有的接口。

从前面的介绍中我们可以看出，TMDS 连接从逻辑功能上可以划分成两个阶段：编码和并串转换。在编码阶段，编码器将视频源中的像素数据、HDMI 的音频/附加数据，以及行同步和场同步信号分别编码成 10 位的字符流。然后在并串转换阶段将上述的字符流转换成串行数据流，并将其从三个差分输出通道发送出去。

DVI 编码器在视频有效数据段输出像素数据，在消隐期输出控制数据，如图 12-4 所示。其中 VDE（Video Data Enable）为高电平时表示视频数据有效，为低电平代表当前处于视频消隐期。

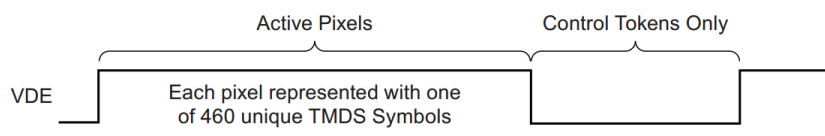


图 12-4 DVI 编码输出示意图

图 12-5 给出了三个通道的 DVI 编码器示意图。对于像素数据的 RGB 三个颜色通道，编码器的逻辑是完全相同的。VDE 用于各个通道选择输出视频像素数据还是控制数据，HSYNC 和 VSYNC 信号在蓝色通道进行编码得到 10 位字符，然后在视频消隐期传输。绿色和红色通道的控制信号 C0 和 C1 同样需要进行编码，并在消隐期输出。但是 DVI 规范中这两个通道的控制信号是预留的（未用到），因此将其置为 2'b00。

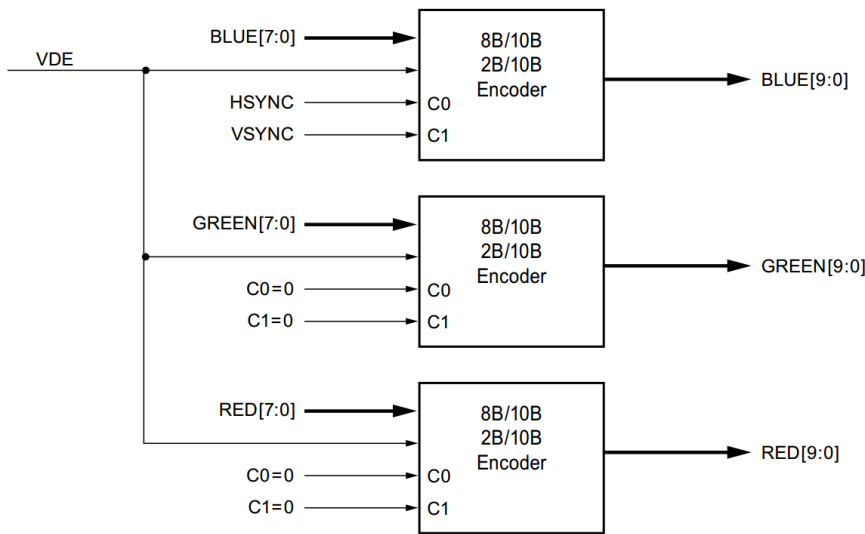


图 12-5 DVI 编码示意图

每个通道输入的视频像素数据都要使用 DVI 规范中的 TMDS 编码算法进行编码。每个 8-bit 的数据都将被转换成 460 个特定 10-bit 字符中的一个。这个编码机制大致上实现了传输过程中的直流平衡，即一段时间内传输的高电平（数字“1”）的个数大致等于低电平（数字“0”）的个数。同时，每个编码后的 10-bit 字符中状态跳转（“由 1 到 0”或者“由 0 到 1”）的次数将被限制在五次以内。

除了视频数据之外，每个通道 2-bit 控制信号的状态也要进行编码，编码后分别对应四个不同的 10-bit 控制字符，分别是 10'b1101010100，10'b0010101011，10'b0101010100，和 10'b1010101011。可以看出，每个控制字符都有七次以上的状态跳转。视频字符和控制字符状态跳转次数的不同将会被用于发送和接收设备的同步。

在编码之后 3 个通道的 10-bit 字符将进行并串转换，这一过程是使用 7 系列 FPGA 中专用的硬件资源来实现的。ZYNQ PL 部分与 7 系列的 FPGA 是等价的，它提供了专用的并串转换器——OSERDESE2。单一的 OSERDESE2 模块可以实现 8:1 的并串转换，通过位宽扩展可以实现 10:1 和 14:1 的转换率。

2.硬件设计

开发板上 HDMI 接口部分的原理图如下图所示：

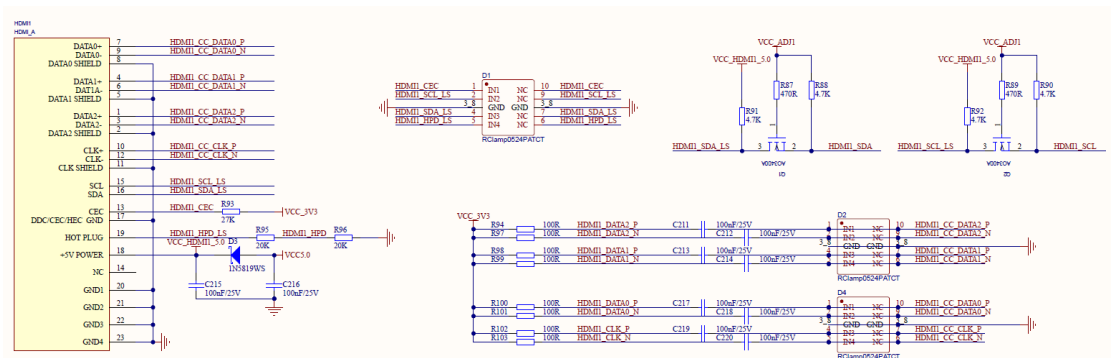


图 12-6 HDMI 接口原理图

从图 12-6 中可以看出，其中 HDMI 的三个数据通道 HDMI_D[2:0]和一个时钟通道 HDMI_CLK，通过图中 D2 和 D4 为 RCLAMP0524PATCT 瞬态抑制二极管 (TVS) 与 ZYNQ PL 端的 TMDS 差分引脚相连，而瞬态抑制二极管用于保护高速数据接口，防止电流倒灌损毁 FPGA。RCLAMP0524PATCT 是一种独特的设计，包括 ESD 额定、超低容量转向二极管和箝位单元的独特设计，钳位单元是单个封装中的等效 TVS 二极管。在瞬态条件下，转向二极管将瞬态引导至内部 ESD 线路或接地线。如果大家还想深入的学习 RCLAMP0524PATCT，可以网上自行查阅 RCLAMP0524PATCT 芯片数据手册。

HDMI_HPD 指的是热拔插检测 (Hot Plug Detect), 当视频设备与接收设备通过 HDMI 连接时, 接收设备将 HPD 置为高电平, 通知发送设备。当发送设备检测到 HPD 为低电平时, 表明断开连接。HDMI HPD 通过 R95 与 R96 这两个电阻实现分压转换。

HDMI_SCL_LS 和 HDMI_SDA_LS 是 HDMI 接口的显示数据通道 (DDC, Display Data Channel), 用于 HDMI 发送端和接收端之间交换一些配置信息, 通过 I2C 协议通信。发送端通过 DDC 通道, 读取接收端保存在 HDMI 显示器 EEPROM 中的 EDID 数据, 获取接收端的信息, 确认接收端终端显示的设置和功能, 决定跟接收端之间以什么格式传输音/视频数据。需要说明的是, HDMI 的 IIC 引脚是 5V 电平的, 需要将其转换为 ZYNQ PL 端所需的 3.3V 电压。

其中, HDMI1_SCL 和 HDMI1_SDA 连接到 ZYNQ PL 侧的 IO 口上, HDMI_SCL_LS 和 HDMI_SDA_LS 连接到 HDMI 插座上。当总线空闲即 HDMI1_SCL 和 HDMI1_SDA 都被 ZYNQ 驱动为高电平时, MOS 管截止, HDMI_SCL_LS, HDMI_SDA_LS 也被上拉电阻 R91 和 R92 上拉到+5V 的高电平。当 HDMI1_SCL 或 HDMI1_SDA 被 ZYNQ 驱动为低电平时, MOS 管导通, 此时 HDMI_SCL_LS 或 HDMI_SDA_LS 也被下拉到了 GND。若 HDMI_SDA_LS 需要向 ZYNQ 发送低电平时, HDMI_SDA_LS 为低, MOS 管内部二极管导通 HDMI1_SDA 被下拉到了低电平。以此实现了双向电平转换的功能。

3.实验任务

本章的实验任务是驱动 ZYNQ 开发板上的 HDMI 接口, 在显示器上显示 720p 彩条图案 (720p 分辨率为 1280*720, 像素时钟大约为 75MHz)。

4.程序设计

本次实验的系统框图如下所示:

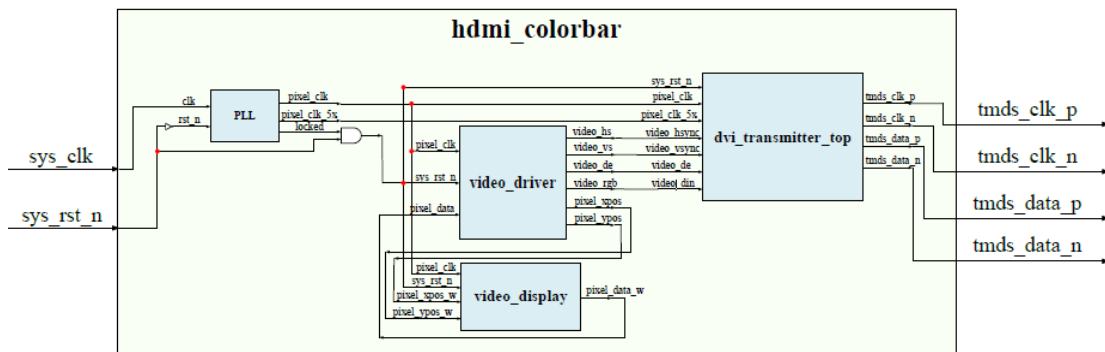


图 12-7 HDMI 系统总统框架图

4.1 video_driver 驱动模块

将输入模块的 RGB888 格式的视频图像转换成 TMDs 数据输出，其中 video_driver 模块主要用于控制 HDMI 显示器的行场同步信号，HDMI 数据使能信号及像素位置信号。

(1) 为了对常用的显示器进行兼容，首先对常用的显示器分辨率进行定义，如 1280*720，这说明一行有 1280 个像素点，一共有 720 行。

```
parameter H_SYNC    = 11'd40;    // 行同步
parameter H_BACK    = 11'd220;   // 行显示后沿
parameter H_DISP    = 11'd1280;  // 行有效数据
parameter H_FRONT   = 11'd110;   // 行显示前沿
parameter H_TOTAL   = 11'd1650;  // 行扫描周期

parameter V_SYNC    = 11'd5;     // 场同步
parameter V_BACK    = 11'd20;    // 场显示后沿
parameter V_DISP    = 11'd720;   // 场有效数据
parameter V_FRONT   = 11'd5;     // 场显示前沿
parameter V_TOTAL   = 11'd750;   // 场扫描周期
```

(2) 首先对行场信号用计数器进行控制，以控制像素点显示的位置。

```
// 行计数器对行计数
always @(posedge pixel_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)
        cnt_h <= 11'd0;
    else begin
        if(cnt_h < H_TOTAL - 1'b1)
            cnt_h <= cnt_h + 1'b1;
        else
            cnt_h <= 11'd0;
    end
end

// 场计数器对场计数
always @(posedge pixel_clk or negedge sys_rst_n) begin
```

```

    if (!sys_rst_n)
        cnt_v <= 11'd0;
    else if(cnt_h == H_TOTAL - 1'b1) begin
        if(cnt_v < V_TOTAL - 1'b1)
            cnt_v <= cnt_v + 1'b1;
        else
            cnt_v <= 11'd0;
    end
end
end

```

(3) 在显示器显示的部分请求颜色数据的输入。

```

always @(posedge pixel_clk or negedge sys_rst_n) begin
    if(!sys_rst_n)
        data_req <= 1'b0;
    else if(((cnt_h >= H_SYNC + H_BACK - 2'd2) && (cnt_h < H_SYNC +
H_BACK + H_DISP - 2'd2))
            && ((cnt_v >= V_SYNC + V_BACK) && (cnt_v < V_SYNC +
V_BACK+V_DISP))))
        data_req <= 1'b1;
    else
        data_req <= 1'b0;
end
end

```

(4) 在请求颜色输出的同时，控制在像素点的坐标。

```

//像素点x 坐标
always@ (posedge pixel_clk or negedge sys_rst_n) begin
    if(!sys_rst_n)
        pixel_xpos <= 11'd0;
    else if(data_req)
        pixel_xpos <= cnt_h + 2'd2 - H_SYNC - H_BACK ;
    else
        pixel_xpos <= 11'd0;
end

//像素点y 坐标
always@ (posedge pixel_clk or negedge sys_rst_n) begin
    if(!sys_rst_n)
        pixel_ypos <= 11'd0;
    else if((cnt_v >= (V_SYNC + V_BACK)) && (cnt_v < (V_SYNC + V_BACK +
V_DISP)))
        pixel_ypos <= cnt_v + 1'b1 - (V_SYNC + V_BACK) ;
    else
        pixel_ypos <= 11'd0;
end
end

```

对于驱动模块产生显示器位置的代码，大体上的代码驱动如上述所示，对于拉高 hs，vs 等信号是通过 assign 语句拉高，为了减少教程篇幅则没有写，具体代码可以参考提供的代码例程。

4.2 video_display 模块

该模块主要是通过驱动模块输出的像素点位置，产生对应的颜色信息，整个模块十分的简单。

(1) 将每个颜色以 RGB888 的形式定义，如下所示：

```
//parameter define
parameter H_DISP = 11'd1280;           //分辨率一行
parameter V_DISP = 11'd720;           //分辨率一列

localparam WHITE = 24'b11111111_11111111_11111111; //RGB888 白色
localparam BLACK = 24'b00000000_00000000_00000000; //RGB888 黑色
localparam RED   = 24'b11111111_00001100_00000000; //RGB888 红色
localparam GREEN = 24'b00000000_11111111_00000000; //RGB888 绿色
localparam BLUE  = 24'b00000000_00000000_11111111; //RGB888 蓝色
```

(2) 依据像素点位置，将颜色进行赋值。

```
//根据当前像素点坐标指定当前像素点颜色数据，在屏幕上显示彩条
always @(posedge pixel_clk ) begin
    if (!sys_rst_n)
        pixel_data <= 16'd0;
    else begin
        if((pixel_xpos >= 0) && (pixel_xpos < (H_DISP/5)*1))
            pixel_data <= WHITE;
        else if((pixel_xpos >= (H_DISP/5)*1) && (pixel_xpos <
(H_DISP/5)*2))
            pixel_data <= BLACK;
        else if((pixel_xpos >= (H_DISP/5)*2) && (pixel_xpos <
(H_DISP/5)*3))
            pixel_data <= RED;
        else if((pixel_xpos >= (H_DISP/5)*3) && (pixel_xpos <
(H_DISP/5)*4))
            pixel_data <= GREEN;
        else
            pixel_data <= BLUE;
    end
end
end
```

4.3 RGB2DVI 模块

在设计 RGB2DVI 模块框图之前，先了解一下 RGB2DVI 顶层模块的设计框图：

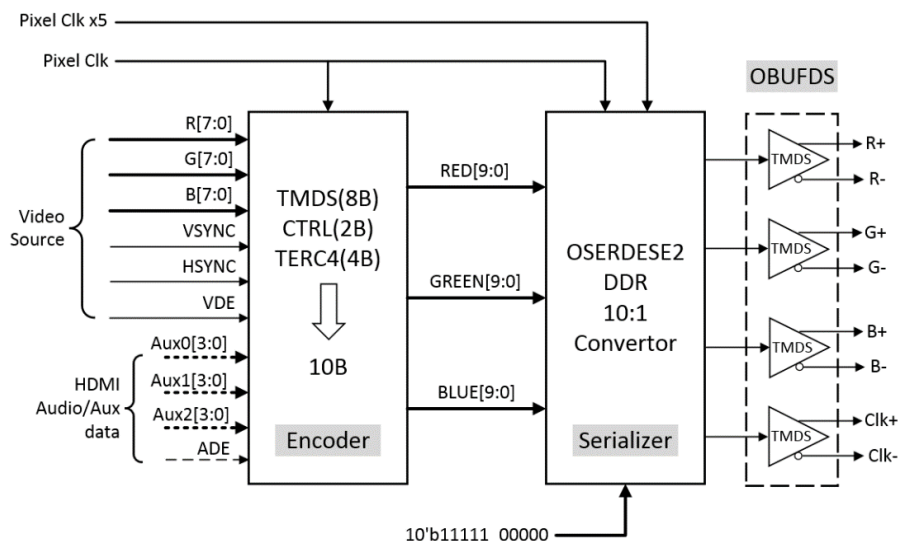


图 12-8 RGB2DVI 模块框图

从图 12-8 中，可以看出 Encoder 模块负责对数据进行编码，Serializer 模块对编码后的数据进行并串转换，最后通过 OBUFDS 转化成 TMDs 差分信号传输。

整个系统需要两个输入时钟，一个是视频的像素时钟 Pixel Clk，另外一个时钟是像素时钟的五倍——Pixel Clk x5。由前面的简介部分可以知道，并串转换过程的实现的是 10:1 的转换率，理论上转换器需要一个 10 倍像素时钟频率的串行时钟。这里我们只用了一个 5 倍的时钟频率，这是因为 OSERDESE2 模块可以实现 DDR 的功能，即它在五倍时钟频率的基础上又实现了双倍数据速率。

TMDs 连接的时钟通道我们采用与数据通道相同的并转串逻辑来实现。通过对 10 位二进制序列 10'b11111_00000 在 10 倍像素时钟频率下进行并串转换，就可以得到像素时钟频率下的 TMDs 参考时钟。

OSERDESE2 模块要求复位信号高电平有效，并且需要将异步复位信号同步到并行时钟域。因此，我们生成一个同步复位模块，将低电平有效的异步复位信号转换成高有效，同时对其进行异步复位，同步释放处理。

另外需要注意的是，图中左下脚 HDMI 的音频/附加数据输入在本次实验中并未用到，因此以虚线表示。

整体框图如下所示：

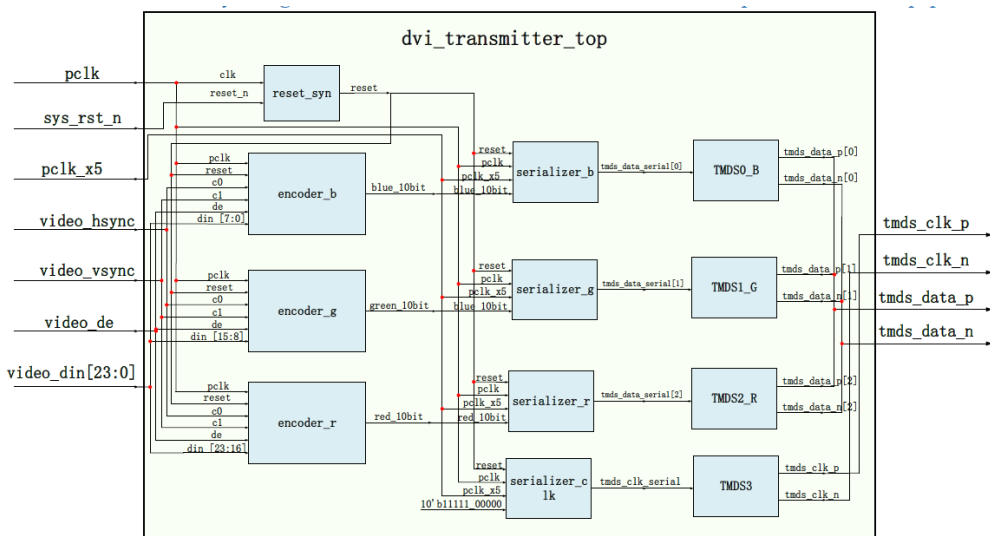


图 12-9 RGB2DVI 模块总体框图

根据模块的设计分析以及我们绘制的 RGB2DVI 模块框图，我们可以编写 RGB2DVI 顶层的整体代码 RGB2DVI 顶层模块就是 RGB2DVI 驱动控制部分的顶层模块，内部实例化编码模块和并行转串行模块，连接各自对应信号，代码编写较为简单，整个模块主要是将数据进行例化，例化代码十分长，在这部分逻辑则不在这进行大篇幅描写，推荐参考提供的代码历程。

在 dvi_transmitter_top 模块中，不仅例化了编码模块和并转串模块，同时还例化了四个 OBUFDS 原语，用于将三路数据和一路时钟信号转换成差分信号输出，OBUFDS 是差分输出缓冲器，用于将来自 FPGA 内部逻辑的信号转换成差分信号输出，支持 TMDS 电平标准。OBUFDS 原语示意图如下所示：

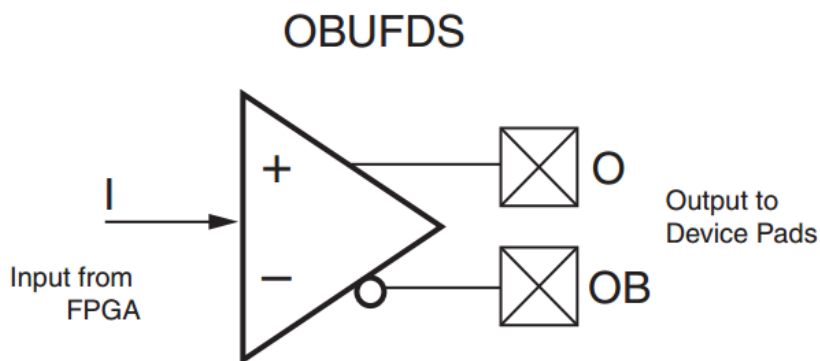


图 12-10 OBUFDS 原语示意图

4.4 编码模块

dvi_encoder 编码模块就是为了完成 RGB 图像数据 8b 转 10b 的编码，关于 8b 转 10b 编码的理论知识在前面的 HDMI 简介部分已经做了详细介绍。

根据上述分析，可以绘制出 dvi_encoder 编码模块的框图：

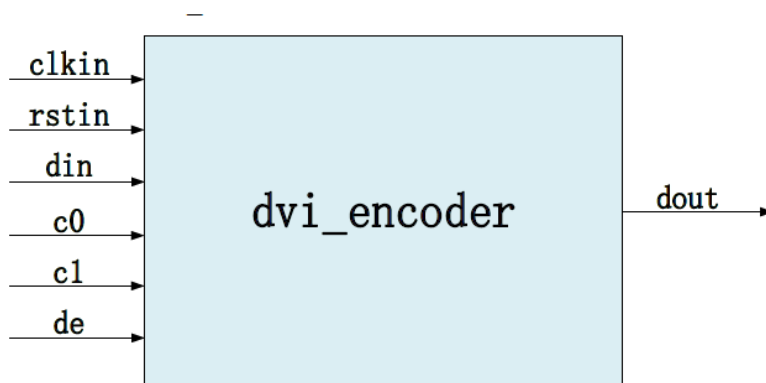


图 12-11 编码模块框图

TMDS 的编码模块主要采用的是 Xilinx 官方提供的编码程序，编码模块的算法流程如下图所示：

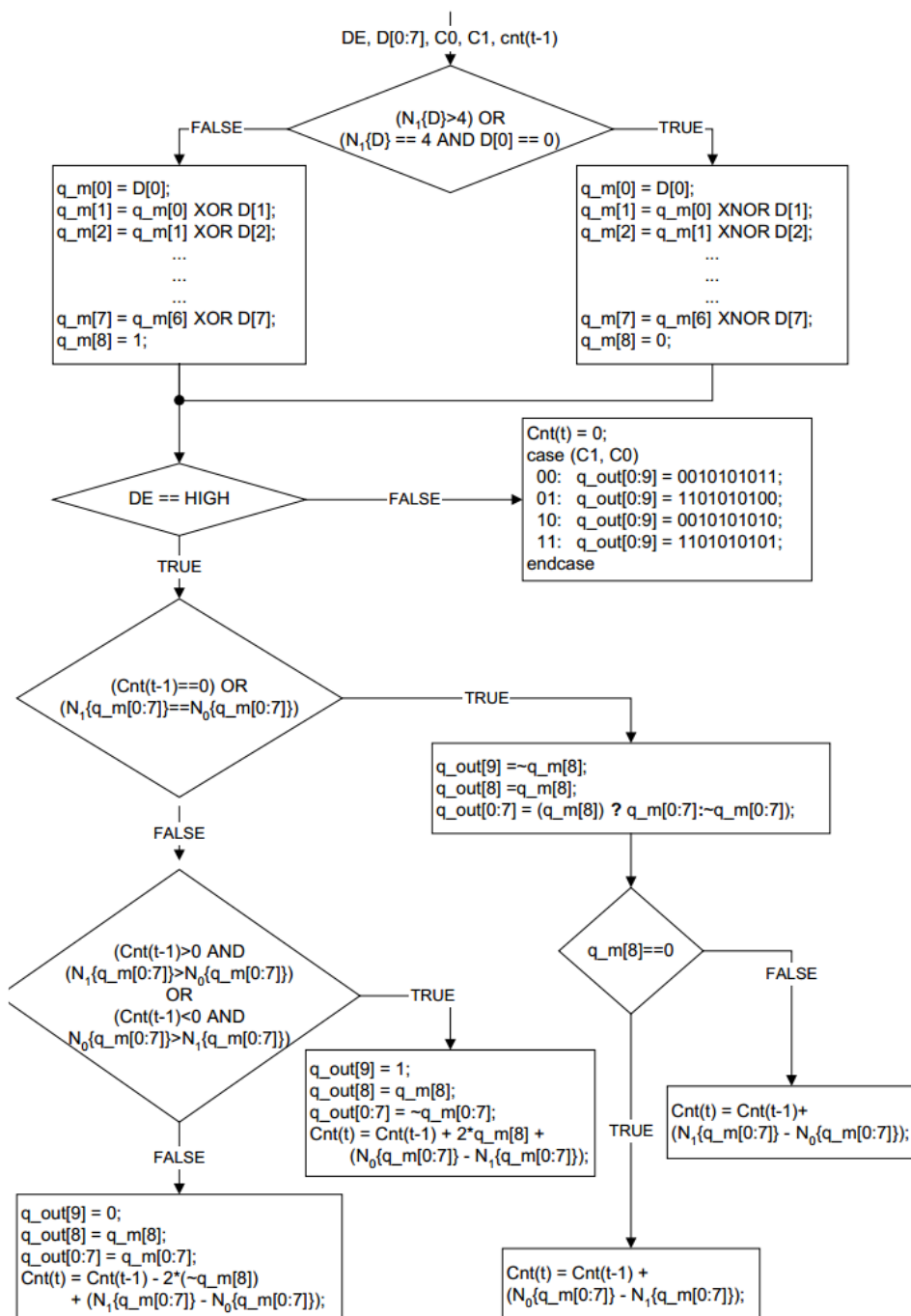


图 12-12 TMDS 编码算法

TMDS 通过逻辑算法将 8 位字符数据通过编码转换为 10 位字符数据，前 8 位数据由原始信号经运算后获得，第 9 位表示运算的方式，1 表示异或 0 表示异或非。经过 DC 平衡后（第 10 位），采用差分信号传输数据。第 10 位实际是一个反转标志位，1 表示进行了反转而 0 表示没有反转，从而达到 DC 平衡。

接收端在收到信号后，再进行相反的运算。TMDS 和 LVDS、TTL 相比有较好的电磁兼容性能。这种算法可以减小传输信号过程的上冲和下冲，而 DC 平衡使信号对传输线的电磁干扰减少，可以用低成本的专用电缆实现长距离、高质量的数字信号传输。

结合仿真波形可以看一下 xilinx 官方提供的编码模块：

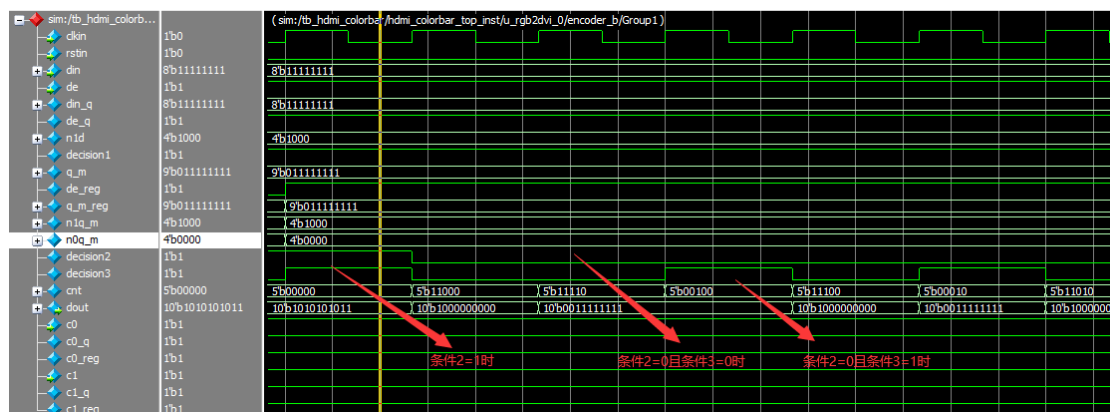


图 12-12 编码模块仿真图

如上图所示当“decision2= 1”时，对应代码中的第 101 到 105 行，dout 等于 10'b10_0000_0000，cnt 等于 5'b11000；当“decision2= 0 且条件 decision3 = 0”时，对应代码的第 114 到 118 行，dout 等于 10'b00_1111_1111，cnt 等于 5'b11110，下一个时钟周期 cnt 等于 5'b00100，因为 q_m_reg 值保持不变，所以 dout 也保持不变；当“decision2=0 且 decision3 = 1 时”，对应代码的第 108 到 112 行，dout 等于 10'b10_0000_0000，cnt 等于 5'b11100。

因为 cnt 和 dout 为时序逻辑，所以在输出的结果会延迟一个时钟周期。这里 dout 以及 cnt 的计算过程，大家如果有兴趣可以参考对应行号的公式进行计算一下，这里就不做一个详细的展开了。

4.5 Serializer 模块设计

下面来看一下 Serializer (并串转换) 模块的设计，本次设计的并串转换模块是通过 Xilinx 官方提供的 OSERDESE2 原语，对编码模块输出的 10bit 数据进行并串转换然后输出，模块框图如下图所示：

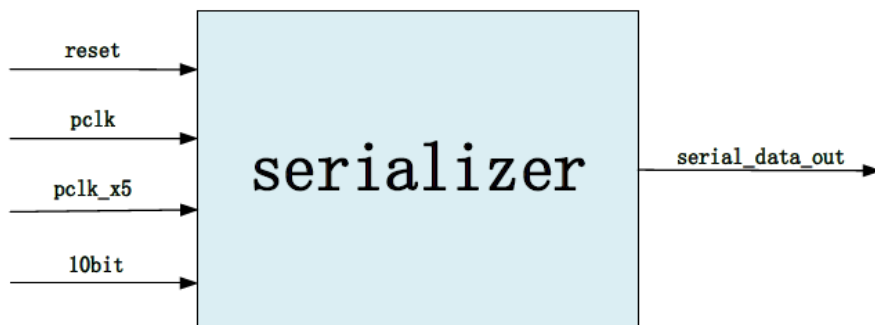


图 12-13 并串转换模块

在教大家调用 Xilinx OSERDESE2 的原语之前，首先带领大家一起了解一下它的相关概念。

OSERDESE2 是一种专用的并-串转换器，每个 OSERDESE2 模块都包括一个专用串行化程序用于数据和 3 状态控制。数据和 3 状态序列化程序都可以工作在 SDR 和 DDR 模式。数据串行化的位宽可以达到 8:1 (如果使用原语模块级联，则可以到 10:1 和 14:1)。3 状态序列化最高可达 14:1，有一个专用的 DDR3 模式可用于支持高速内存应用程序。OSERESE2 框图如下图所示：

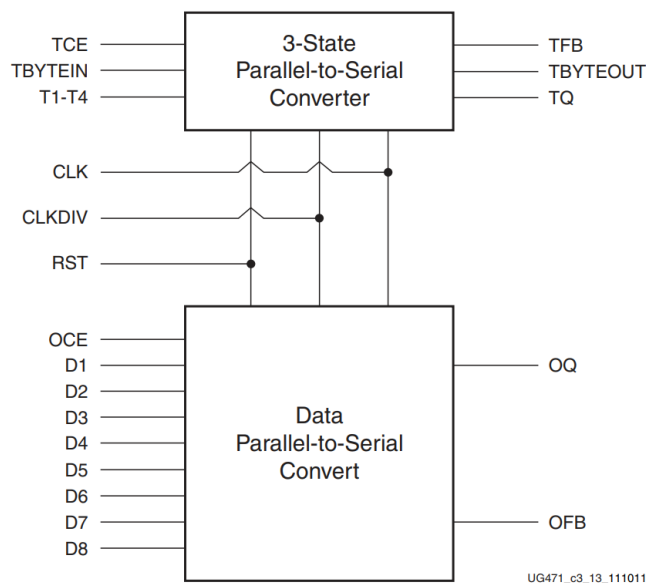


图 12-13 OSERDESE2 框图

需要注意的是，一个 OSERDESE2 只能实现最多 8:1 的转换率，在这里我们通过位宽扩展实现了 10:1 的并串转换，如下图所示：

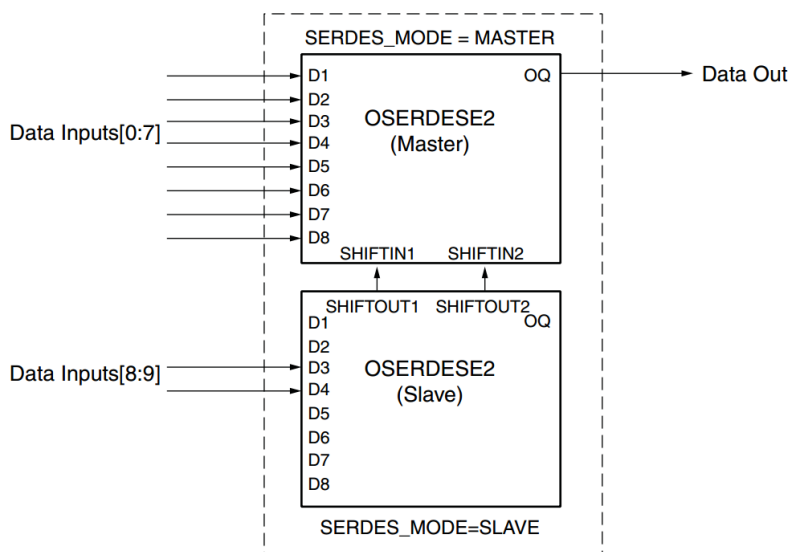


图 12-14 OSERDESE2 位宽拓展

SERDESE2 位宽扩展通过两个 OSERDESE2 模块来实现，其中一个作为 Master，另一个作为 Slave，通过这种方式最多可实现 14:1 的并串转换。需要注意的是，在位宽扩展时，Slave 模块的数据输入端只能使用 D3 至 D8。

接下来例化一个原语来实现 10 位数据的并串转换，采用 DDR 输出。例化 2 个原语 OSERDESE2 级联（打开 vivado--Tools--Language Templates，搜索“OSERDESE2”，选择 A7 系列，可以找到 Xilinx 提供的原语模板），具体步骤如下图所示：

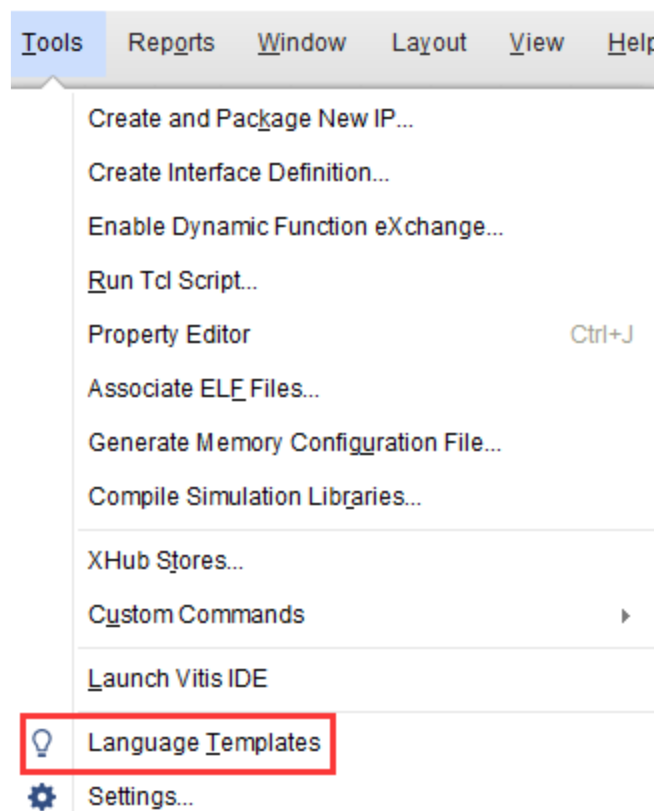


图 12-15 打开 Tools 工具栏

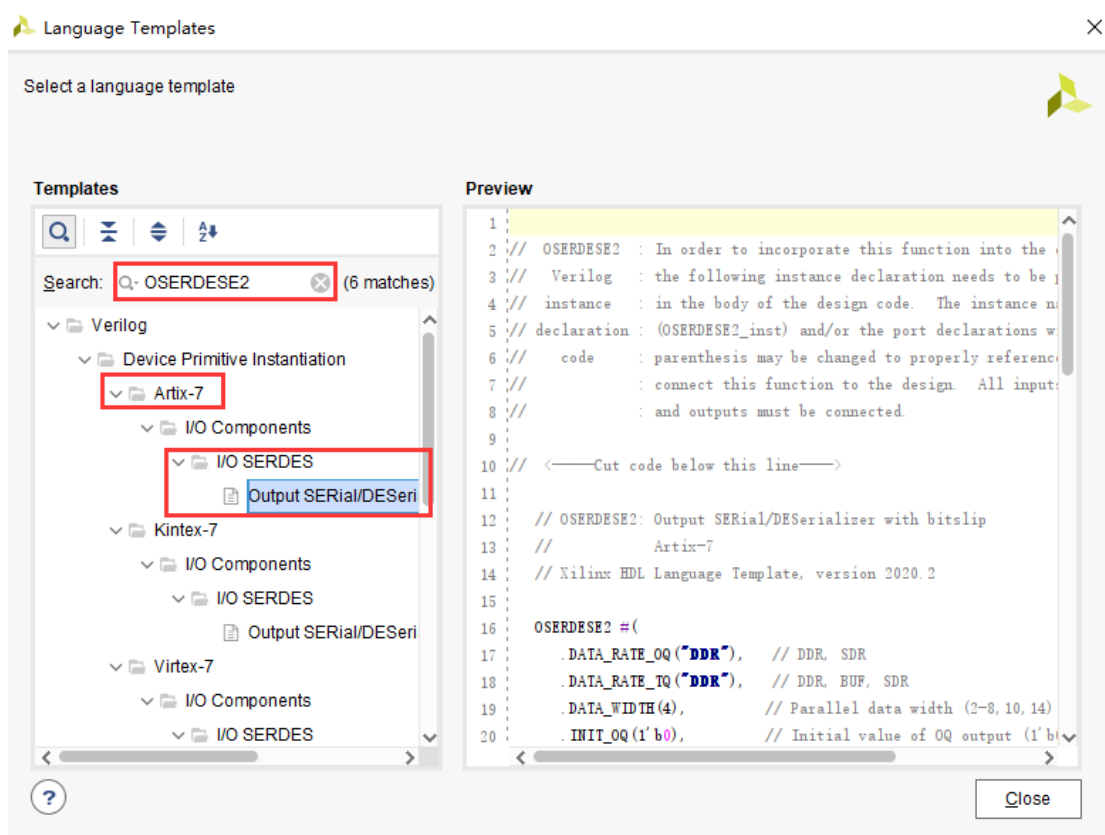


图 12-16 OSERDESE2 原语

接下来将原语例化到 serializer 模块中，由于 oserdes 模块占用的篇幅过长，因此不在该

历程中进行编写，可以参考提供的例程。

本次例化 OSERDESE2 原语需要注意的是，根据 UG471 数据手册的说明，配置 10 转 1 的并串转换需要将 DATA_RATE_OQ 配置为“DDR”和 DATA_RATE_TQ“SDR”，具体详见下图所示：

INTERFACE_TYPE	DATA_RATE_OQ	DATA_RATE_TQ	DATA_WIDTH	TRISTATE_WIDTH
DEFAULT	SDR	SDR	1, 2, 3, 4, 5, 6, 7, 8	1
	DDR	DDR	4	4
		SDR	2, 6, 8, 10, 14	1

图 12-17 OSERDESE2 属性配置

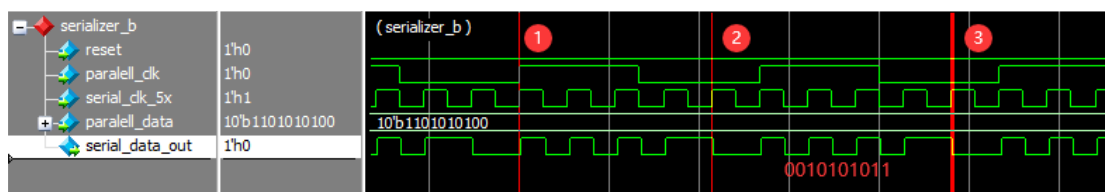


图 12-18 并转串模块仿真波形

从仿真图中可以看出，在①处的上升沿采集到的数据为 10'b1101010100，在②处也就是 4 个串行时钟的延迟后，串行输出开始有数据，分别为 0-0-1-0-1-0-1-0-1-1，可以看出是第 1 个上升沿采集到的数据输出（11010_10100 从低位往高位输出）。

4.6 顶层模块设计

实验工程的各子功能模块均已讲解完毕，在本小节对顶层模块做一下介绍。hdmi_colorbar 顶层模块主要是对各个子功能模块的实例化，以及对应信号的连接。代码编写较为容易。

顶层模块 hdmi_colorbar_top 的代码如下：

(1) 首先定义输入和输出的信号

```
module  hdmi_colorbar_top(
input      sys_clk,
input      sys_rst_n,

output     tmds_clk_p,    // TMDS 时钟通道
output     tmds_clk_n,
output [2:0] tmds_data_p, // TMDS 数据通道
output [2:0] tmds_data_n
);
```

(2) 随后例化显示驱动模块，显示模块以及并串转换模块。

// 例化视频显示驱动模块

```
video_driver  u_video_driver(
    .pixel_clk      ( pixel_clk ),
    .sys_rst_n      ( sys_rst_n ),

    .video_hs       ( video_hs ),
```

```

        .video_vs      ( video_vs ),
        .video_de      ( video_de ),
        .video_rgb     ( video_rgb ),
        .data_req      (),

        .pixel_xpos    ( pixel_xpos_w ),
        .pixel_ypos    ( pixel_ypos_w ),
        .pixel_data     ( pixel_data_w )
    );

// 例化视频显示模块
video_display u_video_display(
    .pixel_clk      (pixel_clk),
    .sys_rst_n      (sys_rst_n),

    .pixel_xpos     (pixel_xpos_w),
    .pixel_ypos     (pixel_ypos_w),
    .pixel_data     (pixel_data_w)
);

// 例化 HDMI 驱动模块
dvi_transmitter_top u_rgb2dvi_0(
    .pclk           (pixel_clk),
    .pclk_x5        (pixel_clk_5x),
    .reset_n        (sys_rst_n & clk_locked),

    .video_din      (video_rgb),
    .video_hsync    (video_hs),
    .video_vsync    (video_vs),
    .video_de       (video_de),

    .tmbs_clk_p     (tmbs_clk_p),
    .tmbs_clk_n     (tmbs_clk_n),
    .tmbs_data_p    (tmbs_data_p),
    .tmbs_data_n    (tmbs_data_n),
    .tmbs_oen       () // 预留的端口，本次实验未用到
);

```

上述基本是对产生的模块进行例化的过程，其中 PLL 的时钟模块，产生一个 75M 时钟与 375M 时钟，PLL 中设置如下图所示：

Clocking Options								
Output Clocks								
Port Renaming								
MMCM Settings								
Summary								
The phase is calculated relative to the active input clock.								
Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives
		Requested	Actual	Requested	Actual	Requested	Actual	
☒ clk_out1	clk_out1	75.000	75.00000	0.000	0.000	50.000	50.0	BUFG
☒ clk_out2	clk_out2	375.000	375.00000	0.000	0.000	50.000	50.0	BUFG
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	BUFG

图 12-19 PLL 输出时钟

5.HDMI 彩条实验仿真及仿真结果

(1) 实验工程的各子功能模块均已讲解完毕，在本小节对顶层模块做一下介绍。
hdmi_colorbar 顶层模块主要是对各个子功能模块的实例化，以及对应信号的连接。

```
module SIM_TB_HDMI();
    reg sys_clk ;
    reg sys_rst_n ;
    wire tmds_clk_p ; // TMD5 时钟通道
    wire tmds_clk_n ;
    wire [2:0] tmds_data_p ; // TMD5 数据通道
    wire [2:0] tmds_data_n ;

    initial begin
        sys_clk = 1'b1;
        sys_rst_n <= 1'b0;
        #201
        sys_rst_n <= 1'b1;
    end

    always #10 sys_clk <= ~sys_clk;

    hdmi_colorbar_top hdmi_colorbar_top_inst(
        .sys_clk (sys_clk ),
        .sys_rst_n (sys_rst_n ),
        .tmds_clk_p (tmds_clk_p ), // TMD5 时钟通道
        .tmds_clk_n (tmds_clk_n ),
        .tmds_data_p (tmds_data_p), // TMD5 数据通道
        .tmds_data_n (tmds_data_n)
    );
endmodule
```

(2) 接下来打开 vivado 对代码进行仿真，仿真的波形如下图所示：

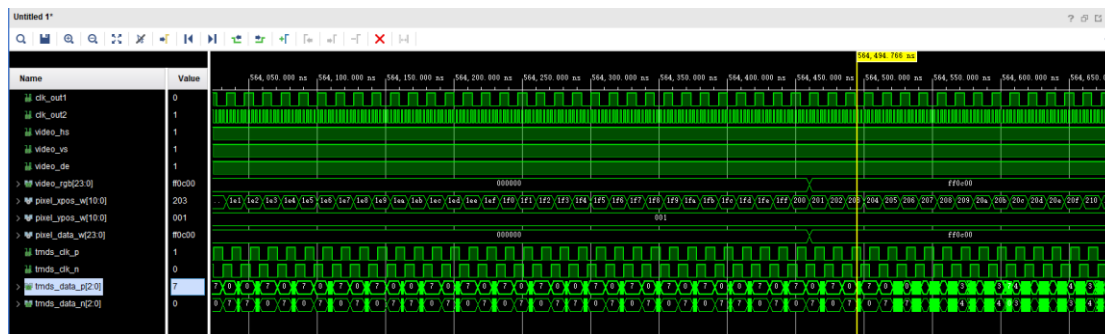


图 12-19 HDMI 彩条仿真波形

从波形图中可以看出，HDMI 顶层差分时钟以及差分数据信号结果是没有问题的。而 lcd_rgb 在 lcd_hs 拉高的范围内，将 RGB888 颜色分为五种颜色，分别是白 (24'hffffff)、黑 (24'h000000)、红 (24'hff0000)、绿 (24'h00ff00) 和蓝 (24'h0000ff)。与我们设计的代码是一致的，编码模块的波形以及并转串的仿真，在前面的子功能模块已经进行了验证分析。而驱动以及显示模块的波形，可以参考 RGB-LCD 彩条显示实验部分的模块介绍。至此本章 HDMI 彩条实验的设计已经完成，接下来就可以进行下载验证了。

6. 添加内部逻辑分析仪

(1) 在 HDMI 的彩条实验中最主要的是查看是否按生成的时序拉高行场同步信号、DE 信号，并且在信号拉高的同时是否产生数据，Debug 信号添加如下：

```

28  (* mark_debug = "true" *)
29  wire [10:0] pixel_xpos_w;
30  (* mark_debug = "true" *)
31  wire [10:0] pixel_ypos_w;
32  (* mark_debug = "true" *)
33  wire [23:0] pixel_data_w;
34
35  (* mark_debug = "true" *)
36  wire video_hs;
37  (* mark_debug = "true" *)
38  wire video_vs;
39  (* mark_debug = "true" *)
40  wire video_de;
41  (* mark_debug = "true" *)
42  wire [23:0] video_rgb;

```

图 12-20 添加 Debug 信号

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。

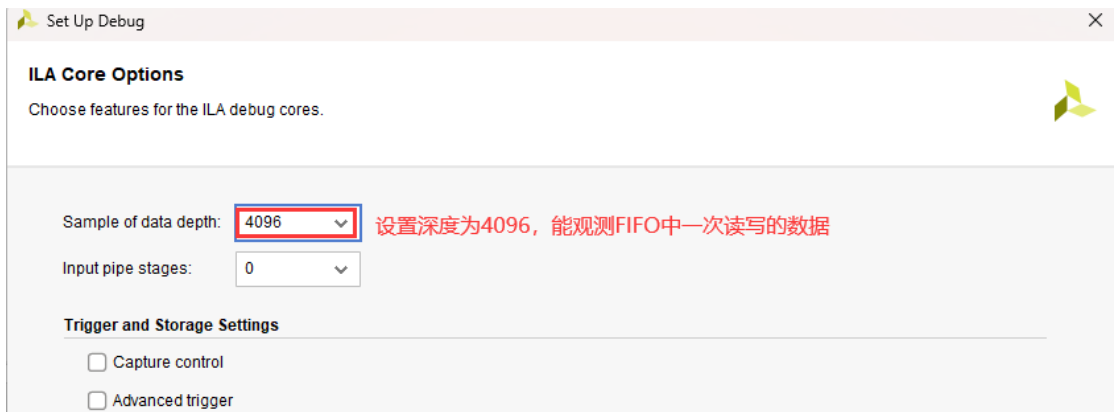


图 12-21 设置 Debug 信号的深度

(3) 查看是否写入约束文件



图 12-22 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

7. 添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按键就可以添加时钟约束了，如下图所示：



图 12-23 添加主时钟时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

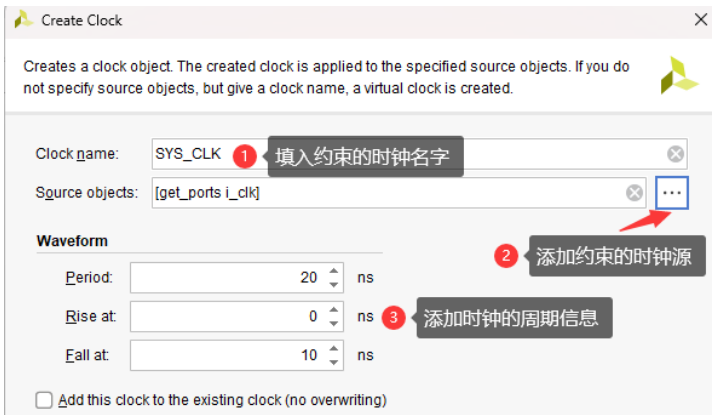


图 12-24 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

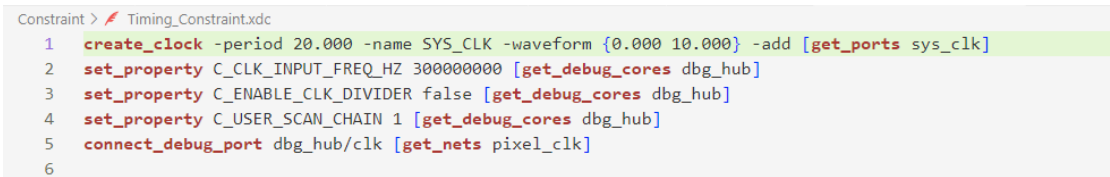


图 12-25 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

8.添加管脚约束

在上述步骤中添加了时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

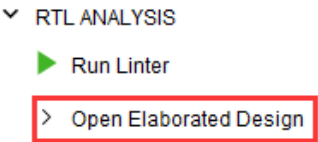


图 12-26 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Net/Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
tmnds_data_p[5]	OUT	tmnds_data_n		✓	33	TMD0_33*	3.300				NONE	FP_3.3_50	
tmnds_data_p[2]	OUT	tmnds_data_n[2] U15		✓	33	TMD0_33*	3.300				NONE	FP_3.3_50	
tmnds_data_p[1]	OUT	tmnds_data_n[1] U17		✓	33	TMD0_33*	3.300				NONE	FP_3.3_50	
tmnds_data_p[0]	OUT	tmnds_data_n[0] A417		✓	33	TMD0_33*	3.300				NONE	FP_3.3_50	
Scalar ports (4)													
sys_clk	IN		W17	✓	33	LVCMD0533	3.300				NONE	NONE	
sys_rst_n	IN		W18	✓	33	LVCMD0533	3.300				NONE	NONE	
tmnds_clk_p	OUT	tmnds_clk_n	W16	✓	33	TMD0_33*	3.300				NONE	FP_3.3_50	

图 12-27 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
Constraint > PHY_Pin.xdc
1  # 主时钟
2  set_property PACKAGE_PIN W17 [get_ports sys_clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports sys_clk]
4
5  set_property PACKAGE_PIN W18 [get_ports sys_rst_n]
6  set_property IOSTANDARD LVCMOS33 [get_ports sys_rst_n]
7
8  # HDMI管脚
9  set_property PACKAGE_PIN W16 [get_ports tmds_clk_p]
10 set_property IOSTANDARD TMDS_33 [get_ports tmds_clk_p]
11
12 set_property PACKAGE_PIN AA17 [get_ports tmds_data_p[0]]
13 set_property IOSTANDARD TMDS_33 [get_ports tmds_data_p[0]]
14
15 set_property PACKAGE_PIN U17 [get_ports tmds_data_p[1]]
16 set_property IOSTANDARD TMDS_33 [get_ports tmds_data_p[1]]
17
18 set_property PACKAGE_PIN U15 [get_ports tmds_data_p[2]]
19 set_property IOSTANDARD TMDS_33 [get_ports tmds_data_p[2]]
```

图 12-28 约束文件中写入管脚约束

9.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

Timing		Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	7.605 ns			
Total Negative Slack (TNS):	0 ns			
Number of Failing Endpoints:	0			
Total Number of Endpoints:	5043			
Implemented Timing Report				

图 12-29 时序结果

首先将电脑通过数据线与 FPGA Debug 接口进行连接，随后使用 HDMI 连接线将 HDMI 显示器连接到开发板上的 HDMI 接口。最后将本次实验生成的 BIT 文件下载下开发板中，下载完成之后 HDMI 显示器上显示彩条图案，说明本次实验在开发板上下载验证成功。

实验结果如下图所示：



图 12-30 HDMI 彩条

抓取 ILA 数据如下图所示:

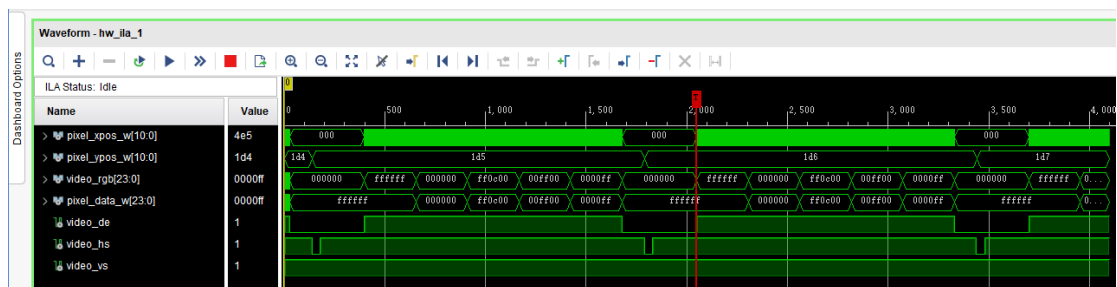


图 12-31 HDMI 彩条 ILA 数据

从 ILA 抓取的数据中可以看出，产生的数据是各个颜色的 24 bit 数据。

10.实验总结

到这里，本章节 HDMI 彩条实验的讲解就完毕。通过本章的实验，相信大家对于 HDMI 显示的基本知识和概念、TMDS 传输原理以及 OSERDESE2 原语的使用已经掌握了。希望大家可以多加练习，做到学以致用。

十三. HDMI 方块移动实验

在 HDMI 彩条显示实验中，成功地在显示器上显示出了静态的彩条图案。本章我们将带领大家基于上一章进行一个拓展实验，通过在屏幕上显示一个移动的小方块，来给大家演示如何使用 HDMI 接口显示动态图案。

关于 HDMI 的简介，硬件设计的内容与“HDMI 彩条显示实验”是完全一致的，这部分不过多介绍，如果有不了解的可以在上一章 HDMI 彩条显示实验找到介绍。那么接下来直接开始程序设计。

1.程序设计

1.1 HDMI 显示模块设计

根据实验任务可知，本章只需要修改 video_display 模块内部的代码。在本次实验中，为了完成方块的显示，需要同时使用像素点的横坐标和纵坐标来绘制方块所在的矩形区域，另外还需要知道矩形区域左上角的顶点坐标。由于 RGB 显示的图像在行场同步信号的同步下不停的刷新，因此只要连续改变方块左上角顶点的坐标，并在新的坐标点处重新绘制方块，即可实现方块移动的效果。

HDMI 显示模块框图与彩条显示模块完全一致，模块框图如下图所示：

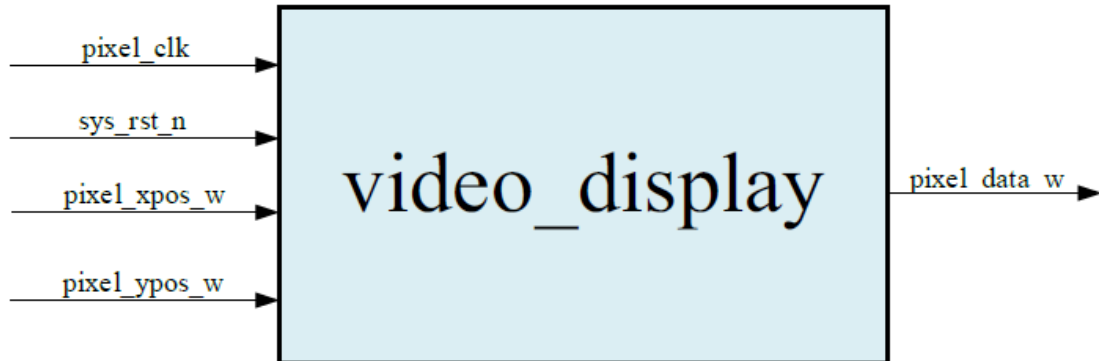


图 13-1 HDMI 显示模块框图

本次方块的大小为 40x40，且该方块的显示范围为 x 坐标为 (40~1239)，y 坐标为 (40~679)。其中 block_x 为方块当前横坐标，block_y 为当前方块纵坐标。当方块的宽度确定时，如果我们知道方块左上方顶点的坐标，就能轻而易举的画出整个方块区域。因此，我们将方块的移动简化为其左上角顶点的移动。

由于像素时钟相对于方块移动速度而言过快，我们通过计数器对时钟计数，得到一个频率为 100hz 的脉冲信号 move_en，用它作为使能信号来控制方块的移动。方块的移动方向分为水平方向 h_direct 和竖直方向 v_direct，当方块移动到上下边框时，竖直移动方向改变；当方块移动到左右边框时，水平移动方向改变。当 move_en 的频率为 100hz 时，方块每秒钟在水平和竖直方向上分别移动 100 个像素点的距离，也可以通过调整 move_en 的频率，来加快或减慢方块移动的速度。

(1) 首先产生一个 100hz 的时钟，在每次达到 100hz 时，将方块移动 100 个像素点，在

当前时钟下进行计数，代码如下

//通过对div 驱动时钟计数，实现时钟分频

```
always @(posedge pixel_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)
        div_cnt <= 22'd0;
    else begin
        if(div_cnt < DIV_CNT)
            div_cnt <= div_cnt + 1'b1;
        else
            div_cnt <= 22'd0;           //计数达10ms 后清零
    end
end
```

(2) 当方块移动到显示器边界时，需要改变当前方块的移动方向，其中每个边界依据像素点的位置已经确定了，那么只需要到达像素边界即可改变像素的移动方向。

//当方块移动到边界时，改变移动方向

```
always @(posedge pixel_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        h_direct <= 1'b1;           //方块初始水平向右移动
        v_direct <= 1'b1;           //方块初始竖直向下移动
    end
    else begin
        if(block_x == SIDE_W + 1'b1)           //到达左边界时，水平向右
            h_direct <= 1'b1;
        else           //到达右边界时，水平向左
            if(block_x == H_DISP - SIDE_W - BLOCK_W + 1'b1)
                h_direct <= 1'b0;
            else
                h_direct <= h_direct;

        if(block_y == SIDE_W + 1'b1)           //到达上边界时，竖直向下
            v_direct <= 1'b1;
        else           //到达下边界时，竖直向上
            if(block_y == V_DISP - SIDE_W - BLOCK_W + 1'b1)
                v_direct <= 1'b0;
            else
                v_direct <= v_direct;
    end
end
```

(3) 控制方块的移动边界后，每个时钟都会重新判断方块的移动位置，并将显示方块的位置，代码如下：

//根据方块移动方向，改变其纵横坐标

```
always @(posedge pixel_clk or negedge sys_rst_n) begin
```

```

if (!sys_rst_n) begin
    block_x <= SIDE_W + 1'b1;           // 方块初始位置横坐标
    block_y <= SIDE_W + 1'b1;           // 方块初始位置纵坐标
end
else if(move_en) begin
    if(h_direct)
        block_x <= block_x + 1'b1;     // 方块向右移动
    else
        block_x <= block_x - 1'b1;     // 方块向左移动

    if(v_direct)
        block_y <= block_y + 1'b1;     // 方块向下移动
    else
        block_y <= block_y - 1'b1;     // 方块向上移动
end
else begin
    block_x <= block_x;
    block_y <= block_y;
end
end
end

```

(4) 根据 RGB 驱动模块输出的纵横坐标判断当前像素点所在的区域, 对不同区域中的像素点赋以不同的颜色值, 从而实现边框、方块以及背景颜色的绘制。。

```

// 给不同的区域绘制不同的颜色
always @(posedge pixel_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)
        pixel_data <= BLACK;
    else begin
        if( (pixel_xpos < SIDE_W) || (pixel_xpos >= H_DISP - SIDE_W)
            || (pixel_ypos <= SIDE_W) || (pixel_ypos > V_DISP - SIDE_W))
            pixel_data <= BLUE;         // 绘制屏幕边框为蓝色
        else
            if( (pixel_xpos >= block_x - 1'b1) && (pixel_xpos < block_x +
BLOCK_W - 1'b1)
                && (pixel_ypos >= block_y) && (pixel_ypos < block_y +
BLOCK_W))
                pixel_data <= BLACK;    // 绘制方块为黑色
            else
                pixel_data <= WHITE;    // 绘制背景为白色
        end
    end
end
end

```

1.2 HDMI 方块移动顶层模模块

实验工程的其他各子功能模块均与上一章“HDMI 彩条实验”完全一致，在本小节我们只需要按照“HDMI 彩条实验”对顶层模块进行例化即可。

顶层模块 hdmi_block_move 的代码如下：

(1) 首先定义输入和输出的信号

```
module top_hdmi_block_move(
    input        sys_clk,
    input        sys_rst_n,

    output       tmds_clk_n,
    output       tmds_clk_p,
    output [2:0] tmds_data_n,
    output [2:0] tmds_data_p
);
```

(2) 随后例化显示驱动模块，显示模块以及并串转换模块。

```
// 例化 RGB 数据驱动模块
video_driver u_video_driver(
    .pixel_clk    ( pixel_clk ),
    .sys_rst_n    ( sys_rst_n ),

    .video_hs     ( video_hs ),
    .video_vs     ( video_vs ),
    .video_de     ( video_de ),
    .video_rgb    ( video_rgb ),
    .data_req     ( ),

    .pixel_xpos   ( pixel_xpos_w ),
    .pixel_ypos   ( pixel_ypos_w ),
    .pixel_data   ( pixel_data_w )
);

// 例化 RGB 数据显示模块
video_display u_video_display(
    .pixel_clk    ( pixel_clk ),
    .sys_rst_n    ( sys_rst_n ),

    .pixel_xpos   ( pixel_xpos_w ),
    .pixel_ypos   ( pixel_ypos_w ),
    .pixel_data   ( pixel_data_w )
);
```

```
// 例化 HDMI 驱动模块
dvi_transmitter_top u_rgb2dvi_0(
    .pclk          (pixel_clk),
    .pclk_x5       (pixel_clk_5x),
    .reset_n       (sys_rst_n ),

    .video_din     (video_rgb),
    .video_hsync   (video_hs),
    .video_vsync   (video_vs),
    .video_de      (video_de),

    .tmads_clk_p   (tmads_clk_p),
    .tmads_clk_n   (tmads_clk_n),
    .tmads_data_p  (tmads_data_p),
    .tmads_data_n  (tmads_data_n),
    .tmads_oen     () // 预留的端口，本次实验未用到
);
```

上述基本是对产生的模块进行例化的过程，其中 PLL 的时钟模块，产生一个 75M 时钟与 375M 时钟，PLL 中设置如下图所示：

Clocking Options Output Clocks Port Renaming MMCM Settings Summary									
The phase is calculated relative to the active input clock.									
Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives	
		Requested	Actual	Requested	Actual	Requested	Actual		
☒ clk_out1	clk_out1	75.000	75.00000	0.000	0.000	50.000	50.0	BUFG	
☒ clk_out2	clk_out2	375.000	375.00000	0.000	0.000	50.000	50.0	BUFG	
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	BUFG	

图 13-2 PLL 输出时钟

2.HDMI 方块移动仿真及仿真结果

(1) 对于 HDMI 方块移动实验来说，只需要产生一个时钟以及复位信号给模块即可，内部产生了所需要的时钟，图像数据等，为了加快仿真速度，将分频计数器设置为 75，仿真代码如下：

```
module TB_SIM_HDMI_MOVE();

reg          sys_clk      ;
reg          sys_rst_n    ;

wire         tmads_clk_n  ;
```

```

wire      tmds_clk_p ;
wire [2:0] tmds_data_n;
wire [2:0] tmds_data_p;

initial begin
    sys_clk = 1'b1;
    sys_rst_n <= 1'b0;
    #201
    sys_rst_n <= 1'b1;
end

always #10 sys_clk <= ~sys_clk;

defparam top_hdmi_block_move_inst.u_video_display.DIV_CNT = 75;

top_hdmi_block_move top_hdmi_block_move_inst(
    .sys_clk      (sys_clk      ),
    .sys_rst_n    (sys_rst_n    ),

    .tmds_clk_n   (tmds_clk_n   ),
    .tmds_clk_p   (tmds_clk_p   ),
    .tmds_data_n  (tmds_data_n  ),
    .tmds_data_p  (tmds_data_p  )
);

```

(2) 首先查看 DIV 分频模块，div_cnt 的最大计数为 75，当计数器计数到 74 是 move_en 拉高一个时钟周期，当计数到最大值后计数器清零。



图 13-3 div_cnt 仿真波形图

(3) 从行场信号像素点的仿真图，可以看出横坐标的像素点为 1~1280，纵坐标的像素点为 1~720，与我们编写的代码数据是一致的。



图 13-4 行场信号像素点

(4) 当 block_y 等于 41 时，v_direct 的使能拉高，当计数到 640 时，v_direct 的使能拉低。



图 13-5 方块垂直移动方向波形 a



图 13-6 方块垂直移动方向波形 b

(5) 方块水平移动的使能信号 h_direct 同理，当 block_x 等于 41 时，h_direct 使能拉高；当 block 等于 1200 时，hdirect 的使能拉低。



图 13-7 方块水平移动方向波形 a



图 13-8 方块水平移动方向波形 b

(6) 方块的值为 24'hff_ffff，从有效值区域的仿真图可以看出，横坐标的像素有效值区域为 41 到 1240。HDMI 显示模块与编写的代码是一致的，下面就可以进行下载验证了。



图 13-9 方块有效值区域 a

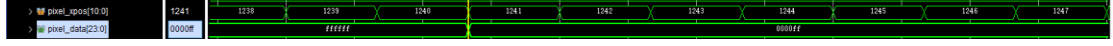


图 13-10 方块有效值区域 b

3.上板验证

注：方块移动实验采用的约束与“HDMI 彩条显示”实验的约束是一致的，因此可以将上一章中写好的约束直接复制到此章节的约束中，因此在此不进行重复编写。

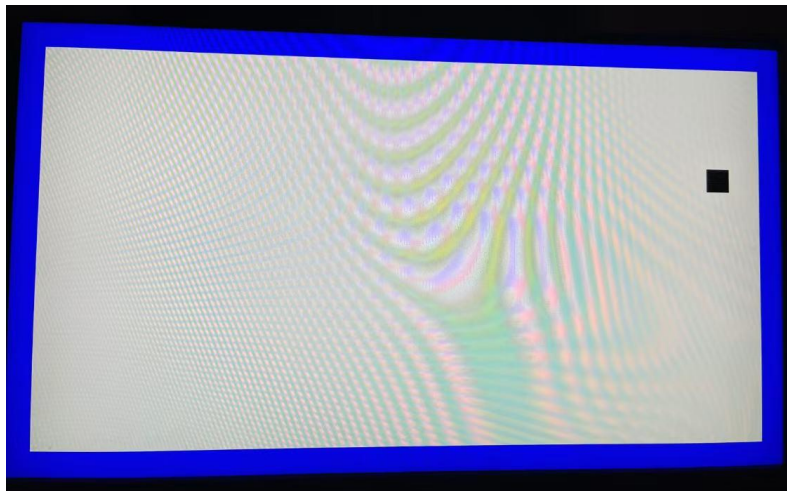
对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

Timing		Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	7.105 ns			
Total Negative Slack (TNS):	0 ns			
Number of Failing Endpoints:	0			
Total Number of Endpoints:	5104			
Implemented Timing Report				

图 13-11 时序结果

首先将电脑通过数据线与 FPGA Debug 接口。然后使用 HDMI 连接线将 HDMI 显示器连接到开发板上的 HDMI 接口。最后连接开发板的电源，并打开电源开关，然后将本次实验生成的 BIT 文件下载下开发板中，下载完成之后 HDMI 显示器上显示彩条图案，说明本次实验在开发板上下载验证成功。

实验结果如下图所示：



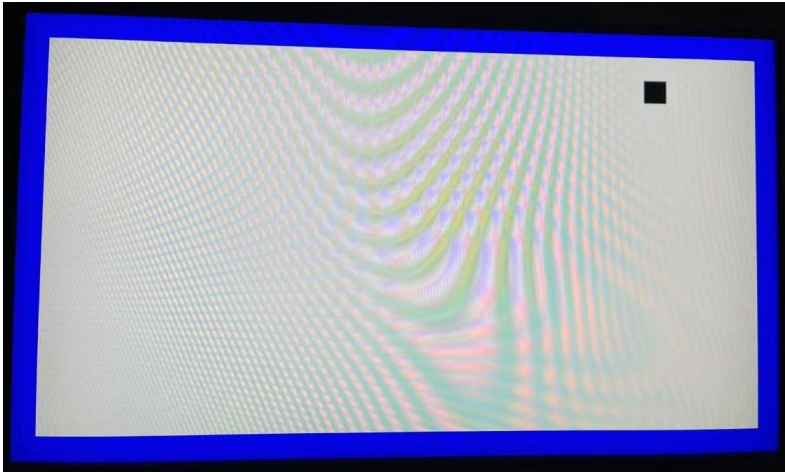


图 13-13 HDMI 方块移动效果图

抓取 ILA 数据如下所示：

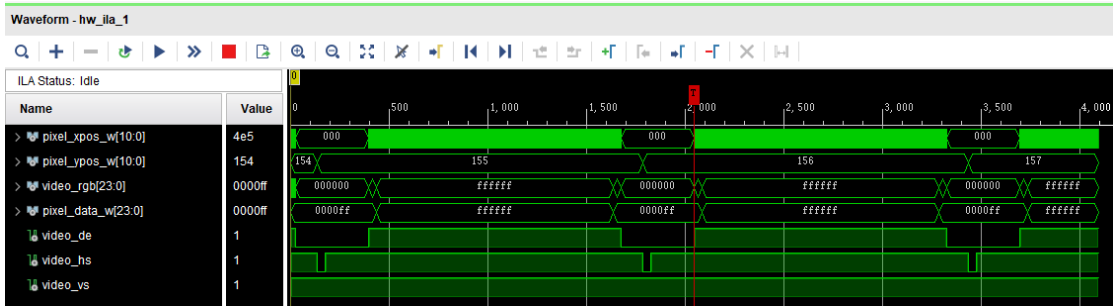


图 13-14 HDMI 方块 ILA 结果

从 ILA 结果中可以看出产生了对应的方块数据，以及背景颜色数据。

4.实验总结

本章节主要是基于“HDMI 彩条实验”进行了一个方块移动弹跳的实验，通过本章的实验，相信大家已经掌握了像素图像弹跳的方法。在实验中我们可以通过改变 DIV_CNT 参数改变方块弹跳的速度，DIV_CNT 的参数越大移动速度越慢，这个参数不是固定的，大家有兴趣可以自己尝试修改一下。

十四. IIC 协议的 EEPROM 读写实验

EEPROM 是一种用于计算机系统的非易失性存储器，也常在嵌入式领域中作为数据的存储设备，在物联网及可穿戴设备等需要存储少量数据的场景中也有广泛应用。本章我们学习设计一个 IIC 读写控制器，实现 FPGA 对 EEPROM 存储器的读写控制实验。虽然开发板的 PL 端没有板载 EEPROM，但作为一块开发板的教程，是需要有对 IIC 协议的支持，因此，对于 IIC 协议的 EEPROM 实验也是采用外接模块的方式，开始此次实验。

1.EEPROM 及 IIC 简介

EEPROM(Electrically Erasable Programmable Read Only Memory, E2PROM)是指带电可擦可编程只读存储器，是一种常用的非易失性存储器（掉电数据不丢失），E2PROM 有多种类型的产品，此次选用的模块为 ATMEL 公司生产的 AT24C 系列的 AT24C64 这一型号。AT24C64 具有高可靠性，可对所存数据保存 100 年，并可多次擦写，擦写次数达一百万次。

一般而言，对于存储类型的芯片，其存储容量是值得关注的，这次实验所用的 AT24C64 存储容量为 64Kbit，内部分成 256 页，每页 32 字节，共有 8192 个字节，且其读写操作都是以字节为基本单位。可以把 AT24C64 看作一本书，那么这本书有 256 页，每页有 32 行，每行有 8 个字，总共有 $256 \times 32 \times 8 = 65536$ 个字，对应着 AT24C64 的 $64 \times 1024 = 65536$ 个 bit。芯片资料可以在该实验提供的例程的文件夹中找到。那么，知道了 AT24C64 的存储容量，就知道了读写的空间大小。那么应该如何对 AT24C64 进行读写操作呢？

由于 AT24C64 采用两线串行接口的双向数据传输协议——I2C 协议实现读写操作，所以有必要了解一下 I2C 协议。

(1) IIC 物理层

I2C 即 Inter-Integrated Circuit(集成电路总线)，是由 Philips 半导体公司（现在的 NXP 半导体公司）在八十年代初设计出来的一种简单、双向、二线制总线标准。多用于主机和从机在数据量不大且传输距离短的情况下的主从通信。主机启动总线，并产生时钟用于传送数据，此时任何接收数据的器件均被认为是从机。

I2C 总线由数据线 SDA 和时钟线 SCL 构成通信线路，既可用于发送数据，也可接收数据。在主机与被控 IC 之间可进行双向数据传送，数据的传输速率在标准模式下可达 100kbit/s，在快速模式下可达 400kbit/s，在高速模式下可达 3.4Mbit/s，但目前大多 I2C 设备尚不支持高速模式。各种被控器件均并联在总线上，通过器件地址（SLAVE ADDR，具体可查器件手册）识别，I2C 总线物理拓扑结构如下图所示。

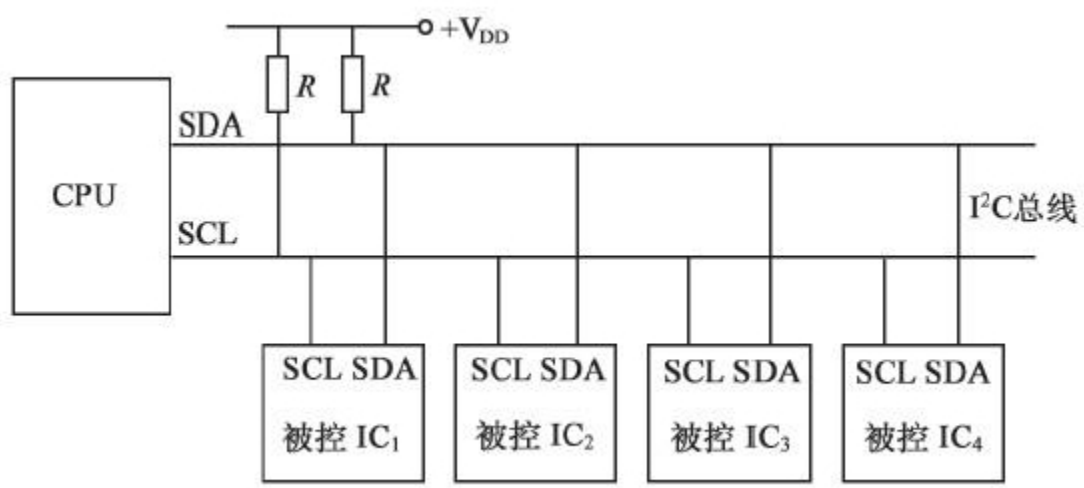


图 14-1 I2C 总线物理拓扑结构图

图中的 I2C_SCL 是串行时钟线，I2C_SDA 是串行数据线，由于 I2C 器件一般采用开漏结构与总线相连，所以 I2C_SCL 和 I2C_SDA 均需接上拉电阻，也正因此，当总线空闲时，这两条线路都处于高电平状态，当连到总线上的任一器件输出低电平，都将使总线拉低，即各器件的 SDA 及 SCL 都是“线与”关系。

I2C 总线支持多主和主从两种工作方式，通常工作在主从工作方式，我们的开发板就采用主从工作方式。在主从工作方式中，系统中只有一个主机，其它器件都是具有 I2C 总线的外围从机。在主从工作方式中，主机启动数据的发送（发出启动信号）并产生时钟信号，数据发送完成后，发出停止信号。

I2C 总线结构虽然简单，使用两线传输，然而要实现器件间的通信，需要通过控制 SCL 和 SDA 的时序，使其满足 I2C 的总线传输协议，方可实现器件间的数据传输。那么 I2C 协议的时序是怎样的呢？

(2) 协议层

IIC 协议的整体时序图如下图所示：

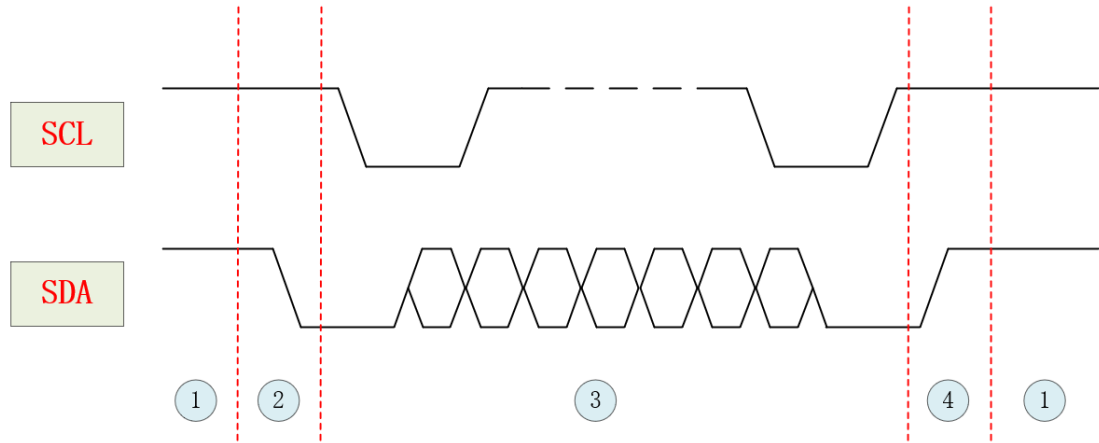


图 14-2 I2C 总体时序图

仔细观察图 14-2 可知 IIC 协议的整体时序由四个部分构成，上图中分别使用①、②、③、④进行区分。

图中①：总线空闲状态，在 I2C 器件开始通信（传输数据）之前，串行时钟线 SCL 和串行数据线 SDA 线由于上拉的原因处于高电平状态，此时 I2C 总线处于空闲状态。

图中②：起始信号，如果主机（此处指 FPGA）想开始传输数据，只需在 SCL 为高电平

时将 SDA 线拉低，产生一个起始信号。

图中③：数据传输状态，主机可以向从机写数据，也可以读取从机输出的数据，数据的传输由双向数据线（SDA）完成。

图中④：停止状态，当数据传输完成，主机只需产生一个停止信号，告诉从机数据传输结束，停止信号的产生是在 SCL 为高电平时，SDA 从低电平跳变到高电平，从机检测到停止信号后，停止接收数据，并且 I2C 总线跳转回总线空闲状态。

由于只有一根数据线进行数据的传输，如果不规定好传输规则肯定会导致信息错乱，如同在单条道路上驾驶，没有交通规则，再好的道路也会发生拥堵甚至更糟。采用两线结构的 I2C 虽然只有一根数据线，但由于还有一条时钟线，可以让数据线在时钟线的带领下有顺序的传送，就好像单条道路上的车辆在交警或信号指示灯的指示下有规则的通行。那么 I2C 遵循怎样的规则呢？

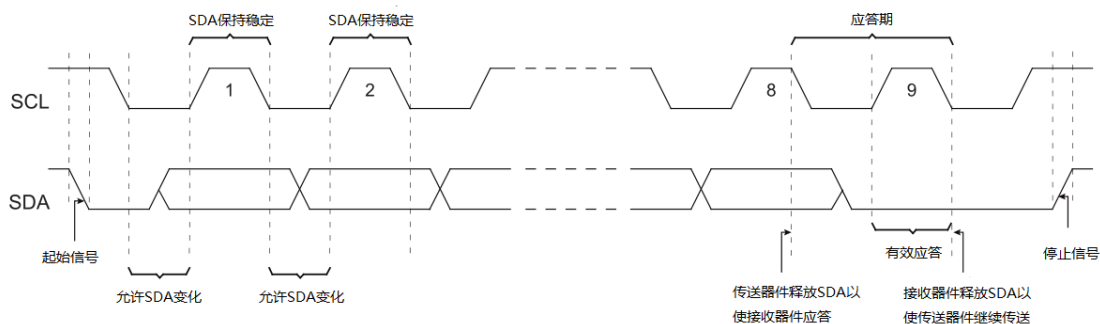


图 14-3 I2C 具体时序图

由图 14-3 可知，在 I2C 发送起始信号之后，主机开始发送传输的数据；在串行时钟线 SCL 为低电平状态时，SDA 允许改变传输的数据位（1 为高电平，0 为低电平），在 SCL 为高电平状态时，SDA 要求保持稳定，相当于一个时钟周期传输 1bit 数据，经过 8 个时钟周期后，传输了 8bit 数据，即一个字节。第 8 个时钟周期末，主机释放 SDA 以使从机应答，在第 9 个时钟周期，从机将 SDA 拉低以应答；如果第 9 个时钟周期，SCL 为高电平时，SDA 未被检测到为低电平，视为非应答，表明此次数据传输失败。第 9 个时钟周期末，从机释放 SDA 以使主机继续传输数据，如果主机发送停止信号，此次传输结束。需要注意的是数据以 8bit 即一个字节为单位串行发出，其最先发送的是字节的最高位。

I2C 的时序部分已经基本介绍完了，但还有一个小问题，就是当多个 I2C 器件挂在总线上时，怎样才能与我们想要传输数据的器件进行通信。这就涉及到了器件地址（也称从机地址，SLAVE ADDRESS）。

(3) IIC 器件地址

每个 I2C 器件都有一个器件地址，有些 I2C 器件的器件地址是固定的，而有些 I2C 器件的器件地址由一个固定部分和一个可编程的部分构成，这是因为很可能在一个系统中有几个同样的器件，器件地址的可编程部分能最大数量的使这些器件连接到 I2C 总线上，例如 E2PROM 器件，为了增加系统的 E2PROM 容量，可能需要多个 E2PROM。器件可编程地址位的数量由它可使用的管脚决定，比如 E2PROM 器件一般会留下 3 个管脚用于可编程地址位。但有些 I2C 器件在出厂时器件地址就设置好了，用户不可以更改（如实时时钟 PCF8563 的器件地址为固定的 7'h51）。所以当主机想给某个器件发送数据时，只需向总线上发送接收器件的器件地址即可。

对于 AT24C64 而言，其器件地址为 1010 加 3 位的可编程地址，3 位可编程地址由器件上的 3 个管脚 A2、A1、A0（见图 33.2.2）的硬件连接决定。当硬件电路上分别将这三个管脚连接到 GND 或 VCC 时，就可以设置不同的可编程地址。关于从机地址，是可以通过模

块上 A2, A1 及 A0 管脚的电平状态进行确认, 若这三个管脚接到地, 则该 E2PROM 的器件地址为 7'h50。

进行数据传输时, 主机首先向总线上发出开始信号, 对应开始位 S, 然后按照从高到低的位序发送器件地址, 一般为 7bit, 第 8bit 位为读写控制位 R/W, 该位为 0 时表示主机对从机进行写操作, 当该位为 1 时表示主机对从机进行读操作, 然后接收从机响应。对于 AT24C64 来说, 其传输器件地址格式如下图所示。

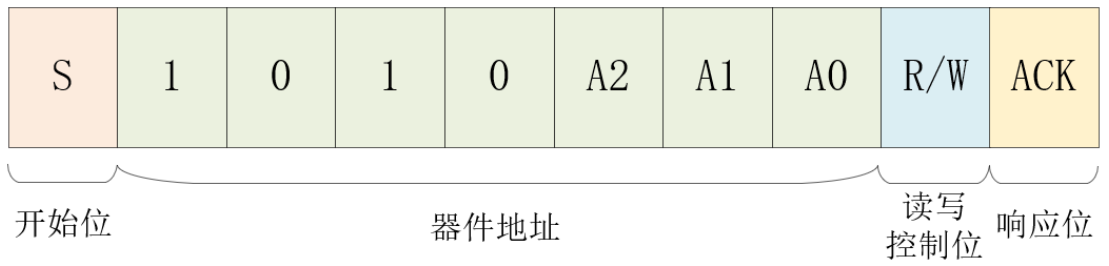


图 14-4 I2C 器件地址格式示意图

(4) 存储地址

发送完第一个字节 (7 位器件地址和一位读写控制位) 并收到从机正确的应答后就开始发送字地址 (Word Address)。一般而言, 每个兼容 I2C 协议的器件, 内部总会有可供读写的寄存器或存储器, 对于此次实验用到的 E2PROM 存储器, 内部就是一系列顺序编址的存储单元。所以, 当我们对一个器件中的存储单元 (包括寄存器) 进行读写时, 首先要指定存储单元的地址即字地址, 然后再向该地址写入内容。该地址为一个或两个字节长度, 具体长度由器件内部的存储单元的数量决定, 当存储单元数量不超过一个字节所能表示的最大数量 ($2^8=256$) 时, 用一个字节表示, 超过一个字节所能表示的最大数量时, 就需要用两个字节来表示, 例如同是 E2PROM 存储器, AT24C02 的存储单元容量为 2Kbit=256Byte (一般 bit 缩写为 b, Byte 缩写为 B), 用一个字节地址即可寻址所有的存储单元, 而 AT24C64 的存储单元容量为 64Kb=8KB, 需要 13 位 ($2^{13}=8KB$) 的地址位, 而 I2C 又是以字节为单位进行传输的, 所以需要两个字节地址来寻址整个存储单元。图 14-5 和图 14-6 分别为单字节字地址和双字节字地址器件的地址分布图, 其中单字节字地址的器件是以存储容量为 2Kb 的 E2PROM 存储器 AT24C02 为例, 双字节字地址的器件是以存储容量为 64Kb 的 E2PROM 存储器 AT24C64 为例, WA7 即字地址 Word Address 的第 7 位, 以此类推, 用 WA 是为了区别前面器件地址中的 A。



图 14-5 单字节字地址分布

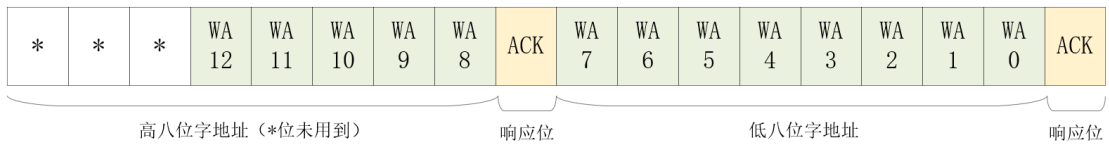


图 14-5 双字节字地址分布

(5) IIC 读写时序

主机发送完字地址，从机正确应答后就把内部的存储单元地址指针指向该单元。如果读写控制位 R/W 位为“0”即写命令，从机就处于接收数据的状态，此时，主机就开始写数据了，写数据可分为单次写（对于 E2PROM 而言，称为字节写）和连续写（对于 E2PROM 而言，称为页写）。下面我们先分别介绍 E2PROM 的单次写与连续写。

E2PROM 单次写（字节写）的时序如下图所示：

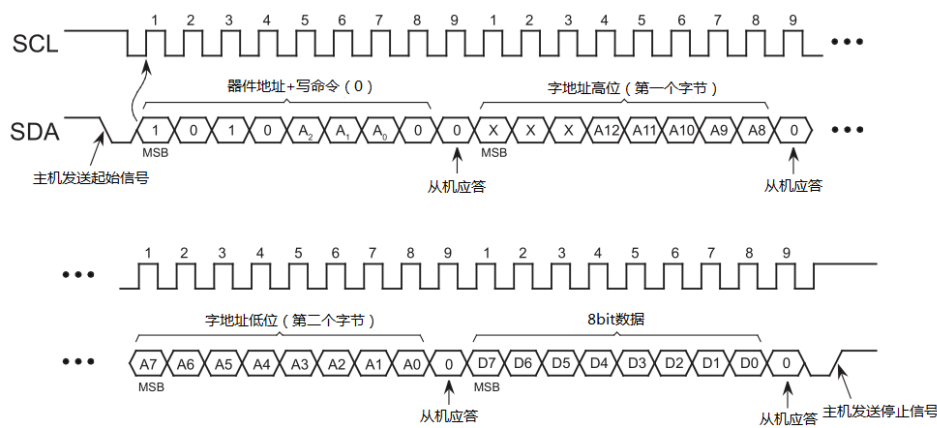


图 14-6 单次写（字节写）时序

参照时序图，列出单次写（字节写）操作流程如下：

- a. 主机产生并发送起始信号到从机，并且将写控制命令发送给从机设备，读写控制位设置为低电平，表示对从机进行写数据操作。注意，写控制命令的发送是高位在前低位在后；
- b. 从机接收到写控制指令后，回传应答信号，如果从机没有应答则会输出 I2C 通信错误信号，如果主机接收到应答信号，就开始字地址的写入。根据器件类型，我们需要先判断使用的器件是单字节地址还是双字节地址，若为双字节地址，先向从机写入高 8 位地址，且高位在前低位在后；待接收到从机回传的应答信号，再写入低 8 位地址，且高位在前低位在后，双字节字地址写入完成后执行步骤 d；若为单字节地址跳转到步骤 c。
- c. 按高位在前低位在后的顺序写入单字节的存储地址，在单字节字地址写入完成后执行步骤 d；
- d. 字地址写入完成，主机接收到从机回传的应答信号后，开始单字节数据的写入；
- e. 单字节数据写入完成，主机接收到应答信号后，向从机发送停止信号，单次写（字节写）完成。

E2PROM 单次写（字节写）的时序介绍完成，接下来我们开始介绍 E2PROM 的连续写（页写）时序。

E2PROM 连续写（页写）的时序如下图所示：

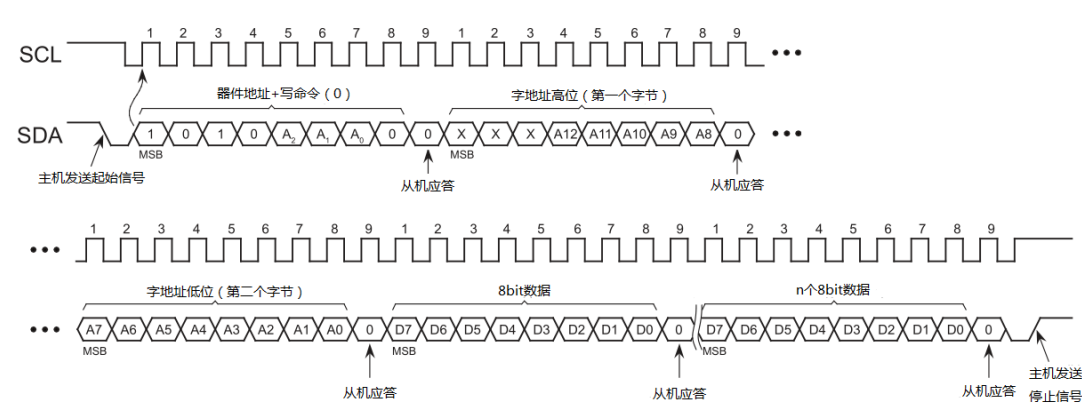


图 14-7 连续写（页写）时序

参照时序图，连续写（页写）操作流程如下：

- a. 主机产生并发送起始信号到从机，并且将写控制命令发送给从机设备，读写控制位设置为低电平，表示对从机进行写数据操作。注意，写控制命令的发送是高位在前低位在后；
- b. 从机接收到写控制指令后，回传应答信号，如果从机非应答，则会输出 I2C 通信错误信号，如果主机接收到从机应答信号，就开始字地址的写入。根据器件类型，我们需要先判断使用的器件是单字节地址还是双字节地址，若为双字节地址，先向从机写入高 8 位地址，且高位在前低位在后；待接收到从机回传的应答信号，再写入低 8 位地址，且高位在前低位在后，双字节地址写入完成后执行步骤 d；若为单字节地址跳转到步骤 c；
- c. 按高位在前低位在后的顺序写入单字节的存储地址，在单字节地址写入完成后执行步骤 d；
- d. 地址写入完成，主机接收到从机回传的应答信号后，开始第一个单字节数据的写入；
- e. 数据写入完成，主机接收到从机回传应答信号后，开始下一个单字节数据的写入；
- f. 直到所有数据写入完成，主机接收到从机回传应答信号后，执行步骤 g。若数据未完成写入，跳回到步骤 e；
- g. 主机向从机发送停止信号，连续写（页写）操作完成。

上面关于 E2PROM 的连续写（页写）时序也介绍完成，接下来总结下 E2PROM 单次写（字节写）与 E2PROM 连续写（页写）的区别：

对比图 14-6 和图 14-7 可知，两者的区别在于发送完一字节数据后，是发送结束信号还是继续发送下一字节数据，如果发送的是结束信号，就称为单次写，如果继续发送下一字节数据，就称为连续写。图 14-6 是 AT24C64 的单次写（字节写）时序，对于字地址为单字节的 I2C 器件而言，在发送完字地址（对应图 14-6 的字地址高位），且从机应答后即可串行发送 8bit 数据。图 14-7 是 AT24C64 连续写（页写）时序。要注意的是，所有 I2C 设备均支持单字节数据写入操作，但只有部分 I2C 设备支持页写操作，对于 AT24C64 的页写，是不能发送超过一页的单元容量的数据的，而 AT24C64 的一页的单元容量为 32Byte，当写完一页的最后一个单元时，地址指针指向该页的开头，如果再写入数据，就会覆盖该页的起始数据。

I2C 写时序介绍完毕后，接下来我们开始 I2C 读时序部分的介绍。根据一次读操作读取数据量的多少，读操作可分为随机读操作和顺序读操作，详细讲解如下。

在发送控制命令时，如果读写控制位 R/W 位为“1”即读命令，主机就处于接收数据的状态，从机从该地址单元输出数据。读数据有三种方式：当前地址读、随机读和连续读。当前地址读是指在一次读或写操作后发起读操作。由于 I2C 器件在读写操作后，其内部的地址指针自动加一，因此当前地址读可以读取下一个字地址的数据。也就是说上次读或写操作的单元地址为 02 时，当前地址读的内容就是地址 03 处的单元数据，时序图如图 14-8 所示。

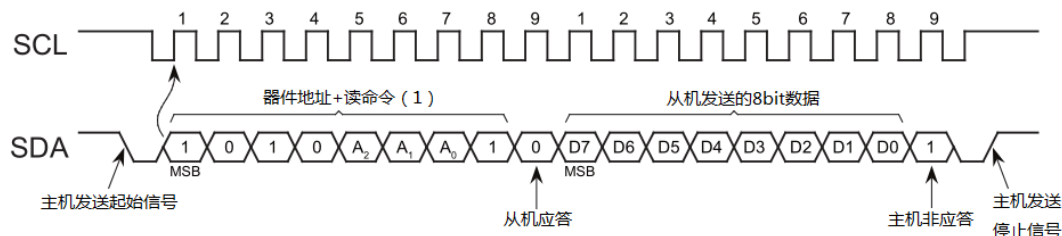


图 14-8 当前地址读时序

参照时序图，当前地址读操作流程如下：

- a. 主机产生并发送起始信号到从机，并且将读控制命令发送给从机设备，读写控制位设置为高电平，表示对从机进行读数据操作。注意，读控制命令的发送是高位在前低位在后；
- b. 从机接收到读控制指令后，如果回传非应答信号，则会输出 I2C 通信错误信号，如果回传应答信号，主机接收到从机回传的应答信号后，开始接收从机传回的单字节数据；

- c.数据接收完成后，主机产生一个时钟的高电平无应答信号；
- d.主机向从机发送停止信号，当前地址读操作完成。

由于当前地址读极不方便读取任意的地址单元的数据，所以就有了随机读，随机读的时序有点奇怪，见图 14-9。

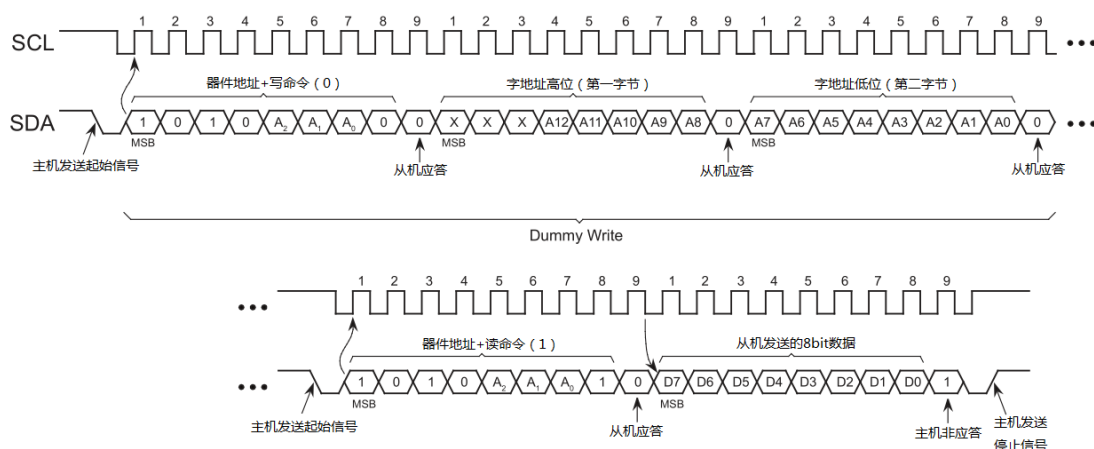


图 14-9 随机地址读时序

参照时序图，当前地址读操作流程如下：

- a.主机产生并发送起始信号到从机，并且将写控制命令发送给从机设备，读写控制位设置为低电平，表示对从机进行写数据操作。注意，写控制命令的发送是高位在前低位在后；
- b.从机接收到读控制指令后，如果回传非应答信号，则会输出 I2C 通信错误信号，如果回传应答信号，主机接收到应答信号后开始字地址的写入。根据器件类型，我们需要先判断使用的器件是单字节地址还是双字节地址，若为双字节地址，先向从机写入高 8 位地址，且高位在前低位在后；待接收到从机回传的应答信号，再写入低 8 位地址，且高位在前低位在后，双字节字地址写入完成后执行步骤 d；若为单字节地址跳转到步骤 c；
- c.按高位在前低位在后的顺序写入单字节的存储地址，单字节字地址写入完成后，则执行步骤 d；
- d.字地址写入完成，主机接收到从机回传的应答信号后，主机再次向从机发送一个起始信号；
- e.主机向从机发送读控制命令，读写控制位设置为高电平，表示对从机进行数据读操作；
- f.主机接收到从机回传的应答信号后，开始接收从机传回的单字节数据；
- g.数据接收完成后，主机产生一个时钟的高电平无应答信号；
- h.主机向从机发送停止信号，单字节读操作完成。

至此随机地址读时序的操作步骤介绍完成，需要注意理解的是随机地址读在发送完器件地址和字地址后，竟然又发送起始信号和器件地址，而且第一次发送器件地址时后面的读写控制位为“0”，也就是写命令，第二次发送器件地址时后面的读写控制位为“1”，也就是读。为什么会有这样奇怪的操作呢？这是因为我们需要使从机内的存储单元地址指针指向我们想要读取的存储单元地址处，所以首先发送了一次 Dummy Write 也就是虚写操作，之所以称为虚写，是因为我们并不是真的要写数据，而是通过这种虚写操作使地址指针指向虚写操作中字地址的位置，等从机应答后，就可以以当前地址读的方式读数据了，如图 14-9 所示，随机地址读是没有发送数据的单次写操作和当前地址读操作的结合体。

随机地址读时序介绍完成，接下来我们开始介绍连续读。当前地址读和随机读都是一次读取一个字节，连续读是将当前地址读或随机读的主机非应答改成应答，表示继续读取数据，图 14-10 是在当前地址读下的连续读。

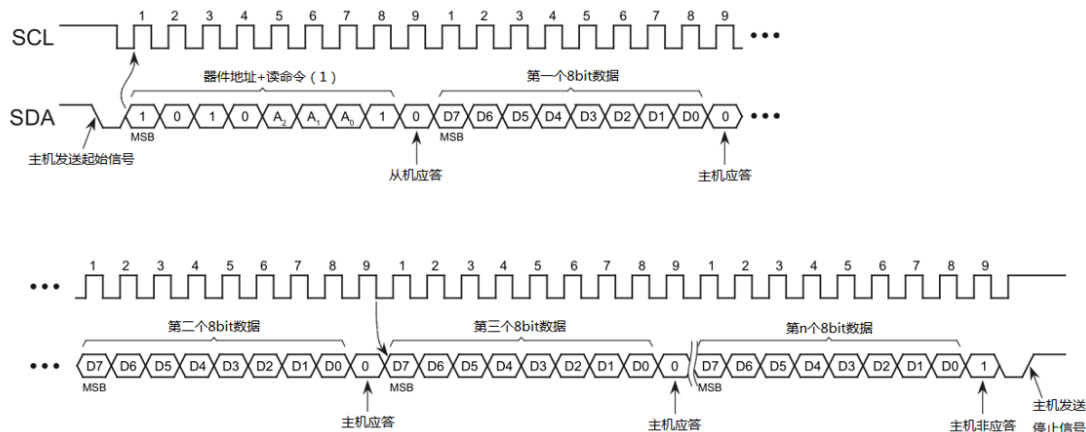


图 14-10 当前地址连续读时序

- 主机产生并发送起始信号到从机，并且将读控制命令发送给从机设备，读写控制位设置为高电平，表示对从机进行读数据操作。注意，读控制命令的发送是高位在前低位在后；
- 从机接收到读控制指令后，如果回传非应答信号，则会输出 I2C 通信错误信号，如果回传应答信号，主机接收到从机回传的应答信号后，开始接收从机传回的单字节数据；
- 数据接收完成后，主机产生应答信号回传给从机，从机接收到应答信号开始下一字节数据的传输，若数据传输完成，主机产生非应答信号回传给从机，执行下一操作步骤；若数据传输未完成，再次执行步骤 c；
- 主机向从机发送停止信号，当前地址连续读时序操作完成。

有当前地址连续读，当然也有随机地址连续读，随机地址连续读是在图 14-9 随机地址读时序的基础上，主机非应答改成应答，表示继续读取数据，图 14-11 是随机地址连续读。

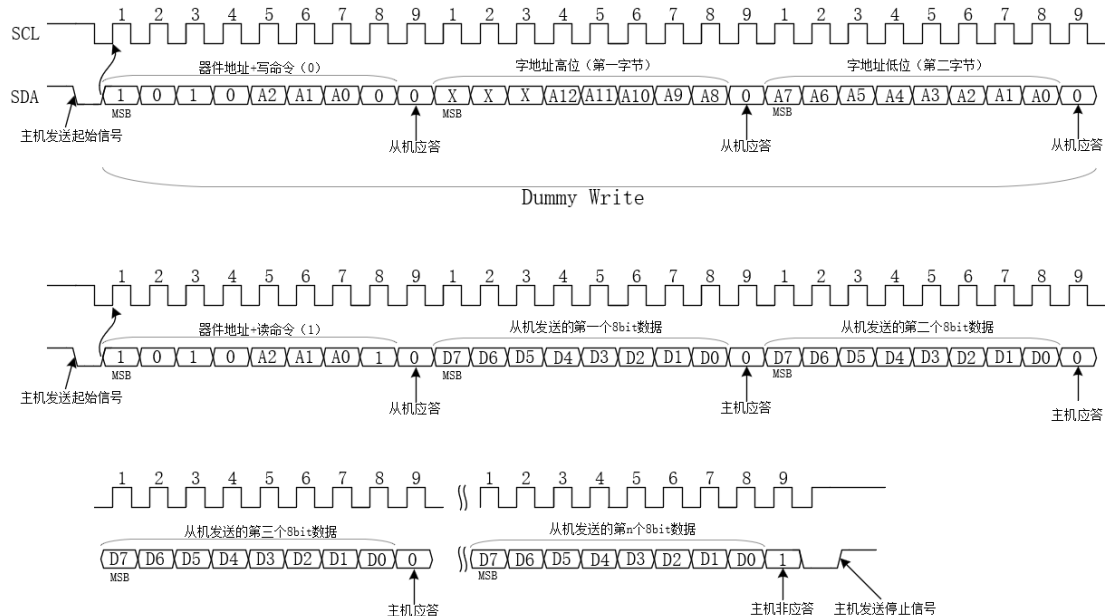


图 14-11 随机地址连续读时序

参照时序图，随机地址连续读操作流程如下：

- 主机产生并发送起始信号到从机，并且将写控制命令发送给从机设备，读写控制位设置为低电平，表示对从机进行写数据操作。注意，写控制命令的发送是高位在前低位在后；
- 从机接收到读控制指令后，如果回传非应答信号，则会输出 I2C 通信错误信号，如果回传应答信号，主机接收到应答信号后开始字地址的写入。根据器件类型，我们需要先判断

使用的器件是单字节地址还是双字节地址，若为双字节地址，先向从机写入高 8 位地址，且高位在前低位在后；待接收到从机回传的应答信号，再写入低 8 位地址，且高位在前低位在后，双字节地址写入完成后执行步骤 d；若为单字节地址跳转到步骤 c；

c.按高位在前低位在后的顺序写入单字节的存储地址，在单字节地址写入完成后执行步骤 d；

d.字地址写入完成，主机接收到从机回传的应答信号后，主机再次向从机发送一个起始信号；

e.主机向从机发送读控制命令，读写控制位设置为高电平，表示对从机进行数据读操作；

f.主机接收到从机回传的应答信号后，开始接收从机传回的单字节数据；

g.数据接收完成后，主机产生应答信号回传给从机，从机接收到应答信号开始下一字节数据的传输，若数据传输完成，主机产生非应答信号回传给从机，执行下一操作步骤；若数据传输未完成，再次执行步骤；

h.主机向从机发送停止信号，随机地址连续读时序操作完成。

(6) 上面几种 IIC 读写方法简单总结如下图所示：

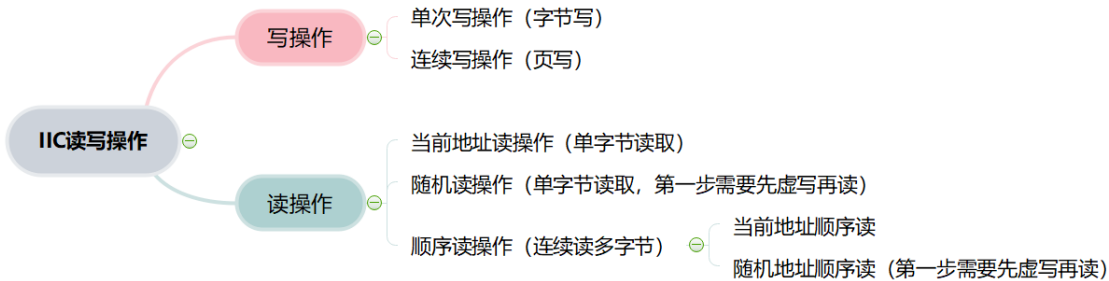


图 14-12 IIC 读写操作方法

至此，I2C 协议就基本讲完了，本章我们主要采用单次写和随机读的方式进行 E2PROM 读写测试

2.硬件设计

此次选用的 AT24C64 芯片模块如下图所示：



图 14-13 AT24C64 模块

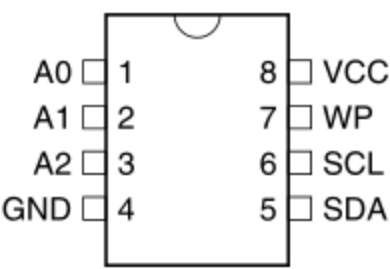


图 14-14 AT24C64 芯片引脚图

AT24C64 的引脚功能如下：

A2，A1，A0：可编程地址输入端；

GND：电源地引脚；

SDA：SDA（Serial Data，串行数据）是双向串行数据输入/输出端；

SCL：SCL（Serial clock，串行时钟）串行时钟输入端；

WP（写保护）：AT24C64 有一个写保护引脚用于提供数据保护，当写保护引脚连接至 GND 时，芯片可以正常写，当写保护引脚连接至 VCC 时，使能写保护功能，此时禁止向芯片写入数据，只能进行读操作；

在使用该模块时，需要将 VCC 管脚连接至 40 Pin 拓展 IO 的电源 VCC 管脚，将 GND 连接至 40 Pin 拓展 IO 的电源 GND 管脚。

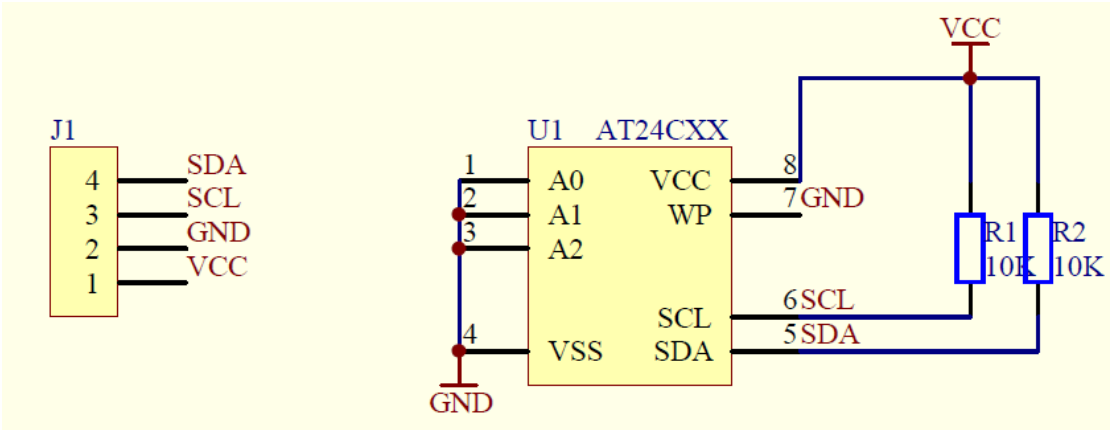


图 14-15 AT24C64 模块原理图

由上图可知，该 E2PROM 模块的可编程地址 A2、A1、A0 连接到地，所以 AT24C64 的器件地址为 1010000，如下图所示：



图 14-16 AT24C64 器件地址

3.实验任务

本节的实验任务是先向 E2PROM(AT24C64)的存储器地址 0 至 255 分别写入数据 0~255; 写完之后再读取存储器地址 0~255 中的数据, 若读取的值全部正确则 LED 灯常亮, 否则 LED 灯闪烁。

4.程序设计

由实验任务可知, 功能是对 E2PROM 模块进行读写测试, 并将读写测试的结果使用 LED 灯显示出来。对 E2PROM 模块进行读写测试, 是以 FPGA 为主机, E2PROM 模块为从机, FPGA 通过 IIC 协议对 E2PROM 模块进行读写控制, 所以在模块划分时需要一个 IIC 驱动模块和一个 E2PROM 读写模块, 两个模块分别命名为 i2c_dri 与 e2prom_rw; 对于读写测试的结果是使用 LED 显示结果表示的, 既读取的值全部正确则 LED 灯常亮, 否则 LED 灯闪烁, 所以还需要一个 E2PROM 读写测试结果显示模块, 该模块命名为 rw_result_led。最后, 需要顶层模块将各子功能模块例化起来, 连接个功能模块对应信号。本次实验需要使用系统时钟, 系统复位在 FPGA 系统中也是必不可少的, 当程序出现跑飞等异常情况时, 可以使程序恢复至默认状态; 由以上分析可知, 本次实验需要 2 个输入的端口, 分别为系统时钟、系统复位, 输出为 LED 灯端口、IIC 串行时钟信号 SCL 与 IIC 串行数据信号 SDA。综上所述, 实验工程整体框图, 如图 4-17 所示。

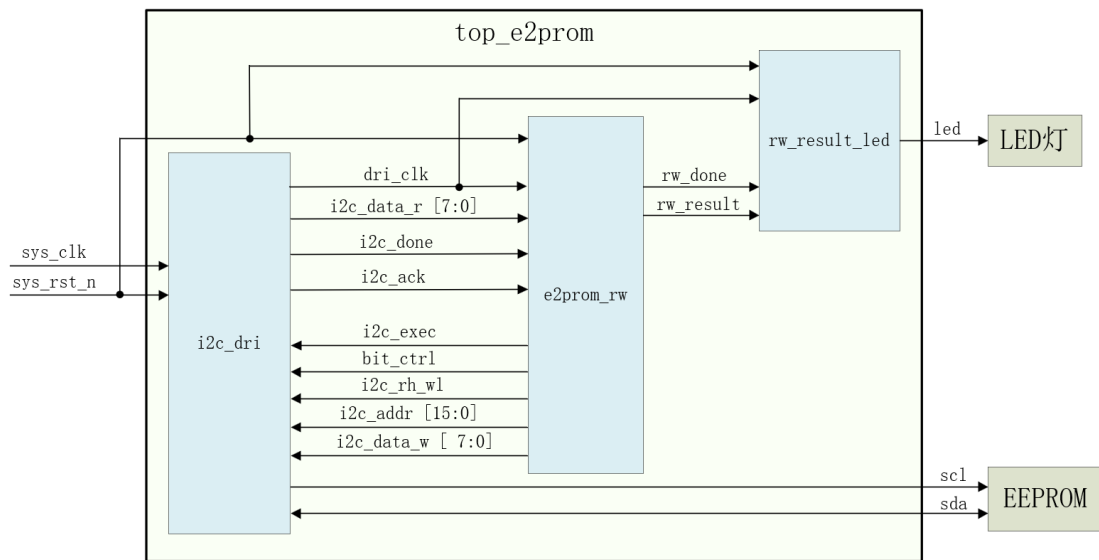


图 14-17 E2PROM 读写测试系统框图

4.1 IIC 驱动模块设计

首先介绍 IIC 驱动模块的设计, I2C 驱动模块的主要功能是按照 I2C 协议对 E2PROM 存储芯片执行数据读写操作。I2C 驱动模块框图见图 14-18。

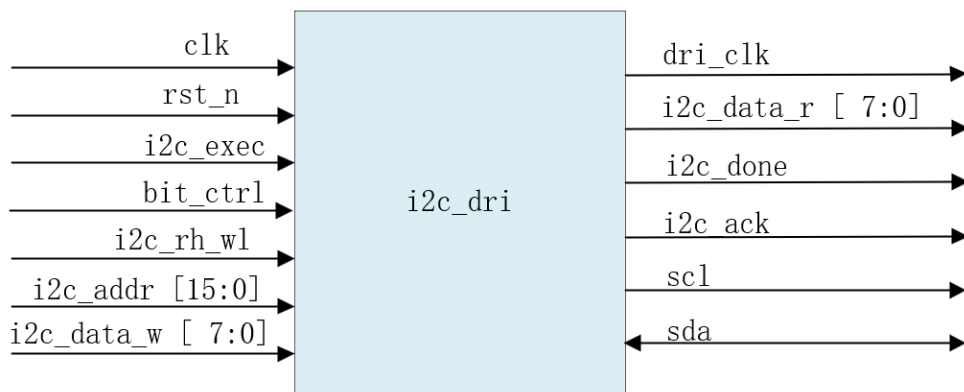


图 14-18 IIC 驱动模块框图

由图 14-18 可知，I2C 驱动模块包括 13 路输入输出信号，其中 7 路输入信号、5 路输出信号，还有一路 sda 既可以做输出，也可以做输入。输入信号中，clk、rst_n 是从顶层例化到 I2C 驱动模块的系统时钟和复位信号；i2c_exec 是 I2C 触发信号，由 e2prom 读写模块生成并传入，高电平有效；i2c_rh_wl 是 I2C 读写控制信号，i2c_rh_wl 为 1 表示进行读操作，i2c_rh_wl 为 0 表示进行写操作；与 i2c_exec 信号同时传入的还有字地址 i2c_addr[15:0] 和待写入字节数据 i2c_data_w[7:0]；当 I2C 触发执行信号有效，并且 i2c_rh_wl 信号为 0，模块执行单次写操作，按照 I2C 器件字地址 i2c_addr，向 E2PROM 对应地址写入数据 i2c_data_w；当 i2c_rh_wl 为 1，模块执行随机数据读操作，按照 I2C 器件字地址 i2c_addr 读取 E2PROM 对应地址中的数据；在前文中提到过，I2C 设备字地址有单字节和双字节两种，为了应对这一情况，我们向模块输入 bit_ctrl 信号，bit_ctrl 信号为字地址位控制信号，是顶层模块定义参数通过例化传入的 I2C 驱动模块，bit_ctrl 为 1 时表示是双字节字地址，在进行数据读写操作时要写入数据字地址 i2c_addr 的全部 16 位，bit_ctrl 为 0 时表示是单字节字地址，在进行数据读写操作时只写入数据字地址 i2c_addr 的低 8 位。

输出信号中，dri_clk 是本模块的工作时钟，由系统时钟 sys_clk 分频而来，它的时钟频率为串行时钟 scl 频率的 4 倍。I2C 起始信号是在 scl 为高电平时拉低 sda 信号产生的，I2C 停止信号是在 scl 为高电平时，sda 从低电平跳变到高电平产生的，使用 dri_clk 检测该起始信号与结束信号的波形如下图所示：

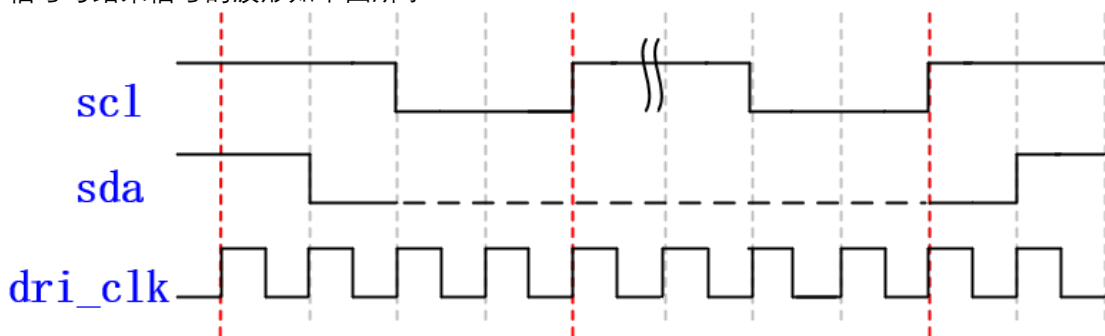


图 14-19 scl 的 4 倍频时钟 dri_clk

从图 14-19 中得知，当 dri_clk 是 scl 的 4 倍以上的频率时，能正确产生 I2C 起始信号和停止信号。

时钟信号 dri_clk 要传入 e2prom 读写模块（e2prom_rw）作为模块的工作时钟；输出给 e2prom 读写模块（e2prom_rw）的 I2C 一次操作完成信号 i2c_done，高电平有效，表示 I2C 一次操作完成；i2c_data_r[7:0] 信号表示自 E2PROM 读出的单字节数据，输出至 e2prom 读写模块（e2prom_rw）；scl、sda 分别是串行时钟信号和串行数据信号，由模块产生传入 E2PROM 存储芯片。

从 I2C 设备单次写操作和随机读操作的操作流程，使用状态机来实现 I2C 设备的读/写操作是十分方便的。参照 I2C 设备单次写操作和随机读操作的流程图，I2C 读/写操作状态转移图如下。

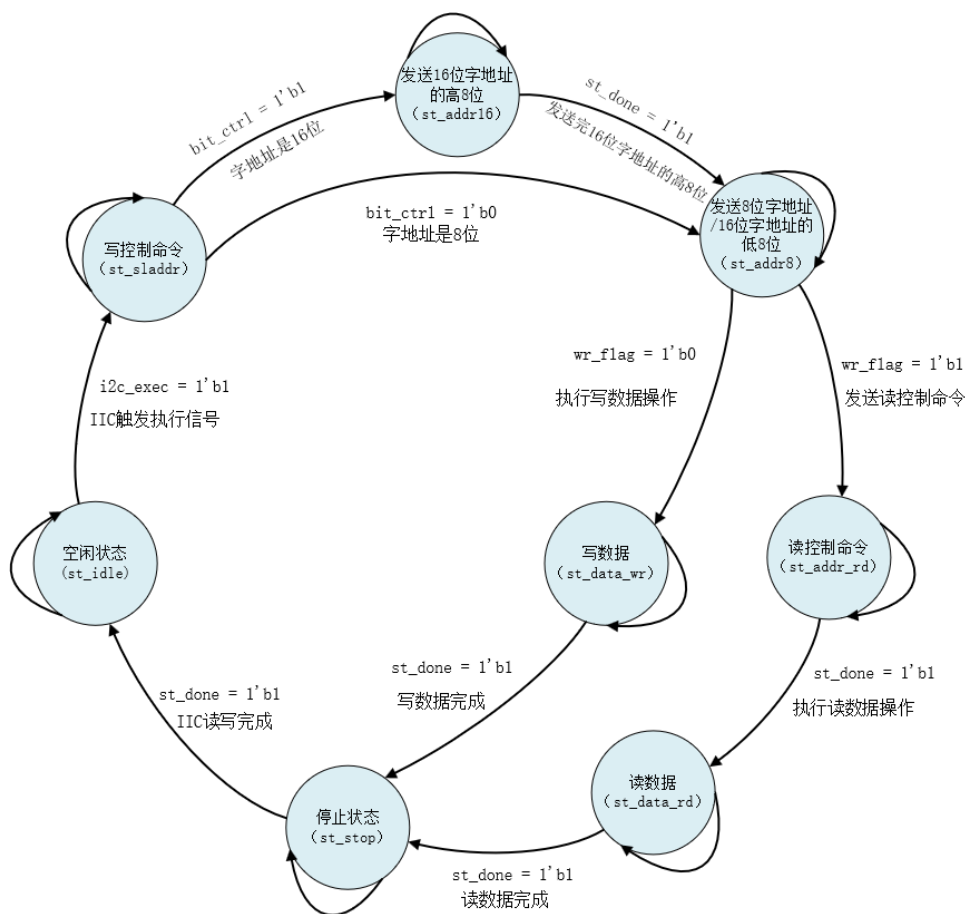


图 14-20 I2C 驱动模块状态跳转图

图 14-20 中各状态跳转如下所示：

(1) I2C 驱动模块接收到 I2C 触发执行信号 (i2c_exec) 后，IIC 控制总线从空闲状态跳转到写控制命令状态 (st_sladdr)，FPGA 开始向 E2PROM 发送写控制命令 (从机地址+写控制)；

(2) 写控制命令发送完成后，FPGA 开始向 E2PROM 发送字地址 (i2c_addr[15:0]) 数据，I2C 驱动模块根据接收到的字地址位控制信号 (bit_ctrl) 进行判断，如果 bit_ctrl 是 0，写单字节的字地址 (st_addr8)，如果 bit_ctrl 是 1，写双字节的字地址 (st_addr16)，此次使用的 E2PROM 是 AT24C64，所以 I2C 驱动模块接收到的字地址位控制信号是 1，从写控制命令状态 (st_sladdr) 跳转到写双字节的字地址 (st_addr16) 状态；

(3) 定义的写地址 (st_addr16) 状态是写双字节器件地址的高八位状态，因为写双字节字地址的低八位状态与写单字节字地址状态是一样的，所以 st_addr8 既是写双字节字地址的低八位状态，也是写单字节字地址状态。在发送完双字节器件地址的高八位后下一个状态跳转到发送双字节字地址的低八位状态；

(4) 器件地址发送完成后，I2C 驱动模块写标志信号 (wr_flag，由 i2c_rh_wl 在 i2c_exec 为高时寄存得到) 来判断下一步是进行读操作还是写操作。如果 i2c_rh_wl 信号为 0，表示即将进入写数据状态 (st_data_wr)，顺序执行第 (5) 步，如果 i2c_rh_wl 信号为 1，跳转到第 (6) 步发送读控制命令状态 st_addr_rd ((1)、(2) 与 (3) 是随机读之前虚写)；

(5) 进入写数据状态 (st_data_wr)，I2C 驱动模块将接收的要写的的数据 (i2c_data_w[7:0])

写入 E2PROM 存储芯片，写操作完成后直接跳到第（8）步进入停止状态；

（6）st_addr_rd 是 FPGA 开始向 E2PROM 发送读控制命令（重新开始+器件地址+读控制），读控制命令发送完成后，开始下一步的读数据操作（st_data_rd）；

（7）读数据操作（st_data_rd）是 I2C 驱动模块将发送的要读的数据（i2c_data_r[7:0]）从 E2PROM 存储芯片中读出，读操作完成后顺序执行下一步；

（8）进入停止状态后（st_stop），FPGA 向 E2PROM 发送 I2C 操作完成信号（st_done），一次 I2C 操作完成，随后状态机跳回 st_idle(初始状态)，等待下一次 I2C 触发执行信号（i2c_exec）。

输出的串行数据 sda 作为一个双向端口，主机通过它向从机发送控制指令、地址以及数据，接收从机回传的应答信号和读取数据。回传给主机的应答信号是校验主机发送数据是否成功的条件之一。

信号 sda_in 作为串行数据 sda 缓存，i2c_ack 信号作为应答信号，如果在主机发送完成后，从机发送非应答信号（sda_in = 1），此时给 i2c_ack 信号赋值一个高电平，代表数据传输错误，会使 led 灯闪烁，提示 I2C 通信错误，接下来清空 cnt 计数跳转到下一个状态开始工作。

i2c_ack 信号只在状态机处于各应答状态时根据 sda_in 信号进行赋值，此时为从机回传的应答信号，其他状态时钟保持低电平。

在 I2C 总线处于空闲状态时，给 I2C 数据方向控制（sda_dir）信号赋值高电平，将 sda 总线的方向设置为主机 FPGA 输出数据。至于为什么要定义一个 I2C 数据方向控制信号，是因为数据线 SDA 是双向的，如图 14-23 所示，为了避免主机、从机同时操作数据线，可以在 FPGA 内部可以使用三态门结构避免此事件发生。sda_dir 表示 I2C 数据方向，为 1 时表示主机（FPGA）输出信号，为 0 时 FPGA 输出高阻态，表示释放控制权。如图 14-24 所示：

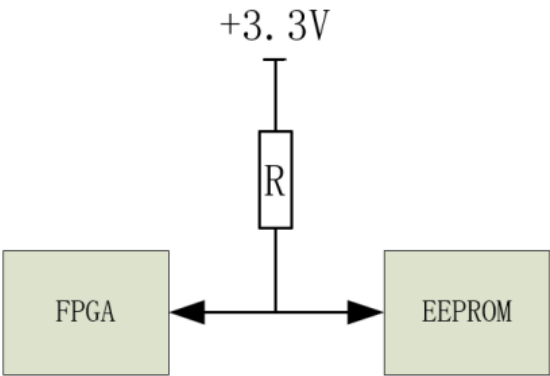


图 14-23 SDA 双向数据线

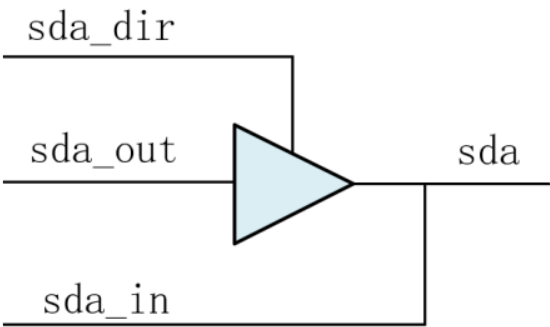


图 14-24 sda_dir 控制 I2C 数据方向

所以在 I2C 单次写操作，既每次 FPGA 在输出数据时，在数据传输之前都需要先将

sda_dir 信号拉高，在数据传输完成后再将 sda_dir 信号拉低，将 SDA 总线的控制权交给从机发送响应数据。

在空闲状态，接收到 I2C 触发信号后进行执行 I2C 操作，并且接收的读写控制信号 i2c_rh_wl 也为低电平时，状态机从空闲状态跳转到发送写命令状态 (st_sladdr)，并且将接收的 I2C 读写控制信号 (i2c_rh_wl) 赋值给写标志 wr_flag，将接收的 I2C 字地址寄存为 addr_t，将接收的 i2c 将写数据寄存为 data_wr_t，I2C 应答信号 i2c_ack 一直处于应答状态。

上述则为 I2C 协议的基本操作过程，那么接下来则对直接对 I2C 协议进行代码编写。

(1) 对上述描述的状态机进行参数定义，以及将控制 SDA 的三态门进行控制，在整个 IIC 传输过程中，SCL 信号是由主模块传输给从模块的时钟信号，SDA 既可以主机发送从机接收，也可以是主机接收从机发送的信号，因而 SDA 线是双向的信号，在 RTL 中用 inout 接口类型来表示该双向信号，在这里即涉及到一个三态门的实现，即当 IIC 进行主机发送从机接收传输时，sda_dir 信号置高，sda 线上信号即为主机发送信号，当从机发送主机接收时，sda_dir 信号拉低，此时 sda 与 sda_out 信号间为高阻状态，视为断开，此时通过 sda_in 信号来获取 sda 信号线上的输入数据。

```
//localparam define
localparam st_idle      = 8'b0000_0001; //空闲状态
localparam st_sladdr    = 8'b0000_0010; //发送器件地址(slave address)
localparam st_addr16    = 8'b0000_0100; //发送16位字地址
localparam st_addr8     = 8'b0000_1000; //发送8位字地址
localparam st_data_wr   = 8'b0001_0000; //写数据(8 bit)
localparam st_addr_rd   = 8'b0010_0000; //发送器件地址读
localparam st_data_rd   = 8'b0100_0000; //读数据(8 bit)
localparam st_stop      = 8'b1000_0000; //结束I2C操作
//SDA控制
assign sda      = sda_dir ? sda_out : 1'bz ; //SDA数据输出或高阻
assign sda_in   = sda ; //SDA数据输入
```

(2) 产生一个4倍频时钟作为 I2C 模块的驱动时钟

```
//生成I2C的SCL的四倍频率的驱动时钟用于驱动i2c的操作
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        dri_clk <= 1'b0;
        clk_cnt <= 10'd0;
    end
    else if(clk_cnt == (clk_divide[8:1] - 9'd1)) begin
        clk_cnt <= 10'd0;
        dri_clk <= ~dri_clk;
    end
    else
        clk_cnt <= clk_cnt + 10'b1;
    end
```

(3) 三段式状态机中状态转移，其中每个状态都可以参考图 4-20 进行跳转，下面状态转移代码是将图 4-20 转换成 RTL 代码。

```

//组合逻辑判断状态转移条件
always @(*) begin
    next_state = st_idle;
    case(cur_state)
        st_idle: begin                                //空闲状态
            if(i2c_exec) begin
                next_state = st_sladdr;
            end
            else
                next_state = st_idle;
        end
        st_sladdr: begin
            if(st_done) begin
                if(bit_ctrl)                            //判断是16位还是8位字地址
                    next_state = st_addr16;
                else
                    next_state = st_addr8 ;
            end
            else
                next_state = st_sladdr;
        end
        st_addr16: begin                                //写16位字地址
            if(st_done) begin
                next_state = st_addr8;
            end
            else begin
                next_state = st_addr16;
            end
        end
        st_addr8: begin                                //8位字地址
            if(st_done) begin
                if(wr_flag==1'b0)                        //读写判断
                    next_state = st_data_wr;
                else
                    next_state = st_addr_rd;
            end
            else begin
                next_state = st_addr8;
            end
        end
        st_data_wr: begin                                //写数据(8 bit)
            if(st_done)

```

```

        next_state = st_stop;
    else
        next_state = st_data_wr;
    end
st_addr_rd: begin                                // 写地址以进行读数据
    if(st_done) begin
        next_state = st_data_rd;
    end
    else begin
        next_state = st_addr_rd;
    end
end
st_data_rd: begin                                // 读取数据(8 bit)
    if(st_done)
        next_state = st_stop;
    else
        next_state = st_data_rd;
    end
end
st_stop: begin                                    // 结束 I2C 操作
    if(st_done)
        next_state = st_idle;
    else
        next_state = st_stop ;
    end
    default: next_state= st_idle;
endcase
end

```

(4) 接下来就是编写三段式状态机的第三段的代码，使用时序逻辑描述状态的输出，主要以写器件地址为例，当 SDA 拉低时，表示一次传输开始，随后发送器件地址，代码如下所示：

```

st_sladdr: begin                                // 写地址( 器件地址和字地址)
    case(cnt)
        7'd1 : sda_out <= 1'b0;                // 开始 I2C
        7'd3 : scl <= 1'b0;
        7'd4 : sda_out <= SLAVE_ADDR[6]; // 传送器件地址
        7'd5 : scl <= 1'b1;
        7'd7 : scl <= 1'b0;
        7'd8 : sda_out <= SLAVE_ADDR[5];
        7'd9 : scl <= 1'b1;
        7'd11: scl <= 1'b0;
        7'd12: sda_out <= SLAVE_ADDR[4];
        7'd13: scl <= 1'b1;
    endcase
end

```

```

7'd15: scl <= 1'b0;
7'd16: sda_out <= SLAVE_ADDR[3];
7'd17: scl <= 1'b1;
7'd19: scl <= 1'b0;
7'd20: sda_out <= SLAVE_ADDR[2];
7'd21: scl <= 1'b1;
7'd23: scl <= 1'b0;
7'd24: sda_out <= SLAVE_ADDR[1];
7'd25: scl <= 1'b1;
7'd27: scl <= 1'b0;
7'd28: sda_out <= SLAVE_ADDR[0];
7'd29: scl <= 1'b1;
7'd31: scl <= 1'b0;
7'd32: sda_out <= 1'b0;           //0:写
7'd33: scl <= 1'b1;
7'd35: scl <= 1'b0;
7'd36: begin
        sda_dir <= 1'b0;
        sda_out <= 1'b1;
    end

```

至此，I2C 驱动模块的代码已经编写完成，接下来我们开始编写 E2PROM 模块的代码。

4.2 E2PROM 读写模块

E2PROM 读写模块主要实现对 I2C 读写过程的控制，包括给出字地址及需要写入该地址中的数据、启动 I2C 读写操作、判断读写数据是否一致等。

E2PROM 读写模块框图和输入输出端口如图 14-25 所示。

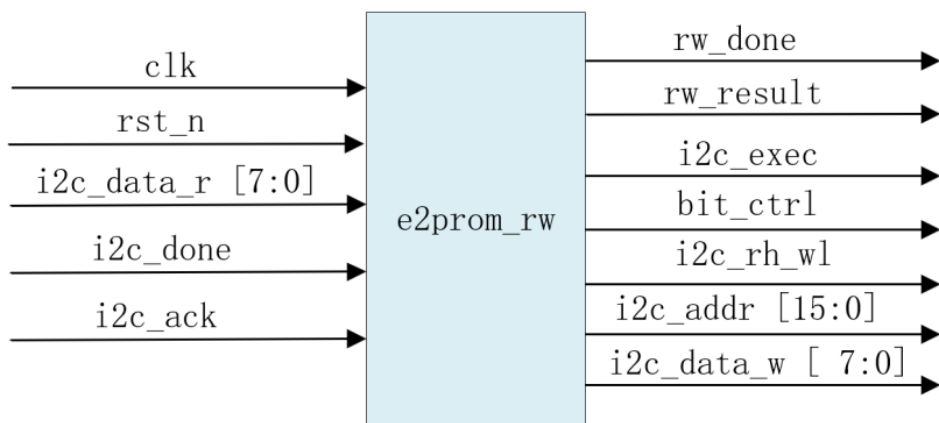


图 14-25 E2PROM 读写模块框图

E2PROM 读写模块包括 11 路输入输出信号，其中 5 路输入信号、6 路输出信号。输入信号中，clk、rst_n 是从顶层例化到 E2PROM 读写模块的 I2C 驱动时钟 dri_clk 和系统复位信号；i2c_data_r 是从 E2PROM 读出的数据，i2c_done 是一次 I2C 操作完成信号，i2c_ack 是 I2C 应答标志。这三个信号都是由 I2C 驱动模块（i2c_dri.v）输入进来。i2c_rh_wl 是 I2C 读

写控制信号，初始值为 0，表示在进行单次写，在写完 256 个数据后，拉高该信号，I2C 开始随机读操作；i2c_exec 是 I2C 触发执行信号，i2c_exe 信号拉高一个周期触发一次 I2C 操作；i2c_addr 是 I2C 器件字地址，i2c_data_w 是 I2C 要写的的数据，初始值都为 0，随着每次单次写操作完成信号 i2c_done 逐次加 1。rw_done 是 E2PROM 读写测试完成信号，在 I2C 读写完成拉高一个周期；rw_result 是 E2PROM 读写测试结果，将读取的数据与写入的数据进行对比，两者一致说明 E2PROM 读写测试成功，将 rw_result 拉为高电平；rw_done 与 rw_result 会传入读写测试结果显示模块 (rw_result_led.v)。

(1) 首先对 I2C 驱动模块发起写操作，即拉高 i2c_exec，拉低 i2c_rh_wl (低电平表示写)，然后分别向 E2PROM 的地址 0 至地址 255 写入数据 0 至 255，并且在每次写操作之间增加 5ms 的延时，由于代码过多，贴出关键部分的代码进行分析。

```
case(flow_cnt)
    2'd0 : begin
        wait_cnt <= wait_cnt + 14'b1;           // 延時計數
        if(wait_cnt == (WR_WAIT_TIME - 14'b1)) begin //EEPROM 写
操作延时完成
            wait_cnt <= 14'b0;
            if(i2c_addr == MAX_BYTE) begin //256 个字节写入完成
                i2c_addr <= 16'b0;
                i2c_rh_wl <= 1'b1;
                flow_cnt <= 2'd2;
            end
        else begin
            flow_cnt <= flow_cnt + 2'b1;
            i2c_exec <= 1'b1;
        end
    end
end
2'd1 : begin
    if(i2c_done == 1'b1) begin //EEPROM 单次写入完成
        flow_cnt <= 2'd0;
        i2c_addr <= i2c_addr + 16'b1; //地址 0~255 分别写入
        i2c_data_w <= i2c_data_w + 8'b1; //数据 0~255
    end
end
end
```

(2) 数据全部写完后，发起读操作，即拉高 i2c_exec，拉高 i2c_rh_wl (高电平表示读)，然后分别从 E2PROM 的地址 0 至地址 255 读出数据，并判断读出的值与写入的值是否一致，如果数据一致并且每次操作 IIC 都有应答信号产生 (i2c_ack)，E2PROM 的读写测试才正确，否则读写测试失败，该部分代码与 (1) 中代码是一部分的代码，所以只贴除了关键代码。

```
2'd2 : begin
    flow_cnt <= flow_cnt + 2'b1;
    i2c_exec <= 1'b1;
end
```

```

        2'd3 : begin
            if(i2c_done == 1'b1) begin          //EEPROM 单次读出完成
                //读出的值错误或者 I2C 未应答, 读写测试失败
                if((i2c_addr[7:0] != i2c_data_r) || (i2c_ack ==
1'b1)) begin
                    rw_done <= 1'b1;
                    rw_result <= 1'b0;
                end
                else if(i2c_addr == (MAX_BYTE - 16'b1))begin //读写测
试成功
                    rw_done <= 1'b1;
                    rw_result <= 1'b1;
                end
                else begin
                    flow_cnt <= 2'd2;
                    i2c_addr <= i2c_addr + 16'b1;
                end
            end
        end
    end
    default : ;
endcase

```

4.3 E2PROM 读写测试结果显示模块

读写测试完成后, 输出 rw_done 信号和 rw_result 信号, rw_done 为 E2PROM 读写测试完成信号, rw_result 为读写测试的结果, 0 表示读写失败, 1 表示读写正确。在 E2PROM 读写测试结果显示模块, 我们通过 LED 灯的状态来展示 E2PROM 读写测试的结果, led 灯保持常亮表示 E2PROM 读写测试成功, led 灯以 0.25s 间隔闪烁表示 E2PROM 读写测试成功失败。

读写测试完成后, 输出 rw_done 信号和 rw_result 信号, rw_done 为 E2PROM 读写测试完成信号, rw_result 为读写测试的结果, 0 表示读写失败, 1 表示读写正确。这部分代码十分简单, 只需要将读写结果代码输入该模块, 并以此驱动 LED 即可。

```

//读写测试完成标志
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        rw_done_flag <= 1'b0;
    else if(rw_done)
        rw_done_flag <= 1'b1;
end

//错误标志为1时PL_LED0 闪烁, 否则PL_LED0 常亮
always @(posedge clk or negedge rst_n) begin

```

```

    if(!rst_n) begin
        led_cnt <= 17'd0;
        led <= 1'b0;
    end
    else begin
        if(rw_done_flag) begin
            if(rw_result)                //读写测试正确
                led <= 1'b1;            //led 灯常亮
            else begin                    //读写测试错误
                led_cnt <= led_cnt + 17'd1;
                if(led_cnt == (L_TIME - 17'b1)) begin
                    led_cnt <= 17'd0;
                    led <= ~led;        //led 灯闪烁
                end
            end
        else
            led <= led;
        end
    end
end
else
    led <= 1'b0;                        //读写测试完成之前,led
灯熄灭
end
end

```

最后只需将三个模块同时在 top 层进行例化即可。

5.基于 I2C 协议的 EEPROM 仿真

5.1 添加 EEPROM 仿真文件

(1) 对于仿真代码的编写与前面的仿真是一致的, 首先产生时钟及复位, 其次在第 11 行代码定义的 led 灯闪烁间隔时间从比较长的时间 0.25s 修改为 1us, 第 12 行代码将读写的字节个数从 256 个修改为读写三个字节, 节约仿真时间, 提高仿真效率。

```

module TB_SIM_EEPROM;

//parameter define
parameter T = 20                                ; //时钟周期为 20ns
parameter SLAVE_ADDR = 7'b1010000                ; //器件地址(SLAVE_ADDR)
parameter BIT_CTRL = 1'b1                        ; //字地址位控制参数(16b/8b)
parameter CLK_FREQ = 26'd50_000_000              ; //i2c_dri 模块的驱动时钟频率
(CLK_FREQ)
parameter I2C_FREQ = 18'd250_000                ; //I2C 的 SCL 时钟频率

```

```
parameter L_TIME          = 17'd1          ; //led 闪烁时间参数
parameter MAX_BYTE        = 16'd3          ; //读写测试的字节个数

//reg define
reg          sys_clk      ;                //时钟信号
reg          sys_rst_n;                //复位信号

//wire define
wire         iic_scl;
wire         iic_sda;
wire         led      ;
```

(2) 随后是给输入信号进行初始化赋值，生成一个 50Mhz 的时钟，即系统时钟 sys_clk。接着将 SDA 数据线为高阻态时上拉为高电平，板子上是通过连接一个上拉电阻实现的上拉效果。

```
//给输入信号初始值
initial begin
    sys_clk          = 1'b0;
    sys_rst_n        = 1'b0;    //复位
    #(T+1) sys_rst_n = 1'b1;    //在第 21ns 的时候复位信号信号拉高
end

//50Mhz 的时钟，周期则为 1/50Mhz=20ns, 所以每 10ns，电平取反一次
always #(T/2) sys_clk = ~sys_clk;

//将 SDA 数据线上拉
pullup(iic_sda);
```

(3) 最后是例化 e2prom_top 模块，因为 I2C 传输数据时需要从机应答，所以在仿真时需要一个 E2PROM 的模型配合才能进行仿真，该模型（E2PROM_AT24C64.v）需要与模块的器件同型号，可以从官网获取。需要注意的是，在仿真的同时也需要将该模型也添加至仿真文件由于篇幅原因，这里就不贴该模型的代码了，该模型的存放路径如下图所示：

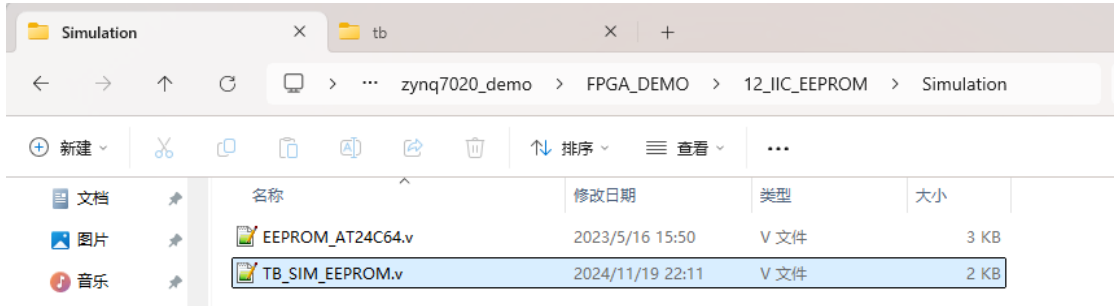


图 14-26 E2PROM 仿真模型

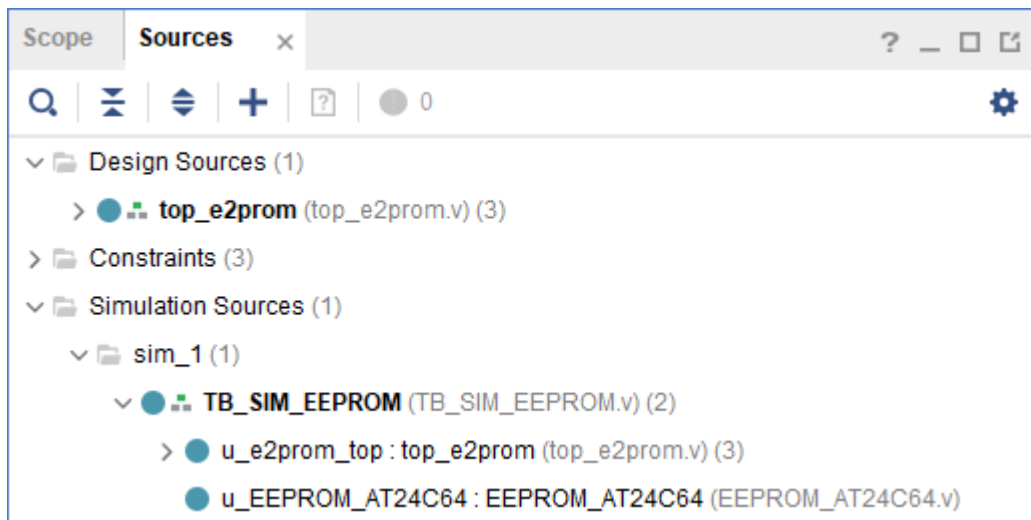


图 14-27 添加 E2PROM 仿真模型至仿真文件

I2C 驱动模块激励文件编写完成, E2PROM 的仿真模型添加成功, 此时就进行代码仿真。

5.2 EEPROM 仿真结果

(1) 首先查看 E2PROM 读写测试顶层模块在运行仿真 30ms 后, 仿真的波形如下图所示:

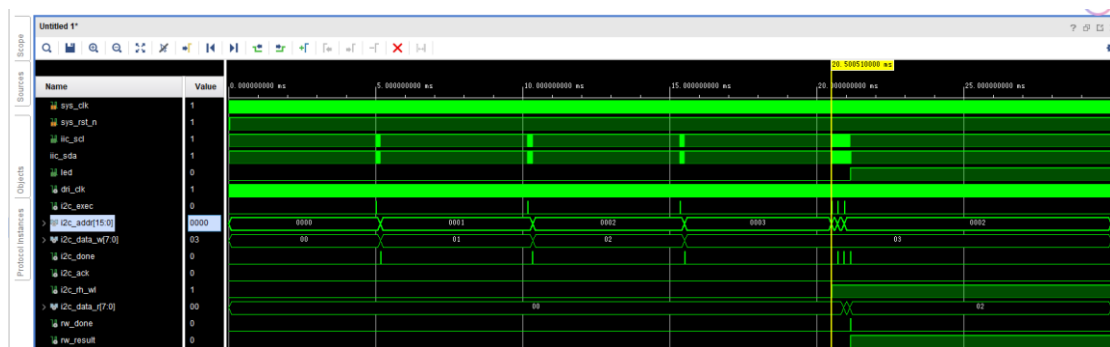


图 14-28 E2PROM 读写测试顶层模块仿真图

通过观察上图, 读出的数据与写入的数据一致, 读写结束后输出读写结束信号 rw_done, 并且输出读写对比结果 rw_result 信号, led 灯保持常亮。

(2) 接下来, 对 EEPROM 写操作波形进行观测, 仿真波形如下:

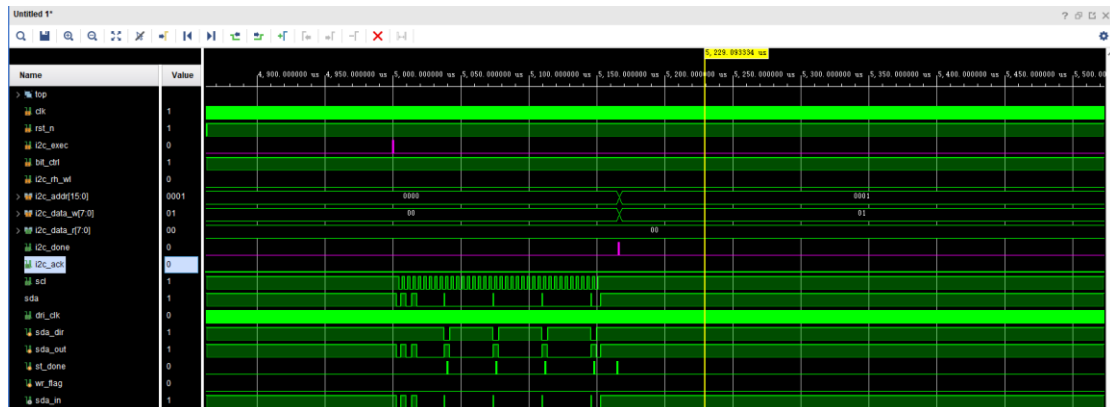


图 14-29 E2PROM 写操作仿真波形图

从该波形图中可以看到读写控制信号 i2c_rh_wl 为低电平, 表示处于写操作状态。当 I2C

触发执行信号 `i2c_exec` 为高电平时开始执行 I2C 写操作。在 IIC 操作结束后，拉高 `i2c_done` 信号。另外，在 IIC 写操作期间，`i2c_ack` (IIC 应答标志) 一直处于低电平，说明 E2PROM 响应了主机并应答。

(3) 最后，对 EEPROM 读操作波形进行观测，仿真波形如下：

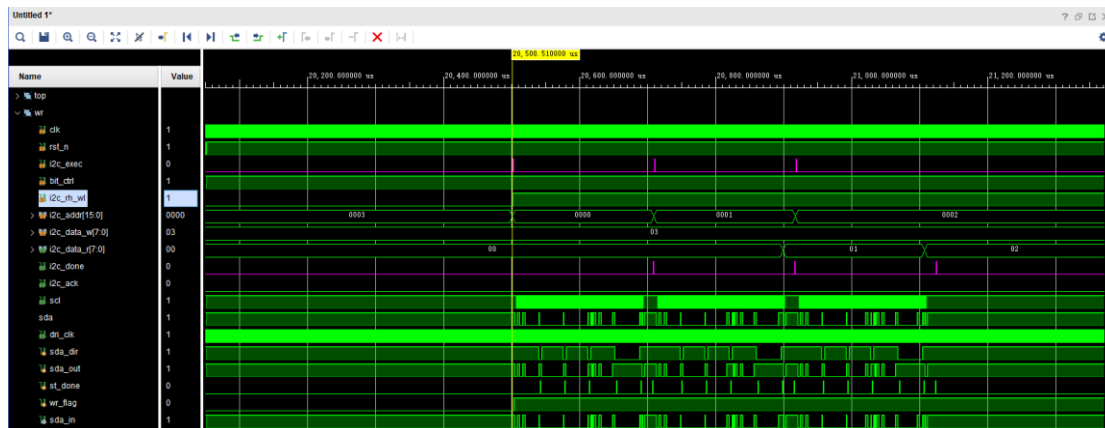


图 14-30 E2PROM 读操作仿真波形图

从该波形图中可以看出读写控制信号 `i2c_rh_wl` 为高电平，表示处于读操作状态。当 I2C 触发执行信号 `i2c_exec` 为高电平时开始执行 I2C 读操作。在 IIC 操作结束后，拉高 `i2c_done` 信号。另外，在 IIC 读操作期间，`i2c_ack` (IIC 应答标志) 一直处于低电平，说明 E2PROM 响应了主机并应答。仿真结果与我们绘制的波形显示一致，仿真验证成功，仿真至此结束。

6. 添加内部逻辑分析仪

(1) 在上一章中可以看出，`i2c_rh_wl` 信号主要是控制读写状态，同时在控制读写状态的同时是需要有触发信号 `i2c_exec`，换言之，可以观测这两个信号的状态，去观测地址以及数据是否写入，那么 Debug 信号添加如下：

```
//wire define
wire      dri_clk    ; //I2C操作时钟
(* mark_debug = "true" *)
wire      i2c_exec   ; //I2C触发控制
(* mark_debug = "true" *)
wire [15:0] i2c_addr  ; //I2C操作地址
(* mark_debug = "true" *)
wire [7:0] i2c_data_w ; //I2C写入的数据
(* mark_debug = "true" *)
wire      i2c_done   ; //I2C操作结束标志
(* mark_debug = "true" *)
wire      i2c_ack    ; //I2C应答标志 0:应答 1:未应答
(* mark_debug = "true" *)
wire      i2c_rh_wl  ; //I2C读写控制
(* mark_debug = "true" *)
wire [7:0] i2c_data_r ; //I2C读出的数据
(* mark_debug = "true" *)
wire      rw_done    ; //E2PROM读写测试完成
(* mark_debug = "true" *)
wire      rw_result  ; //E2PROM读写测试结果 0:失败 1:成功
```

图 14-31 添加 IIC Debug 信号

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。



图 14-32 设置 Debug 信号的深度

(3) 需要注意的是，抓取 IIC 的时钟不能选择由分频器产生的时钟，需要采用系统晶振产生的时钟，时钟信号如下图所示：

Name	Clock Domain	Driver Cell	Probe Type	
> i2c_addr (16)	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
> i2c_data_r (8)	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
> i2c_data_w (8)	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
i2c_ack	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
i2c_done	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
i2c_exec	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
i2c_rh_wl	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
rw_done	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼
rw_result	sys_clk_IBUF_BUFG	FDCE	Data and Trigger	▼

图 14-33 设置抓取 ILA 时钟

(4) 查看是否写入约束文件

```
Constraint > Debug.xdc
1 create_debug_core u_ila_0 ila
2 set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
3 set_property ALL_PROBE_SAME_CNT 1 [get_debug_cores u_ila_0]
4 set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
5 set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
6 set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
7 set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
8 set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
9 set_property C_TRIGGER_EN false [get_debug_cores u_ila_0]
10 set_property port_width 1 [get_debug_ports u_ila_0/clk]
11 connect_debug_port u_ila_0/clk [get_nets {list u_i2c_dri/dri_clk}]
12 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
13 set_property port_width 16 [get_debug_ports u_ila_0/probe0]
14 connect_debug_port u_ila_0/probe0 [get_nets {list {i2c_addr[0]} {i2c_addr[1]} {i2c_addr[2]} {i2c_addr[3]} {i2c_addr[4]} {i2c_addr[5]} {i2c_addr[6]} {i2c_addr[7]} {i2c_addr[8]} {i2c_addr[9]} {i2c_addr[10]} {i2c_addr[11]} {i2c_addr[12]} {i2c_addr[13]} {i2c_addr[14]} {i2c_addr[15]}}]
15 create_debug_port u_ila_0 probe
16 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
17 set_property port_width 8 [get_debug_ports u_ila_0/probe1]
18 connect_debug_port u_ila_0/probe1 [get_nets {list {i2c_data_r[0]} {i2c_data_r[1]} {i2c_data_r[2]} {i2c_data_r[3]} {i2c_data_r[4]} {i2c_data_r[5]} {i2c_data_r[6]} {i2c_data_r[7]}}]
19 create_debug_port u_ila_0 probe
20 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
21 set_property port_width 8 [get_debug_ports u_ila_0/probe2]
22 connect_debug_port u_ila_0/probe2 [get_nets {list {i2c_data_w[0]} {i2c_data_w[1]} {i2c_data_w[2]} {i2c_data_w[3]} {i2c_data_w[4]} {i2c_data_w[5]} {i2c_data_w[6]} {i2c_data_w[7]}}]
```

图 14-34 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

7.添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会

打开“Timing Constraints”界面，点击该界面下的“+”号按钮就可以添加时钟约束了，如下图所示：

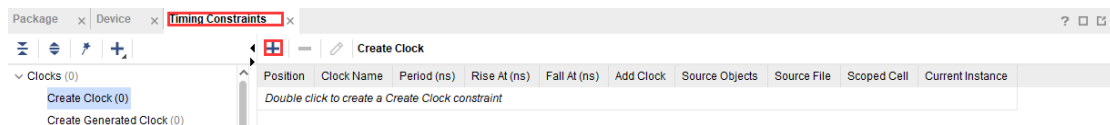


图 14-35 添加主时钟时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

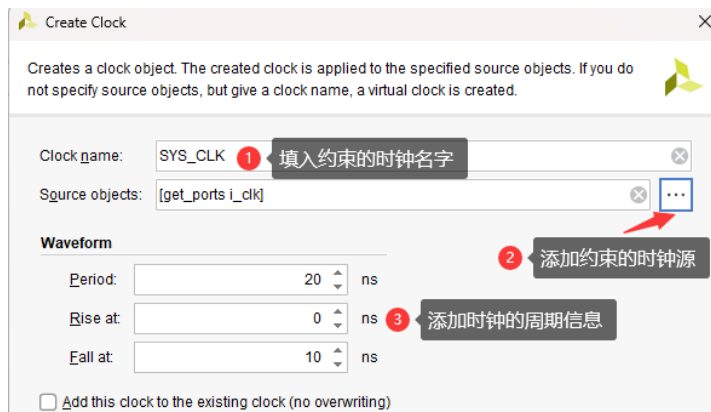


图 14-36 约束主时钟

(3) 保存后，查看约束文件是否写入时钟约束，如下图所示：

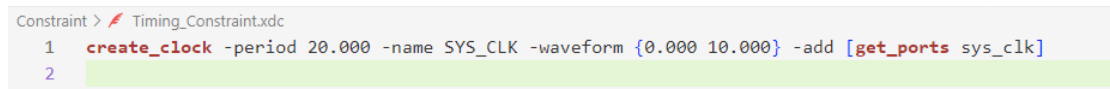


图 14-37 写入主时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

8.添加管脚约束

在上述步骤中添加了时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

需要注意的是，由于开发板 PL 端并没有集成 AT24C64 芯片，因此采用的方案是 AT24C64 模块通过 40 Pin 拓展模块连接到开发板 PL 端，其中模块的 VCC 接开发板 3.3V 电源，GND 接开发板 GND，SDA 接 V12 管脚，SCL 接 W12 管脚，且电平标准都选择 LVCMOS33（SCL 与 SDA 可以在 40 Pin 拓展接口中随意选择两个管脚）。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

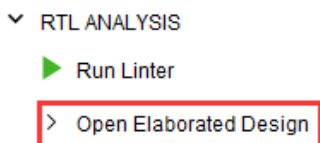


图 14-38 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Net Diff Pair	Package Pin	Fixed	Bank	IO Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
Scalar ports (5)													
ic_jd1	OUT		W12	☒	13	LVCNMOS33*	3.300		12		NONE	FP_VTT_50	
ic_sda	INOUT		V12	☒	13	LVCNMOS33*	3.300		12		NONE	FP_VTT_50	
led	OUT		V15	☒	33	LVCNMOS33*	3.300		12		NONE	FP_VTT_50	
sys_clk	IN		W17	☒	33	LVCNMOS33*	3.300				NONE	NONE	
sys_rst_n	IN		W18	☒	33	LVCNMOS33*	3.300				NONE	NONE	

图 14-39 约束管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
1  # 主时钟
2  set_property PACKAGE_PIN W17 [get_ports sys_clk]
3  set_property IOSTANDARD LVCNMOS33 [get_ports sys_clk]
4
5  set_property PACKAGE_PIN W18 [get_ports sys_rst_n]
6  set_property IOSTANDARD LVCNMOS33 [get_ports sys_rst_n]
7
8  # IIC管脚
9  set_property PACKAGE_PIN W12 [get_ports iic_scl]
10 set_property IOSTANDARD LVCNMOS33 [get_ports iic_scl]
11
12 set_property PACKAGE_PIN V12 [get_ports iic_sda]
13 set_property IOSTANDARD LVCNMOS33 [get_ports iic_sda]
14
15 set_property PACKAGE_PIN V15 [get_ports led]
16 set_property IOSTANDARD LVCNMOS33 [get_ports led]
```

图 14-40 约束文件中写入管脚约束

9.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

Timing		Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	13.928 ns			
Total Negative Slack (TNS):	0 ns			
Number of Failing Endpoints:	0			
Total Number of Endpoints:	4536			
Implemented Timing Report				

图 14-41 时序结果

(1) 首先将电脑通过数据线与 FPGA Debug 接口，同时将模块连接至 FPGA 40 Pin 拓展口，最后连接开发板的电源，并打开电源开关，硬件连接如下图所示：

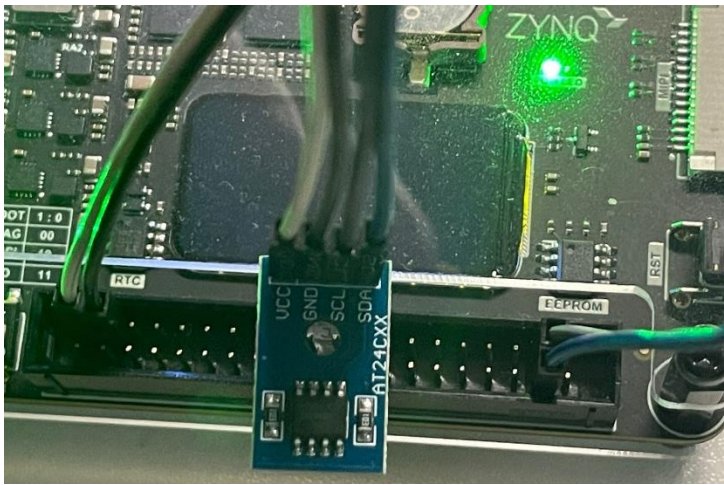


图 14-42 硬件连接图

在开发板背面，是印有 PL 部分 40 Pin 管脚约束的，在使用时，建议核对丝印后再连接模块。

(2) 将本次实验生成的 BIT 文件下载下开发板中，在 ILA 中，触发条件为当 i2c_rh_wl 为低电平，i2c_exec 为高电平时，写入的地址，以及数据。

由于写入速度较快，当设置完抓取条件，并开始触发时，需要按下复位，随后松开，即可抓取到写操作。同时若设置 ILA 只有一个窗口，那么只能看到在 0 地址写入 0，因此需要将窗口设置为 2，窗口设置如图 14-42 所示，内部逻辑分析仪抓取的写结果如图 14-43 所示。



图 14-43 设置 ILA 窗口

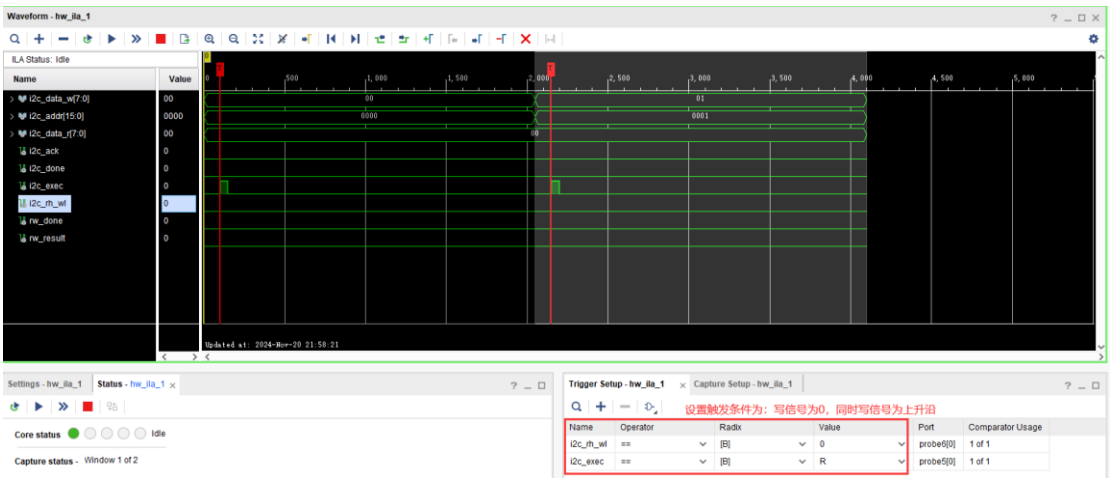


图 14-44 写操作信号抓取

从图 14-43 中可以看出，当进行写操作时，并触发了 IIC 信号，那么随后对 0 地址开始

写入数据 0，随后对地址 1 写入数据 1，这是符合仿真设计的。，因此写地址与写数据是正确的。

(3) 读取数据的 ILA 抓取操作是与写操作是一致的，只是将窗口设置为 4，因为设置为 2，读出的数据是 0，无法查看数据是否正确。窗口设置如图 14-44 所示，内部逻辑分析仪抓取读的结果如图 14-45 所示：



图 14-45 设置 ILA 窗口

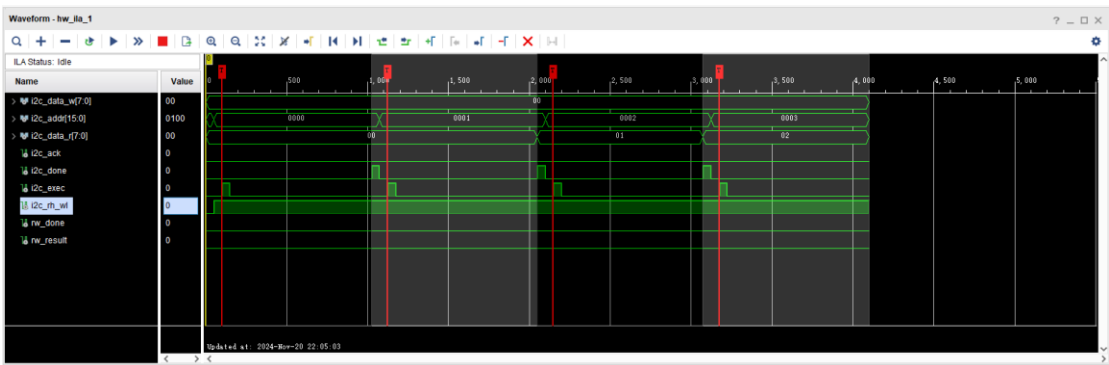


图 14-46 写操作信号抓取

从图 14-45 中可以看出，当进行读操作时，并触发了 IIC 信号，那么随后对 0 地址开始读，当 done 信号拉高后，则会输出 0 地址写入的数据，其余位置读出的结果与写入的地址、数据是一致的。

(4) 最后，对 EEPROM 的测试结果进行抓取，当其拉高时间，则表示写入读出正确，抓取结果如图 14-46 所示：

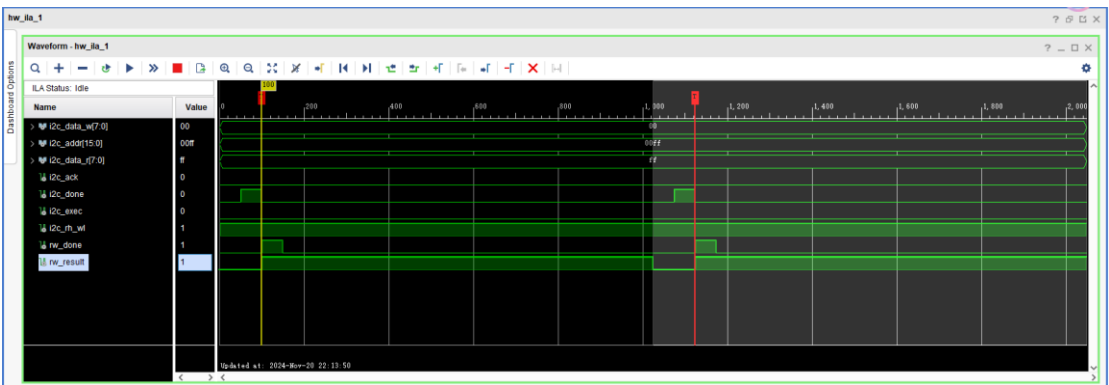


图 14-47 EEPROM 读写结果

10.实验总结

因为 FPGA 的特点是并行执行，所以如果需要处理一些具有前后顺序的事件，就需要使用状态机。此时利用状态机进行开发，可以将一个复杂的控制流程分解成多个互相独立的状态，从而简化设计过程并提高了系统的可靠性和性能。

十五. 以太网 UDP 协议栈实验

UDP 协议是一种面向无连接的传输层协议，属于 TCP/IP 协议簇的一种，UDP 具有消耗资源少、通信效率高等优点，通常用来传输音频、视频等对实时性要求高的场合。

1.网络传输格式简介

UDP (User Datagram Protocol)，即用户数据报协议，是一种面向无连接的传输层协议。无连接是指在传输数据时，数据的发送端和接收端不建立逻辑连接。简单来说，当一台计算机向另外一台计算机发送数据时，发送端不会确认接收端是否存在，就会发出数据，同样接收端在收到数据时，也不会向发送端反馈是否收到数据。由于使用 UDP 协议消耗资源小，通信效率高，所以通常都会用于音频、视频和普通数据的传输（如视频会议等）都会采用 UDP 协议进行传输，这种情况即使偶尔丢失一两个数据包，也不会对接收结果产生太大影响。

UDP 和 TCP 是传输层中非常重要的两个协议，位于 OSI（OpenSystemInterconnection，开放式系统互联）参考模型中的第四层（传输层），是一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务，位于 IP 协议层（网络层）之上。OSI 将计算机网络体系结构分为七层：物理层、数据链路层、网络层、传输层、会话层、表示层和应用层，OSI 参考模型如下图所示。

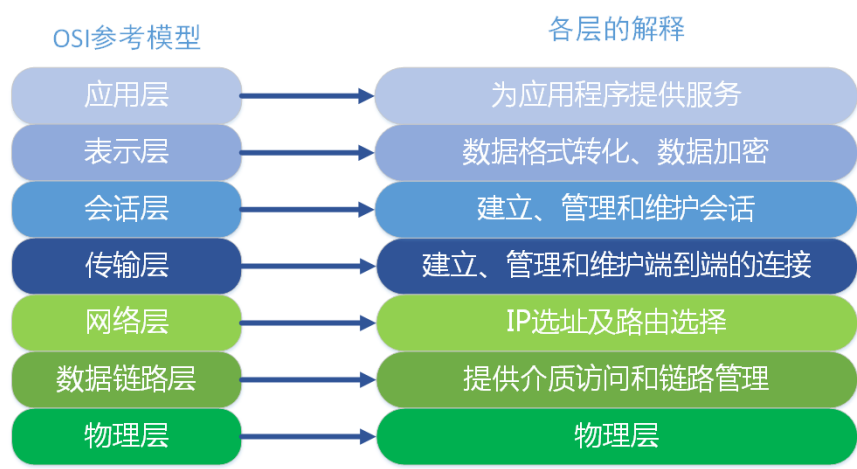


图 15-1 OSI 参考模型

以太网 UDP 传输单包数据的格式如图 15-2 所示。从图中可以看出，以太网的数据包就是对各层协议的逐层封装来实现数据的传输。用户数据打包在 UDP 协议中，UDP 协议又是基于 IP 协议之上的，IP 协议又是通过 MAC 层发送的，即从包含关系来说：MAC 帧中的数据段为 IP 数据报，IP 报文中的数据段为 UDP 报文，UDP 报文中的数据段为用户希望传输的数据内容。接下来对一层一层的协议介绍不同层的数据格式。

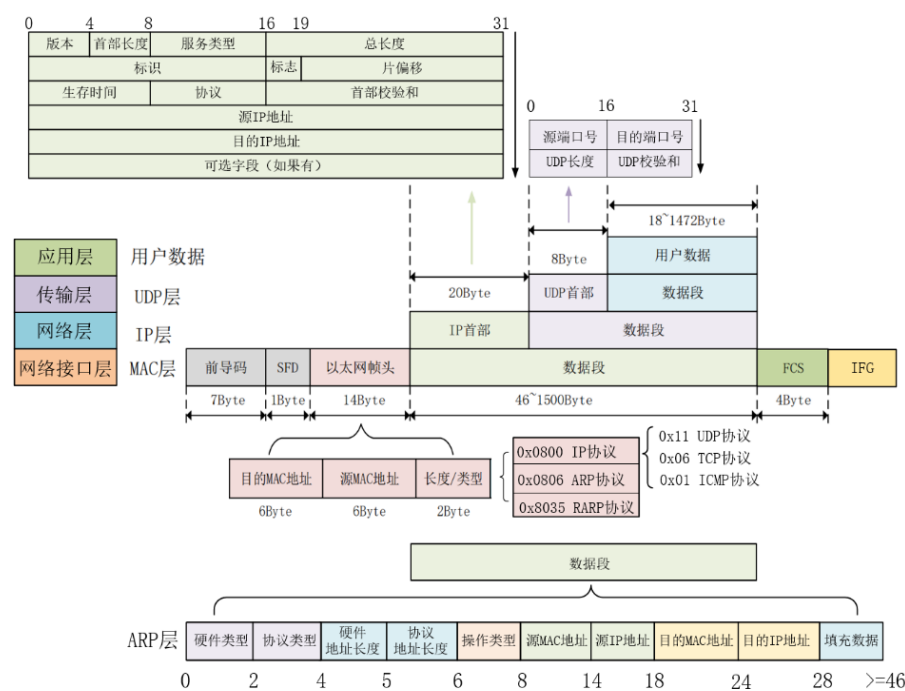


图 15-2 以太网 UDP 传输数据包格式

在以太网通信中，数据是以“帧”的格式进行传输的，第一次看到图 15-2 的图，都有点蒙圈，那么接下来则对以太网帧进行详细介绍。

(1) 以太网 MAC 层

以太网技术的正式标准是 IEEE802.3，它规定了以太网传输数据的帧结构，可以将以太网 MAC 层理解成高速公路，必须遵循它的规则才能在上面通行，以太网 MAC 层帧格式如图 15-3 所示。

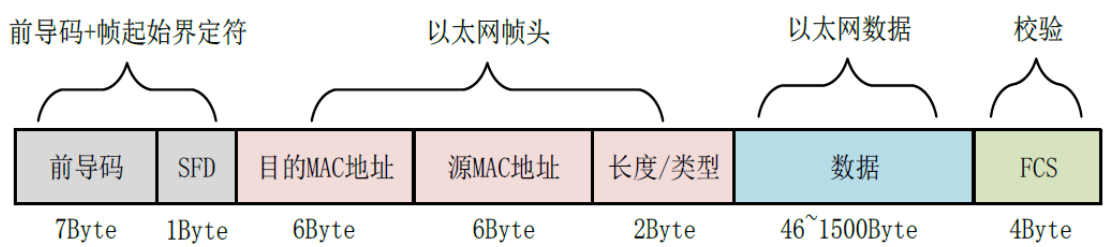


图 15-3 MAC 层帧格式

前导码 (Preamble): 为了实现底层数据的正确同步，物理层使用 7 个字节同步码 (0 和 1 交替 (55-55-55-55-55-55-55)) 实现数据的同步。

帧起始界定符 (SFD, Start Frame Delimiter): 使用 1 个字节的 SFD (固定值为 0xd5) 来表示一帧的开始，即后面紧跟着传输的就是以太网的帧头。

目的 MAC 地址: 即接收端物理 MAC 地址，占用 6 个字节。MAC 地址从应用上可分为单播地址、组播地址和广播地址。单播地址：第一个字节的最低位为 0，比如 00-00-00-11-11-11，一般用于标志唯一的设备；组播地址：第一个字节的最低位为 1，比如 01-00-00-11-11-11，一般用于标志同属一组的多个设备；广播地址：所有 48bit 全为 1，即 FF-FF-FF-FF-FF-FF，它用于标志同一网段中的所有设备。

源 MAC 地址: 即发送端物理 MAC 地址，占用 6 个字节。

长度/类型: 上图中的长度/类型具有两个意义，当这两个字节的值小于 1536 (十六进制为 0x0600) 时，代表该以太网中数据段的长度；如果这两个字节的值大于 1536，则表示该以太网中的数据属于哪个上层协议，例如 0x0800 代表 IP 协议 (网际协议)、0x0806 代表

ARP 协议（地址解析协议）等。

数据：以太网中的数据段长度最小 46 个字节，最大 1500 个字节。最大值 1500 称为以太网的最大传输单元（MTU，Maximum Transmission Unit），之所以限制最大传输单元是因为在多个计算机的数据帧排队等待传输时，如果某个数据帧太大的话，那么其它数据帧等待的时间就会加长，导致体验变差，这就像一个十字路口的红绿灯，你可以让绿灯持续亮一小时，但是等红灯的人一定不愿意的。另外还要考虑网络 I/O 控制器缓存区资源以及网络最大的承载能力等因素，因此最大传输单元是由各种综合因素决定的。为了避免增加额外的配置，通常以太网的有效数据字段小于 1500 个字节。

帧检验序列（FCS，Frame Check Sequence）：为了确保数据的正确传输，在数据的尾部加入了 4 个字节的循环冗余校验码（CRC 校验）来检测数据是否传输错误。CRC 数据校验从以太网帧头开始即不包含前导码和帧起始界定符。通用的 CRC 标准有 CRC-8、CRC-16、CRC-32、CRC-CCIT，其中在网络通信系统中应用最广泛的是 CRC-32 标准。

在这里还有一个要注意的地方就是以太网相邻两帧之间的时间间隔，即帧间隙（IFG，Interpacket Gap）。帧间隙的时间就是网络设备和组件在接收一帧之后，需要短暂的时间来恢复并为接收下一帧做准备的时间，IFG 的最小值是 96 bit time，即在媒介中发送 96 位原始数据所需要的时间，在不同媒介中 IFG 的最小值是不一样的。不管 10M/100M/1000M 的以太网，两帧之间最少要有 96bit time，IFG 的最少间隔时间计算方法如下：

10Mbit/s 最小时间为： $96 \times 100\text{ns} = 9600\text{ns}$ ；
100Mbit/s 最小时间为： $96 \times 10\text{ns} = 960\text{ns}$ ；
1000Mbit/s 最小时间为： $96 \times 1\text{ns} = 96\text{ns}$ 。

(2) 以太网 IP 层

TCP/IP 是网络使用中最基本的通信协议，虽然从名字看上去 TCP/IP 包括两个协议，TCP 和 IP，但 TCP/IP 实际上是一组协议，它包括上百个各种功能的协议，如：TCP、IP、ARP、UDP 等。而 TCP 协议和 IP 协议是保证数据完整传输的两个重要的协议，因此 TCP/IP 协议用来表示 Internet 协议簇。

IP 协议是 TCP/IP 协议簇中的核心协议，也是 TCP/IP 协议的载体，IP 协议规定了数据传输时的基本单元和格式。从图 15-2 中可以看出，IP 协议位于以太网 MAC 帧格式的数据段，IP 协议内容由 IP 首部和数据字段组成。所有的 TCP、UDP 及 ICMP 数据都以 IP 数据报格式传输，IP 数据包格式如图 15-4 所示。

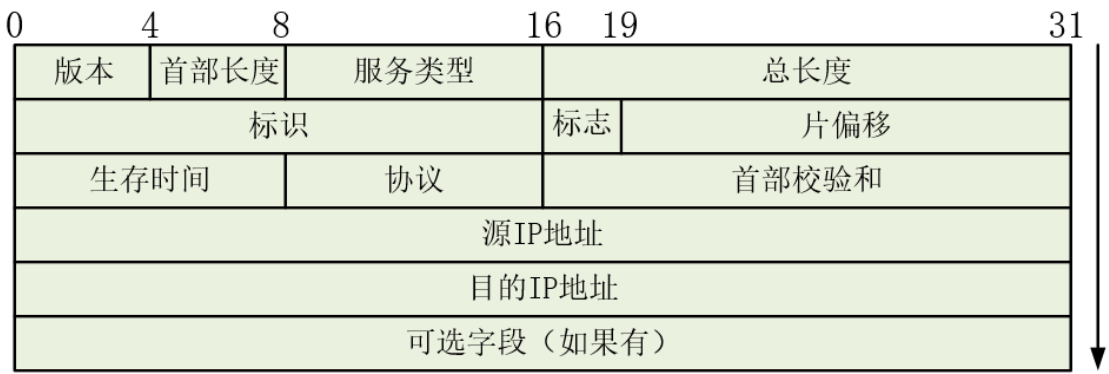


图 15-4 数据报格式

前 20 个字节和紧跟其后的可选字段是 IP 数据报的首部，前 20 个字节是固定的，后面可选字段是可有可无的，首部的每一行以 32 位（4 个字节）为单位。每个字节传输规则为由比特最高位到最低位的顺序逐一进行发送，4 字节的 32bit 值按照以下次序进行传输：首先 7-0bit，其次 15-8bit，然后 23-16bit，最后是 31-24bit。

版本：4 位 IP 版本号 (Version)，这个值设置为二进制的 0100 时表示 IPv4，设置为 0110 时表示 IPv6，目前使用比较多的 IP 协议版本号是 4。

首部长度：4 位首部长度 (IHL, Internet Header Length)，表示 IP 首部一共有多少个 32 位 (4 个字节)。在没有可选字段时，IP 首部长度为 20 个字节，因此首部长度的值为 5。

服务类型：8 位服务类型 (TOS, Type of service)，该字段被划分成两个子字段：3 位优先级字段 (现在已经基本忽略掉了) 和 4 位 TOS 字段，最后一位固定为 0。服务类型为 0 时表示一般服务。

总长度：16 位 IP 数据报总长度 (Total Length)，包括 IP 首部和 IP 数据部分，以字节为单位。我们利用 IP 首部长度和 IP 数据报总长度，就可以知道 IP 数据报中数据内容的起始位置和长度。由于该字段长 16bit，所以 IP 数据报最长可达 65535 字节。尽管理论上可以传输长达 65535 字节的 IP 数据报，但实际上还要考虑网络的最大承载能力等因素。

标识字段：16 位标识 (Identification) 字段，用来标识主机发送的每一份数据报。通常每发送一份报文它的值就会加 1。

标志字段：3 位标志 (Flags) 字段，第 1 位为保留位；第 2 位表示禁止分片 (1 表示不分片 0：允许分片)；第 3 位标识更多分片 (除了数据报的最后一个分片外，其它分片都为 1)。

片偏移：13 位片偏移 (Fragment Offset)，在接收方进行数据报重组时用来标识分片的顺序。

生存时间：8 位生存时间字段，TTL (Time To Live) 域防止丢失的数据包在无休止的传播，一般被设置为 64 或者 128。

协议：8 位协议 (Protocol) 类型，表示此数据报所携带上层数据使用的协议类型，ICMP 为 1，TCP 为 6，UDP 为 17。

首部校验和：16 位首部校验和 (Header Checksum)，该字段只校验数据报的首部，不包含数据部分；校验 IP 数据报头部是否被破坏、篡改和丢失等。

源 IP 地址：32 位源 IP 地址 (Source Address)，即发送端的 IP 地址，如 192.168.1.123。

目的 IP 地址：32 位目的 IP 地址 (Destination Address)，即接收端的 IP 地址，如 192.168.1.102。

可选字段：是数据报中的一个可变长度的可选信息，选项字段以 32bit 为界，不足时插入值为 0 的填充字节，保证 IP 首部始终是 32bit 的整数倍。

以上内容是对 IP 首部格式的详细阐述，还需要补充的内容是 IP 首部校验和的计算方法，其计算步骤如下：

将 16 位校验和字段置为 0，然后将 IP 首部按照 16 位分成多个单元；

对各个单元采用反码加法运算 (即高位溢出位会加到低位，通常的补码运算是直接丢掉溢出的高位)；

此时仍然可能出现进位的情况，将得到的和再次分成高 16 位和低 16 位进行累加；

最后将得到的和的反码填入校验和字段。

例如，使用 IP 协议发送一个 IP 数据报总长度为 60 个字节 (有效数据为 40 个字节) 的数据包，发送端 IP 地址为 192.168.1.10，接收端 IP 地址为 192.168.1.102，则 IP 首部数据如下：

	0	4	8	16	31
IP首部	版本	首部长度	服务类型	总长度	
	4'h4	4'h5	8'h00	16'h003C（60个字节）	
	标识			标志	片偏移
	16'h0			3'h2	13'h0
	生存时间	协议	首部校验和		
	8'h40	8'h01（ICMP协议）	Ip_check_sum		
	源IP地址				
	{8'hc0, 8'ha8, 8'h01, 8'h66}（192.168.1.102）				
	目的IP地址				
	{8'hc0, 8'ha8, 8'h01, 8'h0a}（192.168.1.10）				

图 15-5 IP 首部数据

按照上述提到的 IP 首部校验和的方法计算 IP 首部校验和，即：

0x4500 + 0x003C+ 0x0000 + 0x4000 + 0x4001 + 0x0000(计算时强制置 0) + 0xc0a8 + 0x010a + 0xc0a8 + 0x0166 = 0x248FD
0x0002 + 0x48FD= 0x000048FF（此种情况并未出现进位）
0x0000+ 0x48FF= 0x48FF（此种情况并未出现进位）
check_sum = ~0x48FF（按位取反）= 0xb700

在 IP 层通常也会传输 ICMP 协议，或者 ARP 协议，在这不多介绍，这边只介绍每个层的数据类型。

(3) 以太网 UDP 层

首先回答为什么还需要 UDP 协议？事实上数据是可以直接封装在 IP 协议里而不使用 TCP、UDP 或者其它上层协议的。然而在网络传输中同一 IP 服务器需要提供各种不同的服务，各种不同的服务类型是使用端口号来区分的，例如用于浏览网页服务的 80 端口，用于 FTP（文件传输协议）服务的 21 端口等。TCP 和 UDP 都使用两个字节的端口号，理论上可以表示的范围为 0~65535，足够满足各种不同的服务类型。

然后是为什么不选择传输更可靠的 TCP 协议，而是 UDP 协议呢？TCP 协议与 UDP 协议作为传输层最常用的两种传输协议，这两种协议都是使用 IP 作为网络层协议进行传输。下面是 TCP 协议与 UDP 协议的区别：

- a. TCP 协议面向连接，是流传输协议，通过连接发送数据，而 UDP 协议传输不需要连接，是数据报协议；
- b. TCP 为可靠传输协议，而 UDP 为不可靠传输协议。即 TCP 协议可以保证数据的完整和有序，而 UDP 不能保证；
- c. UDP 由于不需要连接，故传输速度比 TCP 快，且占用资源比 TCP 少；
- d. 应用场合：TCP 协议常用在对数据文件完整性较高的一些场景中，如文件传输等。UDP 常用于对通讯速度有较高要求或者传输数据较少时，比如对速度要求较高的视频直播和传输数据较少的 QQ 等。

首先可以肯定的告诉大家，使用 FPGA 实现 TCP 协议是完全没有问题的，但是，FPGA 发展到现在，却鲜有成功商用的 RTL 级的 TCP 协议设计，大部分以太网传输都是基于比较简单的 UDP 协议。TCP 协议设计之初是根据软件灵活性设计的，如果使用硬件逻辑实现，工程量会十分巨大，而且功能和性能无法得到保证，因此，TCP 协议设计并不适合使用硬件逻辑实现。UDP 协议是一种不可靠传输，发送方只负责数据发送出去，而不管接收方是否正确的接收。在很多场合，是可以接受这种潜在的不可靠性的，例如视频实时传输显示等。UDP 数据格式如图 15-6 所示：

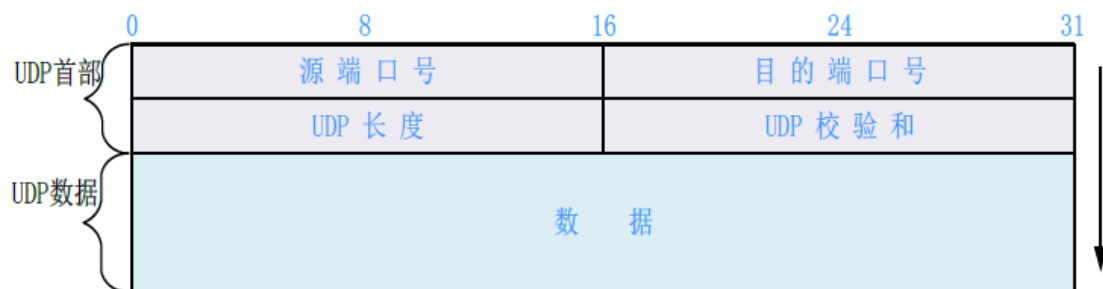


图 15-6 UDP 数据格式

UDP 首部共 8 个字节，同 IP 首部一样，也是一行以 32 位（4 个字节）为单位。

源端口号：16 位发送端端口号，用于区分不同服务的端口，端口号的范围从 0 到 65535。

目的端口号：16 位接收端端口号。

UDP 长度：16 位 UDP 长度，包含 UDP 首部长度+数据长度，单位是字节（byte）。

UDP 校验和：16 位 UDP 校验和。UDP 计算校验和的方法和计算 IP 数据报首部校验和的方法相似，但不同的是 IP 数据报的校验和只检验 IP 数据报的首部，而 UDP 校验和包含三个部分：UDP 伪首部，UDP 首部和 UDP 的数据部分。伪首部的数据是从 IP 数据报头和 UDP 数据报头获取的，包括源 IP 地址，目的 IP 地址，协议类型和 UDP 长度，其目的是让 UDP 两次检查数据是否已经正确到达目的地，只是单纯为了做校验用的。在大多数使用场景中接收端并不检测 UDP 校验和，因此这里不做过多介绍。

用户数据打包在 UDP 协议中，UDP 协议又是基于 IP 协议之上的，IP 协议又是走 MAC 层发送的，即从包含关系来说：MAC 帧中的数据段为 IP 数据报，IP 报文中的数据段为 UDP 报文，UDP 报文中的数据段为用户希望传输的数据内容。现在再回过头看图 15-2 的内容就非常容易理解了。

2. 硬件设计

开发板上有两个千兆网口，一个是 PL 端网口，另一个是 PS 端网口，本次实验使用 PL 端网络进行试验。RJ45 接口用于连接网线，其原理图（以 RTL8211-F 芯片为例）如下图所示：

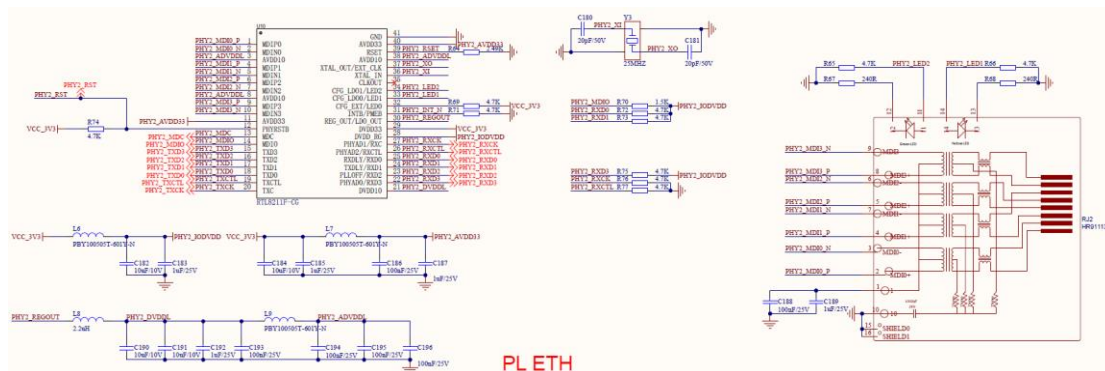


图 15-7 PL 端网口硬件设计

上图中的 PHY_LED1 和 PHY_LED2 是 PHY 输出的 LED 信号，用于控制 RJ45 接口上的 LED 灯，PHY2_MDIO_P/0_N~PHY2_MDIO3_P/3_N 是 PHY 和 RJ45 接口间的差分信号线，用于传输数据。

以太网的数据传输离不开以太网 PHY（物理层）芯片的支持，物理层定义了数据发送与接收所需要的电信号、线路状态、时钟基准、数据编码和电路等，并向数据链路层设备提

供标准接口。开发板上使用的 PHY 芯片为瑞昱半导体公司生产的 RTL8211F。RTL8211F 是一个千兆以太网物理层收发器，支持 1000/100/10Mbps 通信速率，该芯片内部的参数可以通过 MDIO 接口进行配置。由图 15-7 可知，开发板上的以太网 PHY 芯片 PHYAD2、PHYAD1 接下拉电阻，PHYAD0 接上拉电阻，因此 PHY 地址为 5'h01。

3. 实验任务

本节实验任务是上位机通过网口调试助手发送数据给 FPGA，FPGA 通过以太网接口接收数据并将接收到的数据发送给上位机，完成以太网 UDP 数据的环回。同时电脑可以通过命令行窗口发送 ping 命令给 FPGA，FPGA 通过以太网接口接收数据并将接收到的数据发送给电脑，完成以电脑 ping 开发板的实验测试。

4. 程序设计

4.1 MAC 层模块

在前面关于以太网协议帧的介绍中，可以得知 MAC 层是位于整个通信协议栈的最低层，那么首先则从 MAC 层进行代码编写，MAC 的代码编写主要包括 GMII 转 RGMII，以及数据的组包。关于 GMII 与 RGMII 之间的区别，在此不多做叙述，该接口是 FPGA 与 PHY 芯片的通信接口，若有兴趣，可自行学习。

首先设计一下 GMII_TO_RGMII 模块，该模块实现了双沿 (DDR) 数据和单沿 (SDR) 数据之间的转换。GMII_TO_RGMII 模块主要是例化了 rgmii_rx (RGMII 发送) 模块和 rgmii_tx (RGMII 接收) 模块两个模块，gmii_to_rgmii 模块框图如下：

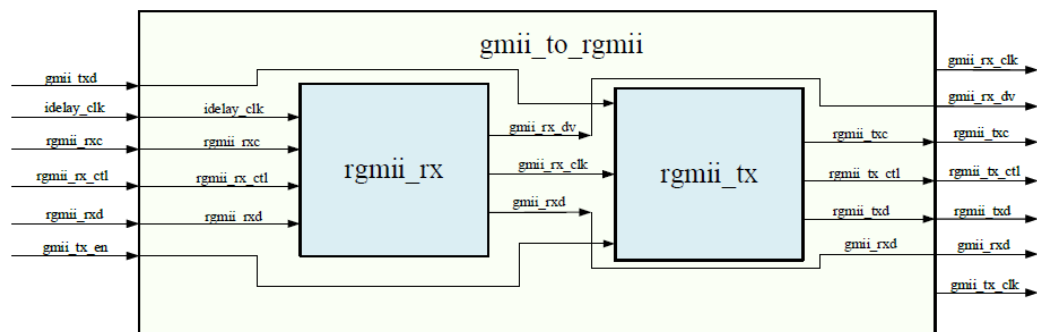


图 15-8 GMII_TO_RGMII 模块

(1) rgmii_rx 模块主要是通过调用 BUFG、BUFIO、IDDR、IDELAYCTRL 和 IDELAY2 原语，实现了 RGMII 接口输入的 DDR 数据到 SDR 数据的转换，输入的 rgmii_rx_ctl 控制信号的转换方法同样类似。rgmii_rx 模块信号转换示意图如下图所示：

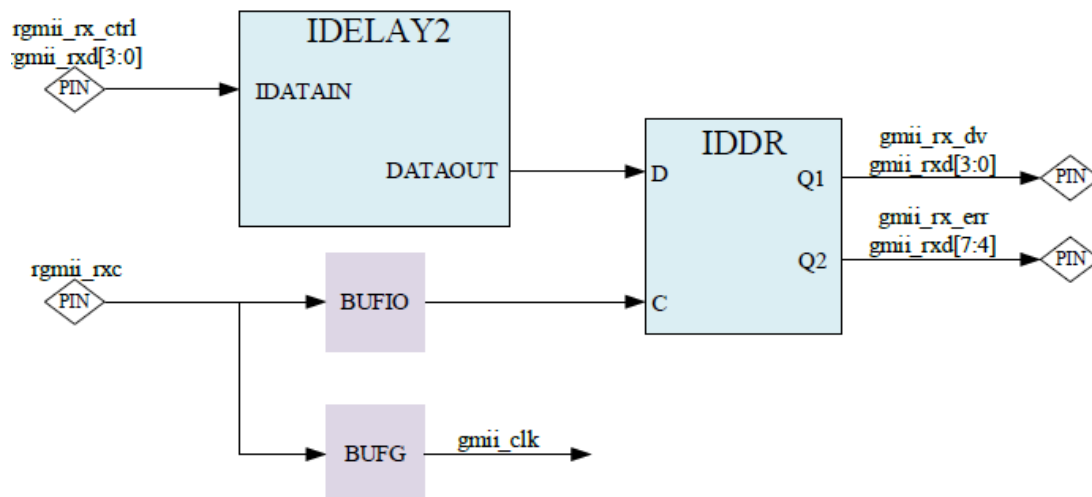


图 15-9 rgmii_rx 模块信号转换示意图

时钟专用引脚输入的 rgmii_rxc 时钟经过 BUFG 后, 得到 rgmii_rxc_bufg (gmii_rx_clk), 该时钟为全局缓冲时钟, 其到达 FPGA 内部的 IOB、CLB、块 RAM 的时钟延迟和抖动非常小, 为其他模块提供操作时钟; 另外 rgmii_rxc 时钟也经过 BUFIO, 专门用于采集 IO 端口的数据。

rgmii_rx_ctl 控制信号和 4 位 rgmii_rxd 数据先经过 IDELAY2 进行延时, 再经过 IDDR 将双沿 1 位数据转换成单沿两位数据。其中 IDELAY2 原语必须搭配 IDELAYCTRL 原语进行使用。

该部分代码过长, 只将关键代码贴出

```
//rgmii_rxd 输入延时与双沿采样
genvar i;
generate for (i=0; i<4; i=i+1)
    (* IDELAY_GROUP = "rgmii_rx_delay" *)
    begin : rxdata_bus
        //输入延时
        (* IDELAY_GROUP = "rgmii_rx_delay" *)
        IDELAYE2 #(
            .IDELAY_TYPE      ("FIXED"),           //
            FIXED, VARIABLE, VAR_LOAD, VAR_LOAD_PIPE
            .IDELAY_VALUE      (IDELAY_VALUE),     // Input delay tap
            setting (0-31)
            .REFCLK_FREQUENCY(200.0)               // IDELAYCTRL clock input
            frequency in MHz
        )
        u_delay_rxd (
            .CNTVALUEOUT      (),                  // 5-bit output: Counter
            value output
            .DATAOUT           (rgmii_rxd_delay[i]), // 1-bit output: Delayed
            data output
        )
    end
end generate
```

```

        .C          (1'b0),          // 1-bit input: Clock
input
        .CE          (1'b0),          // 1-bit input: enable
increment/decrement
        .CINVCTRL    (1'b0),          // 1-bit input: Dynamic
clock inversion
        .CNTVALUEIN  (5'b0),          // 5-bit input: Counter
value input
        .DATAIN      (1'b0),          // 1-bit input: Internal
delay data input
        .IDATAIN     (rgmii_rxd[i]),  // 1-bit input: Data
input from the I/O
        .INC          (1'b0),          // 1-bit input:
Inc/Decrement tap delay
        .LD           (1'b0),          // 1-bit input: Load
IDELAY_VALUE input
        .LDPIPEEN     (1'b0),          // 1-bit input: Enable
PIPELINE register
        .REGRST       (1'b0)          // 1-bit input: Active-
high reset tap-delay
    );

    // 输入双沿采样寄存器
    IDDR #(
        .DDR_CLK_EDGE("SAME_EDGE_PIPELINED"), // "OPPOSITE_EDGE",
        "SAME_EDGE"
                                                // or
        "SAME_EDGE_PIPELINED"
        .INIT_Q1  (1'b0),          // Initial value of Q1:
1'b0 or 1'b1
        .INIT_Q2  (1'b0),          // Initial value of Q2:
1'b0 or 1'b1
        .SRTYPE    ("SYNC")        // Set/Reset type: "SYNC"
or "ASYN"
    ) u_iddr_rxd (
        .Q1        (gmii_rxd[i]),  // 1-bit output for
positive edge of clock
        .Q2        (gmii_rxd[4+i]), // 1-bit output for
negative edge of clock
        .C          (rgmii_rxc_bufio), // 1-bit clock input
rgmii_rxc_bufio

```

```

        .CE      (1'b1),           // 1-bit clock enable
input
        .D       (rgmii_rxd_delay[i]), // 1-bit DDR data input
        .R       (1'b0),           // 1-bit reset
        .S       (1'b0)            // 1-bit set
    );
end
endgenerate

```

代码通过 generatefor 语句实现对 IDELAY2 和 IDDR 的例化，由于输入的数据引脚为 4 位数据，因此这里共例化了 4 次。其等效于分别对输入的 IDELAY2 和 IDDR 例化 4 次，这里采用 generatefor 的写法可以减少很多的代码量，当需要对某个模块例化较多次数时，这种写法能够大大提高效率。

这儿采用调整数据延迟的方式，是当 PCB 上数据线没有做等长的时候，数据接收会出现错误，采用 idelay2 对收回的数据进行调整，保证数据在时钟双边沿的时候都能稳定采集数据。

(2) 接下来看一下 rgmii_tx 模块，rgmii_tx 模块通过调用 ODDR 原语将输入的单沿 8 位数据 (gmii_txd) 转换成双沿采样的 4 位数据 (rgmii_txd)，gmii_tx_en 和 rgmii_tx_ctl 信号的处理方法同样类似。rgmii_tx 模块信号转换示意图如下图所示：

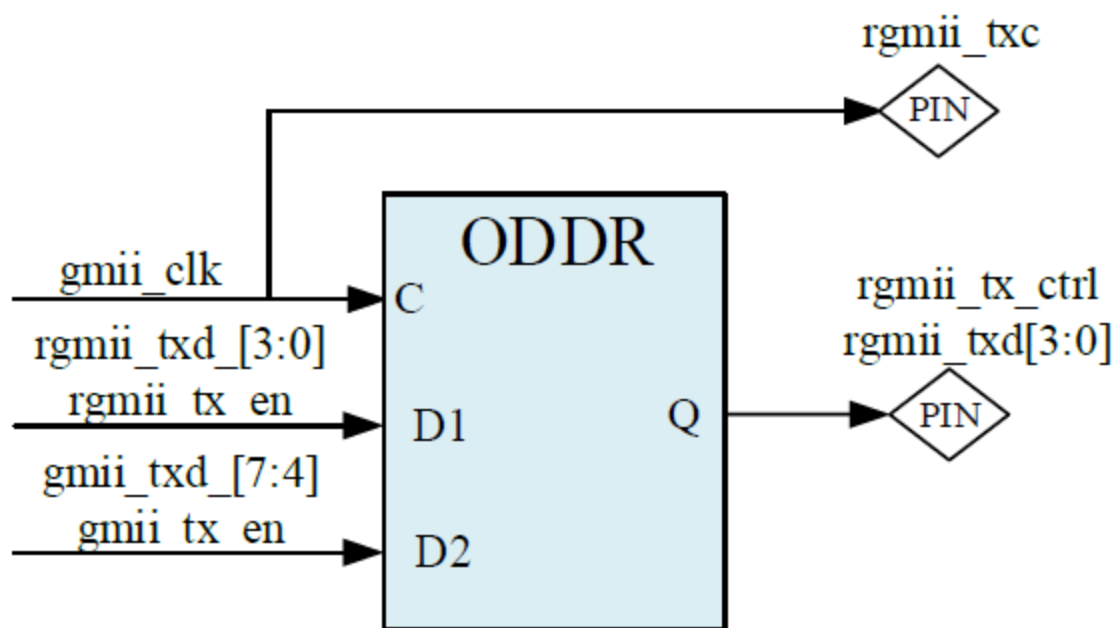


图 15-10 rgmii_tx 模块信号转换示意图

gmii_tx_en 数据使能信号和 8 位 gmii_txd 数据经过 ODDR 将单沿 2 位数据转换成双沿 1 位数据。代码如下：

```

genvar i;
generate for (i=0; i<4; i=i+1)
    begin : txdata_bus
        // 输出双沿采样寄存器 (rgmii_txd)
        ODDR #(

```

```

        .DDR_CLK_EDGE ("SAME_EDGE"), // "OPPOSITE_EDGE" or
"SAME_EDGE"
        .INIT          (1'b0),        // Initial value of Q: 1'b0 or
1'b1
        .SRTYPE        ("SYNC")       // Set/Reset type: "SYNC" or
"ASYNC"
    ) ODDR_inst (
        .Q              (rgmii_txd[i]), // 1-bit DDR output
        .C              (gmii_tx_clk),  // 1-bit clock input
        .CE             (1'b1),         // 1-bit clock enable input
        .D1             (gmii_txd[i]),  // 1-bit data input (positive
edge)
        .D2             (gmii_txd[4+i]), // 1-bit data input (negative
edge)
        .R              (1'b0),         // 1-bit reset
        .S              (1'b0)         // 1-bit set
    );
end
endgenerate

```

(3) 上述代码模块，是将输入的数据源进行处理，以组成 FPGA 能处理的单沿数据，而 MAC 层组包则通过状态机控制，代码如下：

```

//时序电路描述状态输出，发送以太网数据
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        skip_en <= 1'b0;
        cnt <= 6'd0;
        data_cnt <= 5'd0;
        crc_en <= 1'b0;
        gmii_tx_en <= 1'b0;
        gmii_txd <= 8'd0;
        tx_done_t <= 1'b0;

        //初始化数组
        //前导码 7 个 8'h55 + 1 个 8'hd5
        preamble[0] <= 8'h55;
        preamble[1] <= 8'h55;
        preamble[2] <= 8'h55;
        preamble[3] <= 8'h55;
        preamble[4] <= 8'h55;
        preamble[5] <= 8'h55;
        preamble[6] <= 8'h55;
        preamble[7] <= 8'hd5;
    end
end

```

```

// 以太网帧头
eth_head[0] <= DES_MAC[47:40]; // 目的MAC 地址
eth_head[1] <= DES_MAC[39:32];
eth_head[2] <= DES_MAC[31:24];
eth_head[3] <= DES_MAC[23:16];
eth_head[4] <= DES_MAC[15:8];
eth_head[5] <= DES_MAC[7:0];
eth_head[6] <= BOARD_MAC[47:40]; // 源MAC 地址
eth_head[7] <= BOARD_MAC[39:32];
eth_head[8] <= BOARD_MAC[31:24];
eth_head[9] <= BOARD_MAC[23:16];
eth_head[10] <= BOARD_MAC[15:8];
eth_head[11] <= BOARD_MAC[7:0];
eth_head[12] <= ETH_TYPE[15:8]; // 以太网帧类型
eth_head[13] <= ETH_TYPE[7:0];
// ARP 数据
arp_data[0] <= HD_TYPE[15:8]; // 硬件类型
arp_data[1] <= HD_TYPE[7:0];
arp_data[2] <= PROTOCOL_TYPE[15:8]; // 上层协议类型
arp_data[3] <= PROTOCOL_TYPE[7:0];
arp_data[4] <= 8'h06; // 硬件地址长度, 6
arp_data[5] <= 8'h04; // 协议地址长度, 4
arp_data[6] <= 8'h00; // OP, 操作码 8'h01: ARP 请求
8'h02: ARP 应答
arp_data[7] <= 8'h01;
arp_data[8] <= BOARD_MAC[47:40]; // 发送端(源)MAC 地址
arp_data[9] <= BOARD_MAC[39:32];
arp_data[10] <= BOARD_MAC[31:24];
arp_data[11] <= BOARD_MAC[23:16];
arp_data[12] <= BOARD_MAC[15:8];
arp_data[13] <= BOARD_MAC[7:0];
arp_data[14] <= BOARD_IP[31:24]; // 发送端(源)IP 地址
arp_data[15] <= BOARD_IP[23:16];
arp_data[16] <= BOARD_IP[15:8];
arp_data[17] <= BOARD_IP[7:0];
arp_data[18] <= DES_MAC[47:40]; // 接收端(目的)MAC 地址
arp_data[19] <= DES_MAC[39:32];
arp_data[20] <= DES_MAC[31:24];
arp_data[21] <= DES_MAC[23:16];
arp_data[22] <= DES_MAC[15:8];
arp_data[23] <= DES_MAC[7:0];
arp_data[24] <= DES_IP[31:24]; // 接收端(目的)IP 地址

```

```

arp_data[25] <= DES_IP[23:16];
arp_data[26] <= DES_IP[15:8];
arp_data[27] <= DES_IP[7:0];
end

```

上述代码则为 FPGA 发送数据是 MAC 端的初始值，其中每一个时钟都会输出一个 8bit 的数据，该数据是以字节为单位，且符合 MAC 层的数据帧结构。

接收模块的数据与发送模块的数据是一致的，这边不多解析占文档空间，有兴趣的小伙伴可以通过代码自行理解。

4.2 ARP 模块

在设计 ARP 模块之前，首先的了解下什么是 ARP 协议，ARP 协议的作用是什么？下面则对 ARP 协议进行简单介绍：

ARP 协议的基本功能是通过目的设备的 IP 地址，查询目的设备的 MAC 地址，以保证通信的顺利进行。MAC 地址在网络中表示网卡的 ID，每个网卡都需要并有且仅有一个 MAC 地址。在获取到目的 MAC 地址之后，将目的 MAC 地址更新至 ARP 缓存表中，称为 ARP 映射，下次通信时，可以直接从 ARP 缓存表中获取，而不用重新通过 ARP 获取 MAC 地址。但一般 ARP 缓存表会有过期时间，过期后需要重新通过 ARP 协议进行获取。

ARP 映射是指将 IP 地址和 MAC 地址映射起来，分为静态映射和动态映射。

静态映射指手动创建一张 ARP 表，把 IP 地址和 MAC 地址关联起来。手动绑定之后，源主机在通信之前，就可以直接从 ARP 表中直接找到 IP 地址对应的 MAC 地址，但这样做有一定的局限性，因为 MAC 地址可能会变化，比如：

机器可能更换 NIC（网络适配器），结果变成一个新的物理地址；

在某些局域网中，每当计算机加电时，他的物理地址都要改变一次。

移动电脑可以从一个物理网络转移到另一个物理网络，这样会改变物理地址。

要避免这些问题出现，必须定期维护更新 ARP 表，此类比较麻烦而且会影响网络性能。

动态映射指使用协议来获取相对应的物理地址，之所以用动态这个词是因为这个过程是自动完成的，一般应用程序的用户或系统管理员不必关心。已经设计出用于实现动态映射协议的有 ARP 和 RARP（逆地址解析协议）两种，如下图所示：

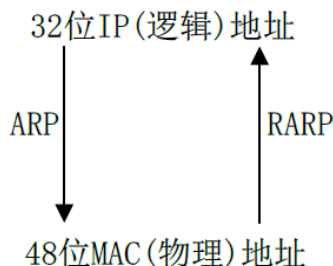


图 15-11 地址解析协议：ARP 和 RARP

ARP 把 IP 地址映射为物理地址，RARP 把物理地址映射为 IP 地址。RARP 是被那些没有磁盘驱动器的系统使用（一般是无盘工作站或 X 终端），此类应用较少，本章不做讨论。

ARP 协议分为 ARP 请求和 ARP 应答，源主机发起查询目的 MAC 地址的报文称为 ARP 请求，目的主机响应源主机并发送包含本地 MAC 地址的报文称为 ARP 应答。

当主机需要找出这个网络中的另一个主机的物理地址时，它就可以发送一个 ARP 请求报文，这个报文包含了发送方的 MAC 地址和 IP 地址以及接收方的 IP 地址。因为发送方不知道接收方的物理地址，所以这个查询分组会在网络层中进行广播，即 ARP 请求时发送的

接收方物理地址为广播地址，用 48'hff_ff_ff_ff_ff_ff 表示。ARP 请求的示意图如下图所示：

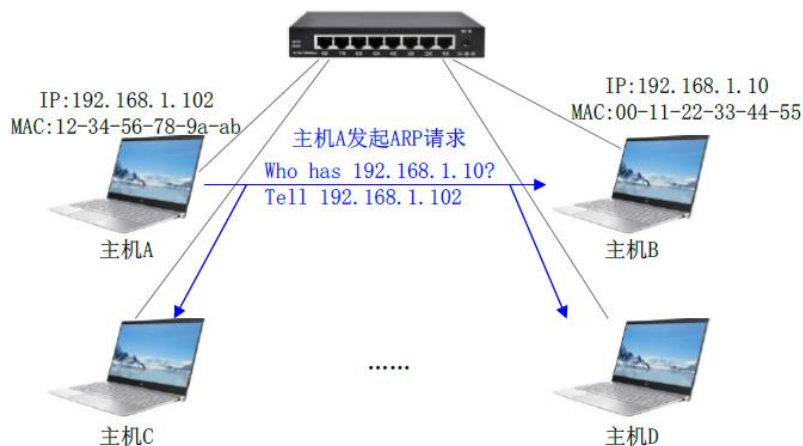


图 15-12 ARP 请求示意图

上图中的主机 A 发起 ARP 请求，由于发送的目的 MAC 地址为广播地址，所以此时局域网中的所有主机都会进行接收并处理这个 ARP 请求报文，然后进行验证，查看接收方的 IP 地址是不是自己的地址。是则返回 ARP 应答报文，不是则不响应。

只有验证成功的主机才会返回一个 ARP 应答报文，这个应答报文包含接收方的 IP 地址和物理地址。ARP 应答的示意图如下图所示：

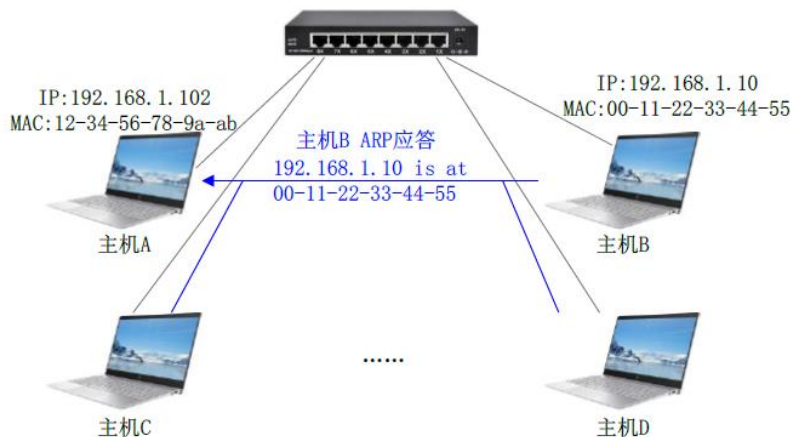


图 15-13 ARP 应答示意图

主机 B 利用收到的 ARP 请求报文中的请求方物理地址，以单播的方式直接发送给主机 A，主机 A 将收到的 ARP 应答报文中的目的 MAC 地址解析出来，将目的 MAC 地址和目的 IP 地址更新至 ARP 缓存表中。当再次和主机 A 通信时，可以直接从 ARP 缓存表中获取，而不用重新发起 ARP 请求报文。需要说明的是，ARP 缓存表中的表项有过期时间（一般为 20 分钟），过期之后，需要重新发起 ARP 请求以获取目的 MAC 地址。

ARP 协议是与 IP 协议是统同一层的，也就是说 ARP 协议的传输格式与 IP 层的传输都是 MAC 的上层协议，ARP 数据包格式如下：

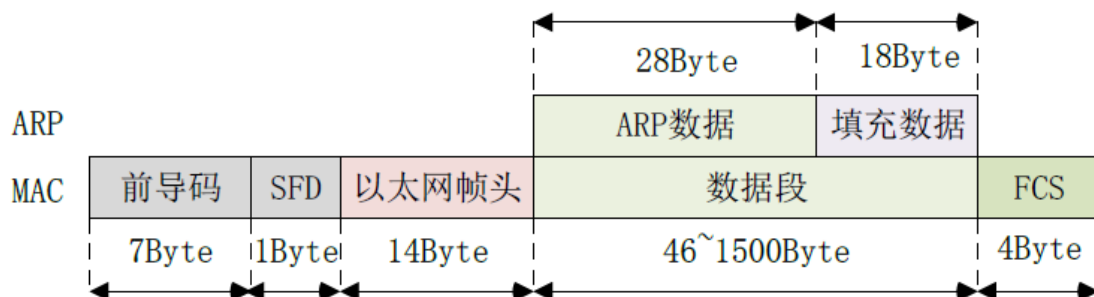


图 15-14 ARP 数据帧格式

ARP 顶层模块实现了整个以太网帧格式与 ARP 协议的功能，它大致可以由 ARP 接收模块（arp_rx）、ARP 发送模块（arp_tx）和 CRC 校验模块（crc32_d8）组成。

(1) 下面代码为发送 ARP 数据的状态，我们前面讲过以太网帧格式的数据部分最少是 46 个字节，ARP 数据只有 28 个字节，因此在发送完 ARP 数据之后补充发送 18 个字节，填充的数据为 0。

```

st_arp_data : begin                                //发送ARP 数据
    crc_en <= 1'b1;
    gmii_tx_en <= 1'b1;
    //至少发送46 个字节
    if (cnt == MIN_DATA_NUM - 1'b1) begin
        skip_en <= 1'b1;
        cnt <= 1'b0;
        data_cnt <= 1'b0;
    end
    else
        cnt <= cnt + 1'b1;
        if(data_cnt <= 6'd27) begin
            data_cnt <= data_cnt + 1'b1;
            gmii_txd <= arp_data[data_cnt];
        end
        else
            gmii_txd <= 8'd0;                        //Padding, 填充0
    end
end

```

(2) 下面为发送 CRC 校验部分代码，发送模块的 CRC 校验是由 crc32_d8 模块完成的，发送模块将输入的 crc 的计算结果每 4 位高低位互换，按位取反发送出去，crc 计算部分在后面阐述。

```

st_crc      : begin                                //发送CRC 校验值
    gmii_tx_en <= 1'b1;
    cnt <= cnt + 1'b1;
    if(cnt == 6'd0)
        gmii_txd <= {~crc_next[0], ~crc_next[1],
~crc_next[2],~crc_next[3],
~crc_next[4], ~crc_next[5],
~crc_next[6],~crc_next[7]};

```

```

        else if(cnt == 6'd1)
            gmii_txd <= {~crc_data[16], ~crc_data[17],
~crc_data[18],
                        ~crc_data[19], ~crc_data[20],
~crc_data[21],
                        ~crc_data[22], ~crc_data[23]};
        else if(cnt == 6'd2) begin
            gmii_txd <= {~crc_data[8], ~crc_data[9],
~crc_data[10],
                        ~crc_data[11], ~crc_data[12],
~crc_data[13],
                        ~crc_data[14], ~crc_data[15]};

        end
        else if(cnt == 6'd3) begin
            gmii_txd <= {~crc_data[0], ~crc_data[1],
~crc_data[2], ~crc_data[3],
                        ~crc_data[4], ~crc_data[5],
~crc_data[6], ~crc_data[7]};
            tx_done_t <= 1'b1;
            skip_en <= 1'b1;
            cnt <= 1'b0;
        end
    else;
end
default ;;

```

(3) ARP 接收模块 (arp_rx): ARP 接收模块负责解析以太网的数据, 判断目的 MAC 地址和目的 IP 地址是否为开发板的地址, 然后按照 ARP 协议将数据解析出来。当解析到正确的 ARP 数据包后, 拉高 arp_rx_done 信号, 持续一个时钟周期。arp_rx_type 用于表示 ARP 数据包的类型, 0 表示收到 ARP 请求包, 1 表示收到 ARP 应答包。src_mac 和 src_ip 分别是解析出的对端设备 MAC 地址和 IP 地址。

接收模块使用三段式状态机来解析以太网包, 从上图可以比较直观的看到每个状态实现的功能以及跳转到下一个状态的条件。这里需要注意的一点是, 在中间状态如前导码错误、MAC 地址错误以及 IP 地址等错误时跳转到 st_rx_end 状态, 而不是跳转到 st_idle 状态。因为中间状态在解析到数据错误时, 单包数据的接收还没有结束, 如果此时跳转到 st_idle 状态会误把有效数据当成前导码来解析, 所以状态跳转到 st_rx_end。而 gmii_rx_dv 信号为 0 时, 单包数据才算接收结束, 所以 st_rx_end 跳转到 st_idle 的条件是 eth_rxdv=0, 准备接收下一包数据。因为代码较长, 只粘贴了第三段状态机的接收 ARP 数据状态和接收结束状态源代码, 代码如下:

```

st_arp_data : begin
    if(gmii_rx_dv) begin
        cnt <= cnt + 5'd1;
    end
end

```

码

址

地址和操作码

begin

```

        if(cnt == 5'd6)
            op_data[15:8] <= gmii_rxd;           //操作

        else if(cnt == 5'd7)
            op_data[7:0] <= gmii_rxd;
        else if(cnt >= 5'd8 && cnt < 5'd14)       //源MAC 地址
            src_mac_t <= {src_mac_t[39:0],gmii_rxd};
        else if(cnt >= 5'd14 && cnt < 5'd18)       //源IP 地址
            src_ip_t<= {src_ip_t[23:0],gmii_rxd};
        else if(cnt >= 5'd24 && cnt < 5'd28)       //目标IP 地

            des_ip_t <= {des_ip_t[23:0],gmii_rxd};
        else if(cnt == 5'd28) begin
            cnt <= 5'd0;
            if(des_ip_t == BOARD_IP) begin         //判断目的IP

                if((op_data == 16'd1) || (op_data == 16'd2))

                    skip_en <= 1'b1;
                    arp_rx_done <= 1'b1;
                    src_mac <= src_mac_t;
                    src_ip <= src_ip_t;
                    src_mac_t <= 48'd0;
                    src_ip_t <= 32'd0;
                    des_mac_t <= 48'd0;
                    des_ip_t <= 32'd0;
                    if(op_data == 16'd1)
                        arp_rx_type <= 1'b0;       //ARP 请求
                    else
                        arp_rx_type <= 1'b1;       //ARP 应答
                    end
                else
                    error_en <= 1'b1;
                end
            else
                error_en <= 1'b1;
            end
        else;
    end
end
st_rx_end : begin
    cnt <= 5'd0;

```

```

//单包数据接收完成
if(gmii_rx_dv == 1'b0 && skip_en == 1'b0)
    skip_en <= 1'b1;
else;
end
default : ;

```

st_arp_data 状态根据 ARP 协议解析数据，代码中会判断目的 IP 地址和 OP 操作码是否正确，如果错误，则丢弃该包数据。随后根据操作码为 arp_rx_type（接收到的 ARP 数据包类型）赋值，当接收到 ARP 请求包时，arp_rx_type 等于 0；当接收到 ARP 应答包时，arp_rx_type 等于 1。

4.3 UDP 模块

(1) UDP 接收模块 (udp_rx)：UDP 接收模块较为简单，因为不需要对数据做 IP 首部校验也不需要 CRC 循环冗余校验，只需要判断目的 MAC 地址与开发板 MAC 地址、目的 IP 地址与开发板 IP 地址是否一致即可。接收模块的解析顺序是：前导码+帧起始界定符→以太网帧头→IP 首部→UDP 首部→UDP 数据（有效数据）→接收结束。

接收模块使用三段式状态机来解析以太网包，从上图可以比较直观的看到每个状态实现的功能以及跳转到下一个状态的条件。这里需要注意的一点是，在中间状态如前导码错误、MAC 地址错误、协议类型错误、不是 UDP 协议错误以及 IP 地址错误时跳转到 st_rx_end 状态而不是跳转到 st_idle 状态。因为中间状态在解析到数据错误时，单包数据的接收还没有结束，如果此时跳转到 st_idle 状态会误把有效数据当成前导码来解析，所以状态跳转到 st_rx_end。而 eth_rxdv 信号为 0 时，单包数据才算接收结束，所以 st_rx_end 跳转到 st_idle 的条件是 eth_rxdv=0，准备接收下一包数据。因为代码较长，只粘贴了第三段状态机的接收数据状态和接收结束状态源代码，代码如下：

```

st_rx_data : begin //接收有效数据
    if(skip_en)
        next_state = st_rx_end;
    else
        next_state = st_rx_data;
    end
st_rx_end : begin //接收结束
    if(skip_en)
        next_state = st_idle;
    else
        next_state = st_rx_end;
    end
default : next_state = st_idle;

```

st_rx_data 状态表示接收 UDP 的有效数据，程序中的第 230 行代码将接收的数据传递给 rec_data。在接收完有效数据后，拉高 rec_pkt_done（单包有效数据接收完成）信号，如程序中第 236 行代码所示。

(2) UDP 发送模块 (udp_tx)：UDP 发送模块和接收模块比较类似，但是多了 IP 首部校验和和 CRC 循环冗余校验的计算。CRC 的校验并不是在发送模块完成，而是在 CRC 校验

模块 (crc32_d8) 里完成的。发送模块的发送顺序是前导码+帧起始界定符→以太网帧头→IP 首部→UDP 首部→UDP 数据 (有效数据) →CRC 校验。

输出信号中, tx_done 为以太网发送完成信号, 当一包数据发送完成后且 CRC 校验完成后, tx_done 会拉高一个时钟周期, 表示 UDP 发送完成; gmii_tx_en 为 GMII 输出数据有效信号, gmii_txd 为 GMII 输出数据, 当 gmii_tx_en 信号拉高, GMII 开始发送数据, 当 gmii_tx_en 信号拉低表示数据发送完成; crc_en 为 CRC 开始校验使能信号, 当 CRC 开始校验时 crc_en 信号拉高, CRC 校验完成后, crc_en 信号拉低; crc_clr 为 CRC 数据复位信号, 当 UDP 发送完成后会拉高一个时钟周期, 表示对 CRC 数据进行清零复位。

// 发送数据

```
always @(posedge clk or negedge rst_n) begin
```

```
    if(!rst_n) begin
```

```
        skip_en <= 1'b0;
```

```
        cnt <= 5'd0;
```

```
        check_buffer <= 32'd0;
```

```
        ip_head[1][31:16] <= 16'd0;
```

```
        tx_bit_sel <= 2'b0;
```

```
        crc_en <= 1'b0;
```

```
        gmii_tx_en <= 1'b0;
```

```
        gmii_txd <= 8'd0;
```

```
        tx_req <= 1'b0;
```

```
        tx_done_t <= 1'b0;
```

```
        data_cnt <= 16'd0;
```

```
        real_add_cnt <= 5'd0;
```

// 初始化数组

// 前导码 7 个 8'h55 + 1 个 8'hd5

```
preamble[0] <= 8'h55;
```

```
preamble[1] <= 8'h55;
```

```
preamble[2] <= 8'h55;
```

```
preamble[3] <= 8'h55;
```

```
preamble[4] <= 8'h55;
```

```
preamble[5] <= 8'h55;
```

```
preamble[6] <= 8'h55;
```

```
preamble[7] <= 8'hd5;
```

// 目的 MAC 地址

```
eth_head[0] <= DES_MAC[47:40];
```

```
eth_head[1] <= DES_MAC[39:32];
```

```
eth_head[2] <= DES_MAC[31:24];
```

```
eth_head[3] <= DES_MAC[23:16];
```

```
eth_head[4] <= DES_MAC[15:8];
```

```
eth_head[5] <= DES_MAC[7:0];
```

// 源 MAC 地址

```
eth_head[6] <= BOARD_MAC[47:40];
```

```

eth_head[7] <= BOARD_MAC[39:32];
eth_head[8] <= BOARD_MAC[31:24];
eth_head[9] <= BOARD_MAC[23:16];
eth_head[10] <= BOARD_MAC[15:8];
eth_head[11] <= BOARD_MAC[7:0];
// 以太网类型
eth_head[12] <= ETH_TYPE[15:8];
eth_head[13] <= ETH_TYPE[7:0];

```

(3) 下面代码为初始状态，根据输入的发送类型、目的 MAC 地址和 IP 地址，重新更新数组里的值。

```

st_idle      : begin
    if(trig_tx_en) begin
        skip_en <= 1'b1;
        // 版本号: 4 首部长度: 5(单位:32bit, 20byte/4=5)
        ip_head[0] <= {8'h45, 8'h00, total_num};
        // 16 位标识, 每次发送累加 1
        ip_head[1][31:16] <= ip_head[1][31:16] + 1'b1;
        // bit[15:13]: 010 表示不分片
        ip_head[1][15:0] <= 16'h4000;
        // 协议: 17(udp)
        ip_head[2] <= {8'h40, 8'd17, 16'h0};
        // 源 IP 地址
        ip_head[3] <= BOARD_IP;
        // 目的 IP 地址
        if(des_ip != 32'd0)
            ip_head[4] <= des_ip;
        else
            ip_head[4] <= DES_IP;
        // 16 位源端口号: 1234 16 位目的端口号:
1234

        ip_head[5] <= {16'd1234, 16'd1234};
        // 16 位 udp 长度, 16 位 udp 校验和
        ip_head[6] <= {udp_num, 16'h0000};
        // 更新 MAC 地址
        if(des_mac != 48'b0) begin
            // 目的 MAC 地址
            eth_head[0] <= des_mac[47:40];
            eth_head[1] <= des_mac[39:32];
            eth_head[2] <= des_mac[31:24];
            eth_head[3] <= des_mac[23:16];
            eth_head[4] <= des_mac[15:8];
            eth_head[5] <= des_mac[7:0];

```

```
end
```

(4) 下面为发送数据部分代码:

```

st_tx_data : begin                                //发送数据
    crc_en <= 1'b1;
    gmii_tx_en <= 1'b1;
    gmii_txd <= tx_data;
    tx_bit_sel <= tx_bit_sel + 3'd1;
    if(data_cnt < tx_data_num - 16'd1)
        data_cnt <= data_cnt + 16'd1;
    else if(data_cnt == tx_data_num - 16'd1)begin
        //如果发送的有效数据少于18个字节,在后面填补充位
        //补充的值为最后一次发送的有效数据
        if(data_cnt + real_add_cnt < real_tx_data_num -
16'd1)

            real_add_cnt <= real_add_cnt + 5'd1;
        else begin
            skip_en <= 1'b1;
            data_cnt <= 16'd0;
            real_add_cnt <= 5'd0;
            tx_bit_sel <= 3'd0;
        end
    end
end
else ;

    if(data_cnt == tx_data_num - 16'd2)
        tx_req <= 1'b0;
    else ;

end
end

```

4.4 TOP 层模块

TOP 层中主要是对数据进行各个模块进行例化, 其中最主要的是生成了两个 IP, 一个是通过 PLL 生成了 200Mhz 的时钟, 该时钟作为 IDELAY 模块参考时钟以及一个异步 FIFO, 以异步 FIFO 作为接收模块与发送模块的跨时钟处理。

PLL 模块设置如下图所示:

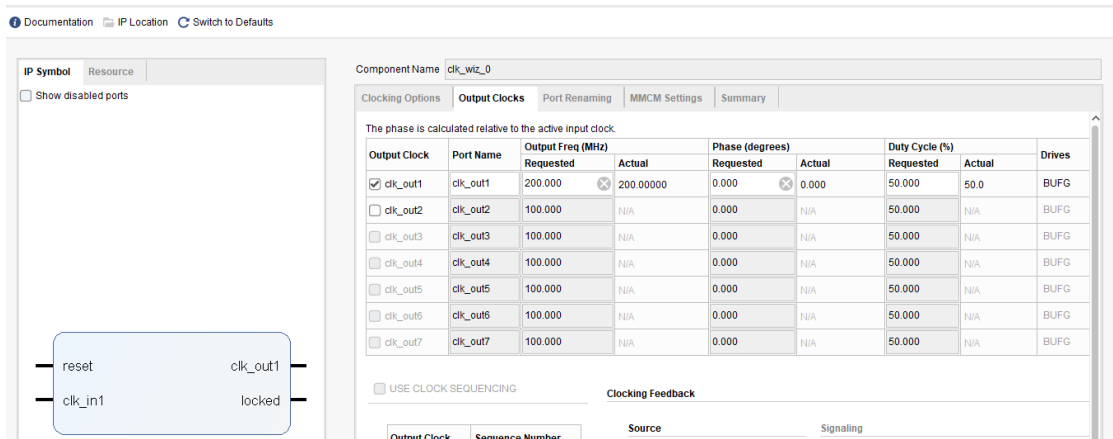


图 15-15 PLL IP 核设置

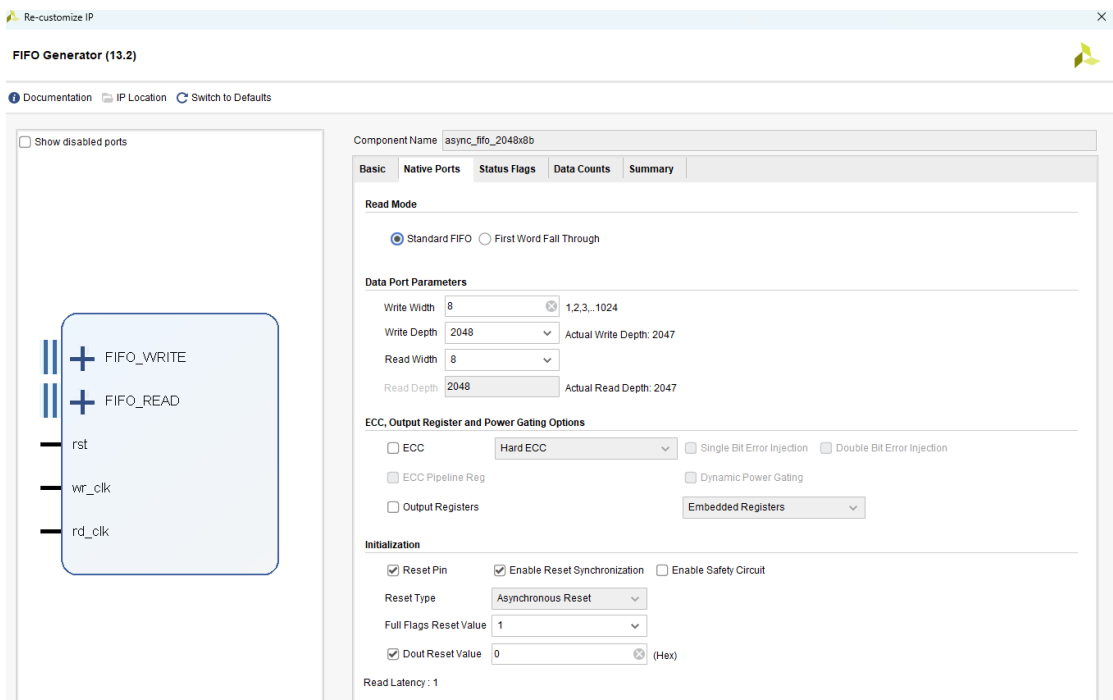


图 15-16 异步 FIFO IP 核设置

这设置 FIFO 的深度为 2048，UDP 协议栈发送与接收的一帧以太网数据的最大字长为 1472，因此设置为 2048 是完全够用的。

5.协议栈模块仿真

由于协议栈的模块过多，因此需要对协议栈中每个模块进行单独仿真。

5.1 ARP 仿真验证

根据 ARP 模块的介绍，通过例化 ARP 顶层模块的代码，并通过激励输入端的信号完成 ARP 仿真代码的编写。需要注意的是为了体现 ARP 接收和发送代码请求到应答的状态，仿真代码中我们将开发板的 IP 地址与目的 IP 地址设置为相同的地址。

(1) ARP 仿真代码如下：

```
//parameter define
```

```

parameter T = 8; //时钟周期为8ns
parameter OP_CYCLE = 100; //操作周期

//开发板MAC 地址 00-11-22-33-44-55
parameter BOARD_MAC = 48'h00_11_22_33_44_55;
//实际开发板 IP 地址 192.168.1.10
//为了返回应答, 仿真代码将开发板地址设置与目的 IP 地址相同
parameter BOARD_IP = {8'd192,8'd168,8'd1,8'd102};
//目的MAC 地址 ff_ff_ff_ff_ff_ff
parameter DES_MAC = 48'hff_ff_ff_ff_ff_ff;
//目的 IP 地址 192.168.1.102
parameter DES_IP = {8'd192,8'd168,8'd1,8'd102};

//给输入信号初始值
initial begin
    gmii_clk = 1'b0;
    sys_rst_n = 1'b0; //复位
    #(T+1) sys_rst_n = 1'b1; //在第(T+1)ns 的时候复位信号信号拉高
end

//125Mhz 的时钟, 周期则为 1/125Mhz=8ns, 所以每4ns, 电平取反一次
always #(T/2) gmii_clk = ~gmii_clk;

always @(posedge gmii_clk or negedge sys_rst_n) begin
    if(!sys_rst_n) begin
        arp_tx_en <= 1'b0;
        arp_tx_type <= 1'b0;
        delay_cnt <= 1'b0;
        flow_cnt <= 1'b0;
    end
    else begin
        case(flow_cnt)
            'd0 : flow_cnt <= flow_cnt + 1'b1;
            'd1 : begin
                arp_tx_en <= 1'b1;
                arp_tx_type <= 1'b0; //发送ARP 请求
                flow_cnt <= flow_cnt + 1'b1;
            end
            'd2 : begin
                arp_tx_en <= 1'b0;
                flow_cnt <= flow_cnt + 1'b1;
            end
        endcase
    end
end

```

```

        'd3 : begin
            if(tx_done)
                flow_cnt <= flow_cnt + 1'b1;
            end
        'd4 : begin
            delay_cnt <= delay_cnt + 1'b1;
            if(delay_cnt == OP_CYCLE - 1'b1)
                flow_cnt <= flow_cnt + 1'b1;
            end
        'd5 : begin
            arp_tx_en <= 1'b1;
            arp_tx_type <= 1'b1; //发送ARP 应答
            flow_cnt <= flow_cnt + 1'b1;
        end
        'd6 : begin
            arp_tx_en <= 1'b0;
            flow_cnt <= flow_cnt + 1'b1;
        end
        'd7 : begin
            if(tx_done)
                flow_cnt <= flow_cnt + 1'b1;
            end
        default;;
    endcase
end
end

//ARP 通信
arp
#(
    .BOARD_MAC      (BOARD_MAC),           //参数例化
    .BOARD_IP        (BOARD_IP ),
    .DES_MAC          (DES_MAC ),
    .DES_IP           (DES_IP )
)
u_arp(
    .rst_n            (sys_rst_n ),

    .gmii_rx_clk      (gmii_rx_clk),
    .gmii_rx_dv       (gmii_rx_dv ),
    .gmii_rxd         (gmii_rxd ),
    .gmii_tx_clk      (gmii_tx_clk),

```

```

.gmii_tx_en    (gmii_tx_en ),
.gmii_txd      (gmii_txd  ),

.arp_rx_done   (arp_rx_done),
.arp_rx_type   (arp_rx_type),
.src_mac       (src_mac   ),
.src_ip        (src_ip    ),
.arp_tx_en     (arp_tx_en  ),
.arp_tx_type   (arp_tx_type),
.des_mac       (des_mac   ),
.des_ip        (des_ip    ),
.tx_done       (tx_done   )
);

```

(2) ARP 模块的仿真波形如下:

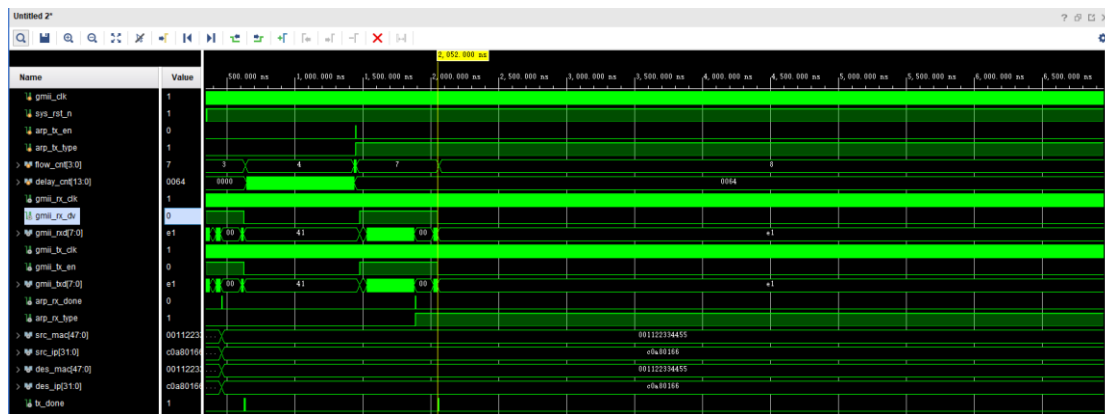


图 15-17 ARP 仿真模块的数据波形

从整体的仿真波形图中可以看出, flow_cnt 的计数范围为 0~8, 进行了一次完整的 ARP 数据发送。

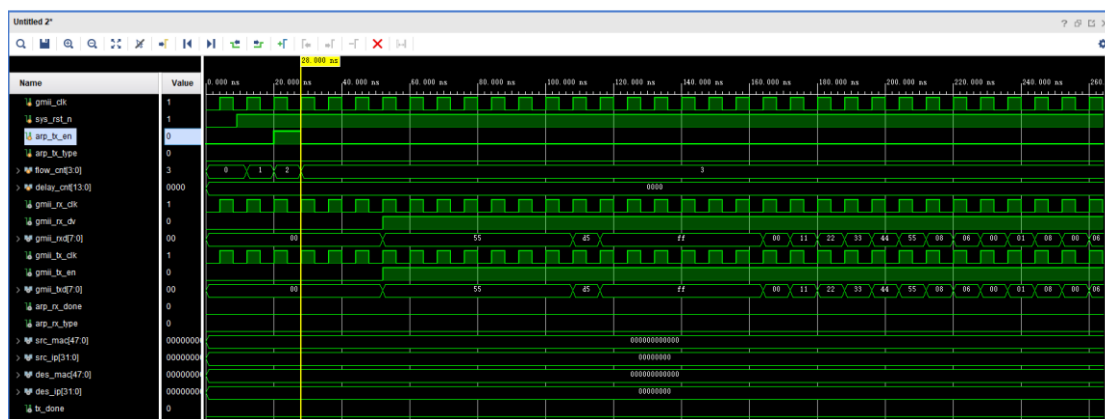


图 15-18 发送 ARP 请求仿真波形

当 flow_cnt 等于 1 时, arp_tx_en (ARP 发送使能信号) 拉高一个周期, arp_tx_type (ARP 发送类型) 电平保持低有效, ARP 进入发送请求状态。

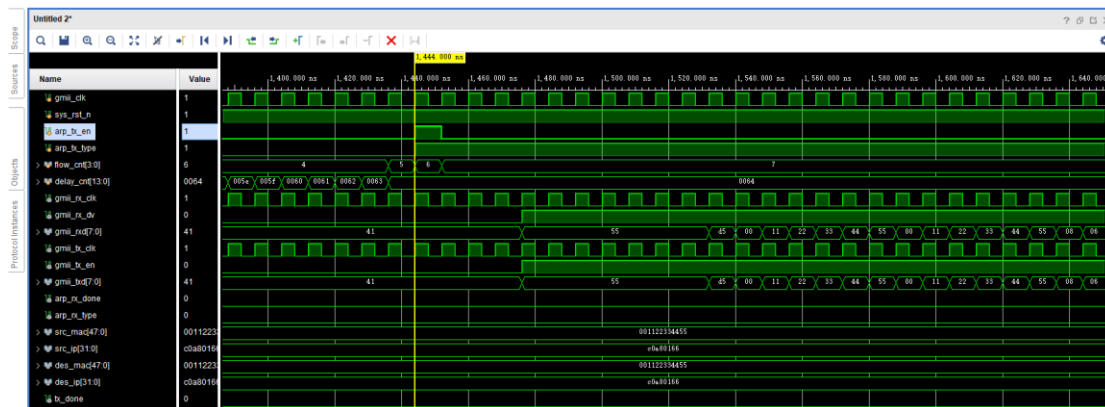


图 15-19 发送 ARP 应答仿真波形

当 flow_cnt 等于 5 时, arp_tx_en (ARP 发送使能信号) 拉高一个周期, arp_tx_type (ARP 发送类型) 电平拉高, ARP 进入发送应答状态。

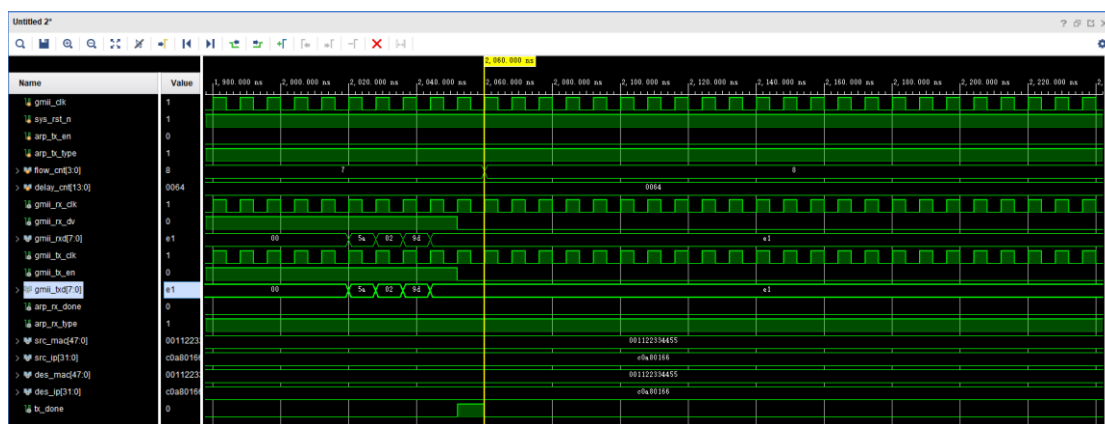


图 15-20 发送完成信号仿真波形

当发送完成后, tx_done 信号会拉高一个时钟周期, 表示完成了一次 ARP 的发送的整个流程。

5.2 UDP 仿真验证

根据 UDP 模块的介绍, 我们可以通过例化 UDP 顶层模块的代码, 并通过激励输入端的信号完成 UDP 仿真代码的编写。需要注意的是与 ICMP 仿真代码相同, 在仿真 udp 协议的接收过程中, 我们需要将开发板的 IP 地址与目的 IP 地址设置为相同的地址。

(1) UDP 模块仿真代码如下:

```
module TB_SIM_UDP();

//parameter define
parameter T = 8; //时钟周期为 8ns
parameter OP_CYCLE = 100; //操作周期(发送周期间隔)

//开发板 MAC 地址 00-11-22-33-44-55
parameter BOARD_MAC = 48'h00_11_22_33_44_55;
//开发板 IP 地址 192.168.1.10
parameter BOARD_IP = {8'd192,8'd168,8'd1,8'd10};
```

```

//目的MAC 地址 ff_ff_ff_ff_ff_ff
parameter DES_MAC    = 48'hff_ff_ff_ff_ff_ff;
//目的IP 地址 192.168.1.102
parameter DES_IP      = {8'd192,8'd168,8'd1,8'd10};

//给输入信号初始值
initial begin
    gmii_clk          = 1'b0;
    sys_rst_n          = 1'b0;    //复位
    #(T+1) sys_rst_n    = 1'b1;    //在第(T+1)ns 的时候复位信号信号拉高
end

//125Mhz 的时钟，周期则为1/125Mhz=8ns,所以每4ns，电平取反一次
always #(T/2) gmii_clk = ~gmii_clk;

always @(posedge gmii_clk or negedge sys_rst_n) begin
    if(!sys_rst_n) begin
        tx_start_en <= 1'b0;
        tx_data <= 8'h00;
        tx_byte_num <= 1'b0;
        des_mac <= 1'b0;
        des_ip <= 1'b0;
        delay_cnt <= 1'b0;
        flow_cnt <= 1'b0;
    end
    else begin
        case(flow_cnt)
            'd0 : flow_cnt <= flow_cnt + 1'b1;
            'd1 : begin
                tx_start_en <= 1'b1;    //拉高开始发送使能信号
                tx_byte_num <= 16'd10; //设置发送的字节数
                flow_cnt <= flow_cnt + 1'b1;
            end
            'd2 : begin
                tx_start_en <= 1'b0;
                flow_cnt <= flow_cnt + 1'b1;
            end
            'd3 : begin
                if(tx_req)
                    tx_data <= tx_data + 8'h11;
                if(tx_done) begin

```

```

        flow_cnt <= flow_cnt + 1'b1;
        tx_data <= 8'h00;
    end
end
'd4 : begin
    delay_cnt <= delay_cnt + 1'b1;
    if(delay_cnt == OP_CYCLE - 1'b1)
        flow_cnt <= flow_cnt + 1'b1;
    end
'd5 : begin
    tx_start_en <= 1'b1; //拉高开始发送使能信号
    tx_byte_num <= 16'd30; //设置发送的字节数
    flow_cnt <= flow_cnt + 1'b1;
end
'd6 : begin
    tx_start_en <= 1'b0;
    flow_cnt <= flow_cnt + 1'b1;
end
'd7 : begin
    if(tx_req)
        tx_data <= tx_data + 8'h11;
    if(tx_done) begin
        flow_cnt <= flow_cnt + 1'b1;
        tx_data <= 8'h00;
    end
end
default;;
endcase
end
end

//例化 UDP 模块
udp
#(
    .BOARD_MAC    (BOARD_MAC),      //参数例化
    .BOARD_IP     (BOARD_IP ),
    .DES_MAC      (DES_MAC ),
    .DES_IP       (DES_IP )
)
u_udp(
    .rst_n        (sys_rst_n ),

```

```

.gmii_rx_clk  (gmii_rx_clk ),
.gmii_rx_dv   (gmii_rx_dv  ),
.gmii_rxd     (gmii_rxd   ),
.gmii_tx_clk  (gmii_tx_clk ),
.gmii_tx_en   (gmii_tx_en),
.gmii_txd     (gmii_txd),

.rec_pkt_done  (),
.rec_en       (),
.rec_data     (),
.rec_byte_num  (),
.tx_start_en  (tx_start_en),
.tx_data      (tx_data    ),
.tx_byte_num  (tx_byte_num),
.des_mac      (des_mac    ),
.des_ip       (des_ip     ),
.tx_done      (tx_done    ),
.tx_req       (tx_req     )
);

```

(2) UDP 模块的仿真波形如下：

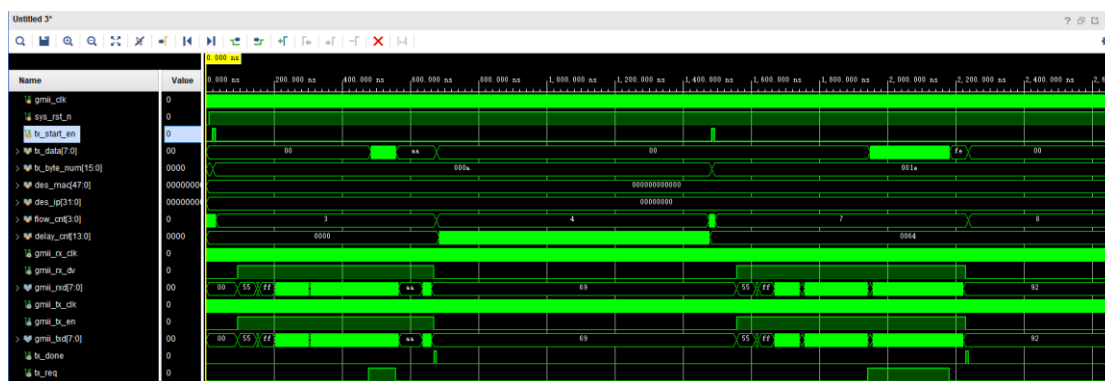


图 15-21 UDP 仿真模块的数据波形

从整体的仿真波形图中可以看出，flow_cnt 的计数范围为 0~8，进行了两次完整的 UDP 数据发送。第一次发送的字节数为 10 个，第二次发送 30 个。当发送完成之后 tx_done 信号会拉高一个时钟周期表示发送已完成。

6. 添加内部逻辑分析仪

(1) 在 UDP 协议栈中，主要可以通过抓取发送帧以及接收帧观测数据是否正确，那么可以直接将逻辑分析信号添加在输出端口，但 FPGA 是无法对双沿数据进行信号抓取的，那么只能对 GMII 信号进行数据抓取，添加的 Debug 信号如下：

```

43 wire          gmii_rx_clk          ; //GMII接收时钟
44 (* mark_debug = "true" *)
45 wire          gmii_rx_dv          ; //GMII接收数据有效信号
46 (* mark_debug = "true" *)
47 wire [7:0]    gmii_rxd            ; //GMII接收数据
48 wire          gmii_tx_clk          ; //GMII发送时钟
49 (* mark_debug = "true" *)
50 wire          gmii_tx_en           ; //GMII发送数据使能信号
51 (* mark_debug = "true" *)
52 wire [7:0]    gmii_txd            ; //GMII发送数据

```

图 15-22 UDP 发送数据以及接收数据 Debug

(2) 添加完 Debug 信号后，会对当前工程进行综合并设置需要抓取信号的深度。

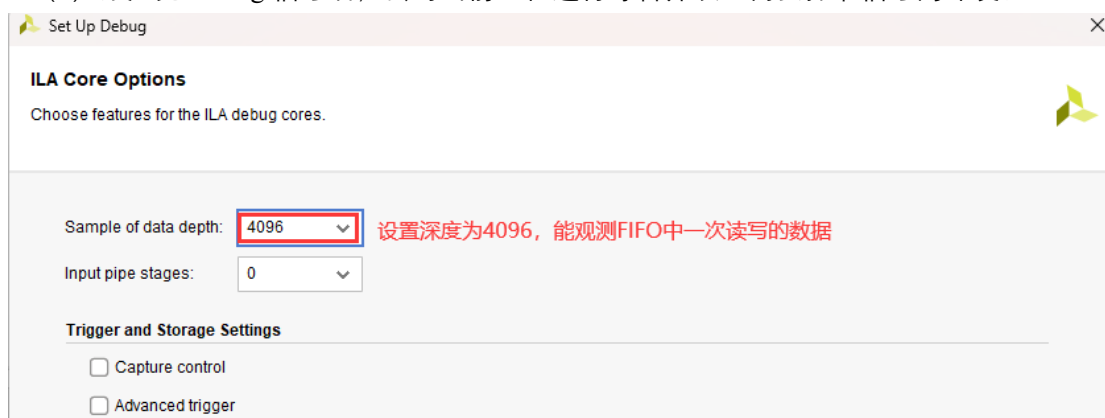


图 15-23 设置 Debug 信号的深度

(3) 查看是否写入约束文件

```

1 create_debug_core u_ila_0 ila
2 set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
3 set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
4 set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
5 set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
6 set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
7 set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
8 set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
9 set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
10 set_property port_width 1 [get_debug_ports u_ila_0/clk]
11 connect_debug_port u_ila_0/clk [get_nets [list u_gmii_to_rgmii/u_rgmii_rx/gmii_tx_clk]]
12 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
13 set_property port_width 8 [get_debug_ports u_ila_0/probe0]
14 connect_debug_port u_ila_0/probe0 [get_nets [list {gmii_txd[0]} {gmii_txd[1]} {gmii_txd[2]} {gmii_txd[3]} {gmii_txd[4]} {gmii_txd[5]} {gmii_txd[6]} {gmii_txd[7]}]]
15 create_debug_port u_ila_0 probe
16 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
17 set_property port_width 8 [get_debug_ports u_ila_0/probe1]
18 connect_debug_port u_ila_0/probe1 [get_nets [list {gmii_rxd[0]} {gmii_rxd[1]} {gmii_rxd[2]} {gmii_rxd[3]} {gmii_rxd[4]} {gmii_rxd[5]} {gmii_rxd[6]} {gmii_rxd[7]}]]
19 create_debug_port u_ila_0 probe
20 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
21 set_property port_width 1 [get_debug_ports u_ila_0/probe2]
22 connect_debug_port u_ila_0/probe2 [get_nets [list gmii_rx_dv]]
23 create_debug_port u_ila_0 probe
24 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe3]
25 set_property port_width 1 [get_debug_ports u_ila_0/probe3]
26 connect_debug_port u_ila_0/probe3 [get_nets [list gmii_tx_en]]
27 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
28 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
29 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
30 connect_debug_port dbg_hub/clk [get_nets gmii_rx_clk]

```

图 15-24 约束文件写入 Debug 信息

Debug 信号添加完毕，随后可以进行时序约束

7.添加时序约束

(1) 综合完成后，点开时序约束按钮，点击“Edit Timing Constraints”按钮后，Vivado 就会打开“Timing Constraints”界面，点击该界面下的“+”号按钮就可以添加时钟约束了，如下图所示：

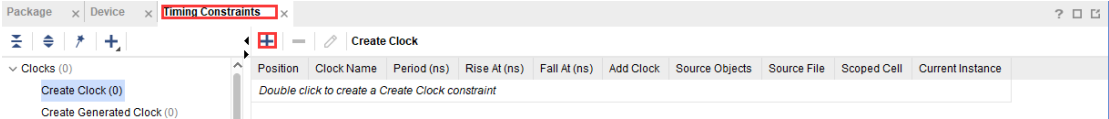


图 15-25 添加主时钟时序约束

(2) 点击“+”号后，在弹出的“CreateClock”界面，填入当前需要约束的时钟信息，如下图所示：

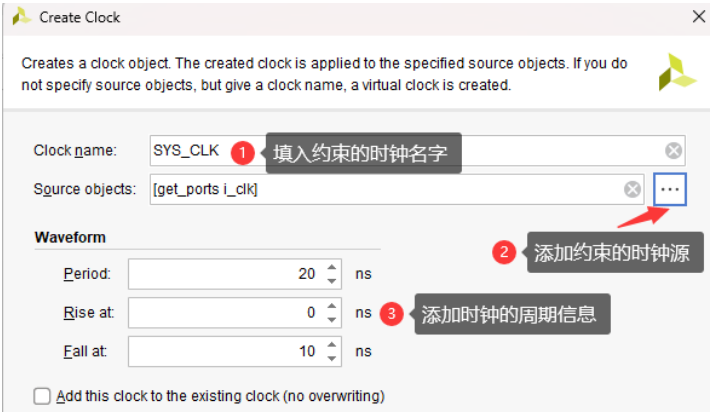


图 15-26 约束主时钟

(3) 对主时钟约束完毕后，需要对输入的 RXC 时钟进行约束，与主时钟的约束是一致的。点击图 15-26 中的 2 后，则会出现图 15-27 所示的流程：

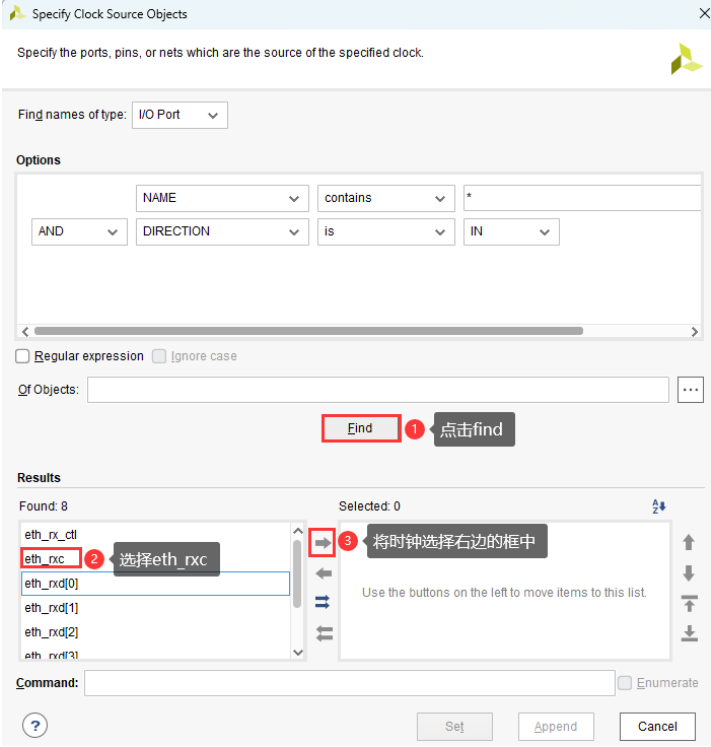


图 15-27 约束以太网输入 RXC 时钟

(4) 保存后，查看约束文件是否写入时钟约束，如下图所示：

```
1 create_clock -period 20.000 -name SYS_CLK -waveform {0.000 10.000} -add [get_ports sys_clk]
2 create_clock -period 8.000 -name eth_rxc -waveform {0.000 4.000} [get_ports eth_rxc]
```

图 15-28 写入时钟约束

查看约束文件后，可以看出当前关于时钟的约束已经写入了约束文件，接下来则是对管脚进行约束。

8.添加管脚约束

在上述步骤中添加了时序约束，但是并未对需要使用的管脚进行约束，接下来则添加管脚约束。

(1) 在综合后点开“Open Elaborated Design”，如下图所示：

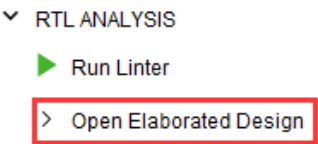


图 15-29 打开管脚约束

(2) 管脚约束，如下图所示：

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	IO Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination (Multiple)	IN_TERM
read_clk_write_clk_33166 (2)	(Multiple)			☒	33	LVCMO533*	3.300				NONE		
Scalar ports (2)													
eth_rxc	IN		Y19	☒	33	LVCMO533*	3.300				NONE	NONE	
eth_txc	OUT		AB22	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
eth_rxd (4)	IN			☒	33	LVCMO533*	3.300				NONE	NONE	
eth_rxd[3]	IN		V20	☒	33	LVCMO533*	3.300				NONE	NONE	
eth_rxd[2]	IN		U20	☒	33	LVCMO533*	3.300				NONE	NONE	
eth_rxd[1]	IN		W21	☒	33	LVCMO533*	3.300				NONE	NONE	
eth_rxd[0]	IN		W20	☒	33	LVCMO533*	3.300				NONE	NONE	
eth_txd (4)	OUT			☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
eth_txd[3]	OUT		AA21	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
eth_txd[2]	OUT		AA22	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
eth_txd[1]	OUT		U21	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
eth_txd[0]	OUT		T21	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
Scalar ports (5)													
eth_rst_n	OUT		Y21	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
eth_rx_n	IN		V19	☒	33	LVCMO533*	3.300				NONE	NONE	
eth_tx_n	OUT		AB21	☒	33	LVCMO533*	3.300	12			NONE	FP_VTT_50	
sys_clk	IN		W17	☒	33	LVCMO533*	3.300				NONE	NONE	
sys_rst_n	IN		W18	☒	33	LVCMO533*	3.300				NONE	NONE	

图 15-30 约束以太网管脚

(3) 管脚分配好之后，进行保存，保存后可以查看当前的管脚约束是否写入约束文件。如下图所示：

```
1  # 主时钟
2  set_property PACKAGE_PIN W17 [get_ports sys_clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports sys_clk]
4
5  set_property PACKAGE_PIN W18 [get_ports sys_rst_n]
6  set_property IOSTANDARD LVCMOS33 [get_ports sys_rst_n]
7
8  #UDP RGMII管脚
9  #RX管脚
10 set_property PACKAGE_PIN Y19 [get_ports eth_rxc]
11 set_property IOSTANDARD LVCMOS33 [get_ports eth_rxc]
12
13 set_property PACKAGE_PIN V19 [get_ports eth_rx_ctl]
14 set_property IOSTANDARD LVCMOS33 [get_ports eth_rx_ctl]
15
16 set_property PACKAGE_PIN V20 [get_ports eth_rxd[3]]
17 set_property IOSTANDARD LVCMOS33 [get_ports eth_rxd[3]]
18
19 set_property PACKAGE_PIN U20 [get_ports eth_rxd[2]]
20 set_property IOSTANDARD LVCMOS33 [get_ports eth_rxd[2]]
21
22 set_property PACKAGE_PIN W21 [get_ports eth_rxd[1]]
23 set_property IOSTANDARD LVCMOS33 [get_ports eth_rxd[1]]
24
25 set_property PACKAGE_PIN W20 [get_ports eth_rxd[0]]
26 set_property IOSTANDARD LVCMOS33 [get_ports eth_rxd[0]]
```

图 15-31 约束文件中写入管脚约束

9.上板验证

对整个 FPGA 工程约束完成后，可以生成 Bit 文件进行上板验证，在 Bit 文件生成后，需要注意是否有时序错误，可以发现此次设计时序没有错误，如下图所示：

Timing	Setup Hold Pulse Width
Worst Negative Slack (WNS):	1.226 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	7342
Implemented Timing Report	

图 15-32 时序结果

9.1 硬件连接及修改 IP 地址

(1) 硬件连接

将网线一端连接至 FPGA 开发板的 PL 网口，一端连接至上位机端。连接完成后对开发板上电。连接图如下图所示：

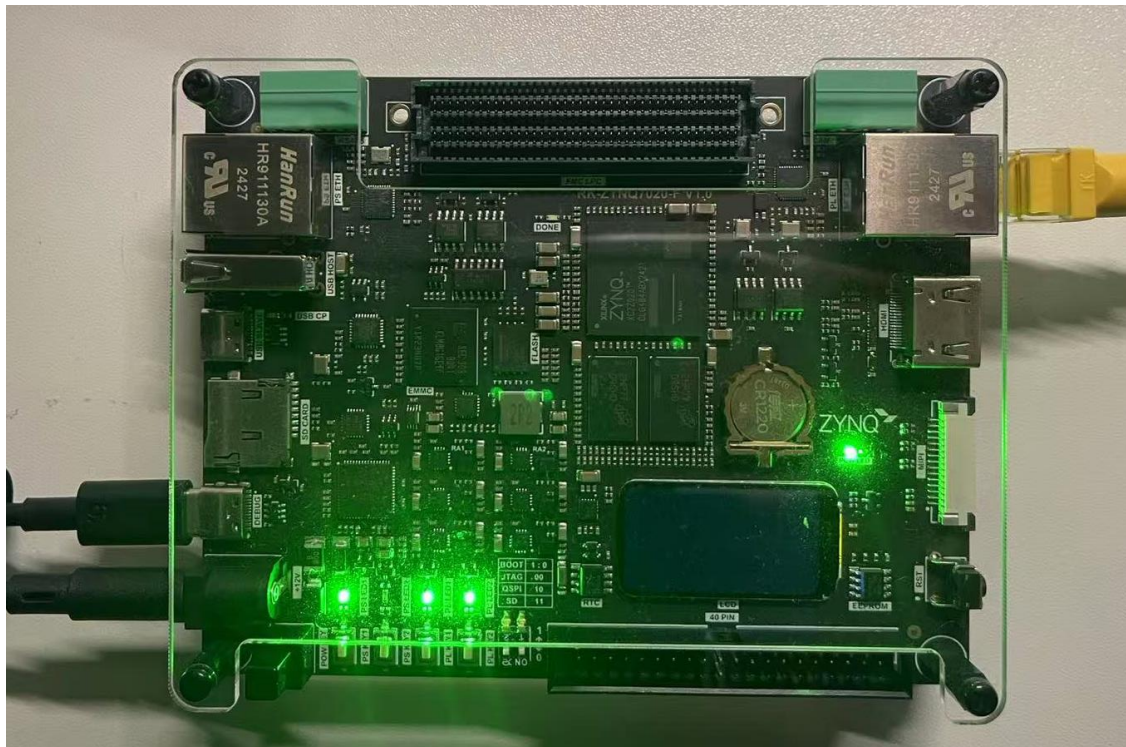


图 15-33 开发板连接图

当 FPGA 上电后，可以检查 RTL8211 PHY 芯片与电脑的协商速率，以 Windows 11 环境为例，检查方式如下：

(2) 在开始菜单输入控制面板，并点击进入控制面板

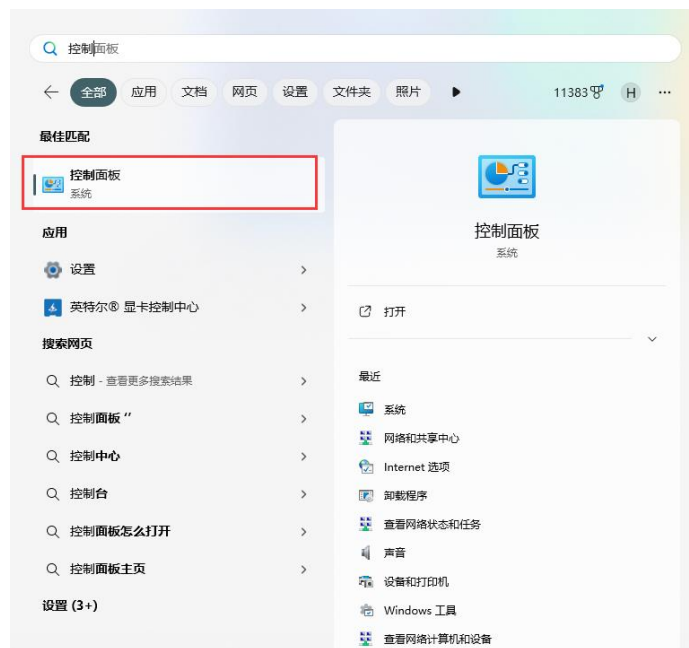


图 15-34 查找控制面板

(3) 点击网络和 Internet。



图 15-35 点击网络

(4) 点击网络和共享中心



图 15-35 点击网络和共享中心

(5) 点击适配器设置



图 15-36 点击适配器设置

(6) 依据识别的网卡，双击进入选项，如下图所示：

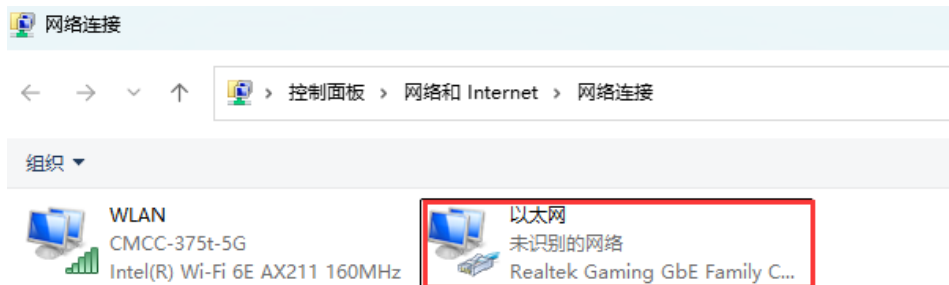


图 15-37 点击以太网

(7) 如下图红框处所示，可以发现协商速度为 1Gbps



图 15-38 以太网协商速率

(8) 依据程序设计的 IP 地址，对电脑的 IP 地址进行修改，代码中编写的目标 IP 为 192.168.1.102，那么就需要将上位机 IP 修改为 192.168.1.102，点击下图中属性进行修改。



图 15-39 点击属性

(9) 双击 Internet 协议版本。



图 15-40 双击 Internet 协议版本 4

(10) 将 IP 地址修改后，子网掩码默认即可。



图 15-41 修改 IP

9.2 ICMP 验证

(1) 下载完 bit 文件后，通过点开“开始”后搜索 CMD 进入控制面板，如下图所示：



图 15-42 搜索 CMD

(2) 打开 CMD，执行 ping 命令，输入“ping 192.168.1.10”命令如下图所示打印如下信息表示电脑 ping 开发板成功。

```
C:\Users\Peng>ping 192.168.1.10

正在 Ping 192.168.1.10 具有 32 字节的数据:
来自 192.168.1.10 的回复: 字节=32 时间=2ms TTL=128
来自 192.168.1.10 的回复: 字节=32 时间=1ms TTL=128
来自 192.168.1.10 的回复: 字节=32 时间=2ms TTL=128
来自 192.168.1.10 的回复: 字节=32 时间<1ms TTL=128

192.168.1.10 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 2ms, 平均 = 1ms
```

图 15-43 ping 命令输出信息

从上图可以看到开发板正常回复，电脑 ping 开发板成功。

(3) 其次可以在发送 pin 命令之前，将 ILA 打开，观测收到 FPGA 收到的数据是否符合 ICMP 协议格式，ILA 中数据如下所示：

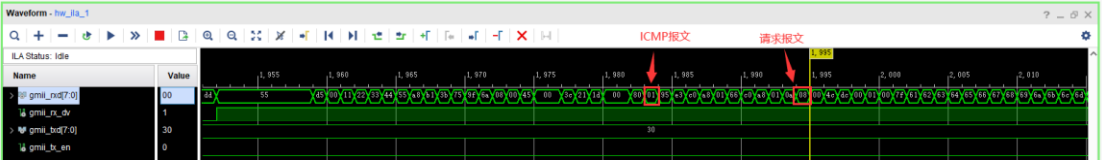


图 15-44 ILA GMII 接收数据

从上图可以看出，当电脑进行 ping 命令时，FPGA 开发板接收到一条报文，该报文是符合 MAC 层接收数据格式，其中图 15-44 中红框标注的表示这是一条 ICMP 请求报文。

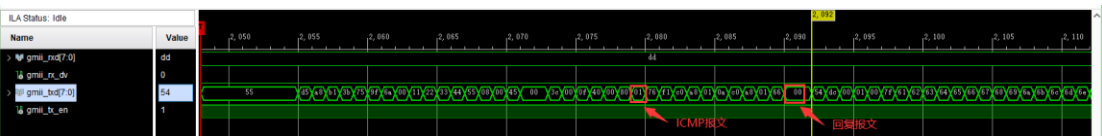


图 15-45 ILA GMII 接收数据

从上图可以看出，当电脑进行 ping 命令时，FPGA 开发板接收到一条报文后，会回复一

条报文该报文是符合 MAC 层发送数据格式, 其中图 15-45 中红框标注的表示这是一条 ICMP 回复报文。

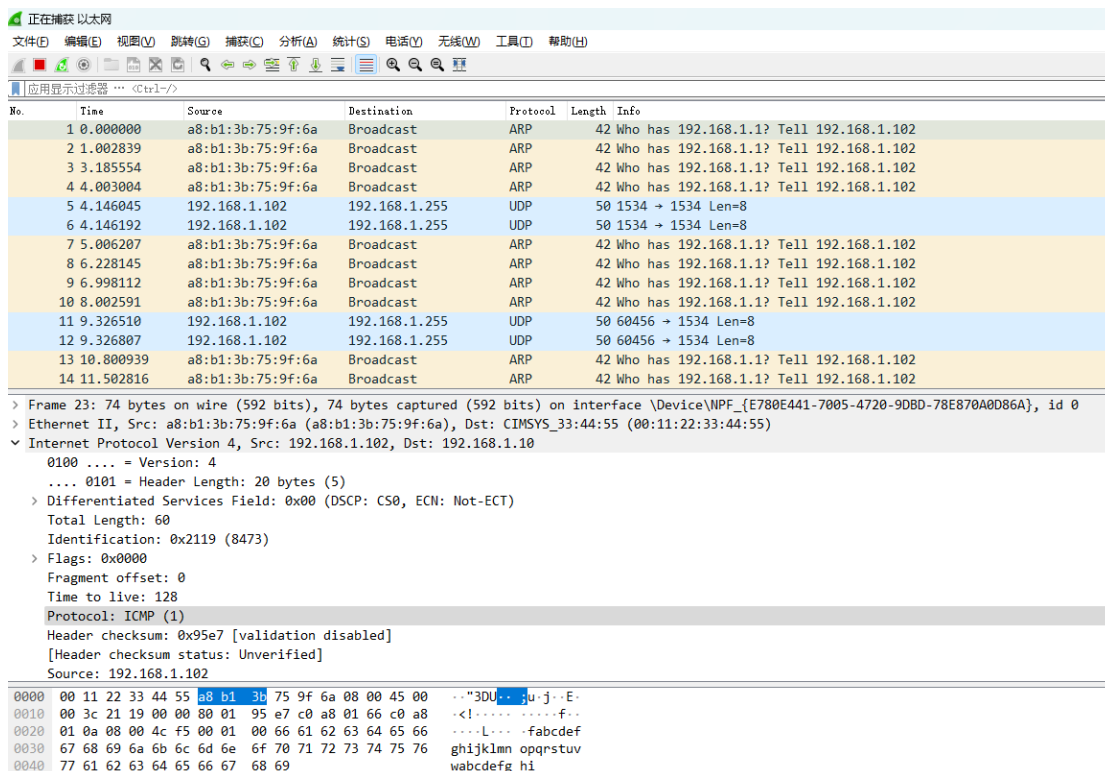


图 15-46 wireshark 以太网打开界面

从上图可以看到, 已经抓取到其它应用程序使用以太网发送的数据包, 但是这些数据包并不是开发板发送的数据包, 我们这个时候重新在电脑命令提示符框中发送“ping 192.168.1.10”命令, 可以看到 Wireshark 软件中抓取的数据, 如下图所示。

23	17.503751	192.168.1.102	192.168.1.10	ICMP	74	Echo (ping) request	id=0x0001, seq=102/26112, ttl=128 (reply in 24)
24	17.505065	192.168.1.10	192.168.1.102	ICMP	74	Echo (ping) reply	id=0x0001, seq=102/26112, ttl=128 (request in 23)
25	18.506454	192.168.1.102	192.168.1.10	ICMP	74	Echo (ping) request	id=0x0001, seq=104/26624, ttl=128 (reply in 26)
26	18.507783	192.168.1.10	192.168.1.102	ICMP	74	Echo (ping) reply	id=0x0001, seq=104/26624, ttl=128 (request in 25)
27	19.153069	192.168.1.102	192.168.1.255	UDP	50	1534 → 1534 Len=8	
28	19.153274	192.168.1.102	192.168.1.255	UDP	50	1534 → 1534 Len=8	
29	19.523491	192.168.1.102	192.168.1.10	ICMP	74	Echo (ping) request	id=0x0001, seq=105/26880, ttl=128 (reply in 30)
30	19.524848	192.168.1.10	192.168.1.102	ICMP	74	Echo (ping) reply	id=0x0001, seq=105/26880, ttl=128 (request in 29)
31	20.530637	192.168.1.102	192.168.1.10	ICMP	74	Echo (ping) request	id=0x0001, seq=106/27136, ttl=128 (reply in 32)
32	20.532026	192.168.1.10	192.168.1.102	ICMP	74	Echo (ping) reply	id=0x0001, seq=106/27136, ttl=128 (request in 31)

图 15-46 wireshark 中抓取数据

上图中第 23 行是上位机发送的 ICMP 请求数据包, 第 24 行是开发板返回的 ICMP 应答数据包。点击开发板返回的数据包, 可以看到开发板发送的详细数据, 如下图所示:

23	17.503751	192.168.1.102	192.168.1.10	ICMP	74	Echo (ping) request	id=0x0001, seq=102/26112, ttl=128 (reply in 24)
24	17.505065	192.168.1.10	192.168.1.102	ICMP	74	Echo (ping) reply	id=0x0001, seq=102/26112, ttl=128 (request in 23)
25	18.506454	192.168.1.102	192.168.1.10	ICMP	74	Echo (ping) request	id=0x0001, seq=104/26624, ttl=128 (reply in 26)
26	18.507783	192.168.1.10	192.168.1.102	ICMP	74	Echo (ping) reply	id=0x0001, seq=104/26624, ttl=128 (request in 25)

Fragment offset: 0

Time to live: 128

Protocol: ICMP (1)

Header checksum: 0x95e7 [validation disabled]

[Header checksum status: Unverified]

Source: 192.168.1.102

Destination: 192.168.1.10

Internet Control Message Protocol

Type: 8 (Echo (ping) request)

Code: 0

Checksum: 0x4cf5 [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence number (BE): 102 (0x0066)

0000	00	11	22	33	44	55	a8	b1	3b	75	9f	6a	08	00	45	00	..3DU...u.j..E.
0010	00	3c	21	19	00	00	80	01	95	e7	c0	a8	01	66	c0	a8	<!--...f--
0020	01	0a	08	00	4c	f5	00	01	00	66	61	62	63	64	65	66	L....fabcd
0030	67	68	69	6a	6b	6c	6d	6e	6f	70	71	72	73	74	75	76	ghijklmn opqrstuv
0040	77	61	62	63	64	65	66	67	68	69							wabcdfg hi

图 15-47 Wireshark 抓取到的详细数据

由上图可知，通信协议是 ICMP 协议，源 IP 地址（开发板 IP 地址）为 192.168.1.10，目的 IP 地址（电脑 IP 地址）为 192.168.1.102。上图中下方红框为开发板发送的 16 进制数据（去掉前导码、SFD 和 CRC 值），可以看到，ICMP 的用户数据段对应的 ASCII 码为“abcdefghijklmn opqrstuvwxyzabcdefghijklmnop hi”。

9.3 UDP 环回验证

接下来看一下关于 UDP 环回部分的测试，打开网口调试助手后，协议类型选择：UDP；本地主机地址选择：本地连接的 IP 地址（在这里是 192.168.1.102）；本地主机端口号：1234；设置完成后点击【打开】按钮。如下图所示：设置界面如下图所示：



图 15-48 网络调试助手界面设置

上述设置完成后，即在网络调试助手中输入“RK-ZYNQ7020-F”并以 ASCII 码进行发送，FPGA 也会以同样的字符返回，实验结果如下图所示：

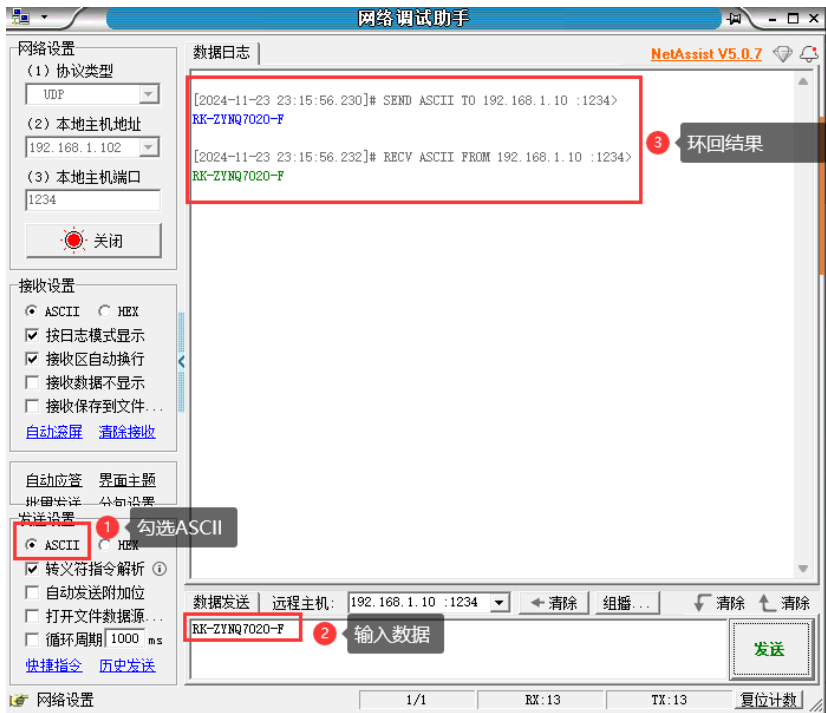


图 15-49 网络助手环回结果

接下来通过 Wireshark 软件抓取网口的数据包，界面如下图所示：

No.	Time	Source	Destination	Protocol	Length	Info
6	4.048167	192.168.1.10	192.168.1.102	UDP	60	1234 → 1234 Len=13
7	6.340372	192.168.1.102	192.168.1.10	UDP	55	1234 → 1234 Len=13
8	6.341789	192.168.1.10	192.168.1.102	UDP	60	1234 → 1234 Len=13
9	6.992693	192.168.1.102	192.168.1.255	UDP	50	60456 → 1534 Len=8
10	6.992996	192.168.1.102	192.168.1.255	UDP	50	60456 → 1534 Len=8
11	9.010928	a8:b1:3b:75:9f:6a	CIMSYS_33:44:55	ARP	42	Who has 192.168.1.10? Tell 192.168.1.102
12	9.012495	CIMSYS_33:44:55	a8:b1:3b:75:9f:6a	ARP	60	192.168.1.10 is at 00:11:22:33:44:55
13	9.479950	192.168.1.102	192.168.1.255	UDP	155	60457 → 1534 Len=113
14	9.480235	192.168.1.102	192.168.1.255	UDP	155	60457 → 1534 Len=113
15	10.512082	192.168.1.102	192.168.1.255	UDP	50	60458 → 1534 Len=8
16	10.512431	192.168.1.102	192.168.1.255	UDP	50	60458 → 1534 Len=8
17	14.134554	192.168.1.102	192.168.1.10	UDP	55	1234 → 1234 Len=13
18	14.135991	192.168.1.10	192.168.1.102	UDP	60	1234 → 1234 Len=13

图 15-50 wireshark 抓取结果

上图中第 11 行是上位机发送的 ARP 请求数据包，第 12 行是开发板返回的 ARP 应答数据包，第 17 行是上位机发送的 UDP 数据包，第 18 行是开发板返回的 UDP 数据包。双击开发板返回的数据包，可以看到开发板发送的详细数据，如下图所示：

17	14.134554	192.168.1.102	192.168.1.10	UDP	55	1234 → 1234	Len=13
18	14.135991	192.168.1.10	192.168.1.102	UDP	60	1234 → 1234	Len=13

> Ethernet II, Src: a8:b1:3b:75:9f:6a (a8:b1:3b:75:9f:6a), Dst: CIMSYS_33:44:55 (00:11:22:33:44:55)

▼ Internet Protocol Version 4, Src: 192.168.1.102, Dst: 192.168.1.10

0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 41
Identification: 0x2125 (8485)
> Flags: 0x0000
Fragment offset: 0
Time to live: 128
Protocol: UDP (17)
Header checksum: 0x95de [validation disabled]
[Header checksum status: Unverified]
Source: 192.168.1.102
Destination: 192.168.1.10

> User Datagram Protocol, Src Port: 1234, Dst Port: 1234

> Data (13 bytes)

0000	00 11 22 33 44 55 a8 b1 3b 75 9f 6a 08 00 45 00	.. "3DU.. ;u-j..E.
0010	00 29 21 25 00 00 80 11 95 de c0 a8 01 66 c0 a8	.)!%....f..
0020	01 0a 04 d2 04 d2 00 15 83 e7 52 4b 2d 5a 59 4e	RK-ZYN
0030	51 37 30 32 30 2d 46	Q7020-F

图 15-50 wireshark 抓取到的详细数据

由上图可知，源 IP 地址（电脑 IP 地址）为 192.168.1.102，目的 IP 地址（开发板 IP 地址）为 192.168.1.10，源端口号和目的端口号都是 1234。可以看到，UDP 的用户数据段对应的 ASIC 码为“RK-ZYNQ7020-F”。

10.本章总结

至此，以太网 UDP 测试实验的讲解就完毕了。通过实验，相信读者对于 UDP 的概念以及 UDP 的协议已经有了一定的了解。希望大家可以多花一些时间对 UDP 协议进行巩固学习，彻底掌握本章节中关于 UDP 协议的内容。